HTML

MDN Web Docs (Mozilla) — Português do Brasil

Fonte oficial: https://developer.mozilla.org/pt-BR/docs/Web/HTML
Gerado para o projeto doc-frog

HTML: Linguagem de Marcação de Hipertexto

HTML (Linguagem de Marcação de HiperTexto) é o bloco de construção mais básico da web. Define o significado e a estrutura do conteúdo da web. Outras tecnologias além do HTML geralmente são usadas para descrever a aparência/apresentação (CSS) ou a funcionalidade/comportamento (JavaScript) de uma página da web.

"Hipertexto" refere-se aos *links* que conectam páginas da Web entre si, seja dentro de um único site ou entre sites. Links são um aspecto fundamental da web. Ao carregar conteúdo na Internet e vinculá-lo a páginas criadas por outras pessoas, você se torna um participante ativo na world wide web.

O HTML usa "Marcação" para anotar texto, imagem e outros conteúdos para exibição em um navegador da Web. A marcação HTML inclui "elementos" especiais, como `<head>`, `<title>`, `<body>`, `<header>`, `<footer>`, `<article>`, `<section>`, `<p>`, `<div>`, `<span>`, `<img>`, `<aside>`, `<audio>`, `<canvas>`, `<datalist>`, `<details>`, `<embed>`, `<nav>`, `<output>`, `<progress>`, `<video>`, `<ul>`, `<ol>`, `<li>` e muitos outros.

Um elemento HTML é separado de outro texto em um documento por "tags", que consistem no nome do elemento entre "`<`" e "`>`". O nome de um elemento dentro de uma tag é insensível a maiúsculas e minúsculas. Isto é, pode ser escrito em maiúsculas, minúsculas ou um mistura. Por exemplo, a tag `<title>` pode ser escrita como `<Title>`, `<TITLE>` ou de qualquer outra forma.

Os artigos abaixo podem ajudá-lo a aprender mais sobre HTML.

- Introdução ao HTML

  Se você é novo no desenvolvimento web, não deixe de ler nosso artigo O Básico do HTML para aprender o que é HTML e como usá-lo.

- Tutoriais HTML

  Para artigos sobre como usar o HTML, assim como tutoriais e exemplos completos, confira nossa Área de Aprendizado de HTML.

- Referência do HTML

  Em nossa extensa seção de Referência do HTML, você encontrará os detalhes sobre cada elemento e atributo do HTML.

Tutoriais para iniciantes

Nossa Área de Aprendizado de HTML apresenta vários módulos que ensinam HTML desde o início — sem necessidade de conhecimento prévio.

Introdução ao HTML

> Este módulo fornece uma base de conhecimento, permitindo que você acostume-se com os conceitos e sintaxes importantes, tais como aplicar HTML para texto, como criar *hyperlinks* e como usar HTML para estruturar uma página web.

Multimídia e como incorporá-la

> Este módulo explora como usar HTML para incluir multimídia em sua página web, incluindo as diferentes maneiras para incluir imagens, e como incorporar vídeos, áudios e até como colocar uma página inteira em outras páginas web.

Tabelas HTML

> Representar dados tabulares em uma página web de maneira entendível e acessível pode ser um desafio. Este módulo cobre marcação tabular básica, junto com outras funcionalidades mais complexas como legendas e sumários.

Formulários HTML

> Formulários são partes muito importantes na Web — proporcionam muitas das funcionalidades que você precisa para interagir em páginas web. Exemplos disso são registrar-se e conectar-se, enviar comentários, comprar produtos e muito mais. Este módulo lhe permite começar a criar as partes visíveis dos formulários, ou seja, as partes que os usuários interagem.

Usando HTML para resolver problemas comuns

Apresenta links para seções com conteúdo explicando como usar HTML para resolver problemas muito comuns, quando se cria uma página web: como trabalhar com títulos, agregar imagens ou vídeos, colocar ênfase em conteúdo, criar um formulário básico, etc.

Tópicos avançados

Habilitando Imagem CORS

O atributo `crossorigin`, em combinação com um cabeçalho CORS adequado, permite definir imagens pelo elemento `<img>` para ser carregado de outras fontes em um elemento `<canvas>` como se estivessem sendo carregados da fonte atual.

Configuração de atributos CORS

Alguns elementos em HTML que suportam CORS, como por exemplo o `<img>` ou `<video>`, têm o atributo `crossorigin` (a propriedade `crossOrigin`), que permite configurar as requisições CORS para os dados recebidos pelo elemento.

Gerenciamento de foco em HTML

O atributo DOM `activeElement` e o método DOM `hasFocus()` lhe ajudam a melhorar a interação entre o usuário e os elementos da página.

Tipos de conexões

Em HTML, vários tipos de links podem ser utilizados para realizar conexões entre documentos, como por exemplo `<a>`, `<area>` e `<link>`.

Formatos de arquivos suportados pelos elementos de áudio e vídeo em HTML

Os elementos `<audio>` e `<video>` permitem que você reproduza áudios e vídeos no seu HTML. Estes elementos fornecem uma alternativa nativa de recursos do navegador, parecidos com a do Adobe Flash Player e outros plug-ins.

Usando o cachê de aplicação

O cachê de aplicação tem como o objetivo executar a aplicação offline. Você pode usar a interface **Application Cache** (*AppCache*) para escolher elementos que o browser possa guardar, deixando-os disponíveis para usuários desconectados. As aplicações que forem armazenadas em cache podem ser utilizadas sem nenhum problema, mesmo que o usuário clique no botão de atualizar a página enquanto estiver offline.

Pré-carregando conteúdo com rel="preload"

O valor `preload` do atributo `rel` do elemento `<link>` permite que você escreva solicitações de busca declarativas em seu elemento `<head>` do HTML, especificando recursos que suas páginas vão utilizar logo após o carregamento. Você também pode necessitar que estes recursos sejam pré-carregados ao início do ciclo de execução da página e depois que o mecanismo de renderização do navegador comece a agir. Isso garante que estarão disponíveis mais cedo e que seja menos provável que sua página seja bloqueada do processo de renderizar. Isso aumenta a performance da página. Este artigo oferece um guia básico de como `preload` funciona.

Referências

Referência do HTML

HTML é formado de **elementos**, onde cada um deles pode ser modificado por alguns **atributos**. Documentos HTML são interconectados através de links.

Referência dos Elementos HTML

Navegue em uma lista com todos os elementos HTML.

Referência de atributos em HTML

Elementos em HTML possuem atributos. Estes configuram ou ajustam o funcionamento e o comportamento dos seus elementos de muitas maneiras.

Atributos Globais

> Os atributos globais podem ser utilizados em todos os elementos HTML, *mesmo aqueles que não são especificados como padrão*. Isso significa que qualquer elemento não padrão vai permitir estes atributos, mesmo que este retorne um documento HTML5 incompatível.

Elementos em linha e Elementos em nível de bloco

> Elementos HTML são normalmente elementos *inline* (em linha) ou *block-level* (em nível de bloco). Um elemento em linha ocupa somente o espaço limitado pelas tags que o definem. Um elemento em nível de bloco ocupa o espaço inteiro do elemento pai (container), portanto criando um bloco.

Tipos de Links

> No HTML, vários tipos de links podem ser utilizados para estabelecer e definir o relacionamento entre dois documentos. Os tipos de elementos de link que podem ser incluídos são `<a>`, `<area>` e `<link>`.

Formatos de mídia suportados pelos elementos vídeo e áudio do HTML

> Os elementos `<audio>` e `<video>` permitem que você reproduza mídias de áudio e vídeo. Estes elementos fornecem uma alternativa nativa para o navegador reproduzir recursos similares aos encontrados no Adobe Flash e outros plug-ins.

Tipos de conteúdo HTML

> O HTML é composto de uma grande diversidade de conteúdos, e cada um pode ser usado em certos contextos e não ser permitido em outros. Da mesma forma, cada um tem um conjunto de outras categorias de conteúdos que podem ou não serem usadas dentro deles. Este é um guia destas categorias.

Os Modos Quirks e o Modo Padrão

> Informações sobre a história dos Modos *Quirks* e o Modo padrão.

Tópicos relacionados

Aplicando cores em elementos HTML usando CSS

> Este artigo mostra as principais formas de se usar CSS para adicionar cores ao seu conteúdo HTML, mostrando que partes dos documentos HTML podem ser coloridas e quais propriedades utilizar. Inclui exemplos e links para ferramentas de *palette-building*, entre outros.

Categorias de conteúdo

Cada elemento HTML deve permanecer, por regras, definindo que tipo de conteúdo ele pode ter. Essas regras estão agrupadas em modelos de conteúdo para muitos elementos. Cada elemento HTML pertence a nenhum, um, ou múltiplos modelos de conteúdo, cada regra de definição que o conteúdo do elemento deve seguir em um documento HTML.

Há três tipos de categorias de conteúdo:

- Principais categorias de conteúdo, as quais descrevem regras comuns de conteúdo compartilhado por vários elementos;
- Categorias de conteúdo relacionado, as quais decrevem regras comuns de conteúdo para elementos relacioados;
- Categorias de conteúdo específico, as quais descrevem categorias raras compartilhadas somente por poucos elementos, às vezes apenas em um contexto específico.



Principais categorias de conteúdo

Conteúdo de metadados

Os elementos pertencentes a categoria *conteúdo de metadados* modificam a apresentação ou o comportamento do resto do documento, define ligações para outros documentos ou transmite outras informações *fora da banda*.

Os elementos pertencentes a essa categoria são `<base>`, `<command>`, `<link>`, `<meta>`, `<noscript>`, `<script>`, `<style>` e `<title>`.

Conteúdo de fluxo

Elementos pertencentes a categoria de conteúdo de fluxo tipicamente contém texto ou conteúdo embutido. Eles são: `<a>`, `<abbr>`, `<address>`, `<article>`, `<aside>`, `<audio>`, `<b>`,`<bdo>`, `<bdi>`, `<blockquote>`, `<br>`, `<button>`, `<canvas>`, `<cite>`, `<code>`, `<command>`, `<data>`, `<datalist>`, `<del>`, `<details>`, `<dfn>`, `<div>`, `<dl>`, `<em>`, `<embed>`, `<fieldset>`, `<figure>`, `<footer>`, `<form>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<header>`, `<hgroup>`, `<hr>`, `<i>`, `<iframe>`, `<img>`, `<input>`, `<ins>`, `<kbd>`, `<label>`, `<main>`, `<map>`, `<mark>`, `<math>`, `<menu>`, `<meter>`, `<nav>`, `<noscript>`, `<object>`, `<ol>`, `<output>`, `<p>`, `<pre>`, `<progress>`, `<q>`, `<ruby>`, `<s>`, `<samp>`, `<script>`, `<section>`, `<select>`, `<small>`, `<span>`, `<strong>`, `<sub>`, `<sup>`, `<svg>`, `<table>`, `<template>`, `<textarea>`, `<time>`, `<ul>`, `<var>`, `<video>`, `<wbr>` e Text.

Alguns outros elementos pertencem a essa categoria, mas somente se uma condição específica é realizada:

- `<area>`, se for um descendente de um elemento `<map>`
- `<link>`, se o atributo **itemprop** estiver presente
- `<meta>`, se o atributo **itemprop** estiver presente
- `<style>`, se o atributo `scoped` estiver presente

Conteúdo de seccionamento

Os elementos pertencentes ao modelo de conteúdo de seccionamento criam uma seção no esboço atual que define o escopo dos elementos `<header>`, elementos `<footer>` e na conteúdo do cabeçalho.

Elementos pertencentes a essa categoria são `<article>`, `<aside>`, `<nav>` e `<section>`.

> **Nota:** Não confunda esse modelo de conteúdo com a categoria de seccionamento raiz que isola seus conteúdos dos esboços regulares.

Conteúdo do cabeçalho

O conteúdo do cabeçalho define o título de uma seção, se é marcada por um explícito elemento do conteúdo de seccionamento ou implicitamente definido pelo próprio conteúdo do cabeçalho.

Os elementos pertencentes a essa categoria são `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>` e `<hgroup>`.

> **Nota:** Embora provavelmente contenha algum conteúdo do cabeçalho, o `<header>` não faz parte do conteúdo do cabeçalho em si.

Conteúdo fraseado

O conteúdo fraseado define o texto e a marcação que ele contém. Séries de conteúdos fraseados compõem parágrafos.

Os elementos pertencentes a essa categoria são `<abbr>`, `<audio>`, `<b>`, `<bdo>`, `<br>`, `<button>`, `<canvas>`, `<cite>`, `<code>`, `<command>`, `<datalist>`, `<dfn>`, `<em>`, `<embed>`, `<i>`, `<iframe>`, `<img>`, `<input>`, `<kbd>`, `<label>`, `<mark>`, `<math>`, `<meter>`, `<noscript>`, `<object>`, `<output>`, `<progress>`, `<q>`, `<ruby>`, `<samp>`, `<script>`, `<select>`, `<small>`, `<span>`, `<strong>`, `<sub>`, `<sup>`, `<svg>`, `<textarea>`, `<time>`, `<var>`, `<video>`, `<wbr>` e texto simples (não consistindo somente de caracteres de espaço em branco).

Alguns outros elementos pertencem a essa categoria, mas somente se uma condição específica é realizada:

- `<a>`, se contém somente conteúdo fraseado
- `<area>`, se for um descendente de um elemento `<map>`
- `<del>`, se contém somente conteúdo fraseado
- `<ins>`, se contém somente conteúdo fraseado
- `<link>`, se o atributo **itemprop** estiver presente
- `<map>`, se contém somente conteúdo fraseado
- `<meta>`, se o atributo **itemprop** estiver presente

Conteúdo embutido

O conteúdo embutido importa outro recurso ou insere conteúdo de uma outra linguagem de marcação no documento. Os elementos que pertencem a essa categoria incluem: `<audio>`, `<canvas>`, `<embed>`, `<iframe>`, `<img>`, `<math>`, `<object>`, `<svg>` e `<video>`.

Conteúdo interativo

O conteúdo interativo inclui elementos que são especificamente desenvolvidos para a interação do usuário. Os elementos que pertencem a essa categoria incluem: `<a>`, `<button>`, `<details>`, `<embed>`, `<iframe>`, `<label>`, `<select>`, e `<textarea>`. Alguns elementos pertencem a essa categoria somente sob condições específicas:

- `<audio>`, se o atributo `controls` estiver presente
- `<img>`, se o atributo `usemap` estiver presente
- `<input>`, se o atributo `type` não estiver no modo escondido
- `<menu>`, se o atributo `type` estiver no modo barra de ferramentas
- `<object>`, se o atributo `usemap` estiver presente
- `<video>`, se o atributo `controls` estiver presente

Conteúdo associado ao form

O conteúdo associado ao **form** compreende elementos que têm um formulário de proprietário, exposto por um atributo **form**. Um formulário de proprietário é ou um elemento `<form>` ou o elemento o qual o ID é especificado no atributo **form**.

- `<button>`
- `<fieldset>`
- `<input>`
- `<label>`
- `<meter>`
- `<object>`
- `<output>`
- `<progress>`
- `<select>`
- `<textarea>`

Essa categoria contém várias sub-categorias:

listed

> Elementos que estão listados nas coleções IDL form.elements e fieldset.elements. Contém `<button>`, `<fieldset>`, `<input>`, `<object>`, `<output>`, `<select>` e `<textarea>`.

labelable

> Elementos que podem ser associados com elementos `<label>`. Contém `<button>`, `<input>`, `<meter>`, `<output>`, `<progress>`, `<select>` e `<textarea>`.

submittable

> Elementos que podem ser usados para construir o formulário de dados quando o formulário é enviado. Contém `<button>`, `<input>`, `<object>`, `<select>`, e `<textarea>`.

resettable

> Elementos que podem ser afetados quando um formulário é reinicializado. Contém `<input>`, `<output>`,`<select>` e `<textarea>`.

Modelo de conteúdo transparente

Se um elemento tem um modelo de conteúdo transparente, então seu conteúdo deve ser estruturado de tal forma que ele seja um HTML5 válido, mesmo se o elemento transparente for removido e substituído pelos elementos filhos.

Por exemplo, os elementos `<del>` e `<ins>` são transparentes:

```
<p>We hold these truths to be <del><em>sacred &amp;amp; undeniable</em></del> <ins>self-evident</ins>.</p>
```

Se aqueles elementos forem removidos, esse fragmento continuaria sendo um HTML válido.

```
<p>We hold these truths to be <em>sacred &amp;amp; undeniable</em> self-evident.</p>
```

Outros modelos de conteúdo

Raiz de secionamento.

Microformatos

Microformatos (ás vezes abreviado como **µF**) são convenções utilizadas para incorporar convenções semânticas em HTML e providenciar uma API a ser usada por mecanismos de pesquisa, agregadores e outras ferramentas. Esses padrões mínimos de HTML são usados para marcar entidades que variam de informações fundamentais a específicas de domínio, como pessoas, organizações, eventos e locais.

Os microformatos são suportados pelos principais mecanismos de pesquisa. Os dados são transportados na propriedade de classe que pode ser adicionada a qualquer elemento HTML. Além de legíveis por máquina, seu formato é projetado para ser facilmente lido por humanos.

Existem bibliotecas de análise para a maioria das linguagens para microformatos2.

Como os Microformatos Funcionam?

Um autor de uma página web pode adicionar microformatos no seu HTML. Por exemplo se ele quer se identificar ele podem utilizar um h-card como:

```
<a class="h-card" href="http://example.com">Joe Bloggs</a>
```

Quando um analisador encontra esse dado, ele saberá que nessa página contém um "card" que descreve uma pessoa ou uma organização chamada `Joe Blogger` com uma URL de `http://example.com/`. O analisador disponibiliza esses dados por meio de APIs que podem ser usadas por diferentes aplicativos.

Como neste exemplo, alguns padrões de marcação requerem apenas um único nome de classe raiz de microformato, que os analisadores usam para encontrar algumas propriedades genéricas, como: `name`, `url`, `photo`.

Prefixos de Microformatos

Todos os microformatos consistem de uma raíz e uma coleção de propriedades. As propriedades são opcionais e potencialmente multivaloradas - aplicações que necessitam de um valor único devem utilizar a primeira instância de uma propriedade. Os dados hierárquicos são representados com microformatos aninhados, normalmente como valores de propriedade.

Todas os nomes de classes de microformatos utilizam prefixos. Os prefixos são Os prefixos são **independentes da sintaxe dos vocabulários**, os quais são desenvolvidos separadamente.

- **"h-*" para nome de classes raíz**, p.ex "h-card", "h-entry", "h-feed", e várias outras. Essas classes de alto nível normalmente indicam um tipo e o vocabulário esperado de propriedades correspondente. Por exemplo:
    - h-card descreve uma pessoa ou uma organização.
    - h-entry descreve conteúdo on-line em série ou com data marcada como uma postagem de blog.
    - h-feed descreve um fluxo de dados ou um feed de postagens.
    - Você pode encontrar vários outros You can find many more vocabulários na wiki de microformatos2.
- **"p-*" para propriedades de texto simples,** p.ex "p-name", "p-summary"
    - Análise de texto simples, texto de elemento em geral. Em certos elementos HTML, use atributos especiais primeiro, por exemplo img / alt, abbr / título.
- **"u-*" para propriedades URL**, p.ex "u-url", "u-photo", "u-logo"
    - Análise especial necessária: prefer a/href, img/src, object/data etc. atributos sobre o conteúdo do elemento.
- **"dt-*" para propriedades de data e hora**, p.ex "dt-start", "dt-end", "dt-bday"
    - Análise especial necessária: value-class-pattern and separate date time value parsing for readability.

- **"e-*" para propriedades da árvore de elementos** em que toda a hierarquia de elementos contidos é o valor, p.ex "e-content". O prefixo "e-" também pode ser lembrado mnemonicamente como "árvore de elementos", "marcação incorporada", ou "marcação encapsulada".

Alguns exemplos de microformatos

h-card

O microformato h-card representa uma pessoa ou uma organização.

O valor de cada propriedade é definido no HTML utilizando a propriedade class, qualquer elemento pode receber.

Exemplo de h-card

```
<p class="h-card">
  <img class="u-photo" src="http://example.org/photo.png" alt="" />
  <a class="p-name u-url" href="http://example.org">Joe Bloggs</a>
  <a class="u-email" href="mailto:joebloggs@example.com"
    >joebloggs@example.com</a
  >,
  <span class="p-street-address">17 Austerstræti</span>
  <span class="p-locality">Reykjavík</span>
  <span class="p-country-name">Iceland</span>
</p>
```

Propriedade	Descrição
p-name	O nome completo/formatado da pessoa ou organização
u-email	endereço de e-mail
u-photo	uma foto da
u-url	página na web ou outra URL representando a pessoa ou a organização
u-uid	identificador universal único, de preferência URL canônico
p-street-address	número da rua + enderço
p-locality	cidade ou vilarejo
p-country-name	nome do país

Exemplo de h-card aninhado

```
<div class="h-card">
  <a class="p-name u-url" href="http://blog.lizardwrangler.com/"
    >Mitchell Baker</a
  >
  (<a class="p-org h-card" href="http://mozilla.org/">Mozilla Foundation</a>)
</div>
```

JSON analisado:

```
{
  "items": [{
  "type": ["h-card"],
  "properties": {
    "name": ["Mitchell Baker"],
    "url": ["http://blog.lizardwrangler.com/"],
    "org": [{
    "value": "Mozilla Foundation",
    "type": ["h-card"],
    "properties": {
      "name": ["Mozilla Foundation"],
      "url": ["http://mozilla.org/"]
    }
    }]
```

```
  }
  }]
}
```

Nota: o h-card aninhado implica nas propriedades 'name' e 'url', assim como qualquer outro h-card apenas com nome de classe raiz em um `<a href>`.

h-entry

O microformato h-entry representa um conteúdo em série ou um conteúdo com data marcada na web. h-entry é frequentemente usado com conteúdo destinado a ser distribuído, p.ex postagens em blog.

Exemplo de h-entry como uma postagem em blog:

```
<article class="h-entry">
  <h1 class="p-name">Microformats are amazing</h1>
  <p>
    Published by
    <a class="p-author h-card" href="http://example.com">W. Developer</a> on
    <time class="dt-published" datetime="2013-06-13 12:00:00"
      >13<sup>th</sup> June 2013</time
    >
  </p>

  <p class="p-summary">In which I extoll the virtues of using microformats.</p>

  <div class="e-content">
    <p>Blah blah blah</p>
  </div>
</article>
```

Propriedades

Propriedade	Descrição
`p-name`	nome/título da entrada
`p-author`	quem escreveu a entrada, h-card opcionalmente incorporado
`dt-published`	quando a entrada foi publicada
`p-summary`	breve resumo da entrada
`e-content`	conteúdo completo da entrada

Exemplo de h-entry analisado

```
<div class="h-entry">
  <p><span class="p-author h-card">
    <a href="https://quickthoughts.jgregorymcverry.com/profile/jgmac1106" ><img class="u-photo"
src="https://quickthoughts.jgregorymcverry.com/file/2d6c9cfed7ac8e849f492b5bc7e6a630/thumb.jpg"/></a>
    <a class="p-name u-url" href="https://quickthoughts.jgregorymcverry.com/profile/jgmac1106">Greg McVerry</a></span>
     Replied to <a class="u-in-reply-to" href="https://developer.mozilla.org/pt-BR/docs/Web/HTML/microformats">a post
on
   <strong>developer.mozilla.org</strong> </a>:
  </p>
   <p class="p-name e-content">Hey thanks for making this microformats resource</p>
   <p> <a href="https://quickthoughts.jgregorymcverry.com/profile/jgmac1106">Greg McVerry</a>
  published this <a class="u-url url" href="https://quickthoughts.jgregorymcverry.com/2019/05/31/hey-thanks-for-
making-this-microformats-resource"><time class="dt-published"
   datetime="2019-05-31T14:19:09+0000">31 May 2019</time></a></p>
</div>
```

```
{
  "items": [
    {
      "type": [ "h-entry" ],
```

```
      "properties": {
        "in-reply-to": [ "https://developer.mozilla.org/pt-BR/docs/Web/HTML/microformats" ],
        "name": [ "Hey thanks for making this microformats resource" ],
        "url": [ "https://quickthoughts.jgregorymcverry.com/2019/05/31/hey-thanks-for-making-this-microformats-
resource" ],
        "published": [ "2019-05-31T14:19:09+0000" ],
        "content": [
          {
            "html": "Hey thanks for making this microformats resource",
            "value": "Hey thanks for making this microformats resource",
            "lang": "en"
          }
        ],
        "author": [
          {
            "type": [ "h-card" ],
            "properties": {
              "name": [ "Greg McVerry" ],
              "photo": [ "https://quickthoughts.jgregorymcverry.com/file/2d6c9cfed7ac8e849f492b5bc7e6a630/thumb.jpg"
],
              "url": [ "https://quickthoughts.jgregorymcverry.com/profile/jgmac1106" ]
            },
            "lang": "en",
            "value": "Greg McVerry"
          }
        ]
      },
      "lang": "en"
    }
```


h-feed

O h-feed é um fluxo de dados ou um feed de posts de h-entry, como postagens completas em uma página inicial ou em páginas de arquivo, sumários ou outras listagens de postagens

Exemplo h-feed

```
<div class="h-feed">
  <h1 class="p-name">Microformats Blogs</h1>
  <article class="h-entry">
    <h2 class="p-name">Microformats are amazing</h2>
    <p>
      Published by
      <a class="p-author h-card" href="http://example.com">W. Developer</a> on
      <time class="dt-published" datetime="2013-06-13 12:00:00"
        >13<sup>th</sup> June 2013</time
      >
    </p>
    <p class="p-summary">
      In which I extoll the virtues of using microformats.
    </p>
    <div class="e-content"><p>Blah blah blah</p></div>
  </article>
</div>
```

Propriedades

Propriedades	Descrição
`p-name`	nome do feed
`p-author`	autor do feed, opcionalmente incorporado com um h-card

Filhos

h-entry aninhado	
objetos representando os itens do feed	

h-event

O `h-event` é para evento na web. O h-event é frequentemente usado com listagens de eventos e páginas de eventos individuais.

```
<div class="h-event">
  <h1 class="p-name">Microformats Meetup</h1>
  <p>
    From
    <time class="dt-start" datetime="2013-06-30 12:00"
      >30<sup>th</sup> June 2013, 12:00</time
    >
    to <time class="dt-end" datetime="2013-06-30 18:00">18:00</time> at
    <span class="p-location">Some bar in SF</span>
  </p>
  <p class="p-summary">
    Get together and discuss all things microformats-related.
  </p>
</div>
```

Propriedades

Propriedade	Descrição
`p-name`	nome do evento (ou título)
`p-summary`	breve sumário do evento
`dt-start`	data e hora de início do evento
`dt-end`	data e hora de termino do evento
`p-location`	local do evento, opcionalmente incorporado com um h-card

Exemplo de h-event analisado

```
<div class="h-event">
  <h2 class="p-name">IndieWeb Summit</h2>
  <time class="dt-start" datetime="2019-06-29T09:00:00-07:00">June 29, 2019 at 9:00am  (-0700)</time><br>through <time
class="dt-end" datetime="2019-06-30T18:00:00-07:00">June 30, 2019 at 6:00pm (-0700)</time><br>
  <div class="p-location h-card">
    <div>
    <span class="p-name">Mozilla</span>
     </div>
     <div>
      <span class="p-street-address">1120 NW Couch St</span>,
      <span class="p-locality">Portland</span>,
      <span class="p-region">Oregon</span>,
      <span class="p-country">US</span>
     </div>
       <data class="p-latitude" value="45.52345"></data>
      <data class="p-longitude" value="-122.682677"></data>
  </div>
    <div class="e-content">Come join us
     </div>
    <div>
     <span class="p-author h-card"><a class="u-url p-name" href="https://aaronparecki.com">Aaron Parecki</a></span>
Published this <a href="https://aaronparecki.com/2019/06/29/1/" class="u-url">event </a>on <time class="dt published"
datetime="2019-05-25T18:00:00-07:00">May 5th, 2019</time>
    </div>
</div>
```

```
{
  "items": [
```

```
  {
    "type": [ "h-event" ],
    "properties": {
      "name": [ "IndieWeb Summit" ],
      "url": [ "https://aaronparecki.com/2019/06/29/1/" ],
      "author": [
        {
          "type": [ "h-card" ],
          "properties": {
            "name": [ "Aaron Parecki" ],
            "url": [ "https://aaronparecki.com"]
          },
          "lang": "en",
          "value": "Aaron Parecki"
        }
      ],
      "start": [ "2019-06-29T09:00:00-07:00" ],
      "end": [ "2019-06-30T18:00:00-07:00" ],
      "published": [ "2019-05-25T18:00:00-07:00" ],
      "content": [
        {
          "html": "Come join us",
          "value": "Come join us",
          "lang": "en"
        }
      ],
      "location": [
        {
          "type": [ "h-card" ],
          "properties": {
            "name": [ "Mozilla" ],
            p-street-address: [ "1120 NW Couch St" ]
            "locality": [ "Portland" ],
            "region": [ "Oregon" ],
            "country": [ "US" ],
            "latitude": [ "45.52345" ],
            "longitude": [ "-122.682677" ]
          },
          "lang": "en",
          "value": "Mozilla"
        }
      ]
    },
    "lang": "en"
  }
],
```

See also

- Microformat na Wikipedia
- Site oficial dos Microformatos
- Motores de busca suportados no site oficial do Microformats
- Microformatos no IndieWebCamp

Quirks Mode e Standards Mode

Nos velhos dias da web, páginas eram tipicamente escritas em duas versões: Uma para o Netscape Navigator, e outra para o Microsoft Internet Explorer. Quando os padrões web foram criados pelo W3C, navegadores não puderam começar a usá-los imediatamente, pois isto iria quebrar a maior parte dos sites existentes na web. Portanto os navegadores introduziram dois modos para tratar os novos padrões em sites condescendentes diferentemente dos antigos sites legados.

Existem agora três modos usados pelos mecanismos de layout nos navegadores web: "quirks mode" ("modo equivocado"), "almost standards mode" ("modo quase padrão"), e "full standards mode" ("modo de padrões completos"). Em **quirks mode**, o layout emula o comportamento não-padrão do Netscape Navigator 4 e do Internet Explorer 5 para Windows que é requerido para não quebrar o conteúdo existente na Web. No **full standards mode**, o comportamento é (espera-se) o descrito pelas

especificações do HTML e CSS. No **almost standards mode**, há apenas um número muito pequeno de peculiaridades não-padrão implementadas.

Como os navegadores determinam qual modo usar?

Para documentos HTML, os navegadores usam um DOCTYPE no início do documento para decidir se os tratarão em *quirks mode* ou *standards mode*. Para garantir que sua página use o *full standards mode*, certifique-se que sua página tenha um DOCTYPE como deste exemplo:

```
<!doctype html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>Olá Mundo!</title>
  </head>
  <body></body>
</html>
```

O DOCTYPE mostrado no exemplo, `<!DOCTYPE html>`, é o mais simples possível, e o recomendado pelo HTML5. As versões anteriores do HTML padrão recomendavam outras variantes, mas todos os navegadores existentes hoje irão usar o *full standards mode* para este DOCTYPE, até mesmo o Internet Explorer 6. Não há razões válidas para usar um DOCTYPE mais complicado. Se você usar outro DOCTYPE, você pode correr o risco de escolher um que acione o almost *standards mode* ou o *quirks mode*.

Certifique-se de colocar o DOCTYPE logo no início de seu documento HTML. Qualquer coisa antes do DOCTYPE, como um comentário ou uma declaração XML irá acionar o *quirks mode* no Internet Explorer 9 e versões anteriores.

Em HTML5, o único propósito do DOCTYPE é de ativar o *full standards mode*. Versões mais antigas do padrão HTML deram significado adicional ao DOCTYPE, mas nenhum navegador jamais usou o DOCTYPE para qualquer outra coisa além de alternar entre o *quirks mode* e o *standards mode*.

Veja também uma descrição detalhada de quando diferentes navegadores escolhem diferentes modos.

XHTML

Se você servir sua página como XHTML usando o MIME type `application/xhtml+xml` no cabeçalho HTTP `Content-Type`, você não precisará de um DOCTYPE para ativar o *full standards mode*, já que tais documentos sempre usam este modo. Note porém que servir suas páginas como `application/xhtml+xml` irá fazer com que o Internet Explorer 8 (e anteriores) mostre uma caixa de diálogo para download de um formato desconhecido ao invés de exibir sua página, pois a primeira versão do Internet Explorer com suporte ao XHTML é o Internet Explorer 9.

Se você servir conteúdo como XHTML usando o MIME type `text/html`, os navegadores irão lê-lo como HTML, e você precisará colocar o DOCTYPE para usar o *standards mode*.

Como faço para ver o modo usado?

No Firefox, selecione *Propriedades da página* no menu de contexto, e procure por *Renderização*.

No Internet Explorer, pressione *F12*, e procure por *Modo de Documento*.

Quais as diferenças entre os modos?

Veja a lista de peculiaridades ("quirks") e "almost standards mode" para entender a diferença entre estes modos.

Imagens responsivas

- Anterior
- Menu: Multimídia e Incorporação
- Próxima

Neste artigo nós iremos aprender sobre o conceito de imagens responsivas — imagens que funcionam em dispositivos com diferentes tamanhos de tela, resoluções e outras funcionalidades — e entrar em contato com quais ferramentas o HTML oferece para ajudar a implementá-las. Imagens responsivas são apenas uma parte do web design responsivo, um futuro tópico de CSS para você aprender.

Pré-requisitos:	**Você deve ter visto a introdução ao HTML e como adicionar imagens estáticas numa página web.**
Objetivo:	Aprender como usar funcionalidades como `srcset` e o elemento `<picture>` para implementar soluções de imagens responsivas em websites.

Por que imagens responsivas?

Então, qual problema nós estamos tentando resolver com imagens responsivas? Vamos examinar um cenário típico. Um típico website provavelmente tem uma imagem de cabeçalho, para ter uma boa aparência para os visitantes e talvez mais algumas imagens no conteúdo abaixo. Você provavelmente quer fazer uma imagem do cabeçalho preencher toda a largura do cabeçalho, e que as imagens no conteúdo caibam dentro de alguma coluna. Vamos dar uma olhada em um exemplo simples disso:

 Our example site as viewed on a wide screen - here the first image works ok, as it is big enough to see the detail in the center.

Isto funciona bem em um dispositivo de tela grande, como um laptop ou desktop (você pode ver o exemplo ao vivo e encontrar o código-fonte no Github.) Nós não vamos discutir muito o CSS, exceto para dizer que:

- O conteúdo do <body> foi colocado para o máximo de 1200 pixels de largura (width) —em viewports acima, o body continua a 1200 pixels e centrado no espaço disponível. Em viewports abaixo, o body vai usar 100% da largura disponível.
- A imagem de cabeçalho foi colocada para estar sempre no centro, não importando a largura do título. Então, se a página está sendo vista em uma tela mais estreita, o detalhe importante no centro da imagem (as pessoas) continuam sendo vistas, o excesso é perdido nos lados e tem 200 pixels de altura.
- As imagens do conteúdo foram configuradas para caso o elemento body se torne menor que as imagens, então elas comecem a diminuir. Assim sempre estarão dentro do body, mesmo que ultrapassando ele.

Isto está bom, mas o problema vem quando você começa a ver a página em uma tela estreita - o cabeçalho parece bom, mas está começando a pegar um tamanho grande para um dispositivo móvel; A primeira imagem do conteúdo por outro lado parece terrível - neste tamanho você mal consegue ver as pessoas nela.

 Our example site as viewed on a narrow screen; the first image has shrunk to the point where it is hard to make out the detail on it.

Seria muito melhor mostrar uma versão cortada da imagem que contenha os detalhes importantes quando a página é vista em uma tela estreita, e talvez algo entre as duas para uma tela de largura média como um tablet - isto é comumente conhecido como o **problema de direção artística**.

Ainda, não é preciso embutir estas imagens grandes em páginas se será visto em pequenas telas de celulares; isto é chamado de problema de mudança de resolução - uma imagem rasterizada é um número de pixels de largura e um número de pixels de altura; como nós vimos quando olhamos para vetores gráficos, uma imagem rasterizada começa a parecer granulada e horrível se é mostrada maior que seu tamanho original (enquanto que um vetor não). E se isto é mostrado muito menor que seu tamanho original, é um desperdício de largura de banda - usuários mobile especialmente não querem ter que gastar seu pacote de dados baixando uma imagem grande feita para desktop, enquanto uma imagem pequena poderia ser feita para seu dispositivo. Uma situação ideal seria ter múltiplas resoluções disponíveis e apresentar tamanhos apropriados dependendo dos diferentes dispositivos que acessam a página.

Para tornar as coisas mais complicadas, alguns dispositivos tem uma alta resolução, que demanda imagens maiores do que as que você espera para ficar bom. Isto é essencialmente o mesmo problema, mas em um contexto diferente.

Você pode pensar que imagens vetorizadas resolveriam estes problemas, e elas resolvem em certo grau - elas têm um tamanho pequeno, se adaptam bem, e você deveria usá-las sempre que possível. No entanto elas não são adequadas para todos os tipos de imagem, embora elas sejam ótimas para gráficos simples, padrões, elementos de interface, etc. Criar uma imagem baseada em vetor com o tipo de detalhe que você encontraria em uma foto é complexo. Por exemplo, formatos de imagens rasterizadas, como JPEGs, são melhores para este tipo de detalhe como nós vemos no exemplo acima.

Este tipo de problema não existia quando a web começou a existir, no começo dos anos 1990 - naquele tempo somente desktops e laptops navegavam a Web, então engenheiros de navegadores e programadores nem pensavam em implementar soluções. *Tecnologias de imagens responsivas* foram implementadas recentemente para resolver os problemas indicados acima, permitindo a você oferecer ao navegador vários arquivos de imagem, todas mostrando a mesma coisa mas contendo diferentes números de pixels (mudança de resolução), ou diferentes imagens para diferentes espaços de alocação (direção de arte).

> **Nota:** As novas funcionalidades discutidas neste artigo — `srcset`/`sizes`/`<picture>` — são todas suportadas nas versões atuais de navegadores mobile e desktop (incluindo Microsoft Edge, embora não suportada pelo Internet Explorer).

Como você faz para criar imagens responsivas?

Nesta seção, nós iremos ver os dois problemas ilustrados acima e mostrar como resolvê-los usando funcionalidades de imagens responsivas do HTML. Você deve notar que nós iremos focar no elemento HTML `<img>` para esta seção, como visto na área de conteúdo do exemplo acima - a imagem no cabeçalho da página é somente para decoração, e assim implementada usando imagens de background CSS. CSS indiscutivelmente tem ferramentas melhores para design responsivo do que HTML, e nós iremos falar sobre estas ferramentas em um módulo futuro de CSS.

Mudança de resolução: Diferentes tamanhos

Então, qual é o problema que nós queremos resolver com a mudança de resolução? Nós queremos mostrar o conteúdo identico da imagem, somente maior ou menor dependendo do dispositivo - esta é a situação que nós temos com a segunda imagem do conteúdo em nosso exemplo. O padrão do elemento `<img>` tradicionalmente permite somente apontar o navegador para uma única fonte:

```
<img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Guides/Responsive_images/elva-fairy-800w.jpg" alt="Elva
dressed as a fairy" />
```

Nós podemos, entretanto, usar dois novos atributos — `srcset` e `sizes` — para fornecer várias fontes adicionais juntamente com sugestões para ajudar o navegador a escolher a correta. Você pode ver um exemplo disso no nosso exemplo responsive.html no Github (ver também o código fonte):

```
<img
  srcset="
    elva-fairy-320w.jpg 320w,
    elva-fairy-480w.jpg 480w,
    elva-fairy-800w.jpg 800w
  "
  sizes="(max-width: 320px) 280px,
            (max-width: 480px) 440px,
            800px"
  src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Guides/Responsive_images/elva-fairy-800w.jpg"
  alt="Elva dressed as a fairy" />
```

Os atributos `srcset` e `sizes` parecem complicados, mas não são difíceis de entender se você formata eles como mostrado acima, com uma parte diferente do valor do atributo para cada linha. Cada valor contém uma lista separada por vírgula, e cada parte da lista é dividida em três sub-partes. Vamos percorrer o conteúdo de cada agora:

`srcset` define o conjunto de imagens que nós iremos permitir ao navegador escolher, e qual tamanho cada imagem tem. Antes de cada vírgula nós escrevemos:

1. Um **nome do arquivo da imagem** (`elva-fairy-480w.jpg`).
2. Um espaço.
3. A **largura da imagem em pixels** (`480w`) — note que é usada a unidade `w`, e não `px` como você pode esperar. Este é o tamanho real da imagem, que pode ser encontrado no arquivo dela no seu computador (por exemplo no Mac você pode selecionar a imagem no Finder, e pressionar

   `Cmd`

   `+`

   `I`

   para mostrar as informações na tela).

`sizes` define um conjunto de condições de mídia (ex.: largura da tela) e indica qual tamanho da imagem deveria ser a melhor escolha, quando certas condições de tela são verdadeiras - Estas são as sugestões que nós falamos antes. Neste caso, antes de cada vírgula nós escrevemos:

1. Uma **condição de mídia** (`(max-width:480px)`) — Você vai aprender mais sobre isso no tema CSS, mas para agora vamos somente dizer que a condição de mídia descreve um possível estado em que a tela pode estar. Neste caso, nós estamos dizendo "quando a largura da tela é 480px ou menos".
2. Um espaço.
3. A **largura do slot** que a imagem irá preencher quando a condição de mídia for verdadeira (`440px`).

> **Nota:** Para a largura do slot, você pode fornecer um tamanho absoluto (`px`, `em`) ou um tamanho relativo (como porcentagem). Você pode ter notado que o último slot de largura não tem condição de mídia - isto é o padrão que será escolhido quando nenhuma condição for verdadeira. O navegador ignora tudo depois da primeira condição satisfeita, então tenha cuidado com a ordem de condições.

Então, com estes atributos no lugar, o navegador irá:

1. Ver a largura do dispositivo.
2. Ver qual condição de mídia na lista `sizes` é a primeira a ser verdadeira.
3. Ver qual é o slot para aquela condição de mídia.
4. Carregar a imagem definida na lista `srcset` que possui o mesmo tamanho do slot ou, se não houver, da primeira imagem que é maior que o tamanho do slot escolhido.

E é isto! Então neste ponto, se um navegador suportado com uma largura de 480px carregar a página, a condição (`max-width: 480px`) será verdadeira, então o slot `440px` será escolhido, então o `elva-fairy-480w.jpg` será carregada, como a largura inerente (`480w`) é a mais próxima de `440px`. A imagem de 800px é 128KB no disco enquanto que a versão de 480px é somente 63KB - economizando 65KB. Agora imagine se fosse uma página que tivesse várias imagens. Usando esta técnica poderia economizar os dados de usuários de celular.

Navegadores antigos que não suportam estas funcionalidades serão ignorados, seguiremos e carregaremos a imagem definida no atributo `src` como normal.

> **Nota:** No `<head>` do documento você encontrará a linha `<meta name="viewport" content="width=device-width">`: isto força os navegadores de celular a adotar a largura real para carregar páginas da web (alguns navegadores mobile mentem sobre sua largura da janela, e em vez carregam páginas em uma largura grande, então encolhem a página carregada, o que é de muita ajuda para imagens e designs responsivos. Nós iremos ensinar mais sobre isso em um módulo futuro).

Ferramentas de desenvolvimento úteis

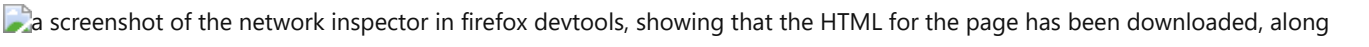
Há algumas ferramentas de desenvolvimento úteis em navegadores para ajudar a exercitar o necessário para o slot de largura, etc, que você precisa usar. Quando eu estava trabalhando neles, eu primeiro carreguei a versão não responsiva do meu exemplo (`not-responsive.html`), então fui no Modo de Design Responsivo (Ferramentas > Desenvolvimento Web > Modo de Design Responsivo), que permite a você ver o layout da sua página como se ela estivesse sendo vista através de uma variedade de diferentes tamanhos de tela.

Eu configurei a largura da janela para 320px e depois 480px; para cada uma eu fui no DOM Inspector, cliquei no elemento `<img>` no qual nós estamos interessados, então olhei o tamanho na aba Box Model view no lado direito da tela. Isto deve dar para você a dica da largura de imagem que você precisa.

 A screenshot of the firefox devtools with an image element highlighted in the dom, showing its dimensions as 440 by 293 pixels.

Próximo, você pode checar se o `srcset` está funcionando configurando a largura da janela para a qual você quiser (coloque para uma largura estreita, por exemplo), abrindo o Network Inspector (Ferramentas > Web Developer > Network), então recarregue a página. Isto deve dar a você uma lista do que foi carregado na página, e aqui você pode checar qual arquivo da imagem foi escolhida para baixar.

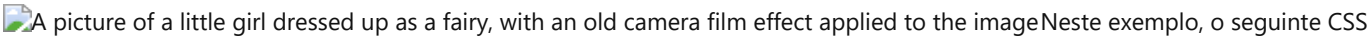
> **Nota:** Use o Mozilla Firefox para testar o `srcset`. O Chrome carrega a melhor imagem se estiver em cache no navegador, anulando o propósito do teste na ferramenta de desenvolvimento.


a screenshot of the network inspector in firefox devtools, showing that the HTML for the page has been downloaded, along with three images, which include the two 800 wide versions of the responsive images

Mudança de Resolução: Mesmo tamanho, diferentes resoluções

Se você está dando suporte a múltiplas resoluções de vídeo, mas todas veem sua imagem no tamanho real na tela, você pode permitir ao navegador escolher uma resolução apropriada para a imagem usando `srcset` com x identificadores e sem `sizes` - uma sintaxe um pouco mais fácil! Você pode encontrar um exemplo de como isto parece em srcset-resolutions.html e (ver também o código fonte):

```
<img
  srcset="elva-fairy-320w.jpg, elva-fairy-480w.jpg 1.5x, elva-fairy-640w.jpg 2x"
  src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Guides/Responsive_images/elva-fairy-640w.jpg"
  alt="Elva dressed as a fairy" />
```


A picture of a little girl dressed up as a fairy, with an old camera film effect applied to the imageNeste exemplo, o seguinte CSS é aplicado na imagem, então ela terá uma largura de 320 pixels na tela (também chamado pixels CSS):

```
img {
  width: 320px;
}
```

Neste caso, `sizes` não é preciso - o navegador simplesmente encontra qual das resoluções será mostrada, e fornece a mais apropriada definida no `srcset`. Então se o dispositivo que acessa a página tem um padrão de baixa resolução, com um pixel do dispositivo representando cada pixel CSS, a imagem `elva-fairy-320w.jpg` será carregada (o 1x é implícito, então você não precisa incluí-lo). Se o dispositivo tem uma alta resolução de dois pixels do dispositivo para pixel CSS ou mais, a imagem `elva-fairy-640w.jpg` será carregada. A imagem 640px é 93KB, enquanto a 320px é somente 39KB.

Direção de Arte

Recapitulando, o problema de direção de arte involve querer mudar a imagem mostrada para se adequar a diferentes tamanhos de tela. Por exemplo, se uma foto de um grande panorama com uma pessoa no centro é mostrada em uma página quando visualizada em um navegador de desktop, então é encolhido quando a página é vista em um navegador de celular, ficará ruim pois a pessoa no centro aparecerá bem pequena e difícil de ver. Provavelmente seria melhor mostrar uma imagem menor e em formato retrato (em pé) no celular, que mostrará a pessoa ampliada. O elemento `<picture>` permite a nós implementar justamente este tipo de solução.

Voltando para o nosso exemplo not-responsive.html, nós temos uma imagem que precisa de uma direção de arte:

```
<img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Guides/Responsive_images/elva-800w.jpg" alt="Chris
standing up holding his daughter Elva" />
```

Vamos consertar isso, com `<picture>`! Como `<video>` e `<audio>`, o elemento `<picture>` é um invólucro contendo muitos elementos `<source>` que fornecem muitas fontes diferentes para o navegador escolher, seguido pelo elemento mais importante `<img>`. O código em responsive.html ficará assim então:

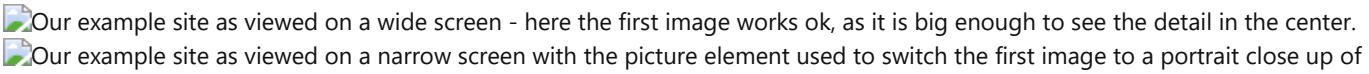
```
<picture>
  <source media="(max-width: 799px)" srcset="elva-480w-close-portrait.jpg" />
  <source media="(min-width: 800px)" srcset="elva-800w.jpg" />
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Guides/Responsive_images/elva-800w.jpg" alt="Chris
standing up holding his daughter Elva" />
</picture>
```

- Os elementos `<source>` incluem um atributo `media` que contem uma condição de mídia - como no nosso primeiro exemplo `srcset`, estas condições são testadas para definir qual imagem será mostrada no dispositivo - a primeira que retornar um

valor verdadeiro, será escolhida. Neste caso, se a largura da janela é 799px ou menor, a primeira imagem do elemento `<source>` será mostrada. Se a largura da janela é 800px ou maior, será escolhida a segunda.

- Os atributos `srcset` contem o caminho para a imagem que será apresentada. Note que como acabamos de ver acima com `<img>`, `<source>` podemos pegar um atributo `srcset` com múltiplas imagens referenciadas, e um atributo `sizes` também. Então você pode oferecer múltiplas imagens via um elemento `<picture>`, mas também oferecer múltiplas resoluções para cada uma. Na prática, você provavelmente não vai querer fazer isso com frequência.
- Em todos os casos, você deve fornecer um elemento `<img>`, com `src` e `alt`, logo antes do `</picture>`, de outra forma não aparecerá imagens. Assim um padrão será aplicado quando nenhuma condição for atendida (você pode remover o segundo elemento `<source>` neste exemplo), e verificar navegadores que não suportam o elemento `<picture>`.

Este código nos permite mostrar uma imagem adequada em ambas extensas e estreitas telas, como podemos ver abaixo:


Our example site as viewed on a wide screen - here the first image works ok, as it is big enough to see the detail in the center.

Our example site as viewed on a narrow screen with the picture element used to switch the first image to a portrait close up of the detail, making it a lot more useful on a narrow screen

> **Nota:** Você deveria usar o atributo `media` somente em cenários de direção de arte; quando você usa `media`, não oferecendo também condições com o atributo `sizes`.

Por que não podemos só fazer isso usando CSS ou JavaScript?

Quando o navegador começa a carregar a página, inicia o download de todas as imagens antes do analisador principal ter começado a carregar e interpretar o JavaScript e o CSS da página. Isto é uma técnica útil, a qual diminui o tempo de carregamento médio em 20%. Entretanto, isto não é útil para imagens responsivas, então é necessário implementar soluções como srcset. Você não pode, por exemplo, carregar o elemento `<img>`, então detectar a largura da janela com JavaScript e mudar dinamicamente o arquivo da imagem para uma menor, caso deseje. Até lá, a imagem original deveria já ter sido carregada, e você iria carregar uma menor, o que é ainda pior em termos de imagens responsivas.

Use bastante formatos de imagens modernos

Há vários novos e excitantes formatos de imagens (como WebP e JPEG-2000) que podem manter um baixo tamanho de arquivo e alta qualidade ao mesmo tempo. Entretanto, o suporte do navegador é menor.

`<picture>` permite-nos continuar dando suporte para navegadores antigos. Você pode suprir tipos MIME dentro de atributos `type`, então o navegador pode rejeitar imediatamente arquivos não suportados:

```
<picture>
  <source type="image/svg+xml" srcset="pyramid.svg" />
  <source type="image/webp" srcset="pyramid.webp" />
  <img
    src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Guides/Responsive_images/pyramid.png"
    alt="regular pyramid built from four equilateral triangles" />
</picture>
```

- *Não* use o atributo `media`, a menos que você também precise de direção de arte.
- No elemento `<source>`, você só pode referenciar imagens de tipos declarados em `type`.
- Como antes, você é encorajado a usar uma lista separada por vírgula com `srcset` e `sizes`, caso precise.

Aprendizado ativo: Implementando suas próprias imagens responsivas

Para esse exercício, nós estamos esperando que você seja corajoso e vá sozinho. Principalmente, nós queremos que você implemente sua própria direção adequada de arte em tela estreita/ampla usando `<picture>`, e um exemplo de mudança de resolução que use `srcset`.

1. Escreva um simples HTML contendo seu código (use `not-responsive.html` como um ponto de partida, se quiser).
2. Encontre uma boa imagem ampla de um panorama com algum detalhe contido em alguma parte. Crie uma versão de tamanho web usando um editor de imagem, então coloque para mostrar uma pequena parte que amplia o detalhe, e salve como uma segunda imagem (algo como 480px está bom).
3. Use o elemento `<picture>` para implementar uma mudança de imagem!
4. Crie múltiplos arquivos de imagem de diferentes tamanhos, cada um mostrando a mesma imagem.

5. Use `srcset`/`size` para criar um exemplo de mudança de resolução, que sirva para os mesmos tamanhos de imagens em diferentes resoluções, ou diferentes tamanhos de imagens em cada largura de janela.

> **Nota:** Use a ferramenta de desenvolvimento do navegador para ajudar a ver os tamanhos que você precisa, como mencionado acima.

Sumário

Isto é um geral sobre imagens responsivas - nós esperamos que você tenha aproveitado estas novas técnicas. Recapitulando, há dois problemas que nós discutimos aqui:

- **Direção de Arte:** O problema consiste em apresentar imagens cortadas para diferentes layouts - por exemplo, uma imagem panorâmica mostrada completa em um layout desktop, e uma imagem retrato mostrando o objeto principal ampliado em um layout mobile. Isto pode ser resolvido usando o elemento `<picture>`.
- **Mudança de resolução:** O problema é apresentar arquivos menores de imagens para dispositivos estreitos, porque eles não precisam de imagens gigantes como em computadores - e também, opcionalmente, que você queira apresentar imagens de diferentes resoluções para altas e baixas densidades de tela. Isto pode resolver usando vector graphics (imagens SVG), e os atributos `srcset` e `sizes`.

Isto também encerra o módulo Multimídia e Incorporação! A única coisa para fazer agora é seguir e tentar nosso Teste de Multimídia e Incorporação, e ver como você está. Divirta-se!

Veja também

- Jason Grigsby's excelente introdução a imagens responsivas
- Imagens respondivas: Se você está mudando de resolução, use srcset — Inclui mais explicação sobre como o navegador resolve qual imagem usar
- `<img>`
- `<picture>`
- `<source>`

- Anterior
- Menu: Multimídia e Incorporação
- Próxima

Dicas para criar páginas HTML de carregamento rápido

Estas dicas são baseadas em conhecimento comum e experimentação.

Uma página web otimizada não apenas provê um site mais responsivo aos visitantes, como também reduz a carga sobre os servidores e a conexão com a Internet. Isso pode ser crucial para sites de grande volume ou sites que tenham um pico de tráfego devido a circunstâncias extraordinárias, como plantões de notícias urgentes.

Otimizar a performance do carregamento de páginas não serve apenas para o conteúdo que será visto por visitantes com linha discada ou aparelhos móveis. É igualmente importante para banda larga e pode levar a melhorias dramáticas até mesmo para seus visitantes com as mais rápidas conexões.

Dicas

Reduza o peso da página

O peso da página é, de longe, o fator mais importante na performance de seu carregamento.

Reduzir o peso da página através da eliminação de espaço em branco desnecessário e comentários, comumente conhecido como minimização, e mover scripts e CSS *inline* para arquivos externos pode melhorar a performance de download sem muita necessidade de outras mudanças na estrutura da página.

Ferramentas como HTML Tidy podem automaticamente cortar espaços em branco desnecessários e linhas vazias de um código HTML validado. Outras ferramentas podem "comprimir" JavaScript ao reformatar o código-fonte ou o obfuscando e substituindo identificadores longos por versões mais curtas.

Minimize o número de arquivos

Reduzir o número de arquivos referenciados por uma página diminui o número de conexões HTTP requeridas para realizar o download da página.

Dependendo das configurações de cache do *browser*, este pode enviar uma requisição `If-Modified-Since` ao servidor para cada arquivo CSS, JavaScript ou de imagem, perguntando se o arquivo foi modificado desde a última vez que foi baixado.

Ao reduzir o número de arquivos referenciados de dentro de uma página, reduz-se o tempo necessário para essas requisições serem enviadas e suas respostas recebidas.

Se você usa muitas imagens de fundo em seu CSS, pode reduzir o número de verificações HTTP combinando imagens em um único arquivo, o que é conhecido como um *sprite* de imagens. Então você apenas utiliza a mesma imagem cada vez que precisá-la, ajustando as coordenadas x/y apropriadamente. Essa técnica funciona melhor com elementos que terão dimensões limitadas, não sendo aplicável para todas as imagens. Contudo, o número menor de requisições HTTP e *caching* de uma única imagem devem ajudar a reduzir o tempo de carregamento.

Muito tempo gasto pesquisando quando foi a modificação mais recente de arquivos referenciados pode atrasar a exibição inicial de uma página, já que o *browser* deve verificar o momento de modificação para cada arquivo CSS ou JavaScript antes de carregar a página.

Reduza pesquisa de domínio

Já que cada domínio distinto demanda tempo durante uma pesquisa de DNS, o tempo de carregamento da página aumentará conforme o número de domínios distintos que aparecem em links de CSS e fontes de JavaScript e imagens.

Pode nem sempre ser prático, mas você deve sempre tomar cuidado para utilizar apenas o mínimo necessário de domínios diferentes nas suas páginas.

Conteúdo em cache reutilizado

Assegure que qualquer conteúdo que possa ser armazenado em cache o seja, e com tempos de expiração adequados.

Em especial, atente ao cabeçalho `Last-Modified`. Ele permite mecanismos de cache eficientes; através desse cabeçalho, informações sobre o arquivo que o agente de usuário quer carregar, como quando foi modificado da última vez, são transferidas. A maioria dos servidores web automaticamente anexam o cabeçalho `Last-Modified` a páginas estáticas (p. ex.: `.html`, `.css`), baseado na data de última modificação armazenada no sistema de arquivos. Com páginas dinâmicas (p. ex: `.php`, `.aspx`), isso não pode ser feito, e o cabeçalho não é enviado.

Então, para essas páginas que são geradas dinamicamente, alguma pesquisa adicional é benéfica. Isso vai salvar muito tempo em requisições nas páginas que normalmente não permitem armazenamento em cache.

Mais informações:

1. Get HTTP Condicional para Hackers RSS
2. HTTP 304: Not Modified
3. Sobre o Last-Modified HTTP e ETag

Estabeleça a ordem dos componentes da página de forma otimizada

Baixe o conteúdo da página primeiro, junto com qualquer CSS ou JavaScript que pode ser requerido para sua exibição inicial, de modo que o usuário receba a resposta mais rápida possível durante o carregamento. Esse conteúdo é tipicamente texto, e portanto pode ser beneficiado por técnicas de compressão de texto durante o tráfego, permitindo uma resposta ainda mais rápida ao usuário.

Quaisquer elementos dinâmicos que requeiram que a página complete seu carregamento antes de serem usados devem ser inicialmente desabilitados, e apenas habilitados após o carregamento completo. Isso fará com que o JavaScript seja carregado após o conteúdo da página, o que melhorará a aparência geral do carregamento.

Reduza o número de scripts inline

Scripts *inline* podem ser custosos para o carregamento, uma vez que o parser deve assumir que o script pode modificar a estrutura da página enquanto o processo de *parsing* está em andamento. Reduzir o número de scripts *inline* no geral e reduzir o

uso de `document.write()` para a saída de conteúdo pode melhorar o carregamento da página. Use métodos AJAX modernos para manipular o conteúdo da página, ao invés de abordagens antigas baseadas em `document.write()`.

Use CSS moderno e marcação validada

O uso de CSS moderno reduz a quantidade de marcação, pode reduzir a necessidade de imagens, em termos de layout, e frequentemente substitui imagens de textos estilizados — que "custam" muito mais do que o texto estilizado com CSS.

Usar marcações validadas tem outras vantagens. Primeiro, *browsers* não precisarão realizar correção de erros durante o *parsing* de HTML (isso é à parte da preocupação filosófica de permitir variação de formato na entrada do usuário, e então programaticamente "corrigir" ou normalizá-la; ou se, ao invés disso, forçar um formato de entrada rígido, sem tolerância a desvios).

Além do mais, marcação válida permite o livre uso de outras ferramentas que podem pré-processar páginas web. Por exemplo, HTML Tidy pode remover espaços em branco e tags finais opcionais; contudo, a ferramenta não será executada em uma página com erros graves de marcação.

Divida seu conteúdo

Layout de tabelas é um método legado que não deve mais ser empregado. Layouts utilizando blocos `<div>` e, no futuro próximo, layout multi-colunas CSS3 ou layout de caixas flexíveis CSS3, devem ser utilizadas ao invés disso.

Tabelas ainda são consideradas marcações válidas, mas devem ser usadas para exibir dados tabulares. Para ajudar o browser a renderizar sua página mais rapidamente, você deve evitar aninhar suas tabelas.

Ao invés de realizar aninhamentos profundos como:

```
<TABLE>
  <TABLE>
    <TABLE>
          ...
    </TABLE>
  </TABLE>
</TABLE>
```

use tabelas não-aninhadas ou divs, como em

```
<TABLE>...</TABLE>
<TABLE>...</TABLE>
<TABLE>...</TABLE>
```

Veja também: Especificações do layout multi-colunas CSS3 e layout de caixas flexíveis CSS3

Especifique tamanhos para imagens e tabelas

Se o *browser* puder imediatamente determinar a altura e/ou largura de suas imagens e tabelas, ele será capaz de exibir uma página sem ter que recalcular o fluxo do conteúdo. Isso não apenas acelera a exibição da página como previne mudanças irritantes no layout ao finalizar o carregamento. Por essa razão, `height` e `width` devem ser especificadas para imagens, sempre que possível.

Tabelas devem usar a combinação CSS selector:property:

```
  table-layout: fixed;
```

e devem especificar as larguras das colunas usando as tags HTML `COL` e `COLGROUP`.

Escolha bem seus requisitos de agente de usuário

Para atingir as maiores melhorias no design de páginas, tenha certeza de que requisitos de agente de usuário razoáveis estejam especificados para os projetos. Não espere que seu conteúdo apareça de forma perfeita, pixel por pixel, em todos os *browsers*, especialmente nos obsoletos.

Idealmente, seus requisitos básicos devem ser baseados em considerações sobre os *browsers* modernos que suportam os padrões relevantes. Isso pode incluir: Firefox 3.6+ em qualquer plataforma, Internet Explorer 8.0+ no Windows, Opera 10+ no Windows, e Safari 4 no Mac OS X.

Note, contudo, que muitas das dicas listadas neste artigo são técnicas de senso comum que se aplicam a qualquer agente-usuário, e podem ser aplicadas a qualquer página web, independentemente de requisitos de compatibilidade em *browsers*.

Exemplo de estrutura de página

- `html`
- `head`
    - `link`... Arquivos CSS requeridos para a aparência da página. Minimize o número de arquivos para performance enquanto mantém CSS não-relacionado em arquivos separados para manutenção.
    - `script`... Arquivos JavaScript para funções **requeridas** durante o carregamento da página, sem qualquer DHTML que só pode ser executado após o carregamento completo.
- Minimize o número de arquivos para performance enquanto mantém JavaScript não-relacionado em arquivos separados para manutenção.
- `body`
- Páginas de conteúdo visíveis ao usuário em pequenas divisões (tabelas / divs) que podem ser exibidas sem esperar a página inteira ser baixada.
    - `script`... Quaisquer scripts que forem usados para realizar DHTML. Um script DHTML geralmente só pode ser executado após o carregamento completo da página e a inicialização de todos os objetos necessários. Não há necessidade de carregar esses scripts antes do conteúdo. Isso apenas desacelera a aparência inicial do carregamento da página.
- Minimize o número de arquivos para performance enquanto mantém CSS não-relacionado em arquivos separados para manutenção.
- Se uma ou mais imagens forem usadas para efeitos de *rollover*, você deve pré-carregá-las aqui após o conteúdo da página ter sido baixado.

Use async and defer, se possível

Faça com que scripts JavaScript sejam compatíveis tanto com async como defer e use async sempre que possível, especialmente se você tiver múltiplas tags de script.

Com isso, a página pode parar de renderizar enquanto o JavaScript ainda estiver sendo carregado. Do contrário, o *browser* não renderizará nada que estiver após as tags de script sem esses atributos.

Nota: Apesar desses atributos ajudarem muito na primeira vez que a página for carregada, você não pode confiar que vai funcionar em todos os *browsers*. Se você seguir todas as orientações para produzir bom código JavaScript, não há necessidade de alterá-lo.

Links Relacionados

- Livro: "Speed Up Your Site" por Andy King
- O excelente e muito completo Melhores Práticas para Acelerar Seu Web Site (Yahoo!)
- Ferramentas para analisar e otimizar a performance: Google PageSpeed

Informações do Documento Original

- Autor(es): Bob Clary, Evangelista de Tecnologia, Netscape Communications
- Última Data de Atualização: Publicado em 04 Abr 2003
- Informações de Copyright: Copyright © 2001-2003 Netscape. Todos os direitos reservados.
- Nota: Este artigo reeditado foi originalmente parte do site DevEdge.

CORS_habilitar_imagens

The HTML specification introduces a `crossorigin` attribute for images that, in combination with an appropriate CORS header, allows images defined by the `<img>` element that are loaded from foreign origins to be used in canvas as if they were being loaded from the current origin.

See CORS settings attributes for details on how the `crossorigin` attribute is used.

O que é um "contaminado" canvas?

Embora você possa usar imagens sem aprovação do CORS em seu canvas, criando contaminação ao canvas. Uma vez o canvas tenhya sido "contaminado" você não pode mais "puxar" informações do canvas.Por exemplo, você pode não mais usar os métodos canvas toBlob(), toDataURL(), ou getImageData(), fazendo isto você irá lançar um erro de sergurança.

Esta proteção de usuário tem tido dados expostos por uso de informações de imagens de site web remoto sem permissão.

Exemplo: Armazenando uma imagem de uma origem estrangeira

Você precisa ter um servidor hospedando as imagenscom o apropriado Access-Control-Allow-Origin cabeçalho. Adicionando atributos crossOrigin faz uma requisição ao cabeçalho. Você pode usar este exemplo das Configurações Apache Server HTML5 Boilerplate para aproximadamente responder com este cabeçalho de resposta.

```
<IfModule mod_setenvif.c>
    <IfModule mod_headers.c>
        <FilesMatch "\.(cur|gif|ico|jpe?g|png|svgz?|webp)$">
            SetEnvIf Origin ":" IS_CORS
            Header set Access-Control-Allow-Origin "*" env=IS_CORS
        </FilesMatch>
    </IfModule>
</IfModule>
```

Dado que está tudo classificado, você permiti salvar àquelas imagens no Armazenamento DOM

```
var img = new Image(),
  canvas = document.createElement("canvas"),
  ctx = canvas.getContext("2d"),
  src = "http://example.com/image"; // insert image url here

img.crossOrigin = "Anonymous";

img.onload = function () {
  canvas.width = img.width;
  canvas.height = img.height;
  ctx.drawImage(img, 0, 0);
  localStorage.setItem("savedImageData", canvas.toDataURL("image/png"));
};
img.src = src;
// make sure the load event fires for cached images too
if (img.complete || img.complete === undefined) {
  img.src =
    "";
  img.src = src;
}
```

Veja também

- Usando imagens de Cross-domínio em WebGL e Chrome 13
- Especificações HTML - o atributo crossorigin

Utilizando data attributes

O HTML5 foi criado pensando na extensibilidade dos dados que precisam ser associados a um determinado elemento mas não necessariamente tem um significado definido. Atributos data-* nos permite armazenar informações extras em elementos HTML

padrões e semânticos, sem a necessidades de hacks como classList, atributos fora do padrão, propriedades extras no DOM ou o método depreciado setUserData.

Sintaxe HTML

A sintaxe é simples. Qualquer atributo de qualquer elemento no qual o nome do atributo inicia com `data-` é um atributo data. Digamos que você possui um article e quer armazenar informações extras que não possuem nenhuma representação visual. Use atributos data para isso:

```
<article
  id="electriccars"
  data-columns="3"
  data-index-number="12314"
  data-parent="cars">
  ...
</article>
```

Acesso no JavaScript

Ler os valores destes atributos via JavaScript é muito simples também. Você pode lê-los usando `getAttribute()` com o nome html completo, mas a forma padrão provê uma alternativa mais simples: um `DOMStringMap` pode ser lido através da propriedade `dataset`.

Para obter o atributo data através do objeto `dataset`, acesse a propriedade utilizando a parte do nome do atributo após o prefixo `data-` (note que o hífen é convertido para camelCase).

```
var article = document.getElementById("electriccars");

article.dataset.columns; // "3"
article.dataset.indexNumber; // "12314"
article.dataset.parent; // "cars"
```

Cada propriedade é uma String e pode ser lida e escrita. No exemplo acima a atribuição `article.dataset.columns = 5` iria alterar esse atributo para "5".

Acesso no CSS

Note que os atributos data são atributos em HTML puro, e você pode inclusive acessá-los via CSS. Por exemplo, para mostrar o elemento pai em um artigo, você pode usar conteúdo gerado em CSS com a função `attr()`:

```
article::before {
  content: attr(data-parent);
}
```

Pode-se também usar os seletores de atributos em CSS para alterar estilos de acordo com o atributo data:

```
article[data-columns="3"] {
  width: 400px;
}
article[data-columns="4"] {
  width: 600px;
}
```

Pode-se tudo isso em funcionamento neste exemplo JSBin.

Atributos data também podem ser utilizados para conter informações que mudam constantemente, como a pontuação em um jogo. Usando seletores CSS e acesso com JavaScript permite que se construa efeitos excelentes sem ter que escrever suas próprias rotinas de display. Veja esta tela para um exemplo utilizando conteúdo gerado e transições CSS (exemplo JSBin).

Uma vez que valores data são strings, todos os valores devem estar entre aspas ou então a estilização não fará efeito.

Issues

Não armazene conteúdo que deve estar visível e acessível nos atributos data porque tecnologias assistivas podem não acessá-los. Além disso, motores de busca podem não indexar os valores dos atributos data.

Os principais issues a serem considerados são com suporte e performance no Internet Explorer. O Internet Explorer 11+ provê suporte para o padrão, mas todas as versões anteriores não suportam `dataset`. Para suporte ao IE 10 e anteriores, deve-se acessar atributos data através de `getAttribute()` . E ainda, a performance de leitura dos atributos data é ruim, comparada com o armazenamento em um data warehouse JS. O uso de `dataset` é até pior que a leitura dos dados com `getAttribute()`.

Apesar do que foi colocado, para metadados customizados associados a elementos, eles são uma ótima solução.

No Firefox 49.0.2 (e talvez em versões anteriores/posteriores), os atributos data que excederem 1022 caracteres não serão lidos pelo Javascript (EcmaScript 4).

Veja também

- Este artigo é uma adaptação de Usando atributos data em JavaScript e CSS no hacks.mozilla.org.
- Como usar atributos data em HTML5 (Sitepoint)

Referência HTML

Esta referência de HTML descreve todos os **elementos** e **atributos** do HTML, incluindo **atributos globais** que se aplicam a todos elementos.

Referência de elementos do HTML

> Esta página lista todos os elementos do HTML que são criados usando tags.

Referência de atributos do HTML

> Os elementos do HTML contêm atributos, que são valores adicionais que configuram os elementos ou ajustam seu comportamento de várias formas, para que se adaptem aos critérios dos usuários.

Atributos globais

> Atributos globais são atributos comuns a todos os elementos do HTML; podem ser usados em todos os elementos, embora possam não ter efeito em alguns elementos.

Tipos de links

> Em HTML, os tipos de links a seguir indicam o relacionamento entre dois documentos, em que um aponta para o outro usando um elemento `<a>`, `<area>` ou `<link>`.

Categorias de conteúdo

> Todo elemento em HTML é membro de uma ou mais categorias de conteúdo — estas categorias agrupam elementos que compartilham características comuns.

Atributos

Nome do Atributo	Elementos	Descrição
accept	`<form>`, `<input>`	Lista de tipos que o servidor aceita, tipicamente um tipo de arquivo.
accept-charset	`<form>`	Lista de conjunto de caracteres suportados.
accesskey	Global attribute	Define um atalho no teclado para ativar ou adicionar foco ao elemento.
action	`<form>`	A URI de um programa que processa a informação submetida através do formulário.
align	`<applet>`, `<caption>`, `<col>`, `<colgroup>`, `<hr>`, `<iframe>`,	Especifica o alinhamento horizontal do elemento.

Nome do Atributo	Elementos	Descrição
	`<img>`, `<table>`, `<tbody>`, `<td>`, `<tfoot>` , `<th>`, `<thead>`, `<tr>`	
alt	`<applet>`, `<area>`, `<img>`, `<input>`	Texto alternativo caso uma imagem não possa ser exibida.
async	`<script>`	Indica que o script deve ser executado assíncrono.
autocomplete	`<form>`, `<input>`	Indica se controles neste formulário podem por padrão ter seus valores completados automaticamente pelo navegador.
autofocus	`<button>`, `<input>`, `<select>`, `<textarea>`	O elemento deve ser focado automaticamente após a página ser carregada.
autoplay	`<audio>`, `<video>`	O aúdio ou vídeo deve ser reproduzido assim que possível.
bgcolor	`<body>`, `<col>`, `<colgroup>`, `<marquee>`, `<table>`, `<tbody>`, `<tfoot>`, `<td>`, `<th>`, `<tr>`	Cor do plano de fundo do elemento.Nota: Este é um atributo legado. Por favor use a propriedade CSS `background-color` em vez disso.
border	`<img>`, `<object>`, `<table>`	A largura da borda.Nota: este é um atributo legado. Por favor use a propriedade CSS `border` em vez disso.
buffered	`<audio>`, `<video>`	Contém o intervalo de tempo da mídia que já foi carregada.
charset	`<meta>`, `<script>`	Declara a codificação dos caracteres da página ou do script.
checked	`<command>`, `<input>`	Indica se o elemento deve ser checado no carregamento da página.
cite	`<blockquote>`, `<del>`, `<ins>`, `<q>`	Contêm uma URI que aponta para a fonte da citação ou alteração.
class	Global attribute	Frequentemente usado com CSS para estilizar os elementos com propriedades comuns.
code	`<applet>`	Especifica a URL do arquivo de classe do miniaplicativo que será carregado e executado.
codebase	`<applet>`	Este atributo fornece a URL absoluta ou relativa do diretório onde os arquivos .class do miniaplicativo são armazenados.
color	`<basefont>`, `<font>`, `<hr>`	Este atributo define a cor do texto usando o nome de uma cor ou uma cor especificada em hexadecimal através do formato #RRGGBB.Nota: este é um atributo legado. Por favor use a propriedade CSS `color` em vez disso.
cols	`<textarea>`	Define o número de colunas numa area de texto.
colspan	`<td>`, `<th>`	O atributo colspan define o número de colunas que uma célula deve conter.
content	`<meta>`	Um valor associado com `http-equiv` ou `name` dependendo do contexto.
contenteditable	Global attribute	Indica se o conteúdo do elemento é editável.
contextmenu	Global attribute	Define o ID de um elemento `<menu>` que servirá como o menu de contexto de um outro elemento.
controls	`<audio>`, `<video>`	Indica se o navegador deve mostrar ou não os controles de reprodução ao usuário.
coords	`<area>`	Uma escolha de valores especificando as coordenadas da região hot-spot.
data	`<object>`	Especifica a URL do recurso.
datetime	`<del>`, `<ins>`, `<time>`	Indica a data e o horário associados ao elemento.
default	`<track>`	Indica que a faixa deve estar habilitada a não ser que as preferências do usuário indiquem algo diferente.

Nome do Atributo	Elementos	Descrição
defer	`<script>`	Indica que o script deve ser executado após a página ter sido analisada.
dir	Global attribute	Defina a direção do texto. Os valores permitidos são ltr (Esquerda para direita) e rtl (Direita para esquerda)
dirname	`<input>`, `<textarea>`	
disabled	`<button>`, `<command>`, `<fieldset>`, `<input>`, `<optgroup>`, `<option>`, `<select>`, `<textarea>`	Indica se o usuário pode ou não interagir com o elemento.
draggable	Global attribute	Define se o elemento pode ser arrastado.
dropzone	Global attribute	Indica que o elemento aceita a soltagem de conteúdo nele.
enctype	`<form>`	Define o tipo de conteúdo da data do formulário quando o `method` é POST.
for	`<label>`, `<output>`	Descreve elementos na qual pertencem a este.
form	`<button>`, `<fieldset>`, `<input>`, `<label>`, `<meter>`, `<object>`, `<output>`, `<progress>`, `<select>`, `<textarea>`	Indica o formulário que é o proprietário do elemento.
headers	`<td>`, `<th>`	IDs dos elementos `<th>` na qual se aplicam a este elemento.
height	`<canvas>`, `<embed>`, `<iframe>`, `<img>`, `<input>`, `<object>`, `<video>`	Nota: Em algumas instâncias, tal como `<div>`, isto é um atributo legado, em cujo caso a propriedade CSS `height` deveria ser usado em vez. Em outros casos, assim como `<canvas>`, a altura deve ser especificado com esse atributo.
hidden	Global attribute	Indica a relevância de um elemento.
high	`<meter>`	Indica o menor limite da faixa superior.
href	`<a>`, `<area>`, `<base>`, `<link>`	A URL de um recurso ligado.
hreflang	`<a>`, `<area>`, `<link>`	Especifica o idioma de um recurso ligado.
http-equiv	`<meta>`	
icon	`<command>`	Especifica uma imagem na qual represente o comando.
id	Global attribute	Frequentemente usado com CSS para estilizar um elemento específico. O valor deste atributo deve ser único.
ismap	`<img>`	Indica que a imagem é parte de um mapa de imagem "sever-side".
itemprop	Global attribute	
kind	`<track>`	Especifica o tipo de caminho de texto.
label	`<track>`	Especifica um título "user-releadable" de um caminho de texto.
lang	Global attribute	Define o idioma usado no elemento.
language	`<script>`	Define o idioma do script usado no elemento.
list	`<input>`	Identifica uma lista de opções pré definidas para sugerir ao usuário.
loop	`<audio>`, `<bgsound>`, `<marquee>`, `<video>`	Indica se a mídia deveria começar tocando do começo quando ela é finalizada.
low	`<meter>`	Indica o maior limite da menor distância.
manifest	`<html>`	Especifica a URL do cache manifest do documento. documento

Nome do Atributo	Elementos	Descrição
max	`<input>`, `<meter>`, `<progress>`	Indica o valor máximo permitido.
maxlength	`<input>`, `<textarea>`	Define o núemro máximo de caracteres permitido no elemento.
media	`<a>`, `<area>`, `<link>`, `<source>`, `<style>`	Especifica uma sugestão da mídia para qual recurso ligado foi designado.
method	`<form>`	Define qual método HTTP usar quando enviar um formulário. Pode ser GET(padrão) ou POST.
min	`<input>`, `<meter>`	Indica o valor mínimo permitido.
multiple	`<input>`, `<select>`	Indica se múltiplos valores podem ser inseridos em uma entrada do tipo `email` ou `file`.
name	`<button>`, `<form>`, `<fieldset>`, `<iframe>`, `<input>`, `<object>`, `<output>`, `<select>`, `<textarea>`, `<map>`, `<meta>`, `<param>`	Nome do elemento. Por exemplo usado pelo servidor para identificar os campos no envio do formulário.
novalidate	`<form>`	Este atributo indica que o formulário não deveria ser validado quando enviado.
open	`<details>`	Indica se os detalhes serão mostrados no carregamento da página.
optimum	`<meter>`	Indica o valor numérico optimal.
pattern	`<input>`	Define uma espreção regular na qual o valor do elemento será validado de encontro.
ping	`<a>`, `<area>`	
placeholder	`<input>`, `<textarea>`	Fornece uma sugestão ao usuário de o que pode ser inserido no campo.
poster	`<video>`	Uma URL indicando uma poster frame para mostrar desde que o usuário toque ou busque.
preload	`<audio>`, `<video>`	Indica se todo o recurso, partes dele ou nada deveria ser pré carregado.
pubdate	`<time>`	Indica se esta data e tempo é a data no mais próximoelemento antecessor`<article>`.
radiogroup	`<command>`	
readonly	`<input>`, `<textarea>`	Indica se o elemento pode ser editado.
rel	`<a>`, `<area>`, `<link>`	Especifica o relacionamento do objeto alvo para o objeto ligado.
required	`<input>`, `<select>`, `<textarea>`	Indica se este elemento é requerido para preencher ou não.
reversed	`<ol>`	Indica se a lista deveria ser mostrada em uma ordem decrescente em vez de uma crescente.
rows	`<textarea>`	Define o número de linhas em uma área de texto.
rowspan	`<td>`, `<th>`	Define o número de linhas que uma célula de tabela deveria abranger mais.
sandbox	`<iframe>`	
spellcheck	Global attribute	Indica se o spell de checagem está permitido para o elemento.
scope	`<th>`	
scoped	`<style>`	
seamless	`<iframe>`	

Nome do Atributo	Elementos	Descrição
selected	`<option>`	Define o valor na qual será selecionado no carregamento da página.
shape	`<a>`, `<area>`	
size	`<input>`, `<select>`	Define a largura do elemento (em pixels). Se o elemento de atributo `type` é `text` ou `password` então este é o número de caracteres.
sizes	`<link>`	
span	`<col>`, `<colgroup>`	
src	`<audio>`, `<embed>`, `<iframe>`, `<img>`, `<input>`, `<script>`, `<source>`, `<track>`, `<video>`	URL de um conteúdo incorporável.
srcdoc	`<iframe>`	
srclang	`<track>`	
start	`<ol>`	Define o primeiro número se não for 1.
step	`<input>`	
style	Global attribute	Define estilos CSS na qual ultrapassarão estilos previamente configurados.
summary	`<table>`	
tabindex	Global attribute	Ultrapassa a ordem de tabulação padrão do navegador e segue o especificado como alternativa.
target	`<a>`, `<area>`, `<base>`, `<form>`	
title	Global attribute	Texto a ser mostrado em uma dica quando suspenso sobre um elemento.
type	`<button>`, `<input>`, `<command>`, `<embed>`, `<object>`, `<script>`, `<source>`, `<style>`, `<menu>`	Define o tipo de elemento.
usemap	`<img>`, `<input>`, `<object>`	
value	`<button>`, `<option>`, `<input>`, `<li>`, `<meter>`, `<progress>`, `<param>`	Define o valor padrão na qual será mostrado no elemento no carregar da página.
width	`<canvas>`, `<embed>`, `<iframe>`, `<img>`, `<input>`, `<object>`, `<video>`	Nota: Em algumas instâncias, tal como `<div>`, este é um atributo legado, no caso da propriedade CSS `width` deveria ser usado ao em vez. em outros casos, tal como `<canvas>`, a largura deve ser especificada com este atributo.
wrap	`<textarea>`	Indica se o texto deveria ser enrolado.

Elementos HTML

Esta página lista todos os elementos HTML. Eles são agrupados por função para ajuda-lo a achar o que quer fácilmente. Apesar deste guia ser escrito para aqueles mais inexperientes com a programação, nós esperamos que seja útil para todos.

Elementos básicos

Elementos básicos são o que constituem a base de todo documento HTML. Se vê esses elementos no código fonte de todas as páginas da web, logo em seguida da declaração de doctype, que está na primeira linha da página. O doctype específica qual a versão de (X)HTML que a página está usando. Os elementos que formam o conteúdo da página são postos entre a tag de abertura `<html>` e a tag de fechamento `</html>`. O elemento `<html>` é também conhecido como o elemento raiz.

Element	Description
`<html>`	O elemento **HTML <html>** (ou *HTML root element*) representa a raiz de um HTML ou XHTML documento. Todos os outros elementos devem ser descendentes desse elemento.

Metadados do documento

Os metadados são onde se guardam várias informações sobre a página, incluindo informações sobre estilos, scripts e dados para auxiliar software (ferramentas de pesquisa, navegadores, etc) usar e renderizar a página. Os metadados de estilos e scripts podem ser definidos dentro da própria página ou escritos em um arquivo externo que é referênciado pela mesma.

Element	Description
`<base>`	O *elemento HTML Base* (**<base>**) especifica o endereço (URL) utilizada por todos os endereços relativos contidos dentro de um documento. Há um número máximo de 1 (um) elemento *Base* <base> do documento.
`<head>`	O elemento **HTML `<head>` providencia informações gerais** (metadados) sobre o documento, incluindo seu título e links para scripts e folhas de estilos.
`<link>`	O **Elemento HTML <*link*>** especifica as relações entre o documento atual e um recurso externo. Possíveis usos para este elemento incluem a definição de uma estrutura relacional para navegação. Este elemento é mais usado para vincular as folhas de estilo.
`<meta>`	O elemento **HTML `<meta>` **define qualquer informação de metadados que não podem ser definidos por outros elementos **HTML.** (`<base>`, `<link>`, `<script>`, `<style>` ou `<title>`).
`<style>`	O **elemento HTML <style>** contém informações de estilo para um documento ou uma parte do documento. As informações de estilo específico estão contidas dentro deste elemento, geralmente no CSS.
`<title>`	O **elemento HTML** `<title>` (*Elemento HTML Título*) define o título do documento, mostrado na barra de título de um navegador ou na aba da página. Pode conter somente texto e quaisquer marcações contidas no texto não são interpretadas.

Sectioning root

Element	Description
`<body>`	O **elemento** `<body>` do **HTML** representa o conteúdo de um documento HTML. è permitido apenas um `<body>` por documento.

Separação de conteúdo

Elementos de separação de conteúdo permitem organizar o conteúdo do documento em partes lógicas. Use elementos de separação para criar um esboço do conteúdo da página, incluindo navegação por cabeçalho e rodapé, e elementos de cabeçalho para identificar seções de conteúdo.

Element	Description
`<address>`	O **elemento HTML `<address>`** fornece informações de contato para seu ancestral `<article>` ou `<body>` mais próximo; no segundo caso, ele se aplica ao documento inteiro.
`<article>`	O *Elemento HTML Article* (<article>) representa uma composição independente em um documento, página, aplicação, ou site, ou que é destinado a ser distribuido de forma independente ou reutilizável, por exemplo, em sindicação. Este poderia ser o post de um fórum, um artigo de revista ou jornal, um post de um blog, um comentário enviado por um usuário, um gadget ou widget interativos, ou qualquer outra forma de conteúdo independente.
`<aside>`	O elemento HTML **`<aside>`** representa uma seção de uma página que consiste de conteúdo que é tangencialmente relacionado ao conteúdo do seu entorno, que poderia ser considerado separado do conteúdo. Essas seções são, muitas vezes, representadas como barras laterais. Elas muitas vezes contem explicações laterais, como a definição de um glossário; conteúdo vagamente relacionado, como avisos; a biografia do autor; ou, em aplicações web, informações de perfil ou links de blogs relacionados.

Element	Description
`<footer>`	O elemento HTML de Rodapé (<footer>) representa um rodapé para o seu sectioning content (conteúdo de seção) mais próximo ou sectioning root elemento (ou seja, seu parente mais próximo `<article>`, `<aside>`, `<nav>`, `<section>`, `<blockquote>`, `<body>`, `<details>`, `<fieldset>`, `<figure>`, `<td>`). Normalmente um rodapé contém informações sobre o autor da seção de dados, direitos autorais ou links para documentos relacionados.
`<header>`	O **elemento HTML `<header>`** representa um grupo de suporte introdutório ou navegacional. Pode conter alguns elementos de cabeçalho mas também outros elementos como um logo, seções de cabeçalho, formulário de pesquisa, e outros.
`<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`	Os **elementos HTML `<h1>`–`<h6>`** representam seis níveis de título de seção. `<h1>` é o nível de seção mais alto e `<h6>` é o mais baixo.
`<main>`	O elemento **`<main>`** define o conteúdo principal dentro do `<body>` em seu documento ou aplicação. Entende-se como conteúdo principal aquele relacionado diretamente com o tópico central da página ou com a funcionalidade central da aplicação. O mesmo deverá ser único na página, ou seja, dentro do elemento <main> não deverão ser incluidas seções da página que sejam comuns a todo o site ou aplicação, tais como mecanismos de navegação, informações de copyright, logotipo e campos de busca (a não ser, é claro, caso a função principal do documento seja fazer algum tipo de busca).
`<nav>`	O *Elemento HTML de Navegação* (`<nav>`) representa uma seção de uma página que aponta para outras páginas ou para outras áreas da página, ou seja, uma seção com links de navegação.
`<section>`	O **elemento HTML `<section>`** representa uma seção genérica contida em um documento HTML, geralmente com um título, quando não existir um elemento semântico mais específico para representá-lo.

Conteúdo textual

Usam-se elementos HTML de conteúdo textual para se organizar blocos ou seções de conteúdo postos entre as tags de abertura `<body>` e fechamento `</body>`. Importantes para accessibilidade e SEO, esses elementos identificam o propósito ou estrutura do conteúdo.

Element	Description
`<blockquote>`	O Elemento HTML <blockquote> (ou Elemento HTML de citação de bloco) indica que o texto incluído é uma longa citação. Normalmente, este é processado visualmente pelo recuo (ver Notas sobre como mudá-lo). A URL para a fonte da citação pode ser dada usando o atributo **cite**, enquanto uma representação de texto da fonte pode ser dada usando o `<cite>` elemento.
`<dd>`	O **elemento HTML <dd>** fornece detalhes ou uma definição mais completa do termo precedente (definido por `<dt>`) numa lista de descrições (`<dl>`).
`<div>`	O **elemento de divisão** **HTML `<div>` **é um container genérico para conteúdo de fluxo, que de certa forma não representa nada. Ele pode ser utilizado para agrupar elementos para fins de estilos (usando **class** ou **id**), ou porque eles compartilham valores de atributos, como **lang**. Ele deve ser utilizado somente quando não tiver outro elemento de semântica (tal como `<article>` ou `<nav>`).
`<dl>`	O elemento HTML *Definition List* (`<dl>`) engloba uma lista de pares de termos e descrições. Um uso comum para este elemento é para implementar um glossário ou exibir metadados(uma lista de pares chave e valor).
`<dt>`	O elemento **HTML `<dt>`** (ou Elemento HTML de Definição de Termo) identifica um termo na lista de definição. Este elemento pode ocorrer somente em um elemento filho de `<dl>`. Geralmente seguido por um elemento `<dd>`; ou multiplos `<dt>` na mesma linha indicam vários termos sendo definidos pelo próximo element `<dd>`.
`<figcaption>`	O **Elemento HTML Figcaption** (`<figcaption>`) representa uma legenda ou uma legenda associada com uma figura ou ilustração descrita pelo resto dos dados do elemento `<figure>` que seu elemento pai.

Element	Description
`<figure>`	O elemento **`<figure>`** HTML representa conteúdo autocontido, potencialmente com uma legenda opcional, que é especificada usando o `<figcaption>` elemento. A figura, sua legenda e seu conteúdo são referenciados como uma única unidade.
`<hr>`	O **elemento HTML `<hr>`** representa uma quebra temática entre elementos de nível de parágrafo (por exemplo , uma mudança da cena de uma história, ou uma mudança de tema com uma seção). Nas versões anteriores do HTML, representava uma linha horizontal. Pode continuar sendo exibida como uma linha horizontal nos navegadores, mas agora está definida em termos semânticos, em vez de termos de apresentação.
`<li>`	O ***elemento HTML*** **`<li>`** (ou a Lista dos Itens de um elemento HTML) é usado para representar um item que faz parte de uma lista. Este item deve estar contido em um elemento pai: uma lista ordenada (`<ol>`), uma lista desordenada (`<ul>`) , ou um menu (`<menu>`) e representa uma única entidade dessa lista. Em menus e listas desordenadas a relação de itens é exibida, normalmente, usando pontos de marcação (as bolinhas). Em listas ordenadas eles são, comumente, mostrados com algum contador ascendente - como um número, ou letra - à sua esquerda.
`<menu>`	A semantic alternative to (`<ul>`, but treated by browsers (and exposed through the accessibility tree) as no different than (`<ul>`. It represents an unordered list of items (which are represented by (`<li>` elements).
`<ol>`	O **Elemento HTML ** (ou *Elemento HTML de lista ordenada*) representa uma lista de itens ordenados. De forma característica esses itens ordenados em uma lista são mostrados com uma contagem que os precede, que pode ser de qualquer tipo, como numerais, letras, algarismos romanos, ou simples símbolos. Esse modelo numerado não é definido na descrição html da página, mas na folha de estilos CSS associada, pela propriedade `list-style-type`.
`<p>`	O **elemento HTML `<p>`** representa um parágrafo. Em mídias visuais, parágrafos são representados como blocos indentados de texto com a primeira letra avançada e separados por linhas em branco. Já em HTML, parágrafos são usados para agrupar conteúdos relacionados de qualquer tipo, como imagens e campos de um formulário.
`<pre>`	*HTML texto preformatado* (**<pre>**) é a tag utilizada para representar texto pré-formatado. Um texto dentro desse elemento é tipicamente exibido em uma fonte não proporcional da mesma maneira em que o texto original foi disposto no arquivo. Espaços em branco são mantidos no texto da mesma forma em que este foi digitado.
`<ul>`	O **elemento HTML`<ul>`** (ou *elemento HTML de Lista desordenada*) representa uma lista de itens sem ordem rígida, isto é, uma coleção de itens que não trazem uma ordenação numérica e as suas posições, nessa lista, são irrelevantes. Caracteristicamente, os itens em uma lista desordenada são exibidos com um marcador que pode ter várias formas, como um ponto, um círculo, ou um quadrado. O tipo de marcador não é definido na descrição HTML da página, mas na CSS associada, utilizando a propriedade `list-style-type`.

Semânticas textuais inline

Usa-se a semântica textual inline para definir o significado, estrutura, ou estilo de uma palavra, linha, ou de qualquer outro tipo de texto.

Element	Description
`<a>`	O **elemento `<a>`** em **HTML** (ou elemento âncora), com o atributo `href` cria-se um hiperligação nas páginas web, arquivos, endereços de emails, ligações na mesma página ou endereços na URL. O conteúdo dentro de cada `<a>` **precisará** indicar o destino do link.
`<abbr>`	O *Elemento* _HTML `<abbr>` _(ou Elemento de Abreviação HTML) representa uma abreviação e opcionalmente fornece uma descrição completa para ela. Se presente, o atributo **`title`** deve conter a descrição completa e apenas ela.
`<b>`	O **elemento HTML <b>** representa um intervalo de texto estilísticamente diferente do texto normal, sem transmitir qualquer importância ou relevância. Ele é geralmente usado para destacar palavras-chaves em um resumo, nomes de produtos em um comentário ou outros vãos de texto cuja a

Element	Description
	apresentação típica seria negrito. Outro exemplo de seu uso é como marcação da frase principal de cada paragrafo de um artigo.
`<bdi>`	O HTML <bdi>elemento (ou elemento de isolamento de Bi-direcional) isola um trechode texto que pode ser formatado em uma direção diferente de outro texto fora dela.
`<bdo>`	O **_elemento_ HTML `<bdo>` **(_bidirectional override_) é usado para substituir a direcionalidade atual do texto. Isso faz com que a direcionalidade dos personagens seja ignorada em favor da direcionalidade especificada.
` `	O elemento HTML _quebra-de-linha_ produz uma quebra de linha em um texto (carriage-return).É útil para escrever poemas ou um endereço, onde a divisão de linha é significante.
`<cite>`	O **elemento HTML <cite>** representa uma referência a um trabalho artístico. Deve incluir o título do trabalho ou uma URL de referência, que pode ser em uma forma abreviada de acordo com as convenções usadas para a adição dos metadados de citação.
`<code>`	O **elemento** **HTML `<code>` **apresenta seu conteúdo estilizado de maneira a indicar que o texto é um pequeno fragmento de código. Por padrão, o conteúdo é exibido utilizando a fonte monoespaçada padrão do user agent's.
`<data>`	The **HTML `<data>` Element** links a given content with a machine-readable translation. If the content is time- or date-related, the `<time>` must be used.
`<dfn>`	O elemento **HTML `<dfn>` **(ou _Elemento Definição HTML_) representa uma instância de definição de um termo.
`<em>`	O **elemento HTML `<em>`** marca o texto que tem ênfase. O elemento pode ser aninhado, com cada nível de aninhamento indicando um grau maior de ênfase.
`<i>`	O **elemento HTML `<i>` **representa uma parte do texto que é destacada do restante por algum motivo, por exemplo, termos técnicos, expressões de outros idiomas ou pensamentos de personagens fictícios. Normalmente, é apresentado com o uso do tipo "itálico".
`<kbd>`	Represents a span of inline text denoting textual user input from a keyboard, voice input, or any other text entry device. By convention, the user agent defaults to rendering the contents of a `<kbd>` element using its default monospace font, although this is not mandated by the HTML standard.
`<mark>`	O **Elemento HMTL `<mark>` **representa um trecho de destaque em um texto, por exemplo, uma sequência de texto marcado como referência, devido à sua relevância em um contexto particular. Por Exemplo, pode ser utilizado em uma página que mostra os resultados de uma busca onde todas as instâncias da palavra pesquisadas receberam destaque.
`<q>`	O elemento HTML <q> indica que o texto dentro da tag é uma pequena citação. Este elemento destina-se a citações curtas que não requerem marcações de parágrafo; para citações maiores use o elemento `<blockquote>`.
`<rp>`	Used to provide fall-back parentheses for browsers that do not support display of ruby annotations using the `<ruby>` element. One `<rp>` element should enclose each of the opening and closing parentheses that wrap the `<rt>` element that contains the annotation's text.
`<rt>`	The **HTML `<rt>` Element** embraces pronunciation of character presented in a ruby annotations, which are for showing pronunciation of East Asian characters and the `<rt>` element is used inside of `<ruby>` element.
`<ruby>`	O **elemento** **HTML `<ruby>` **representa uma anotação ruby. Anotações ruby são para mostrar a pronúncia de caracteres do Leste Asiático
`<s>`	O **elemento HTML `<s>`** renderiza um texto tachado ou uma linha cortando o texto. Use o elemento `<s>` para representar texto que não sejam relevante ou que não estam corretos. Não é apropriado o uso do `<s>` indicar edições no texto; para indicar edições no texto utilize `<del>` e `<ins>`, que são elementos mais apropriados.
`<samp>`	Used to enclose inline text which represents sample (or quoted) output from a computer program. Its contents are typically rendered using the browser's default monospaced font (such as Courier or Lucida Console).

Element	Description
`<small>`	Represents side-comments and small print, like copyright and legal text, independent of its styled presentation. By default, it renders text within it one font-size smaller, such as from `small` to `x-small`.
`<span>`	O elemento **HTML `<span>` é um conteiner generico em linha para conteúdo fraseado** , que não representa nada por natureza. Ele pode ser usado para agrupar elementos para fins de estilo (usando os atributos `class` ou `id`), ou para compartilhar valores de atributos como `lang`. Ele deve ser usado somente quando nenhum outro elemento semântico for apropriado. `<span>` é muito parecido com o elemento `<div>` , entretando `<div>` é um elemento de nível de bloco enquanto `<span>` é um elemento em linha.
`<strong>`	```
`<sub>`	Specifies inline text which should be displayed as subscript for solely typographical reasons. Subscripts are typically rendered with a lowered baseline using smaller text.
`<sup>`	Specifies inline text which is to be displayed as superscript for solely typographical reasons. Superscripts are usually rendered with a raised baseline using smaller text.
`<time>`	O elemento HTML *time* (`<time>`) representa o tempo tanto no formato de 24 horas ou como uma data precisa no calendário Gregoriano (com informações opcionais de tempo e fuso horário)
`<u>`	Represents a span of inline text which should be rendered in a way that indicates that it has a non-textual annotation. This is rendered by default as a simple solid underline, but may be altered using CSS.
`<var>`	O elemento HTML Variable (`<var>`) representa uma variável em uma expressão matemática ou um contexto de programação.
`<wbr>`	O elemento HTML `<wbr>` representa uma posição no texto onde o navegador pode, opcionalmente, quebrar uma linha, embora suas regras de quebra de linha de outra forma não criar uma ruptura naquele local.

Imagem e multimídia

HTML suporta vários recursos multimídia como imagens, audio, e video.

Element	Description
`<area>`	O *HTML `<area>` elemento* define uma região hot-spot em uma imagem, e, opcionalmente, associa-lo com um link de hipertexto. Este elemento é usado somente dentro de um `<map>` elemento.
`<audio>`	O elemento `audio` é utilizado para embutir conteúdo de som em um documento HTML ou XHTML.O elemento `audio` foi adicionado como parte do HTML5.
`<img>`	O **elemento** **HTML `<img>` **(or *HTML Image Element*) representa a inserção de imagem no documento, sendo implementado também pelo HTML5 para uma melhor experiência com o elemento `<figure>` e `<figcaption>`.
`<map>`	O **elemento HTML `<map>`** é usado com os elementos `<area>` para definir um mapa de imagem (a área clicável do link).
`<track>`	O **elemento HTML `<track>`** é usado como filho dos elementos de mídia`<audio>` e `<video>`. Ele permite que você especifique faixas de texto temporizadas (ou dados baseados em tempo), por exemplo, para lidar automaticamente com legendas. As faixas são formatadas em WebVTT format (arquivos `.vtt`) — Web Video Text Tracks or Timed Text Markup Language (TTML).
`<video>`	O elemento HTML `<video>` é utilizado para incorporar conteúdo de vídeo em um documento HTML ou XHTML.

Conteúdo integrado

Além do conteúdo normal de multimídia, HTML pode incluir uma variedade de outros conteúdos, apesar de nem todos serem possuírem fácilidade de interação.

Element	Description
`<embed>`	O **elemento HTML `<embed>` **incorpora conteúdo externo no ponto especificado no documento. Este conteúdo é fornecido por um aplicativo externo ou outra fonte de conteúdo interativo, como um plug-in de navegador.
`<iframe>`	O **elemento HTML <iframe>** (ou *elemento HTML inline frame*) representa um contexto de navegação aninhado, efetivamente incorporando outra página HTML para a página atual. Em HTML 4.01, um documento pode conter uma cabeça e um corpo ou uma cabeça e um conjunto de quadros, mas não tanto um corpo e um conjunto de quadros. No entanto, um <iframe> pode ser usado dentro de um corpo de documento normal. Cada contexto de navegação tem sua própria história de sessão e o documento ativo. O contexto de navegação que contém o conteúdo incorporado é chamado o pai de contexto de navegação. O contexto de navegação de nível superior (que não tem um pai) normalmente é a janela do navegador.
`<object>`	Represents an external resource, which can be treated as an image, a nested browsing context, or a resource to be handled by a plugin.
`<picture>`	O **elemento HTML `<picture>`** é um container usado para especificar múltiplos elementos `<source>` para um elemento específico `<img>` contido nele. O navegador irá escolher a imagem mais adequada de acordo com o layout atual da página, caracteristicas do dispositivo em que será exibido (p.e. um dispositivo normal ou um hiDPI), e a habilidade do navegador de renderizar um certo tipo de imagem (p.e., envie uma imagem WebP para os navegadores baseados no Chromium ou PNG para navegadores não-Chromium); se não houver correspondência entre os elementos `<source>`, o arquivo especificado pelo elemento `<img>` será selecionado. A imagem selecionada é então exibida no espaço ocupado pelo elemento `<img>`.
`<portal>`	Enables the embedding of another HTML page into the current one for the purposes of allowing smoother navigation into new pages.
`<source>`	O elemento `source` é utilizado para especificar múltiplos recursos de mídia de elementos `<picture>`, `<audio>` ou `<video>` em HTML5. É um elemento vazio. É normalmente usado para disponibilizar multiple formats supported by different browsers.

SVG and MathML

You can embed SVG and MathML content directly into HTML documents, using the `<svg>` and `<math>` elements.

Element	Description
`<svg>`	O elemento `svg` é um contêiner que define um novo sistema de coordenadas e janela de visualização. É usado como o elemento mais externo dos documentos SVG, mas também pode ser usado para incorporar um fragmento SVG dentro de um documento SVG ou HTML.
`<math>`	The top-level element in MathML is `<math>`. Every valid MathML instance must be wrapped in `<math>` tags. In addition you must not nest a second `<math>` element in another, but you can have an arbitrary number of other child elements in it.

Scripting

Com o intuito de permitir a criação de conteúdo dinâmico e aplicações Web, HTML foi projetado com suporte a linguagens de scripting, a mais prominiente sendo JavaScript. Certos elementos suportam essa capacidade.

Element	Description
`<canvas>`	O elemento HTML *Canvas* (`<canvas>`) pode ser utilizado para desenhar gráficos utilizando scripts (geralmente JavaScript). Por exemplo, além de desenhar gráficos, ele pode ser usado para fazer composições de fotos e também para animações. Você poderá colocar conteúdos alternativos dentro do bloco `<canvas>`. Este conteúdo será renderizado também em navegadores antigos e em navegadores com JavaScript desabilitado.
`<noscript>`	O **Elemento HTML `<noscript>`** define uma seção de html a ser inserida se um tipo de script não é suportado pela página ou se o script está desativado no navegador.
`<script>`	O **elemento HTML `<script>`** é usado para incluir ou referenciar um script executável.

Demarcando edições

Estes elementos permitem prover indicações que partes específicas de um texto foram alteradas.

Element	Description
`<del>`	O **elemento HTML `<del>`** (ou *Elemento HTML de Texto Excluído*) representa uma parte do texto que foi excluída de um documento. Este elemento é (não necessariamente) renderizado pelos navegadores com uma linha entre o texto.
`<ins>`	The **HTML `<ins>` Element** (or *HTML Inserted Text*) HTML represents a range of text that has been added to a document.

Conteúdo tabulado

Estes elementos são usados para se criar e manipular dados em tabelas.

Element	Description
`<caption>`	O elemento HTML **`caption`** especifica a legenda (ou título) de uma tabela.
`<col>`	O elemento **HTML `<col>`** define uma tabela contendo colunas e sendo utilizada para definições semanticas em todas as colunas comuns. É normalmente encontrada dentro do elemento("colgroup")}} .Este elemento permite a estilização de colunas utilizando-se do CSS, porém apenas um numero pequeno de atributos que terão efeito dentro das colunas (veja as especificações do CSS 2.1 a partir dessa lista).
`<colgroup>`	Defines a group of columns within a table.
`<table>`	O elemento HTML *Table* (`<table>`) representa dados em duas dimensões ou mais.
`<tbody>`	Encapsulates a set of table rows (`<tr>` elements), indicating that they comprise the body of the table (`<table>`).
`<td>`	Defines a cell of a table that contains data. It participates in the *table model*.
`<tfoot>`	O **`<tfoot>`** é um **elemento HTML** que define um conjunto de linhas as quais farão parte do rodapé de uma tabela HTML
`<th>`	O **elemento HTML `<th>`** define uma célula cabeçalho do grupo de células de sua tabela. A exatidão natural deste grupo é definida pelos atributos `scope` e `headers`.
`<thead>`	Defines a set of rows defining the head of the columns of the table.
`<tr>`	Defines a row of cells in a table. The row's cells can then be established using a mix of `<td>` (data cell) and `<th>` (header cell) elements.

Formulários

HTML provê de vários elementos que podem ser usados juntos para criar formulários que o usuário pode preencher e mandar para o website ou aplicativo. Há muito mais informação no guia de formulários HTML.

Element	Description
`<button>`	O **Elemento HTML *`<button>`*** representa um botão clicável.
`<datalist>`	O elemento HTML *Datalist* (<datalist>) contém um conjunto de elementos `<option>` que representam as opções possíveis para o valor de outros controles.
`<fieldset>`	O **elemento HTML `<fieldset>`** é usado para agrupar elementos, assim como labels (`<label>`), dentro de um formulário web.
`<form>`	O **elemento HTML `<form>` **representa uma seção de um documento que contém controles interativos que permitem ao usuário submeter informação a um determinado servidor web.
`<input>`	O **elemento HTML `<input>`** é usado para criar controles interativos para formulários baseados na web para receber dados do usuário. A semântica de um `<input>` varia consideravelmente dependendo do valor de seu atributo `type`.
`<label>`	Um **elemento** **HTML `<label>` **representa uma legenda para um item em uma interface de usuário. Ele pode estar associado com um elemento de controle, colocando este dentro do elemento `label`,

Element	Description
	ou usando o atributo `for`. Tal controle é chamado o *controle etiquetado* do elemento etiqueta. Um input pode ser associado a diversas etiquetas (<label>s).
`<legend>`	O **Elemento HTML <legend> **(ou *Elemento HTML Campo "Legend"*) representa um rótulo para o conteúdo do seu ancestral `<fieldset>`.
`<meter>`	O elemento HTML *meter* (`<meter>`) pode representar um valor escalar dentro de um intervalo conhecido ou um valor fracionário.
`<optgroup>`	Em um Formulário Web, o elemento HTML `<optgroup>` cria um agrupamento de opções dentro do elemento `<select>`.
`<option>`	Em um formulário Web, o **elemento HTML** `<option>` é usado para criar um controle que representa um item dentro de um elemento HTML5 `<select>`, `<optgroup>` ou `<datalist>`.
`<output>`	O elemento de saída (<output>) é um elemento no qual um site ou aplicativo pode injetar os resultados de um cálculo ou o resultado de uma ação do usuário.
`<progress>`	o elemento HTML progress (<progress>) é usado para visualizar o progresso de uma tarefa. Embora as especifidades de como é mostrado ficam a cargo do desenvolvedor, tipicamente, é mostrado como uma barra de progresso.
`<select>`	O elemento HTML *select* (<select>) representa um controle que apresenta um menu de opções. As opções dentro do menu são representadas pelo elemento `<option>`, que podem ser agrupados por elementos `<optgroup>`. As opções podem ser pré-selecionadas para o usuário.
`<textarea>`	O **elemento** HTML `<textarea>` representa um controle de edição para uma caixa de texto, útil quando você quer permitir ao usuário informar um texto extenso em formato livre, como um comentário ou formulário de retorno.

Elementos interativos

HTML oferece uma seleção de elementos que auxiliam na criação de objetos interativos de interface para usuários.

Element	Description
`<details>`	O elemento HTML *details* (`<details>`) é usado como uma ferramenta de onde o usuário irá obter informações adicionais.
`<dialog>`	O **elemento HTML** `<dialog>` representa uma caixa de diálogo ou outro componente interativo, tal como um inspetor ou janela.
`<summary>`	O elemento HTML *summary* (`<summary>`) é utilizado como um sumário ou legenda para o conteúdo de um elemento `<details>`.

Componentes Web

Componentes Web são uma tecnologia relacionada a HTML que faz possível, essencialmente, a criação e customização de elementos como se fossem parte normal do HTML. Além disso, se pode criar versões customizadas de elementos padrão do HTML.

> **Note: Observação:** Os elementos pertencentes aos Componentes Web são definidos na coleção de especificações dos Componentes Web da World Wide Web Consortium (W3C) em vez da especificação do HTML. Além disso, a especificação dos Componentes Web ainda não foi finalizada e está sujeita a mudanças.

Element	Description
`<slot>`	Part of the Web Components technology suite, this element is a placeholder inside a web component that you can fill with your own markup, which lets you create separate DOM trees and present them together.
`<template>`	O elemento **HTML** `<template>` é um mecanismo para encapsular um conteúdo do lado do cliente que não é renderizado quando a página é carregada, mas que pode ser instanciado posteriormente em tempo de execução usando JavaScript.

Elementos obsoletos

> **Warning: Aviso:** Estes elementos HTML antigos são obsoletos e não devem mais ser usados. **Você deve sempre evitar usa-los em novos projetos, e deve substitui-los assim que possível.** Eles estão listados apenas por motivos educacionais.

Element	Description
`<acronym>`	O Elemento HTML Acrônimo (`<acronym>`) permite à autores claramente indicar que uma seqüência de caracteres compõe um acrônimo ou uma abreviação de uma palavra.
`<applet>`	The HTML Applet Element (`<applet>`) identifies the inclusion of a Java applet.
`<bgsound>`	Sets up a sound file to play in the background while the page is used; use `<audio>` instead.
`<big>`	The obsolete **HTML Big Element** (**`<big>`**) renders the enclosed text at a font size one level larger than the surrounding text (`medium` becomes `large`, for example). The size is capped at the browser's maximum permitted font size.
`<center>`	O elemento **Center do HTML** (**`<center>`**) se tornou obsoleto, é um elemento de nível de bloco que exibe o conteúdo em nível de bloco ou em linha centralizado horizontalmente dentro do elemento que o contém. O contêiner `<body>` geralmente está presente, mas não é necessário. Essa tag foi descontinuada no HTML 4 (e XHTML 1) em favor da propriedade CSS `text-align`, que pode ser aplicada ao elemento `<div>` ou à um paragráfo indivídual `<p>`. Para blocos de centralização, use outras propriedades CSS como `margin-left` e `margin-right` e defina-as como `auto` (ou defina `margin` para `0 auto`).
`<content>`	The **HTML `<content>` element**—an obsolete part of the Web Components suite of technologies—was used inside of Shadow DOM as an insertion point, and wasn't meant to be used in ordinary HTML. It has now been replaced by the `<slot>` element, which creates a point in the DOM at which a shadow DOM can be inserted.
`<dir>`	O elemento de diretório HTML obsoleto (**`<dir>`**) é usado como um contêiner para um diretório de arquivos e/ou pastas, potencialmente com estilos e ícones aplicados pelo user agent. Não use este elemento obsoleto; em vez disso, você deve usar o `<ul>` elemento para listas, incluindo listas de arquivos.
`<font>`	Defines the font size, color and face for its content.
`<frame>`	Defines a particular area in which another HTML document can be displayed. A frame should be used within a `<frameset>`.
`<frameset>`	Used to contain `<frame>` elements.
`<image>`	O **elemento HTML `<hgroup>` **destina-se a agrupar cabeçalhos de diferentes níveis para uma seção do documento. Ele agrupa (é um container para) um conjunto de elementos `<h1>`–`<h6>`.
`<marquee>`	Used to insert a scrolling area of text. You can control what happens when the text reaches the edges of its content area using its attributes.
`<menuitem>`	O elemento html <marquee> é usado para inserir uma área de rolagem de texto"scrolling" , similar a um letreiro. Você pode controlar o comportamento do conteúdo fornecendo alguns atributos extras.
`<nobr>`	Prevents the text it contains from automatically wrapping across multiple lines, potentially resulting in the user having to scroll horizontally to see the entire width of the text.
`<noembed>`	`O elemento HTML <nobr>` previne que um texto quebre em uma nova linha automaticamente, de forma que ele seja exibido em uma única grande linha, podendo tornar o *scroll* (horizontal) necessário. Esta tag não é padrão HTML e não deve ser usada. Como alternativa, use a propriedade CSS `white-space` como segue:
`<noframes>`	Provides content to be presented in browsers that don't support (or have disabled support for) the `<frame>` element. Although most commonly-used browsers support frames, there are exceptions, including certain special-use browsers including some mobile browsers, as well as text-mode browsers.
`<param>`	Defines parameters for an `<object>` element.

Element	Description
`<plaintext>`	Renders everything following the start tag as raw text, ignoring any following HTML. There is no closing tag, since everything after it is considered raw text.
`<rb>`	Used to delimit the base text component of a ruby annotation, i.e. the text that is being annotated. One `<rb>` element should wrap each separate atomic segment of the base text.
`<rtc>`	Embraces semantic annotations of characters presented in a ruby of `<rb>` elements used inside of `<ruby>` element. `<rb>` elements can have both pronunciation (`<rt>`) and semantic (`<rtc>`) annotations.
`<shadow>`	An obsolete part of the Web Components technology suite that was intended to be used as a shadow DOM insertion point. You might have used it if you have created multiple shadow roots under a shadow host.
`<spacer>`	Allows insertion of empty spaces on pages. It was devised by Netscape to accomplish the same effect as a single-pixel layout image, which was something web designers used to use to add white spaces to web pages without actually using an image. However, `<spacer>` is no longer supported by any major browser and the same effects can now be achieved using simple CSS.
`<strike>`	Places a strikethrough (horizontal line) over text.
`<tt>`	Creates inline text which is presented using the user agent default monospace font face. This element was created for the purpose of rendering text as it would be displayed on a fixed-width display such as a teletype, text-only screen, or line printer.
`<xmp>`	Renders text between the start and end tags without interpreting the HTML in between and using a monospaced font. The HTML2 specification recommended that it should be rendered wide enough to allow 80 characters per line.

`<a>`

O **elemento** `<a>` em **HTML** (ou elemento âncora), com o atributo `href` cria-se um hiperligação nas páginas web, arquivos, endereços de emails, ligações na mesma página ou endereços na URL. O conteúdo dentro de cada `<a>` **precisará** indicar o destino do link.

Experimente

```
<p>You can reach Michael at:</p>

<ul>
  <li><a href="https://example.com">Website</a></li>
  <li><a href="mailto:m.bluth@example.com">Email</a></li>
  <li><a href="tel:+123456789">Phone</a></li>
</ul>
```

```
li {
  margin-bottom: 0.5rem;
}
```

Atributos

Os atributos do elemento incluem os atributos globais.

`download`

Leva o usuário a salvar a URL em vez de navegar até ela. Pode ser usado com ou sem um valor:

- Sem um valor, o *browser* irá sugerir um nome de arquivo/extensão, gerado a partir de diversas origens:
    - O cabeçalho HTTP `Content-Disposition`
    - O segmento final no path (caminho) da URL
    - The media type (from the (`Content-Type` header, the start of a `data:` URL, or `Blob.type` for a `blob:` URL)
- A definição de um valor o sugere como o nome do arquivo. Caracteres `/` e `\` são convertidos para *underscores* (_).Arquivos de sistema talvez proibam alguns caracteres em nomes de arquivos, então o navegador irá ajustar o

nome sugerido caso necessário.

> **Note: Notes:**- `download` somente funciona para URLs de mesma origem, ou os esquemas `blob:` e `data:`
>
> - Se `Content-Disposition` tiver um diferente `filename` (nome do arquivo) que `download`, o cabeçalho tem prioridade. (Se `Content-Disposition: inline`, Firefox prioriza o cabeçalho enquanto o Chrome escolhe `download`.)

`href`

A URL para a qual o hiperlink aponta. Links não se restrigem a URLs baseadas no protocolo HTTP — eles podem utilizar qualquer tipo de URL suportado pelos browsers:

- Sections of a page with fragment URLs(Seções de página com fragmentos URL)
- Pieces of media files with media fragments (Pedaços de arquivos de mídia com fragmentos da própria mídia)
- Números de telefone com `tel:` URLs
- Email addresses with(Endereço de email com) `mailto:` URLs
- Alguns navegadores talvez não aguentem certos arranjos em URL, para isso os websites fazem uso do `registerProtocolHandler()`

`hreflang`

Hints at the human language of the linked URL. No built-in functionality. Allowed values are the same as the global `lang` attribute.

`ping`

Uma lista de URLs separadas por espaços. Quando o link é seguido, o brrowser enviará requisições `POST` com o corpo `PING` para as URLs. Typically for tracking.

`referrerpolicy`

Quanto do referrer para ser enviado quando acessar o link. Acesse `Referrer-Policy` para possíveis valores e seus efeitos.

`rel`

The relationship of the linked URL as space-separated link types.

`target`

Where to display the linked URL, as the name for a *browsing context* (a tab, window, or `<iframe>`). The following keywords have special meanings for where to load the URL:

- `_self`: No atual contexto de pesquisa. (Default)
- `_blank`: Normalmente uma nova aba, porém usuários podem configurar seus navegadores para abrir em uma nova janela.
- `_parent`: the parent browsing context of the current one. If no parent, behaves as `_self`.
- `_top`: the topmost browsing context (the "highest" context that's an ancestor of the current one). If no ancestors, behaves as `_self`.

> **Nota:** Quando usando `target`, adicione `rel="noreferrer noopener"` para evitar "exploit" para `window.opener` API;

> **Warning: Note:** Linking to another page with `target="_blank"` will run the new page in the same process as your page. If the new page executes JavaScript, your page's performance may suffer. This can also be avoided by using `rel="noreferrer noopener"`.

`type`

Hints at the linked URL's format with a MIME type. No built-in functionality.

Obsolete attributes

`charset`

Hinted at the character encoding of the linked URL.

> **Nota:** This attribute is obsolete and **should not be used by authors**. Use the HTTP `Content-Type` header on the linked URL.

`coords`

Used with the `shape` attribute. A comma-separated list of coordinates.

`name`

Was required to define a possible target location in a page. In HTML 4.01, `id` and `name` could both be used on `<a>`, as long as they had identical values.

> **Nota:** Use the global attribute `id` instead.

`rev`

Specified a reverse link; the opposite of the `rel` attribute. Deprecated for being very confusing.

`shape`

The shape of the hyperlink's region in an image map.

> **Nota:** Use the `<area>` element for image maps instead.

Properties

Content categories	Flow content, phrasing content, interactive content, palpable content.
Permitted content	Transparent, containing either flow content (excluding interactive content) or phrasing content.
Tag omission	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Permitted parents	Any element that accepts phrasing content, or any element that accepts flow content, but not other `<a>` elements.
Implicit ARIA role	`link` when `href` attribute is present, otherwise no corresponding role
Permitted ARIA roles	When `href` attribute is present: • `button` • `checkbox` • `menuitem` • `menuitemcheckbox` • `menuitemradio` • `option` • `radio` • `switch` • `tab` • `treeitem` When `href` attribute is not present: • any

DOM interface	HTMLAnchorElement

Examples

Linking to an absolute URL

HTML

```
<a href="https://www.mozilla.com"> Mozilla </a>
```

Result

Linking to relative URLs

HTML

```
<a href="https://developer.mozilla.org//example.com">Scheme-relative URL</a>
<a href="#mdn-Web-HTML">Origin-relative URL</a>
<a href="./p">Directory-relative URL</a>
```

Result

Linking to an element on the same page

```
<!-- <a> element links to the section below -->
<p><a href="#Section_further_down"> Jump to the heading below </a></p>

<!-- Heading to link to -->
<h2 id="Section_further_down">Section further down</h2>
```

> **Nota:** You can use `href="#top"` or the empty fragment (`href="#"`) to link to the top of the current page, as defined in the HTML specification.

Linking to an email address

To create links that open in the user's email program to let them send a new message, use the `mailto:` scheme:

```
<a href="mailto:nowhere@mozilla.org">Send email to nowhere</a>
```

For details about `mailto:` URLs, such as including a subject or body, see Email links or RFC 6068.

Linking to telephone numbers

```
<a href="tel:+49.157.0156">+49 157 0156</a>
<a href="tel:+1(555)5309">(555) 5309</a>
```

`tel:` link behavior varies with device capabilities:

- Cellular devices autodial the number.
- Most operating systems have programs that can make calls, like Skype or FaceTime.
- Websites can make phone calls with `registerProtocolHandler`, such as `web.skype.com`.
- Other behaviors include saving the number to contacts, or sending the number to another device.

See RFC 3966 for syntax, additional features, and other details about the `tel:` URL scheme.

Using the download attribute to save a <canvas> as a PNG

To save a `<canvas>` element's contents as an image, you can create a link with a `download` attribute and the canvas data as a `data:` URL:

Example painting app with save link

HTML

```
<p>
  Paint by holding down the mouse button and moving it.
  <a href="" download="my_painting.png">Download my painting</a>
</p>

<canvas width="300" height="300"></canvas>
```

CSS

```
html {
  font-family: sans-serif;
}
canvas {
  background: #fff;
  border: 1px dashed;
}
a {
  display: inline-block;
  background: #69c;
  color: #fff;
  padding: 5px 10px;
}
```

JavaScript

```
var canvas = document.querySelector("canvas"),
  c = canvas.getContext("2d");
c.fillStyle = "hotpink";

function draw(x, y) {
  if (isDrawing) {
    c.beginPath();
    c.arc(x, y, 10, 0, Math.PI * 2);
    c.closePath();
    c.fill();
  }
}

canvas.addEventListener("mousemove", (event) =>
  draw(event.offsetX, event.offsetY),
);
canvas.addEventListener("mousedown", () => (isDrawing = true));
canvas.addEventListener("mouseup", () => (isDrawing = false));

document
  .querySelector("a")
  .addEventListener(
    "click",
    (event) => (event.target.href = canvas.toDataURL()),
  );
```

Result

Security and privacy

`<a>` elements can have consequences for users' security and privacy. See `Referer` header: privacy and security concerns for information.

Using `target="_blank"` without `rel="noreferrer"` and `rel="noopener"` makes the website vulnerable to `window.opener` API exploitation attacks (vulnerability description).

Accessibility

Strong link text

The content inside a link should indicate where the link goes, even out of context.

Inaccessible, weak link text

A sadly common mistake is to only link the words "click here" or "here":

```
<p>Learn more about our products <a href="https://developer.mozilla.org/products">here</a>.</p>
```

Strong link text

Luckily, this is an easy fix, and it's actually shorter than the inaccessible version!

```
<p>Learn more <a href="https://developer.mozilla.org/products">about our products</a>.</p>
```

Assistive software have shortcuts to list all links on a page. However, strong link text benefits all users — the "list all links" shortcut emulates how sighted users quickly scan pages.

onclick events

Anchor elements are often abused as fake buttons by setting their `href` to `#` or `javascript:void(0)` to prevent the page from refreshing, then listening for their `click` events .

These bogus `href` values cause unexpected behavior when copying/dragging links, opening links in a new tab/window, bookmarking, or when JavaScript is loading, errors, or is disabled. They also convey incorrect semantics to assistive technologies, like screen readers.

Use a `<button>` instead. In general, **you should only use a hyperlink for navigation to a real URL**.

External links and linking to non-HTML resources

Links that open in a new tab/window via `target="_blank"`, or links that point to a download file should indicate what will happen when the link is followed.

People experiencing low vision conditions, navigating with the aid of screen reading technology, or with cognitive concerns may be confused when a new tab, window, or application opens unexpectedly. Older screen-reading software may not even announce the behavior.

Link that opens a new tab/window

```
<a target="_blank" href="https://www.wikipedia.org">
  Wikipedia (opens in new tab)
</a>
```

Link to a non-HTML resource

```
<a href="2017-annual-report.ppt"> 2017 Annual Report (PowerPoint) </a>
```

If an icon is used to signify link behavior, make sure it has alt text:

```
<a target="_blank" href="https://www.wikipedia.org">
  Wikipedia
  <img alt="(opens in new tab)" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/a/newtab.svg" />
</a>

<a href="2017-annual-report.ppt">
  2017 Annual Report
  <img alt="(PowerPoint file)" src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/a/ppt-
icon.svg" />
</a>
```

- WebAIM: Links and Hypertext - Hypertext Links
- MDN / Understanding WCAG, Guideline 3.2
- G200: Opening new windows and tabs from a link only when necessary
- G201: Giving users advanced warning when opening a new window

Skip links

A **skip link** is a link placed as early as possible in `<body>` content that points to the beginning of the page's main content. Usually, CSS hides a skip link offscreen until focused.

```
<body>
  <a href="#content">Skip to main content</a>

  <header>
    …
  </header>

  <main id="content"> <!-- The skip link jumps to here -->
```

```
.skip-link {
  position: absolute;
  top: -3em;
  background: #fff;
}
.skip-link:focus {
  top: 0;
}
```

Skip links let keyboard users bypass content repeated throughout multiple pages, such as header navigation.

Skip links are especially useful for people who navigate with the aid of assistive technology such as switch control, voice command, or mouth sticks/head wands, where the act of moving through repetitive links can be laborious.

- WebAIM: "Skip Navigation" Links
- How-to: Use Skip Navigation links
- MDN / Understanding WCAG, Guideline 2.4 explanations
- Understanding Success Criterion 2.4.1

Size and proximity

Size

Interactive elements, like links, should provide an area large enough that it is easy to activate them. This helps a variety of people, including those with motor control issues and those using imprecise inputs such as a touchscreen. A minimum size of 44×44 CSS pixels is recommended.

Text-only links in prose content are exempt from this requirement, but it's still a good idea to make sure enough text is hyperlinked to be easily activated.

- Understanding Success Criterion 2.5.5: Target Size
- Target Size and 2.5.5
- Quick test: Large touch targets

Proximity

Interactive elements, like links, placed in close visual proximity should have space separating them. Spacing helps people with motor control issues, who may otherwise accidentally activate the wrong interactive content.

Spacing may be created using CSS properties like `margin`.

- Hand tremors and the giant-button-problem

See also

- `<link>` is similar to `<a>`, but for metadata hyperlinks that are invisible to users.
- `:link` is a CSS pseudo-class that will match `<a>` elements with valid `href` attributes.

<abbr>: O elemento Abbreviation

O elemento HTML **`<abbr>`** representa uma abreviação ou acrônimo.

Ao incluir uma abreviação ou acrônimo, forneça uma expansão completa do termo em texto simples no primeiro uso, junto com o `<abbr>` para marcar a abreviação. Isso informa à pessoa usuária o que a abreviação ou acrônimo significa.

O atributo opcional `title` pode fornecer uma expansão para a abreviação ou acrônimo quando uma expansão completa não estiver presente. Isso fornece uma dica para os agentes da pessoa usuária sobre como anunciar/exibir o conteúdo enquanto informa a todas as pessoas usuárias o que a abreviação significa. Se presente, `title` deve conter esta descrição completa e nada mais.

Experimente

```
<p>
  You can use <abbr>CSS</abbr> (Cascading Style Sheets) to style your
  <abbr>HTML</abbr> (HyperText Markup Language). Using style sheets, you can
  keep your <abbr>CSS</abbr> presentation layer and <abbr>HTML</abbr> content
  layer separate. This is called "separation of concerns."
</p>
```

```
abbr {
  font-style: italic;
  color: chocolate;
}
```

Atributos

Este elemento suporta apenas os atributos globais. O atributo `title` tem um significado semântico específico quando usado com o elemento `<abbr>`; ele *deve* conter uma descrição completa legível para pessoas ou uma expansão da abreviação. Este texto é frequentemente apresentado pelos navegadores como uma dica de ferramenta quando o cursor do mouse é passado sobre o elemento.

Cada elemento `<abbr>` que você usa é independente de todos os outros; fornecer um `title` para um não anexa automaticamente o mesmo texto de expansão a outros com o mesmo texto de conteúdo.

Notas de uso

Casos de uso típicos

Certamente não é necessário que todas as abreviações sejam marcadas usando `<abbr>`. Existem, no entanto, alguns casos em que é útil fazer isso:

- Quando uma abreviação é usada e você deseja fornecer uma expansão ou definição fora do fluxo do conteúdo do documento, use `<abbr>` com um `title` apropriado.
- Para definir uma abreviação que pode ser desconhecida para a pessoa leitora, apresente o termo usando `<abbr>` e um texto em linha fornecendo a definição. Inclua um atributo `title` somente quando a expansão ou definição em linha não estiver disponível.
- Quando a presença de uma abreviação no texto precisa ser semanticamente notada, o elemento `<abbr>` é útil. Isso pode ser usado, por sua vez, para fins de estilo ou script.
- Você pode usar `<abbr>` em conjunto com `<dfn>` para estabelecer definições para termos que são abreviações ou acrônimos. Veja o exemplo Definindo uma abreviação abaixo.

Considerações gramaticais

Em idiomas com número gramatical (ou seja, idiomas em que o número de itens afeta a gramática de uma sentença), use o mesmo número gramatical no seu atributo `title` assim como no seu elemento `<abbr>`. Isso é especialmente importante em idiomas com mais de dois números, como o árabe, mas também é relevante em inglês.

Estilo padrão

O propósito deste elemento é puramente para a conveniência da pessoa autora e todos os navegadores o exibem em linha (`display: inline`) por padrão, embora seu estilo padrão varie de um navegador para outro:

Alguns navegadores adicionam um sublinhado pontilhado ao conteúdo do elemento. Outros adicionam um sublinhado pontilhado ao converter o conteúdo para versaletes. Outros podem não estilizá-lo de forma diferente de um elemento `<span>`. Para controlar esse estilo, use `text-decoration` e `font-variant`.

Acessibilidade

Soletrar a sigla ou abreviação por completo na primeira vez que for usada em uma página é benéfico para ajudar as pessoas a entendê-la, especialmente se o conteúdo for um jargão técnico ou do setor.

Inclua um `title` somente se não for possível expandir a abreviação ou acrônimo no texto. Ter uma diferença entre a palavra ou frase anunciada e o que é exibido na tela, especialmente se for um jargão técnico com o qual a pessoa leitora pode não estar familiarizada, pode ser chocante.

```
<p>
  JavaScript Object Notation (<abbr>JSON</abbr>) is a lightweight
  data-interchange format.
</p>
```

Isso é especialmente útil para pessoas que não estão familiarizadas com a terminologia ou os conceitos discutidos no conteúdo, pessoas que são novas no idioma e pessoas com problemas cognitivos.

Exemplos

Marcando uma abreviação semanticamente

Para marcar uma abreviação sem fornecer uma expansão ou descrição, use `<abbr>` sem nenhum atributo, como visto neste exemplo.

HTML

```
<p>Using <abbr>HTML</abbr> is fun and easy!</p>
```

Resultado

Estilizando abreviações

Você pode usar CSS para definir um estilo personalizado a ser usado para abreviações, como visto neste exemplo básico.

HTML

```
<p>Using <abbr>CSS</abbr>, you can style your abbreviations!</p>
```

CSS

```
abbr {
  font-variant: all-small-caps;
}
```

Resultado

Fornecendo uma expansão

Adicionar um atributo `title` permite que você forneça uma expansão ou definição para a abreviação ou acrônimo.

HTML

```
<p>Ashok's joke made me <abbr title="Laugh Out Loud">LOL</abbr> big time.</p>
```

Resultado

Definindo uma abreviação

Você pode usar `<abbr>` em conjunto com `<dfn>` para definir uma abreviação mais formalmente, como mostrado aqui.

HTML

```
<p>
  <dfn id="html"><abbr title="HyperText Markup Language">HTML</abbr> </dfn> is a
  markup language used to create the semantics and structure of a web page.
</p>

<p>
  A <dfn id="spec">Specification</dfn> (<abbr>spec</abbr>) is a document that
  outlines in detail how a technology or API is intended to function and how it
  is accessed.
</p>
```

Resultado

Resumo técnico

Categorias de conteúdo	Conteúdo de fluxo, conteúdo fraseado, conteúdo palpável
Conteúdo permitido	Conteúdo fraseado
Omissão de tag	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Pais permitidos	Qualquer elemento que aceite conteúdo fraseado
Função ARIA implícita	Nenhuma função correspondente
Funções ARIA permitidas	Qualquer uma
Interface DOM	`HTMLElement`

Veja também

- Usando o elemento `<abbr>`

<acronym>

Resumo

O Elemento HTML Acrônimo (`<acronym>`) permite à autores claramente indicar que uma seqüência de caracteres compõe um acrônimo ou uma abreviação de uma palavra.

> **Nota:** Este elemento foi removido no HTML5 e não deve ser usado mais. Ao invés dele, desenvolvedores devem usar o elemento `<abbr>`.

Atributos

Este elemento não possui qualquer outro atributo além dos atributos globais, comuns à todos os elementos.

Interface DOM

Este elemento implementa a interface `HTMLElement`.

> **Note: Nota de implementação:** Após o Gecko 1.9.2 inclusive, Firefox implementa a interface `HTMLSpanElement` para este elemento.

Exemplo

```
<p>
  A <acronym title="World Wide Web">WWW</acronym> é somente um dos componentes
  da Internet.
</p>
```

Estilo padrão

Embo is purely for the convenira o propósito desta tag seja meramente a conveniência dos autores, seu estilo padrão varia entre diferentes navegadores:

- Alguns navegadores, como o Internet Explorer, não criam qualquer estilo diferente do de um elemento `<span>`.
- Opera, Firefox, e alguns outros adicionam um sublinhado pontilhado no conteúdo do elemento.
- Alguns poucos navegadores não somente adicionam um sublinhado pontilhado, mas também colocam o conteúdo em versalete (small caps); para evitar este estilo, adicione algo como `font-variant: none` no CSS que cuida deste caso.

É portanto fortemente recomendado que autores web não confiem no estilo padrão do navegador para este elemento.

Ver também

- O elemento HTML `<abbr>`

<address>

Resumo

O **elemento HTML `<address>`** fornece informações de contato para seu ancestral `<article>` ou `<body>` mais próximo; no segundo caso, ele se aplica ao documento inteiro.

A informação de contato fornecida por um conteúdo do elemento `<address>` pode tomar qualquer forma que é apropriada para o contexto, e pode incluir qualquer tipo de informação de contato necessária, como endereço físico, URL, email, telefone, mídia social, coordenadas geográficas, por aí vai. Deve incluir o nome da pessoa, grupo, ou organização para qual as informações de contato se referem.

`<address>` pode ser usado em uma variedade de contextos, como fornecer informação de contato de uma empresa no header da página, ou indicar o autor de um artigo incluindo um elemento `<address>` dentro do `<article>`.

- *Categoria de conteúdo* Conteúdo de fluxo, conteúdo palpável.
- *Conteúdo permitido*Conteúdo de fluxo, mas sem nenhum elemento <address> aninhado, e sem conteúdo de cabeçalho (`<hgroup>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`), e nem conteúdo de seção como (`<article>`, `<aside>`, `<section>`, `<nav>`), e sem`<header>` ou o elemento `<footer>`.
- *Omissão de tag* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos-pai permitidos* Qualquer elemento que aceite conteúdo de fluxo, exceto os elementos <address> (de acordo com o princípio lógico da simetria, se a tag <address>, como pai, não pode ter elemento <address> como filho, então o mesmo conteúdo <address> não pode ter tag <address> como pai).
-
- *Regras ARIA permitidas* Nenhuma
- *Interface DOM* `HTMLElement`. Antes do Gecko 2.0 (Firefox 4), Gecko implementava este elemento usando a interface `HTMLSpanElement`

Atributos

Este elemento inclui somente os atributos globais.

Notas de uso

- Para representar um endereço arbitrário, um que não é relacionado à informação para contato, use um elemento `<p>` ao invés do elemento <address>.
- Este elemento não deve conter mais informação além da informação de contato, como uma data de publicação (a qual pertence ao elemento `<time>`).
- Tipicamente um elemento <address> pode ser posicionado dentro do elemento `<footer>` da seção atual, se houver um.

Exemplo

```
<address>
  Voce pode contatar o autor em
  <a href="http://www.somedomain.com/contact">www.somedomain.com</a>.<br />
  Se encontrar qualquer bug, por favor
  <a href="mailto:webmaster@somedomain.com">contate o administrador do site</a
  >.<br />
  Você tambem pode querer nos visitar:<br />
  Mozilla Foundation<br />
  1981 Landings Drive<br />
  Building K<br />
  Mountain View, CA 94043-0801<br />
  USA
</address>
```

Embora o elemento `address` exiba o texto com o mesmo estilo padrão que os elementos `<i>` ou `<em>`, ele é mais apropriado de ser usado quando se lida com informação de contato, já que ele fornece informação semântica adicional.

Ver também

- Outros elementos relacionados à seção: `<body>`, `<nav>`, `<article>`, `<aside>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<hgroup>`, `<footer>`, `<section>`, `<header>`;
- Seções e delineamento de um documento HTML5.

<area>

O *HTML `<area>` elemento* define uma região hot-spot em uma imagem, e, opcionalmente, associa-lo com um link de hipertexto. Este elemento é usado somente dentro de um `<map>` elemento.

- *As categorias de conteúdo* de conteúdo de fluxo , conteúdo fraseado .
- *Permitida conteúdo* vazio; este é um elemento vazio.
- *Tag omissão* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Permitida elementos pai* Qualquer elemento que aceita conteúdo fraseado . O `<area>` elemento deve ter um ancestral `<mapa>`, mas ele não precisa ser um pai direta.
- *Interface DOM* `HTMLAreaElement`

Atributos

Este elemento inclui os atributos globais.

`accesskey`

> Especifica um acelerador de navegação pelo teclado para o elemento. Pressionando ALT ou uma chave semelhante, em associação com o caractere especificado seleciona a forma de controle correlacionada com a seqüência de teclas. Os projetistas de página são avisados para evitar sequências de teclas já vinculados aos navegadores. Este atributo é global desde HTML5.

`alt`

> Uma alternativa seqüência de texto para exibir em navegadores que não exibem imagens. O texto deve ser formulada de modo a que apresenta o usuário com o mesmo tipo de escolha como a imagem iria oferecer quando exibido sem o texto alternativo. Em HTML4, este atributo é necessário, mas pode ser uma string vazia (""). Em HTML5, este atributo é necessário apenas se o **href** atributo é usado.

`coords`

> Um conjunto de valores que especificam as coordenadas da região de hot-spot. O número e o significado dos valores dependem do valor especificado para a **forma** de atributo. Para um `rect` forma ou retângulo, o **coords** valor é de dois pares x, y: left, top, right, e bottom. Para um `círculo` forma, o valor é `x, y, r` onde `x, y` é um par especificando o centro do círculo e `r` é um valor para o raio. Para um `poli` ou polígono <forma>, o valor é um conjunto de pares x, y de cada ponto no polígono: `X1, Y1, X2, Y2, x3, y3`, e assim por diante. Em HTML4, os valores são números de pixels ou porcentagens, se um sinal de porcentagem (%) é anexado; em HTML5, os valores são números de pixels CSS.

`download`

Este atributo, se presente, indica que o autor tem a intenção que o hiperlink seja usado para o download de um recurso. Consulte `<a>` para uma descrição completa da `download` atributo.

`href`

A meta de hyperlink para a área. Seu valor é uma URL válida. Em HTML4, quer este atributo ou o **nohref** atributo deve estar presente no elemento. Em HTML5, este atributo pode ser omitido; em caso afirmativo, o elemento de área não representa um hiperlink.

`hreflang`

Indica o idioma do recurso ligado. Os valores permitidos são determinados por BCP47 . Utilize este atributo somente se a **href** atributo está presente.

`name`

Defina um nome para a área clicável de modo que possa ser programado por navegadores mais antigos.

`media`

Uma dica da mídia para o qual o recurso ligado foi projetado, por exemplo `impressão e tela` . Se omitido, o padrão é `tudo` . Utilize este atributo somente se a **href** atributo está presente.

`nohref`

Indica que não existe hyperlink para a área associada. Ou este atributo ou a **href** atributo deve estar presente no elemento.

> **Note: Nota de Uso:** Este atributo é obsoleto em HTML5, em vez omitindo o atributo **href** é suficiente.

`rel`

Para âncoras contendo o **href** atributo, este atributo especifica a relação do objeto de destino para o objeto link. O valor é uma lista de valores de relacionamento, separados por vírgulas. Os valores e sua semântica será registrado por alguma autoridade que poderia ter significado para o autor do documento. A relação padrão, se nenhum outro é dado, é nula. Utilize este atributo somente se a **href** atributo está presente.

`shape`

A forma do ponto de acesso associado. As especificações para colar 5 e HTML 4 definem os valores `rect` , que define uma região rectangular; `círculo` , o qual define uma região circular; `poli` , que define um polígono; e `padrão` , o que indica toda a região além de quaisquer formas definidas. Muitos navegadores, principalmente o Internet Explorer 4 e superior, apoio `circ` , `polígono` , e `retângulo` como valores válidos para **forma** ; estes valores são .

`tabindex`

Um valor numérico que especifica a posição da área definida na ordem de tabulação browser. Este atributo é global em HTML5.

`target`

Este atributo especifica onde exibir o recurso ligado. Em HTML4, este é o nome de, ou uma palavra-chave para um quadro. Em HTML5, que é um nome ou palavra-chave para um *contexto de navegação* (por exemplo, aba, janela ou quadro embutido). As seguintes palavras-chave têm significados especiais:

- `_self` : Carregar a resposta no mesmo quadro HTML4 (ou contexto de navegação HTML5), como o atual. Este valor é o padrão se o atributo não é especificado.
- `_blank` : Coloque a resposta em uma nova janela sem nome HTML4 ou contexto de navegação HTML5.
- `_parent` : Carregar a resposta para o pai HMTL4 conjunto de quadros do quadro atual ou HTML5 pai contexto de navegação do atual. Se não houver nenhum pai, esta opção se comporta da mesma maneira que `_self` .
- `_top` : Em HTML4: Carregar a resposta para a janela completa, originais, cancelando todos os outros quadros. Em HTML5: Carregar a resposta para o contexto de navegação de nível superior (ou seja, o contexto de navegação que

é um ancestral do atual, e não tem pai). Se não houver nenhum pai, esta opção se comporta da mesma maneira que `_self` .

Utilize este atributo somente se a **href** atributo está presente.

`type`

Este atributo especifica o tipo de mídia na forma de um tipo MIME para o destino do link. Geralmente, este é fornecido informações estritamente como consultivo; no entanto, no futuro, um navegador pode adicionar um pequeno ícone para os tipos de multimédia. Por exemplo, um navegador pode adicionar um pequeno ícone de alto-falante quando o tipo está definido para áudio / wav. Para obter uma lista completa de tipos MIME reconhecidos, consulte https://www.w3.org/TR/html4/references.html # REF-MIMETYPES . Utilize este atributo somente se a **href** atributo está presente.

Exemplo

```
<mapa name="primary">
  <area shape="circle" coords="200,250,25" href="another.htm" />
  <area shape="default" nohref />
</map>
```

Notas

De acordo com o HTML 3.2, 4.0 e 5 especificações, a tag de fechamento `</ área>` é proibido.

A especificação XHTML 1.0 requer uma barra: `<area />` .

Os **ID** , **classe** e **estilo** atributos têm o mesmo significado que os atributos essenciais definidas na especificação HTML 4, mas somente Netscape e Microsoft defini-los.

Netscape 1 de nível não entendem o **alvo** atributo que se refere aos quadros.

HTML 3.2 define apenas **alt** , **coords** , **href** , **nohref** e shape .

article (artigo)

Resumo

O *Elemento HTML Article* (<article>) representa uma composição independente em um documento, página, aplicação, ou site, ou que é destinado a ser distribuido de forma independente ou reutilizável, por exemplo, em sindicação. Este poderia ser o post de um fórum, um artigo de revista ou jornal, um post de um blog, um comentário enviado por um usuário, um gadget ou widget interativos, ou qualquer outra forma de conteúdo independente.

> **Note:** *Notas de uso:*
>
> - Quando um elemento `<article>` está aninhado, o elemento interior representa um artigo relacionado com o elemento exterior. Por exemplo, os comentários do post de um blog podem ser elementos `<article>` aninhados em `<article>` representando o post do blog.
> - Informações sobre o autor de um elemento `<article>` podem ser fornecidas através do elemento `<address>` ,mas ele não se aplica aos elementos `<article>` aninhados.
> - A data e hora de publicação de um elemento `<article>` pode ser descrita usando o atributo `pubdate` de um elemento `<time>`.

Contexto de uso

Conteúdo permitido	Flow content
Omissão de tag	Nenhuma, tanto tag de início quanto de fim são obrigatórias
Elementos pai permitidos	Qualquer elemento que aceita flow content. Note-se que um elemento `<article>` não pode ser descendente de um elemento `<address>`.

Conteúdo permitido	Flow content
Documento normativo	HTML5, section 4.4.4

Atributos

Este elemento não possui outros elementos se não os global attributes, comuns a todos os elementos.

Interface DOM

Este elemento implementa a interface `HTMLElement` .

Exemplos

```
<article>
  <h4>Um artigo realmente impressionante</h4>
  <p>Lotes de texto incrível.</p>
</article>
```

Resultado

<article>

Um artigo realmente impressionante

Lotes de texto incrível.

</article>

Veja também

- Outros elementos de seção relacionados: `<body>`, `<nav>`, `<section>`, `<aside>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<hgroup>`, `<header>`, `<footer>`, `<address>`
- Sections and outlines of an HTML5 document.

<aside>

Sumario

O elemento HTML **`<aside>`** representa uma seção de uma página que consiste de conteúdo que é tangencialmente relacionado ao conteúdo do seu entorno, que poderia ser considerado separado do conteúdo. Essas seções são, muitas vezes, representadas como barras laterais. Elas muitas vezes contem explicações laterais, como a definição de um glossário; conteúdo vagamente relacionado, como avisos; a biografia do autor; ou, em aplicações web, informações de perfil ou links de blogs relacionados.

> **Note:** *Notas de uso:*
>
> - **Não use** o elemento `<aside>` para texto entre parenteses, pois este tipo de texto é considerado parte do fluxo principal.

- *Categoria de conteudo* Conteudo de fluxo, conteudo de secionamento, conteudo palpavel.
- *Conteudo permitido* Conteudo de fluxo.
- *Omissao de tag* Nenhuma, ambas as tags de início e fim são obrigatórias.
- *Elementos pai permitidos* Qualquer elemento que aceita conteudo de fluxo. Note que um elemento `<aside>` nao deve ser descendente de um elemento `<address>`.
- *Interface DOM* `HTMLElement`

Atributos

Este elemento inclui apenas os atributos globais.

Exemplos

```
<aside>
  <p>Algum conteudo relacionado a um &lt;article&gt;</p>
</aside>
```

Veja também

- Outros elementos de seçao relacionados: `<body>`, `<article>`, `<section>`, `<nav>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<hgroup>`, `<header>`, `<footer>`, `<address>`;
- Seçoes e outlines de um documento HTML5.

audio

Sumário

O elemento `audio` é utilizado para embutir conteúdo de som em um documento HTML ou XHTML.O elemento `audio` foi adicionado como parte do HTML5.

Você pode utilizar recursos avançados da API de áudio — que são específicas do Gecko — para gerar e manipular diretamente fluxo de áudio a partir do código JavaScript. Veja Introducing the audio API extension para detalhes.

Contexto de Uso

Conteúdo permitido	**Transparent content, tanto um atributo `src` quanto um ou mais elementos `<source>`, seguidos por um flow content ou phrasing content, sem`<video>` ou elementos `<audio>`.**
Omissão de tag	Nenhuma, tanto as tags iniciais quanto as finais devem estar presentes.
Elementos pai permitidos	Qualquer elemento que aceite flow content, ou qualquer elemento que aceite phrasing content.
Documento normativo	HTML5, section 4.8.7

Atributos

Como todos os elementos HTML, este elemento suporta os global attributes.

`autoplay`

> Um atributo Booleano; se especificado (mesmo se o valor for "false"!), o áudio iniciará automaticamente assim que possível sem parar de carregar os dados.

`autobuffer`

> Um atributo Booleano; se especificado, o audio será baixado automaticamente, mesmo se não está configurado para reprodução automática. Isto continua até que o cache de mídia esteja cheio, ou até que o o arquivo de áudio completo tenha sido baixado, o que vier primeiro. Isto deve ser utilizado apenas quando é esperado que o usuário escolherá tocar o áudio; por exemplo, se o usuário navegou para a página utilizando um link "Reproduzir". Este atributo foi removido no Gecko 2.0 em razão do atributo `preload`.

`buffered`

> Um atributo que pode ser lido para determinar os intervalos do áudio que já foram carregados. Este atributo contém um objeto `TimeRanges`.

`controls`

> Se esse atributo estiver presente, o navegador oferecerá controles para permitir ao usuário controlar a reprodução do áudio, incluindo volume, navegação, e pausa/continuação da reprodução.

`loop`

> Um atributo Booleano; se especificado, ao chegar no fim do áudio, ele voltará automaticamente para o começo.

`mozCurrentSampleOffset`

The offset, specified as the number of samples since the beginning of the audio stream, at which the audio is currently playing.

`muted`

Um atributo Booleano que indica se o áudio será inicializado silenciado.

`played`

Um objeto `TimeRanges`indicando que todo o áudio foi reproduzido.

`preload`

Esse atributo enumerado pretende dar uma sugestão ao navegador sobre o que o autor pensa que proporcionará uma melhor experiência do usuário. Ele pode ter os seguintes valores:

- none: indica que o usuário não necessitará consultar o áudio ou que o servidor quer minimizar seu tráfego; em outros termos indica que o áudio não deve ser pré-carregado;
- metadata: indica que embora o usuário não necessitará consultar o áudio, pegar os meta-dados (ex: comprimento) é interessante;
- auto: indica que o usuário necessita ter prioridade; em outros termos isso significa que, se necessário, o áudio inteiro pode ser baixado, mesmo que não seja esperado a execução;
- Uma *string vazia*: é um sinônimo do valor auto.

Se não definido, seu valor padrão será definido pelo navegador (isto é, cada navegador pode escolher seu valor padrão), embora a especificação recomenda que seja definido para o metadata.

> **Note: Notas de uso:**- O atributo `autoplay` tem precedência sobre o `preload`, pois se é necessário executar o áudio automaticamente, o navegador obviamente o baixará. Definindo ambos `autoplay` e `preload` é permitido pela especificação.
>
> - O navegador não é forçado pela especificação a seguir o valor desse atributo; é apenas uma sugestão.

`src`

A URL do áudio a ser incorporado. Isso é sujeito a HTTP access controls. Isto é opcional; ao invés disso você pode usar o elemento `<source>` dentro do bloco do áudio para especificar o vídeo a ser incorporado .

O tempo de compensação (time offset) entre o áudio e o vídeo está especificado como um valor de ponto flutuante (float) representando o número de segundos da compensação.

> **Nota:** A definição de valor de tempo de compensação ainda não foi completada na especificação do HTML 5 e está sujeita a mudança.

Examples

```
<!-- Reprodução simples de áudio -->
<audio src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/audio/AudioTest.ogg" autoplay>
  <a href="AudioTest.ogg" download="AudioTest.ogg">Baixar áudio OGG</a>.
</audio>

<!-- Reprodução de áudio com legendas -->
<audio src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/audio/foo.ogg">
  <track kind="captions" src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/audio/foo.en.vtt"
srclang="en" label="English" />
  <track kind="captions" src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/audio/foo.sv.vtt"
srclang="sv" label="Svenska" />
</audio>
```

Reproduz o arquivo de áudio anexado a esse artigo.

Interface do DOM

- HTMLAudioElement

Veja também

- Media formats supported by the audio and video elements
- Introducing the audio API extension
- `HTMLAudioElement`
- `nsIDOMHTMLMediaElement`
- `video`
- Using audio and video
- The `audio` element (Especificação HTML5)

`<b>`

Resumo

O **elemento HTML <b>** representa um intervalo de texto estilísticamente diferente do texto normal, sem transmitir qualquer importância ou relevância. Ele é geralmente usado para destacar palavras-chaves em um resumo, nomes de produtos em um comentário ou outros vãos de texto cuja a apresentação típica seria negrito. Outro exemplo de seu uso é como marcação da frase principal de cada paragrafo de um artigo.

> **Note: Notas de Uso:**
>
> - Não confunda o elemento <b> com os elementos `<strong>` , `<em>` ou `<mark>` . O elemento `<strong>` representa um texto com importância, `<em>` coloca alguma ênfase no texto e o elemento `<mark>` representa o texto relevante. O elemento <b> não transmite essa informação semântica especial. Você deve usá-lo apenas quando os outros elementos citados não se encaixam na situação.
> - Também não é recomendado marcar títulos de cabeçalhos usando o elemento <b>. Para isso, use as tags `<h1>` até `<h6>`. usando essas tags você terá liberdade de mudar o estilo padrão dos elementos e não ficará preso ao negrito do elemento <b>.
> - É uma boa prática usar o atributo de classe <b>, a fim de transmitir informação semântica adicional. Isso facilita o desenvolvimento de várias "stylings" de um documento web, sem a necessidade de alterar o seu código HTML.
> - Historicamente, o elemento <b> foi criado para fazer o texto ficar em negrito. Porém as informações de estilo ficou obsoleta desde o HTML4, de modo que significado da tag <b> foi alterado.
> - Se não a propósito semântico sobre o uso do elemento <b>, usar a propriedade css font-weight com algum volume seria uma escolha melhor para fazer o texto ficar em negrito.

- *Content categories* Flow content, phrasing content, palpable content.
- *Permitted content* Phrasing content.
- *Tag omission* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Permitted parent elements* Any element that accepts phrasing content.
- *DOM interface* `HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the `HTMLSpanElement` interface for

Atributos

Este elemento só inclui os atributos globais.

Exemplo

```
<p>
  Este artigo descreve vários <b>níveis de texto</b>. Ele explica a utilização
  do elemento em um documento <b>HTML</b>.
</p>
Palavras-chave são exibidas com o estilo padrão do elemento
<b>, provavelmente em negrito.</b>
```

Resultado

Este artigo descreve vários **níveis de texto**. Ele explica a utilização do elemento em um documento **HTML**.

Palavras-chave são exibidas com o estilo padrão do elemento <b>, provavelmente em negrito.

Veja Também

- Outros elementos de transporte text-level semantics: `<a>`, `<em>`, `<strong>`, `<small>`, `<cite>`, `<q>`, `<dfn>`, `<abbr>`, `<time>`, `<code>`, `<var>`, `<samp>`, `<kbd>`, `<sub>`, `<sup>`, `<i>`, `<mark>`, `<ruby>`, `<rp>`, `<rt>`, `<bdo>`, `<span>`, `<br>`, `<wbr>`.
- Using <b> and <i> elements (W3C)

<base>

Resumo

O *elemento HTML Base* (**<base>**) especifica o endereço (URL) utilizada por todos os endereços relativos contidos dentro de um documento. Há um número máximo de 1 (um) elemento *Base* <base> do documento.

> **Nota:** Se multiplos elementos `<base>` forem especificados, apenas o primeiro valor de **href** e **target** serão utilizados, os demais serão ignorados.

- *Content categories* Metadata content.
- *Permitted content* None, it is an empty element.
- *Tag omission* There must be no closing tag.
- *Permitted parent elements* Any `<head>` that doesn't contain any other `<base>` element
- *DOM interface* `HTMLBaseElement`

Atributos

Este elemento inclui os atributos globais.

- `href`
  - : O URL de base para ser usado em todo o documento para endereços URL relativos.Se este atributo for especificado, este elemento deve vir antes de quaisquer outros elementos com atributos cujos valores são URLs.Endereços (URLs) absolutos e relativos são permitidos (mas ver seção nota abaixo).
- `target`
  - : Esse elemento é uma palavra-chave que indica o local padrão para exibir o resultado quando os hiperlinks causarem navegações, por elementos que não têm referência alvo explícita. Em HTML5, é um nome ou palavra-chave para um contexto de navegação. (por exemplo, guia, janela, ou frame incorporada). As seguintes formas têm significados específicos:
- _self: Coloca i resultado no mesmo quadro HTML4 ou contexto de navegação, caso HTML5, como o atual. Este valor é o padrão se o atributo não for especificado.
- _blank: Coloca o resultado em uma nova janela HTML4 ou janela no contexto de navegação HTML5.
- _parent: Coloca o resultado no conjunto de quadros pai do quadro atual no caso de HTML4 ou contexto de navegação pai no atual HTML5. Se não houver nenhum pai, esta opção se comporta como a forma _self.
- _top: Em HTML4: Coloca o resultado na janela completa, original, cancelando todos os outros quadros. Em HTML5: Coloca o resultado no contexto de navegação de nível superior (ou seja, o contexto de navegação em que é um antepassado do atual, e não o pai). Se não houver nenhum pai, este opção se comporta da mesma forma que _self.

Exemplos

```
<base href="http://www.example.com/" />
<base target="_blank" href="http://www.example.com/" />
```

Notas

- A URL base de um documento pode ser consultada utilizando o seguinte script `document.baseURI`.
- Support of relative URIs for `href` was added in Gecko 2.0 (Firefox 4.0)
- HTML 2.0 and 3.2 define only the `href` attribute
- XHTML requires a trailing slash: `<base />`

<bdi>

Resumo

O HTML <bdi>elemento (ou elemento de isolamento de Bi-direcional) isola um trechode texto que pode ser formatado em uma direção diferente de outro texto fora dela.

Este elemento é útil ao incorporar o texto com uma direção desconhecida, de um banco de dados por exemplo, no interior do texto com uma direção fixa.

Apesar do mesmo efeito visual pode ser alcançado usando a regra de CSS`unicode-bidi`: isolar um `<span>` ou outro elemento de formatação de texto, o significado semântico só é transportado pelo elemento de<bdi>. Especialmente, os navegadores podem ignorar estilos CSS. Nesse caso, o texto seria ainda corretamente exibido usando o elemento HTML, mas se tornará lixo quando usando o estilo CSS para transmitir semântica.

- *Content categories* Flow content, phrasing content, palpable content.
- *Permitted content*Phrasing content.
- *Tag omission* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Permitted parent elements* Any element that accepts phrasing content.
- *DOM interface* `HTMLElement`

Atributos

Como todos os outros elementos do HTML, este elemento tem os atributos globais, com uma pequena diferença de semântica: o atributo dir não é herdado. Se não definido, o valor padrão é o auto que deixa o navegador decidir a direção com base no conteúdo do elemento.

Exemplo

```
<p dir="ltr">
  Esta palavra arábica <bdi>ARABIC_PLACEHOLDER</bdi> é automaticamente voltada
  da direita
</p>
```

Veja Também

- Related HTML element: `<bdo>`
- Related HTML properties: `direction`, `unicode-bidi`

<bdo>

O **elemento* HTML `<bdo>` **(*bidirectional override*) é usado para substituir a direcionalidade atual do texto. Isso faz com que a direcionalidade dos personagens seja ignorada em favor da direcionalidade especificada.

Content categories	Flow content, phrasing content, palpable content.
Permitted content	Phrasing content.
Tag omission	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Permitted parents	Any element that accepts phrasing content.
Permitted ARIA roles	Any
DOM interface	`HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the `HTMLSpanElement` interface for this element.

Atributos

Os atributos desse elemento incluem os atributos globais.

`dir`

Direção de texto neste elemento. Os valores possíveis são:

- `ltr`: Indica que o texto ficará da **Esquerda para a Direita** (**L**eft-**T**o-**R**ight).
- `rtl`: Indica que o texto ficará da **Direita para a Esquerda** (**R**eft-**T**o-**L**ight).

Exemplos

```
<!-- Muda a direção do texto -->
<p>Este texto ficará da esquerda para a direita.</p>
<p><bdo dir="rtl">Este texto ficará da direita para a esquerda.</bdo></p>
```

Resultado

Notas

A especificação HTML 4 não especificam eventos para este elemento; eles foram adicionados no XHTML. Este é provavelmente um descuido.

<big>: Elemento para deixar o texto maio

The obsolete **HTML Big Element** (**`<big>`**) renders the enclosed text at a font size one level larger than the surrounding text (`medium` becomes `large`, for example). The size is capped at the browser's maximum permitted font size.

> **Note: Usage note:** As it was purely presentational, this element has been removed in HTML5 and shouldn't be used anymore. Instead web developers should use the CSS `font-size` property to adjust the font size.

Atributos

This element has no other attributes than the global attributes, common to all elements.

Examples

Here we see examples showing the use of `<big>` followed by an example showing how to accomplish the same results using modern CSS syntax instead.

Usando <code><big></code>

Este elemento usa o comando `<big>` para aumentar o tamanho do texto. O elemento é obsoleto, porém aceitável em todos os navegadores.

HTML

```
<p>
  Este é o primeiro texto.
  <big>Este texto usa big para ficar com a aparência maior.</big>
</p>
```

Resultado

Using CSS <code>font-size</code>

This example uses the CSS `font-size` property to increase the font size by one level.

CSS

```
.bigger {
  font-size: larger;
}
```

HTML

```
<p>
  This is the first sentence.
  <span class="bigger">This whole sentence is in bigger letters.</span>
</p>
```

Result

DOM interface

This element implements the `HTMLElement` interface.

> **Note: Implementation note:** Up to Gecko 1.9.2 inclusive, Firefox implements the `HTMLSpanElement` interface for this element.

See also

- CSS: `font-size`, `font`
- HTML: `<small>`, `<font>`, `<style>`
- HTML 4.01 Specification: Font Styles

<blockquote>

Sumário

O Elemento HTML <blockquote> (ou Elemento HTML de citação de bloco) indica que o texto incluído é uma longa citação. Normalmente, este é processado visualmente pelo recuo (ver Notas sobre como mudá-lo). A URL para a fonte da citação pode ser dada usando o atributo **cite**, enquanto uma representação de texto da fonte pode ser dada usando o `<cite>` elemento.

- *Categorias de conteúdo* Fluxo de conteúdo, raiz selecionamento, conteúdo palpável.
- *Conteúdo permitido* Fluxo de conteúdo.
- *Tag Omissão* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos permitidos* Qualquer elemento que aceita o fluxo de conteúdo.
- *Papéis ARIA permitidos* Qualquer
- *Interface DOM* `HTMLQuoteElement`

Atributos

Este elemento inclui os atributos globais.

`cite`

> Uma URL que designa um documento de origem ou mensagem para a informação citada. Este atributo destina-se a apontar para informações explicando o contexto ou a referência para a cotação.

Notas

Para alterar o recuo de `<blockquote>`, use a propriedade CSS `margin`.

Para citações curtas, use o elemento `<q>`.

Exemplo

```
<blockquote cite="http://developer.mozilla.org">
  <p>Esta é uma citação tirada da Mozilla Developer Center.</p>
</blockquote>
```

O código HTML acima vai resultar em:

> Esta é uma citação tirada da Mozilla Developer Center.

Veja também

- O elemento `<q>` para citações em linha.
- O elemento `<q>` para citações de origem.

<body>

O **elemento** `<body>` do **HTML** representa o conteúdo de um documento HTML. è permitido apenas um `<body>` por documento.

Categorias	Seção raiz.
Conteúdo permitido	Conteúdo de fluxo.
Omissão de tag	A tag inicial pode ser omitida se a primeira coisa dentro do elemento não for um espaço, comentário, elemento `<script>` ou elemento `<style>`. A tag final pode ser omitida se o elemento body possuir conteúdo ou tem uma tag inicial e não é imediatamente seguida por um comentário.
Elementos pais permitidos	Deve ser o segundo elemento de um elemento `<html>`.
DOM interface	`HTMLBodyElement` • O elemento `<body>` expõe a interface `HTMLBodyElement`. • Você pode acessar o elemento body através do atributo `document.body`.

Atributos

Estes elementos incluem os atributos globais.

`alink`

Cor do texto para hyperlinks quando selecionados. Esse método não está em conformidade, utilize a propriedade *`color`* *em conjunto com a pseudo-classe* *`:active`*

`background`

URI de uma imagem para usar como um papel de parede. Esse método não está em conformidade, utilize a propriedade *CSS* *`background`*.

`bgcolor`

Cor de fundo do documento. Esse método não está em conformidade, utilize a propriedade *CSS* *`background-color`*.

`bottommargin`

The margin of the bottom of the body. Esse método não está em conformidade, utilize a propriedade *CSS* *`margin-bottom`*.

`leftmargin`

The margin of the left of the body. Esse método não está em conformidade, utilize a propriedade *CSS* *`margin-left`*.

`link`

Cor de textos de links não visitados. Esse método não está em conformidade, utilize o *`color`* *em conjunto com a pseudo-classe* *`:link`*

`onafterprint`

Função para chamar depois que o usuário solicitou a impressão do documento.

`onbeforeprint`

Função para chamar quando o usuário solicita a impressão do documento.

`onbeforeunload`

Função para chamar quando o documento está prestes a ser descarregado.

`onblur`

Função para chamar quando o documento perde foco.

`onerror`

Função para chamar quando o documento falha ao carregar corretamente.

`onfocus`

Função para chamar quando o documento recebe foco.

`onhashchange`

Function to call when the fragment identifier part (starting with the hash (`'#'`) character) of the document's current address has changed.

`onlanguagechange`

Function to call when the preferred languages changed.

`onload`

Função para chamar quando o documento terminar de carregar

`onmessage`

Função para chamar quando o documento receber uma mensagem.

`onoffline`

Função para chamar quando a conexão com a internet falha.

`ononline`

Função para chamar quando a conexão com a internet é restaurada.

`onpopstate`

Function to call when the user has navigated session history.

`onredo`

Function to call when the user has moved forward in undo transaction history.

`onresize`

Function to call when the document has been resized.

`onstorage`

Function to call when the storage area has changed.

`onundo`

Function to call when the user has moved backward in undo transaction history.

`onunload`

Function to call when the document is going away.

`rightmargin`

The margin of the right of the body. *This method is non-conforming, use CSS `margin-right` property on the element instead.*

`text`

Foreground color of text. *This method is non-conforming, use CSS `color` property on the element instead.*

`topmargin`

The margin of the top of the body. *This method is non-conforming, use CSS `margin-top` property on the element instead.*

`vlink`

Cor do texto para links já visitados. Esse método não está em conformidade, utilize a propriedade *CSS `color` em conjunto com a pseudo-classe `:visited`.*

Veja também

- `<html>`
- `<head>`

Sumário

O elemento HTML *quebra-de-linha*
 produz uma quebra de linha em um texto (carriage-return).É útil para escrever poemas ou um endereço, onde a divisão de linha é significante.

Não use
 para aumentar o espaço entre as linhas de texto; para isso use a propriedade CSS `margin` ou o elemento `<p>`.

Categorias de Conteúdo	Conteúdo de Fluxo, conteúdo fraseado.
Conteúdo permitido	Nenhum, é um empty element.
Omissão de tag	Deve ter uma tag de abertura, e não deve ter uma tag de fechamento. Em documentos XHTML , escreve esse elemento como ` `.
Elementos pai permitidos	Qualquer elemento que aceita conteúdo fraseado.
Interface DOM	`HTMLBRElement`

Atributos

Este elemento inclue os Atributos globais.

`clear`

Indica onde começar a próxima linha depois da quebra.

> **Nota:** Este atributo está obsoleto em HTML 5 e **Não deve ser usado por autores**. Use a propriedade CSS `clear` em vez disso.

Exemplo

```
Mozilla Foundation<br />
1981 Landings Drive<br />
Building K<br />
Mountain View, CA 94043-0801<br />
USA
```

Saída do HTML acima:

Mozilla Foundation
1981 Landings Drive

Building K
Mountain View, CA 94043-0801
USA

Veja também

- O elemento `<address>`
- O elemento `<p>`

<button>

Sumário

O **Elemento HTML *`<button>`*** representa um botão clicável.

- *Categorias de conteúdo* Conteúdo de fluxo, conteúdo fraseado, Conteúdo iterativo, listado, rotulável, e elemento enviável associado a formulário, conteúdo palpável.
- *Conteúdo permitido* Conteúdo fraseado.
- *Omissão de Tag* Nenhuma, as tags de abertura e fechamento são obrigatórias.
- *Elementos pai permitidos* Qualquer elemento que aceite conteúdo fraseado.
- *Interface DOM* `HTMLButtonElement`
- *Tipo de elemento* Inline
-

Atributos

Esse elemento inclui os atributos globais.

`autofocus`

> Esse atributo booleano permite-o especificar que o botão possuirá o foco de entrada assim que a página carrega, a menos que o usuário sobrecreva esse comportamento digitanto um controle diferente. Apenas um elemento de um documento associado a um formulário pode ter esse atributo específico.

`autocomplete`

> O uso desse atributo em um `<button>` não está padronizado nem dentro das especificações do Firefox. Por padrão, diferente de outros navegadores, o Firefox persiste com o estado dinâmico desativado de um `<button>` nas páginas carregadas. Definir o valor desse atributo para `off` (ex: `autocomplete="off"`) desabilita esse recurso. Veja Erro do Firefox 654072.

`disabled`

> Esse atributo booleano indica que o usuário não poderá interagir com o botão. Se esse atributo não for especificado, o botão herdará a configuração do elemento que o contém, por exemplo `<fieldset>`, se não existir nenhum elemento com o atributo **disabled** definido, então o botão estará habilitado.
>
> Firefox irá, diferente de outros navegadores, por padrão, persiste com o estado dinâmico desativado de um `<button>` sob as páginas carregadas. Use o atributo `autocomplete` para controlar esse recurso.

`form`

> O elemento de formulário que o botão está associado (é o *formulário proprietário*). O valor do atributo deve ser o atributo **id** de um elemento `<form>` no mesmo documento. Se esse atributo não for especificado, o elemento `<button>` deve ser descendente de um elemento de formulário. Esse atributo permite que você coloque elementos `<button>` em qualquer lugar do documento, não apenas como descendente de seus elementos `<form>`.

`formaction`

> A URI de um programa que processa a informação submetida pelo botão. Se especificado, ele sobrescreve o atributo `action` do formulário proprietário do botão.

`formenctype`

Se o botão é um botão de envio, esse atributo especifica o tipo de conteúdo que é usado para enviar o formulário para o servidor; Possíveis valores são:

- `application/x-www-form-urlencoded`: O valor padrão se o atributo não está especificado.
- `multipart/form-data`: Use esse valor se você está usando um elemento `<input>` com o atributo `type` definido para o `arquivo`.
- `text/plain`

Se esse atributo fro especificado, ele sobrescreve o atributo `enctype` do formulário proprietário do botão.

`formmethod`

Se o botão for um botão de envio, esse atributo especifica o método HTTP que o navegador usará para enviar o formulário. Possíveis valores são:

- `post`: Os dados obtidos do formulário são incluídos em seu corpo e enviados para o servidor.
- `get`: Os dados obtidos do formulário são anexados aos atributos URI do **formulário**, com uma '?' como separador, e o resultado URI é enviado para o servidor. Use esse método quando o formulário não possui efeitos colaterais e contém apenas caracteres ASCII.

Se especificado, esse atributo sobrescreve o atributo `method` do formulário proprietário do botão.

`formnovalidate`

Se o botão é um botão de envio, esse atributo Booleano especifica que o formulário não é para ser validado quando submetido. Se esse atributo for especificado, ele sobrescreve o atributo `novalidate` do formulário proprietário do botão.

`formtarget`

Se o botão é um botão de envio, esse atributo é um nome ou palavra-chave indicando onde exibir a resposta que é recebida após o envio do formulário. Esse é um nome de, ou palavra-chave para, um *contexto de navegação* (por exemplo, uma aba, janela ou quadro embutido). Se esse atributo é especificado, ele sobrescreve o atributo `target` do formulário proprietário do botão. As seguintes palavras-chaves possuem significados especiais:

- `_self`: Carrega a resposta no mesmo contexto navegação como o atual. Esse valor é o padrão se o atributo não é especificado.
- `_blank`: Carrega a resposta em um contexto de navegação sem nome.
- `_parent`: Carrega a resposta no contexto de navegação pai do atual. Se não há nenhum pai, essa opção passa a ser o mesmo que `_self`.
- `_top`: Carrega a resposta para o contexto de navegação no nível superior (ou seja, o contexto de navegação é um ancestral do atual e não possui nenhum pai). Se não possui nenhum pai, essa opção passa a agir da mesma forma que `_self`.

`name`

O nome do botão que é enviado com os dados do formulário.

`type`

O tipo de botão. O possíveis valores são:

- `submit`: O botão envia os dados do formulário para o servidor. Esse é o padrão se o atributo não for especificado, ou se o atributo é dinamicamente mudado para um valor vazio ou inválido.
- `reset`: O botão restaura todos os controles para seus valores iniciais.
- `button`: O botão não possui comportamento padrão. Ele pode ter scripts do lado do cliente associado com os eventos do elemento, no qual são acionados quando o evento ocorrer.

`value`

O valor inicial do botão.

Notas

É muito mais fácil estilizar elementos `<button>` do que elementos `<input>`. Você pode adicionar conteúdo HTML (como `<em>`, `<strong>`, ou até `<img>`), e usar pseudo-elementos `::after` e `::before` para executar composições complexas, enquanto `<input>`

aceita apenas um atributo `value` do tipo texto.

Exemplo

```
<button name="button">Click me</button>
```

Note que esse botão possui CSS aplicado.

Notas

Elementos `<button>` são muito mais fáceis estilizá-los do que elementos `<input>`. Você pode adicionar dentro do conteúdo do HTML (imagine em `<em>`, `<strong>` ou mesmo `<img>`), e fazer uso do pseudo-elemento `:after` e `:before` para realizar renderizações complexas enquanto `<input>` apenas aceita um atributo com valor textual.

IE7 possui um bug ao enviar um formulário com `<button type="submit" name="myButton" value="foo">Click me</button>`, os dados POST enviados terá como resultado em `myButton=Click me` em vez de `myButton=foo`. IE6 possui um bug ainda pior quando enviado um formulário através de um botão por enviar TODOS os botões do formulário com o mesmo bug do IE7. Esse bug foi corrigido no IE8.

Firefox adicionará, com propósitos de acessibilidade, uma pequena borda pontinhada e um botão focado. Essa borda será declarada por meio de CSS, no estilo do navegador, mas você pode sobrescreve-lo se necessário para adicionar seu próprio estilo de foco usando `button::-moz-focus-inner { }`

Firefox irá, diferente de outros navegadores, por padrão, persistir o estado dinâmico desativado de um `<button>` sob o carregamento das páginas. Definindo o valor do atributo `autocomplete` para `off` desabilita esse recurso. See Erro do Firefox 654072.

Firefox <35 para Android define um padrão `background-image` gradiente em todos os botões (see Erro do Firefox 763671). Isso pode ser desabilitado usando `background-image: none`.

Veja também

Outros elementos que são usados para criar formulários: `<form>`, `<datalist>`, `<fieldset>`, `<input>`, `<label>`, `<legend>`, `<meter>`, `<optgroup>`, `<option>`, `<output>`, `<progress>`, `<select>`, `<textarea>`.

canvas

Sumário

O elemento HTML *Canvas* (`<canvas>`) pode ser utilizado para desenhar gráficos utilizando scripts (geralmente JavaScript). Por exemplo, além de desenhar gráficos, ele pode ser usado para fazer composições de fotos e também para animações. Você poderá colocar conteúdos alternativos dentro do bloco `<canvas>`. Este conteúdo será renderizado também em navegadores antigos e em navegadores com JavaScript desabilitado.

Para mais artigos sobre canvas, veja canvas topic page.

Contexto de uso

Conteúdo permitido	**Transparente, tanto phrasing content ou flow content.**
Omissão de tags	Nenhuma, tanto a tag de início quanto a de fim devem ser incluídas.
Elementos "pai" permitidos	Qualquer elemento que aceite phrasing content ou flow content.
Documento normativo	HTML 5, section 4.8.10

Atributos

Como qualquer outro elemento HTML, este também tem global attributes.

`width`

A largura do espaço em pixels CSS. O padrão é 300.

`height`

A altura do espaço em pixels CSS. O padrão é 150.

> **Nota:** The displayed size of the canvas can be changed using a stylesheet. The image is scaled during rendering to fit the styled size.

Interface DOM

- HTMLCanvasElement

Exemplo

```
<canvas id="canvas" width="300" height="300">
  Desculpe-me, seu navegador não suporta o elemento &lt;canvas&gt;.
</canvas>
```

Veja também

- MDN canvas portal
- Um canvas tutorial
- Canvas cheat sheet
- Canvas-related demos

<caption>

O elemento HTML **`caption`** especifica a legenda (ou título) de uma tabela.

Experimente

```
<table>
  <caption>
    He-Man and Skeletor facts
  </caption>
  <tr>
    <td></td>
    <th scope="col" class="heman">He-Man</th>
    <th scope="col" class="skeletor">Skeletor</th>
  </tr>
  <tr>
    <th scope="row">Role</th>
    <td>Hero</td>
    <td>Villain</td>
  </tr>
  <tr>
    <th scope="row">Weapon</th>
    <td>Power Sword</td>
    <td>Havoc Staff</td>
  </tr>
  <tr>
    <th scope="row">Dark secret</th>
    <td>Expert florist</td>
    <td>Cries at romcoms</td>
  </tr>
</table>
```

```
caption {
  caption-side: bottom;
  padding: 10px;
  font-weight: bold;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
```

```
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

th {
  background-color: rgb(230 230 230);
}

td {
  text-align: center;
}

tr:nth-child(even) td {
  background-color: rgb(250 250 250);
}

tr:nth-child(odd) td {
  background-color: rgb(240 240 240);
}

.heman {
  font: 1.4rem molot;
  text-shadow:
    1px 1px 1px #fff,
    2px 2px 1px #000;
}

.skeletor {
  font: 1.7rem rapscallion;
  letter-spacing: 3px;
  text-shadow:
    1px 1px 0 #fff,
    0 0 9px #000;
}
```

Categoria de conteúdo	Nenhuma.
Conteúdo permitido	Flow content.
Omissão de tag	A tag final pode ser omitida se o elemento não for seguido imediatamente por um espaço em branco ASCII ou um comentário.
Elementos pais permitidos	Um elemento `<table>`, como seu primeiro filho.
Função ARIA implícita	Nenhuma função correspondente
Permitted ARIA roles	No `role` permitted
DOM interface	`HTMLTableCaptionElement`

Atributos

Este elemento inclui os atributos globais.

Atributos obsoletos

Os seguintes atributos são obsoletos e não devem ser usados. Estão documentados abaixo apenas como referência ao atualizar o código, e interesse histórico.

`align`

Esse atributo enumerado indica como a legenda deve ser alinhada em relação à tabela. Ele pode ter um dos seguintes valores:

`left`

A legenda é exibida à esquerda da tabela.

`top`

A legenda é exibida acima da tabela.

`right`

A legenda é exibida à direita da tabela.

`bottom`

A legenda é exibida abaixo da tabela.

> **Aviso:** Não use este atributo já que ele foi depreciado: O elemento `<caption>` deve ser estilizado usando as propriedades CSS. Para dar um efeito similar ao atributo `align`, use as propriedades CSS `caption-side` e `text-align`.

Notas de uso

Se utilizado, o elemento `<caption>` deve ser o primeiro filho de seu elemento `<table>` pai.

Quando o elemento `<table>` que contém o `<caption>` é o único descendente de um elemento `<figure>`, você deve usar o `<figcaption>` elemento em vez de `<caption>`.

Um `background-color` na tabela não incluirá a legenda. Adicione um `background-color` ao elemento `<caption>` também se você deseja que a mesma cor de fundo esteja por trás de ambos.

Exemplo

Esse exemplo simples apresenta uma tabela que inclui uma legenda

```
<table>
  <caption>
    Exemplo de Legenda
  </caption>
  <tr>
    <th>Login</th>
    <th>Email</th>
  </tr>
  <tr>
    <td>usuario1</td>
    <td>usuario1@dominio.com</td>
  </tr>
  <tr>
    <td>usuario2</td>
    <td>usuario2@dominio.com</td>
  </tr>
</table>
```

Veja também

- Propriedades CSS que podem ser especialmente úteis para estilizar o elemento `<caption>`:
  - `text-align`, `caption-side`.

: O elemento de texto centralizado (obsoleto)

O elemento **Center do HTML** (`<center>`) se tornou obsoleto, é um elemento de nível de bloco que exibe o conteúdo em nível de bloco ou em linha centralizado horizontalmente dentro do elemento que o contém. O contêiner `<body>` geralmente está presente, mas não é necessário. Essa tag foi descontinuada no HTML 4 (e XHTML 1) em favor da propriedade CSS `text-align`, que pode ser aplicada ao elemento `<div>` ou à um paragráfo indivídual `<p>`. Para blocos de centralização, use outras propriedades CSS como `margin-left` e `margin-right` e defina-as como `auto` (ou defina `margin` para `0 auto`).

Interface DOM

Este elemento implementa a interface `HTMLElement`.

> **Note: Nota de implementação:** até o Gecko 1.9.2 inclusive, o Firefox implementa a interface `HTMLSpanElement` para este elemento.

Exemplo 1

```
<center>
  Esse texto vai ficar centralizado.
  <p>Este parágrafo também.</p>
</center>
```

O efeito da centralização vai acontecer até onde o elemento `center` for fechado.

Exemplo 2 (Alternativa CSS)

```
<div style="text-align:center">
  Esse texto vai ficar centralizado.
  <p>Este parágrafo também.</p>
</div>
```

Isso acontece por os textos que sofreram as alterações estão dentro dos elementos que foi aplicado a propriedade.

Exemplo 3 (Alternativa CSS)

```
<p style="text-align:center">
  Essa linha vai ficar centralizada.<br />
  E esta linha também.
</p>
```

Nota

Aplicar `text-align:center` a um elemento `<div>` ou `<p>` centraliza o *conteúdo* desses elementos, deixando suas dimensões gerais inalteradas.

Apesar de obsoleto, o elemento é aceito em todos os navegadores do mercado.

Veja também

- `text-align`
- `display`

<cite>

O **elemento HTML <cite>** representa uma referência a um trabalho artístico. Deve incluir o título do trabalho ou uma URL de referência, que pode ser em uma forma abreviada de acordo com as convenções usadas para a adição dos metadados de citação.

Content categories	Flow content, phrasing content, palpable content.
Permitted content	Phrasing content.

Tag omission	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Permitted parents	Any element that accepts phrasing content.
Permitted ARIA roles	Any
DOM interface	`HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the `HTMLSpanElement` interface for this element.

Atributos

Este elemento inclui apenas atributos globais.

Notas

- Um trabalho criativo deve conter a citação de um livro,um documento,um ensaio,um poema,um filme,um programa de televisão,um video game,uma escultura,uma pintura,uma produção cinematográfica,um jogo,uma opera,um musical,uma exibição,um relatório de um caso legal,um programa de computador,um website,uma página da web,um post de um blog ou comentário,um post em um forum ou um comentário no mesmo,um tweet,uma obra ou discurso,etc.
- O W3C específica alguns estados para referenciar um trabalho criativo que deve incluir o nome do autor,enquanto a WHATWG declarou que não deve ser incluso o nome do autor sob nenhuma circustância.
- Use o atributo `cite` em um elemento `<blockquote>` ou `<q>` para referenciar uma fonte online.
- Para evitar que o itálico padrão do estilo seja usado pelo elemento <cite> use a propiedade `font-style` do CSS.

Exemplo

```
Mais informações podem ser encontradas em <cite>[ISO-0000]</cite>.
```

A saída do HTML acima:

Mais informações podem ser encontradas em [ISO-0000].

Veja também

- O elemento `<blockquote>` é para citações longas.
- O elemento `<q>` é para citações curtas ou na mesma linha.

<code>: O Elemento Inline Code

O **elemento** **HTML `<code>` **apresenta seu conteúdo estilizado de maneira a indicar que o texto é um pequeno fragmento de código. Por padrão, o conteúdo é exibido utilizando a fonte monoespaçada padrão do user agent's.

Experimente

```
<p>
  The <code>push()</code> method adds one or more elements to the end of an
  array and returns the new length of the array.
</p>
```

```
code {
  background-color: #eee;
  border-radius: 3px;
  font-family: courier, monospace;
  padding: 0 3px;
}
```

Content categories	Flow content, phrasing content, palpable content.
Permitted content	Phrasing content.
Tag omission	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Permitted parents	Any element that accepts phrasing content.

Permitted ARIA roles	Any
DOM interface	`HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the `HTMLSpanElement` interface for this element.

Atributos

Esse elemento inclui apenas os global attributes.

Exemplo

Um parágrafo que inlcui `<code>`:

```
<p>
  A função <code>selectAll()</code> destaca todo o texto no campo de entrada
  então o usuário pode, por exemplo, copiar ou deletar o texto.
</p>
```

A saída gerada por esse trecho HTML se parece com isso:

Notas

Para representar múltiplas linha de código, envolva o elemento `<code>` dentro do elemento `<pre>`. O elemento `<code>` por si só representa somente uma única senteça ou linha de código.

Uma regra CSS pode ser definida para o seletor `code` para sobrescrever a fonte padrão do navegador. Preferências definidas pelo usuário podem ter precedência sore o CSS especifciado.

Veja também

- `<samp>`
- `<kbd>`
- `<command>` (deprecated)
- `<var>`
- `<pre>`

datalist

Sumário

O elemento HTML *Datalist* (<datalist>) contém um conjunto de elementos `<option>` que representam as opções possíveis para o valor de outros controles.

Contexto de uso

Conteúdo permitido	**Phrasing content ou qualquer quantidade de elementos `<option>` (ou nenhum)**
Omissão de tags	Nenhuma, ambas as tags inicial e final são obrigatórias
Elementos pais permitidos	Qualquer elemento que aceite phrasing content
Documento normativo	HTML5, section 4.10.10

Atributos

Esse elemento não tem nenhum atributo a não ser os global attributes, comuns para todos os elementos.

Interface do DOM

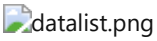
Esse elemento implementa a interface `HTMLDataListElement`.

Exemplos

```
<input list="browsers" />
<datalist id="browsers">
  <option value="Chrome"></option>
  <option value="Firefox"></option>
  <option value="Internet Explorer"></option>
  <option value="Opera"></option>
  <option value="Safari"></option>
</datalist>
```

View Live Examples

Esse código HTML é renderizado assim no Firefox 4:


datalist.png

Polyfill

Inclua este polyfill para fornecer suporte para navegadores mais antigos e atualmente incompatíveis: datalist-polyfill

Veja também

- O elemento `<input>`, especialmente seu atributo `list`;
- O elemento `<option>`.

<dd>: elemento Detalhes da Descrição

O **elemento HTML <dd>** fornece detalhes ou uma definição mais completa do termo precedente (definido por `<dt>`) numa lista de descrições (`<dl>`).

Categorias de conteúdo	Nenhuma.
Conteúdo permitido	Conteúdo de fluxo.
Omissão de Tag	A Tag inicial (`<dd>`) é obrigatória. A Tag final (</dd>) pode ser omitida se seguida imediatamente por outro elemento `<dd>`, ou se o elemento pai não tiver mais conteúdo.
Ancestrais permitidos	`<dl>` ou (em WHATWG HTML) uma `<div>` que está dentro de uma `<dl>`.
Tags irmãs	`<dt>` ou outro elemento `<dd>` .
Papeis ARIA permitidos	Nenhum.
Tipo de elemento DOM	`HTMLElement`

Atributos

Este elemento inclui os atributos globais como seus.

`nowrap`

Se o valor deste atributo for `yes`, o texto de definição não será envolvido. O valor padrão é `no`.

Exemplo

Para exemplos, veja os exemplos de <dl>.

Veja também

- `<dl>`

- `<dt>`

<del>

Sumário

O **elemento HTML** `<del>` (ou *Elemento HTML de Texto Excluído*) representa uma parte do texto que foi excluída de um documento. Este elemento é (não necessariamente) renderizado pelos navegadores com uma linha entre o texto.

- *Content categories*Phrasing content or flow content.
- *Permitted content* Transparent.
- *Tag omission* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Permitted parent elements* Any element that accepts phrasing content.
- *DOM interface* `HTMLModElement`

Attributes

This element includes the global attributes.

`cite`

A URI for a resource that explains the change (for example, meeting minutes).

`datetime`

This attribute indicates the time and date of the change and must be a valid date with an optional time string. If the value cannot be parsed as a date with an optional time string, the element does not have an associated time stamp.

Examples

```
<p><del>This text has been deleted</del></p>
```

Result

~~This text has been deleted~~

See also

- `<ins>` element for insertions into a text

details

Sumário

O elemento HTML *details* (`<details>`) é usado como uma ferramenta de onde o usuário irá obter informações adicionais.

Contexto de uso

Conteúdo permitido	**Um elemento `<summary>` seguido por flow content**
Omissão de tag	Nenhuma, tanto as tas iniciais quanto as finais são obrigatórias.
Elementos pais permitidos	Qualquer elemento que aceite flow content.
Documento normativo	HTML5, section 4.11.1

Atributos

Como todos os elementos HTML, esse elemento aceita os global attributes.

`open`

Esse atributo Booleano indica se os detalhes serão mostrados para o usuário ao carregar a página. Se omitido os detalhes não serão mostrados.

Interface do DOM

Esse elemento implementa a interface`HTMLDetailsElement`.

Exemplos

```
<details>
  <summary>Alguns detalhes</summary>
  <p>Mais informações sobre os detalhes.</p>
</details>
```

Veja também

- `<summary>`

<dfn>

Resumo

O elemento **HTML `<dfn>` **(ou *Elemento Definição HTML*) representa uma instância de definição de um termo.

> **Note: Notas de uso no HTML5:**- O elemento `<dfn>` marca o termo que será definido; a definição do termo deve ser data em torno de `<p>`, `<section>` ou uma lista de definições por grupo (usualmente um `<dt>`, `<dd>` pareado com o `<dt>`).
>
> - O valor exato do termo a ser definido é determinado pelas seguintes regras:
>   1. Se o elemento <dfn> tiver um atributo title, então o termo é o valor deste atributo.
>   2. Senão, se contiver somente um elemento `<abbr>` com um atributo `title`, então o termo é o valor deste atributo.
>   3. Caso contrário, o conteúdo do texto do elemento `<dfn>` é o termo a ser definido.

- *Categorias de conteúdo* Conteúdo de fluxo, Conteúdo de fraseamento, conetúdo palpável.
- *Conteúdo permitido* Conteúdo de fraseamento, mas um elemento `<dfn>` não deve ser um descendente.
- *Omissão de tag* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos pais permitidos* Qualquer elemento que aceite conteúdo de fraseamento.
- *Interface DOM* `HTMLElement`

Atributos

Esse elemento inclui os atributos globais.

No HTML5, o atributo **title** tem um significado especial, como mencionado acima.

Exemplos

Os exemplos a seguir são válidos em HTML5.

```
<!-- Define "A Internet" -->
<p>
  <dfn id="def-internet">A Internet</dfn> é um sistema global de redes
  interconectadas que usam o Internet Protocol Suite (TCP/IP) para servir
  bilhões de usuários no mundo todo.
</p>
```

Posteriormente no mesmo documento:

```
<dl>
  <!-- Define "World-Wide Web" a definição de referência para "A Internet" -->
  <dt>
    <dfn>
      <abbr title="World-Wide Web">WWW</abbr>
    </dfn>
  </dt>
  <dd>
    A World-Wide Web (WWW) é um sistema de documentos de hipertexto interligados
    acessados pela <a href="#def-internet">Internet</a>.
```

```
  </dd>
</dl>
```

Resultado

A Internet é um sistema global de redes interconectadas que usam o Internet Protocol Suite (TCP/IP) para servir bilhões de usuários no mundo todo.

WWW

: A World-Wide Web (WWW) é um sistema de documentos de hipertexto interligados acessados pela Internet.

Veja também

- Elementos relacionados à lista de definições: `<dl>`, `<dt>`, `<dd>`
- `<abbr>`

<dialog>: O elemento Dialog

O **elemento HTML** `<dialog>` representa uma caixa de diálogo ou outro componente interativo, tal como um inspetor ou janela.

Categorias de conteúdo	Flow content, sectioning root
Permitted content	Flow content
Tag omission	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Permitted parents	Qualquer elemento que aceite flow content
Permitted ARIA roles	`alertdialog`
DOM interface	`HTMLDialogElement`

Atributos

Este elemento inclui os atributos globais. O atributo `tabindex` não deve ser utilizado no elemento `<dialog>`.

`open`

: Indica que o Dialog está ativo e pronto para uso. Quando o atributo `open` não for definido, ele não deve ser mostrado ao usuário.

Notas de uso

- Elementos de formulário (`<form>`) podem ser integrados dentro de um elemento `<dialog>`, especificando-os com o atributo `method="dialog"`. Quando esse formulário é submetido, o diálogo é fechado com o seu `returnValue` (valor de retorno) configurado para o valor botão `submit` do formulário que foi usado.
- O pseudo-elemento CSS `::backdrop` pode ser usado para estilizar o fundo de um elemento `<dialog>`, como um escurecer um conteúdo inacessível enquanto uma janela `modal` está aberta, por exemplo. O backdrop(pano de fundo) só está disponível quando um diálogo é exibido com `HTMLDialogElement.showModal()`.

Exemplos

Exemplo simples

```
<dialog open>
  <p>Olá para todos!</p>
</dialog>
```

Exemplo Avançado

Este exemplo abre uma caixa de diálogo contendo um formulário quando o botão "Update details" é clicado.

HTML

```
<!-- Um dialog simples contendo um form -->
<dialog open id="favDialog">
  <form method="dialog">
    <section>
      <p>
        <label for="favAnimal">Favorite animal:</label>
        <select id="favAnimal">
          <option></option>
          <option>Brine shrimp</option>
          <option>Red panda</option>
          <option>Spider monkey</option>
        </select>
      </p>
    </section>
    <menu>
      <button id="cancel" type="reset">Cancel</button>
      <button type="submit">Confirm</button>
    </menu>
  </form>
</dialog>

<menu>
  <button id="updateDetails">Update details</button>
</menu>
```

JavaScript

```
(function () {
  var updateButton = document.getElementById("updateDetails");
  var cancelButton = document.getElementById("cancel");
  var favDialog = document.getElementById("favDialog");

  // O botão Update abre uma Dialog
  updateButton.addEventListener("click", function () {
    favDialog.showModal();
  });

  // O botão cancelButtom fecha uma Dialog
  cancelButton.addEventListener("click", function () {
    favDialog.close();
  });
})();
```

Resultado

Polyfills

Inclua este polyfill para suportar browsers antigos.

dialog-polyfill

Veja também

- O `close` evento
- O `cancel` evento
- Guia de formulários HTML.

<dir>: O elemento obsoleto Directory

O elemento de diretório HTML obsoleto (**`<dir>`**) é usado como um contêiner para um diretório de arquivos e/ou pastas, potencialmente com estilos e ícones aplicados pelo user agent. Não use este elemento obsoleto; em vez disso, você deve usar o `<ul>` elemento para listas, incluindo listas de arquivos.

> **Nota:** Não use este elemento. Embora presente nas primeiras especificações do HTML, foi descontinuado no HTML 4 e, desde então, foi totalmente removido. Nenhum dos principais navegadores suporta esse elemento.

DOM interface

Este elemento implementa a interface `HTMLDirectoryElement`.

Atributos

Como todos os outros elementos HTML, este elemento suporta os global attributes.

`compact`

Este atributo booleano indica que a lista deve ser renderizada em um estilo compacto. A interpretação deste atributo depende do agente do usuário e não funciona em todos os navegadores.

> **Nota:** Não use este atributo, pois ele foi descontinuado: o `<dir>` elemento deve ser estilizado usando CSS. Para dar um efeito semelhante ao alcançado com o atributo `compact`, a propriedade CSS `line-height` pode ser usado com um valor de `80%`.

Veja também

- Outros relacionados a lista de elementos HTML: `<ol>`, `<ul>`, `<li>`, e `<menu>`;
- Propriedades CSS que podem ser especialmente úteis para estilizar o elemento `<dir>` :
  - A propriedade `list-style`, útil para escolher a maneira como o ordinal é exibido.
  - CSS counters, útil para lidar com listas aninhadas complexas.
  - A propriedade `line-height`, útil para simular o atributo obsoleto `compact`.
  - A propriedade `margin`, útil para controlar o recuo da lista.

Resumo

O **elemento de divisão** **HTML `<div>` **é um container genérico para conteúdo de fluxo, que de certa forma não representa nada. Ele pode ser utilizado para agrupar elementos para fins de estilos (usando **class** ou **id**), ou porque eles compartilham valores de atributos, como **lang**. Ele deve ser utilizado somente quando não tiver outro elemento de semântica (tal como `<article>` ou `<nav>`).

- *Categoria de conteúdo* Flow content, palpable content.
- *Conteúdo permitido* Flow content.
- *Omissão de tag* Nenhuma, as tags de abertura e fechamento são obrigatórias. Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos pais permitidos* Qualquer elemento que aceite conteúdo de fluxo.
- *Interface DOM* `HTMLDivElement`

Atributos

Esse elemento inclui atributos globais.

No HTML 5, o atributo **align** no ** ** é obsoleto.

Exemplos

```
<div>
  <p>
    Qualquer tipo de conteúdo aqui. Como &lt;p&gt;, &lt;table&gt;. Você dá o
    nome!
  </p>
</div>
```

Resultado

Qualquer tipo de conteúdo aqui. Como <p>, <table>. Você dá o nome!

Veja também

- Semantic sectioning elements: `<section>`, `<article>`, `<nav>`, `<header>`, `<footer>`
- `<span>` element for styling of phrasing content

dl

Resumo

O elemento HTML *Definition List* (`<dl>`) engloba uma lista de pares de termos e descrições. Um uso comum para este elemento é para implementar um glossário ou exibir metadados(uma lista de pares chave e valor).

Contexto de Uso

Conteúdo permitido	**Zero ou mais elementos `<dt>`, cada um seguido de um ou mais elementos `<dd>`**
Omissão de Tag	Nenhum, este é um elemento vazio.
Permite elementos pai	Qualquer elemento que aceite conteúdo de fluxo.
Documentação normativa	HTML5, section 4.5.8; HTML 4.01, section 10.3

Atributos

Como todo elemento HTML, ele fornece os attributos globais.

`compact`

Obriga a definição da descrição aparecer na mesma linha que a definição do termo. Funciona apenas no Internet Explorer.

Exemplos

Termo e definição única

```
<dl>
  <dt>Firefox</dt>
  <dd>
    A free, open source, cross-platform, graphical web browser developed by the
    Mozilla Corporation and hundreds of volunteers.
  </dd>

  <!-- other terms and definitions -->
</dl>
```

Output:

Image:HTML-dl1.png

Múltiplos termos, definição única

```
<dl>
  <dt>Firefox</dt>
  <dt>Mozilla Firefox</dt>
  <dt>Fx</dt>
  <dd>
    A free, open source, cross-platform, graphical web browser developed by the
    Mozilla Corporation and hundreds of volunteers.
  </dd>

  <!-- other terms and definitions -->
</dl>
```

Saída:



Termo único, múltiplas definições

```
<dl>
  <dt>Firefox</dt>
  <dd>
    A free, open source, cross-platform, graphical web browser developed by the
    Mozilla Corporation and hundreds of volunteers.
  </dd>
  <dd>
    The Red Panda also known as the Lesser Panda, Wah, Bear Cat or Firefox, is a
    mostly herbivorous mammal, slightly larger than a domestic cat (60 cm long).
  </dd>

  <!-- other terms and definitions -->
</dl>
```

Saída:



Múltiplos termos e definições

Também é possível definir vários termos com as definições múltiplas correspondentes, combinando com os exemplos acima.

Interface DOM

- HTMLDListElement

Notas

Não use este elemento, (nor `<ul>` elements), para criar meramente um recuo em uma página. Embora ele funcione, está é uma má prática e obscurece o significado da lista de definição.

Para mudar a indentação de um termo, use a propriedade CSS margin.

Veja também

- Elemento `<dt>`
- Elemento `<dd>`

<dt>

Sumário

O elemento **HTML `<dt>`** (ou Elemento HTML de Definição de Termo) identifica um termo na lista de definição. Este elemento pode ocorrer somente em um elemento filho de `<dl>`. Geralmente seguido por um elemento `<dd>`; ou multiplos `<dt>` na mesma linha indicam vários termos sendo definidos pelo próximo element `<dd>`.

- *Content categories* Vazio.
- *Permitted content* Flow content, but with no `<header>`, `<footer>`, sectioning content or heading content descendants.
- *Tag omission* Must have a start tag. The end tag may be omitted if this element is immediately followed by another `<dd>` element, or if there is no more content in the parent element.
- *Permitted parent elements* Before a `<dt>` or a `<dd>` element, inside a `<dl>`.
- *DOM interface* `HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the HTMLSpanElement interface for this element.

Atributos

Este elemento inclui somente atributos globais.

Exemplo

Para exemplo veja definição de lista.

Veja também

- `<dd>`, `<dl>`

: O elemento de ênfase

O **elemento HTML** `<em>` marca o texto que tem ênfase. O elemento pode ser aninhado, com cada nível de aninhamento indicando um grau maior de ênfase.

Categorias de conteúdo	Conteúdo fluído, conteúdo de frase, conteúdo palpável
Conteúdo permitido	Conteúdo de frase.
Omissão de tag	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Pais permitidos	Qualquer elemento que aceita conteúdo de frase.
Funções de ARIA permitidas	Qualquer uma
Interface DOM	`HTMLElement` Até o Gecko 1.9.2 (Firefox 4) incluso, o Firefox implementa a interface`HTMLSpanElement` para este elemento.

Atributos

Este elemento inclui somente os atributos globais.

Notas de uso

Este elemento geralmente é mostrado no tipo itálico. Contudo, não deve ser usado simplesmente para aplicar estilo itálico; use o estilo CSS para esse propósito. Use o elemento `<cite>` para marcar o título de um trabalho (book, play, song, etc.); também é geralmente estilizado com tipo itálico, mas possui significado diferente. Use o elemento `<strong>` para marcar o texto que tem importância maior do que o texto que está em volta.

Exemplo

O elemento `<em>` é frequentemente usado para indicar um contraste implícito ou explícito.

```
<p>
  Em HTML 5, o que anteriormente era chamado de conteúdo
  <em>block-level</em> agora é chamado de conteúdo <em>flow</em> .
</p>
```

Resultado

Em HTML5, o que anteriormente era chamado de conteúdo block-level (nível de bloco) agora é chamado de *flow* content (conteúdo fluído).

<i> vs <em>

É frequentemente confuso para novos desenvolvedores porque há vários elementos para expressar a ênfase em algum texto. `<i>` e `<em>` são talvez uns dos mais comuns. Por que usar `<em></em>` vs `<i></i>`? Eles produzem o mesmo resultado, certo?

Não exatamente. O resultado visual é, por padrão, o mesmo - ambas as tags renderizam seu conteúdo em itálicos. Mas o significado semântico é diferente. A tag `<em>` representa ênfase importante de seus conteúdos, enquanto que a tag `<i>` representa o texto que é iniciado de uma prosa, como uma palavra estrangeira, pensamentos de um personagem ficcional, ou quando o texto se refere á definição de uma palavra em vez de representar seu significado semântico. (O título de um trabalho, tal como o nome de um livro ou filme, deve usar `<cite>`.)

Um exemplo para `<em>` poderia ser: "Apenas já faça isso!", ou: "Nós temos que fazer algo acerca disso". Uma pessoa ou software lendo o texto pronunciaria as palavras em itálico com uma ênfase.

Um exemplo para `<i>` poderia ser: "A *Rainha Mary* velejou na noite passada". Aqui, não é adicionada ênfase ou importância na palavra "Rainha Mary". É meramente indicado que o objeto em questão não é uma rainha chamada Mary, mas um navio chamado *Rainha Mary*. Outro exemplo para `<i>` poderia ser: "A palavra *o* é um artigo".

Veja também

- `<i>`

<embed>

Sumário

O **elemento HTML `<embed>`** incorpora conteúdo externo no ponto especificado no documento. Este conteúdo é fornecido por um aplicativo externo ou outra fonte de conteúdo interativo, como um plug-in de navegador.

Experimente

```
<embed
  type="video/mp4"
  src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/embed//shared-assets/videos/flower.mp4"
  width="250"
  height="200" />
```

> **Nota:** Este tópico documenta apenas o elemento definido como parte do HTML5. Ele não trata da implementação anterior e não padronizada do elemento.

Lembre-se de que a maioria dos navegadores modernos descontinuou e removeu o suporte para plug-ins de navegador. Portanto, confiar no `<embed>` geralmente não é aconselhável se você deseja que seu site funcione no navegador do usuário comum.

- *Categoria de conteúdo* Flow content, phrasing content, embedded content, interactive content, palpable content.
- *Conteúdo permitido* Nenhum, é um empty element.
- *Omissão de Tag* Deve iniciar com uma tag de início, mas não deve ter uma tag de fechamento.
- *Elementos pai (ancestral) permitidos* Qualquer elemento que aceita incorporação de conteúdo.
- *Interface DOM* `HTMLEmbedElement`

Atributos

Este elemento inclui os atributos globais.

`height`

A altura apresentada do recurso, em CSS pixels. Este deve ser um valor absoluto; porcentagens *não* são permitidas.

`src`

A URL do recurso que está sendo incorporado.

`type`

O tipo MIME a ser usado para a seleção do *plug-in* de instanciação.

`width`

A largura apresentada do recurso, em CSS pixels. Este deve ser um valor absoluto; porcentagens *não* são permitidas.

Exemplo

```
<embed type="video/quicktime" src="https://developer.mozilla.org/pt-
```

```
BR/docs/Web/HTML/Reference/Elements/embed/movie.mov" width="640" height="480" />
```

> **Nota:** Atualmente existe uma diferença na implementação pelos navegadores. Enquanto carrega, no Chrome e no Opera, aparece o conteúdo do recurso em HTML, no Firefox, mostra uma mensagem genérica falando que o conteúdo precisa de um *plug-in* (veja o Erro do Firefox 730768). É recomendado que se use os elementos `<object>` ou `<iframe>`.

Veja também

- Outros elementos usados para incorporar conteúdo de vários tipos, incluem: `<audio>`, `<canvas>`, `<iframe>`, `<img>`, `<math>`, `<object>`, `<svg>`, and `<video>`.
- Posicionamento e dimensionamento de conteúdos incorporados dentro da janela de exibição: `object-position` and `object-fit`

<fieldset>

O **elemento HTML** `<fieldset>` é usado para agrupar elementos, assim como labels (`<label>`), dentro de um formulário web.

Categorias de Conteúdo	Conteúdo de fluxo, sectioning root, listed, form-associated element, palpable content.
Conteúdo permitido	Um elemento `<legend>` opcional, seguido por conteúdo de fluxo.
Omissão deTag	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Elementos pai permitidos	Qualquer elemento que aceita conteúdo de fluxo.
Interface DOM	`HTMLFieldSetElement`

> **Nota:** diferente de quase todo outro elemento, a especificação do WHATWG HTML Rendering sugere `min-width: min-content` como parte do padrão de estilo para `<fieldset>`, e muitos navegadores implementam tal estilização (ou algo que se aproxima disto).

Atributos

Este elemento inclui os atributos globais.

`disabled`

> Se este atributo Booleano está definido, os controles de formulario que são seus descendentes, exceto os descendentes de seu primeiro elemento opcional `<legend>`, estarão desativados, i.e., não editável. Eles não receberão qualquer evento de navegação, como clique do mouse ou relacionado com focus. Frequentemente os navegadores mostram tais controles em cinza.

`form`

> Este atributo tem o valor do atributo **id** do elemento `<form>` com que se relaciona. Seu valor padrão é o **id** do elemento `<form>` mais próximo.

`name`

> O nome associado com o grupo.
>
> > **Nota:** O label para o fieldset é dado pelo primeiro elemento `<legend>` que é um filho do fieldset.

Exemplos

Example #1: Formulário com fieldset, legend, e label

```
<form action="test.php" method="post">
  <fieldset>
    <legend>Title</legend>
    <input type="radio" id="radio" /> <label for="radio">Click me</label>
  </fieldset>
</form>
```

Example #2: Simulando um editável <code><select></code> através de um <code>fieldset</code> de radioboxes e textboxes*

O exemplo a seguinte é feito de puro HTML e CSS. Não contém javascript.

Esteja avisado que leitores de tela (utilizados por cegos) e dispositivos de assistência não interpretarão o seguinte formulário corretamente; este exemplo seria um html invalido se os corretos elementos fossem usados.

```
<!doctype html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Editable [pseudo]select</title>
    <style type="text/css">
      /* Generic form fields */

      fieldset.elist,
      input[type="text"],
      textarea,
      select,
      option,
      fieldset.elist ul,
      fieldset.elist > legend,
      fieldset.elist input[type="text"],
      fieldset.elist > legend:after {
        -webkit-box-sizing: border-box;
        -moz-box-sizing: border-box;
        box-sizing: border-box;
      }

      input[type="text"] {
        padding: 0 20px;
      }

      textarea {
        width: 500px;
        height: 200px;
        padding: 20px;
      }

      textarea,
      input[type="text"],
      fieldset.elist ul,
      select,
      fieldset.elist > legend {
        border: 2px #cccccc solid;
        border-radius: 10px;
      }

      input[type="text"],
      fieldset.elist,
      select,
      fieldset.elist > legend {
        height: 32px;
        font-family: Tahoma;
        font-size: 14px;
```

```
      }

      input[type="text"]:hover,
      textarea:hover,
      select:hover,
      fieldset.elist:hover > legend {
        background-color: #ddddff;
      }

      select {
        padding: 4px 20px;
      }

      option {
        height: 30px;
        padding: 5px 4px;
      }

      option:not(:checked),
      textarea:focus {
        background-color: #ffcccc;
      }

      fieldset.elist > legend:after,
      fieldset.elist label {
        height: 28px;
      }

      input[type="text"],
      fieldset.elist {
        width: 316px;
      }

      input[type="text"]:focus {
        background: #ffcccc

url("
Kyp2dnf39/QAAAK6ursifZHFxcc/Qzu3mxYyMjExCJnV1dc6maO7u7o+Pj2tXNoaGhtfDpKCDVu3lxM+tcaKEV9bW1qOFVWNjY8KrisTExNra2nBbObGxs
by8vO/mu7Kyso9ZAuzs7MSgAIiKhf///8zMzP///wAAAAAAAAAAAAAAAAAAAAAAACH5BAEAADkALAAAAAAQABAAAAaXwJxwSCwOYzWkMpkkZmoAqDQaJdp
qAqw2m53NRjlboAarFczomcE0C99o8DgNMVM8Tm3bbYDr9x11DwkzDG5yc2oQJIRCenx/MxoeETM2Q3pxATMlF4MYlo17OAsdLispMyAioIY0BzMcITMTK
BasjgssFTMqGxItMjYUoTQBBAQHxgE0wZcfMtDRMi/QrA022NnaNg1CQQA7")
           no-repeat 2px center !important;
      }

      input[type="text"]:focus,
      textarea:focus,
      select:focus,
      fieldset.elist > legend {
        border: 2px #ccaaaa solid;
      }

      fieldset {
        border: 2px #af3333 solid;
        border-radius: 10px;
      }

      /* Editable [pseudo]select (i.e. fieldsets with [class=elist]) */

      fieldset.elist {
        display: inline-block;
        position: relative;
        vertical-align: middle;
        overflow: visible;
        padding: 0;
        margin: 0;
        border: none;
      }
```

```
fieldset.elist ul {
  position: absolute;
  width: 100%;
  max-height: 320px;
  padding: 0;
  margin: 0;
  overflow: hidden;
  background-color: transparent;
}

fieldset.elist:hover ul {
  background-color: #ffffff;
  border: 2px #af3333 solid;
  left: 2px;
  overflow: auto;
}

fieldset.elist ul > li {
  list-style-type: none;
  background-color: transparent;
}

fieldset.elist label {
  display: none;
  width: 100%;
}

fieldset.elist input[type="text"] {
  width: 100%;
  height: 30px;
  line-height: 30px;
  border: none;
  background-color: transparent;
  border-radius: 0;
}

fieldset.elist > legend {
  display: block;
  margin: 0;
  padding: 0 0 0 5px;
  position: absolute;
  width: 100%;
  cursor: default;
  background-color: #ccffcc;
  line-height: 30px;
  font-style: italic;
}

fieldset.elist:hover > legend {
  position: relative;
  overflow: hidden;
}

fieldset.elist > legend:after {
  width: 20px;
  content: "\2335";
  float: right;
  text-align: center;
  border-left: 2px #cccccc solid;
  font-style: normal;
  cursor: default;
}

fieldset.elist:hover > legend:after {
  background-color: #99ff99;
}
```

```
    fieldset.elist ul input[type="radio"] {
      display: none;
    }

    fieldset.elist input[type="radio"]:checked ~ label {
      display: block;
      width: 292px;
      background-color: #ffffff;
    }

    fieldset.elist:hover input[type="radio"]:checked ~ label {
      width: 100%;
    }

    fieldset.elist:hover label {
      display: block;
      height: 100%;
    }

    fieldset.elist label:hover {
      background-color: #3333ff !important;
    }

    fieldset.elist:hover input[type="radio"]:checked ~ label {
      background-color: #aaaaaa;
    }
  </style>
</head>
<body>
  <form method="get" action="test.php">
    <fieldset>
      <legend>Order a T-Shirt</legend>
      <p>Write your name (simple textbox): <input type="text" /></p>
      <p>
        Choose your size (simple select):
        <select>
          <option value="s">Small</option>
          <option value="m">Medium</option>
          <option value="l">Large</option>
          <option value="xl">Extra Large</option>
        </select>
      </p>
      <div>
        What address do you want to use? (editable pseudoselect)
        <fieldset class="elist">
          <legend>Address&hellip;</legend>
          <ul>
            <li>
              <input
                type="radio"
                value="1"
                id="address-switch_1"
                checked /><label for="address-switch_1"
                ><input
                  type="text"
                  value="19 Quaker Ridge Rd. Bethel CT 06801"
              /></label>
            </li>
            <li>
              <input type="radio" value="2" id="address-switch_2" /><label
                for="address-switch_2"
                ><input
                  type="text"
                  value="1000 Coney Island Ave. Brooklyn NY 11230"
              /></label>
            </li>
```

```
              <li>
                <input type="radio" value="3" id="address-switch_3" /><label
                  for="address-switch_3"
                  ><input
                    type="text"
                    value="2962 Dunedin Cv. Germantown TN 38138"
                /></label>
              </li>
              <li>
                <input type="radio" value="4" id="address-switch_4" /><label
                  for="address-switch_4"
                  ><input
                    type="text"
                    value="915 E 7th St. Apt 6L. Brooklyn NY 11230"
                /></label>
              </li>
            </ul>
          </fieldset>
        </div>
        <p>
          Write a comment:<br />
          <textarea></textarea>
        </p>
        <p>
          <input type="reset" value="Reset" />
          <input type="submit" value="Send!" />
        </p>
      </fieldset>
    </form>
  </body>
</html>
```

Veja este exemplo em ação

Erros

- Erro do Firefox 504622 - Fieldsets nunca diminui abaixo de sua largura min-intrinsic
- WebKit bug 123507 - `min-width: -webkit-min-content` sobre fieldset
- StackOverflow discussão com soluções alternativas para os bugs acima

Veja também

- Outros elementos de formlários relacionados: `<form>`, `<legend>`, `<label>`, `<button>`, `<select>`, `<datalist>`, `<optgroup>`, `<option>`, `<textarea>`, `<input>`, `<output>`, `<progress>` e `<meter>`.
- Artigo de como usar o fieldset para ajudar na validação do seu formulário html

<figcaption>: O elemento de legenda da figura

O **Elemento HTML Figcaption** (`<figcaption>`) representa uma legenda ou uma legenda associada com uma figura ou ilustração descrita pelo resto dos dados do elemento `<figure>` que seu elemento pai.

Experimente

```
<figure>
  <img
    src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/figcaption//shared-
assets/images/examples/elephant.jpg"
    alt="Elephant at sunset" />
  <figcaption>An elephant at sunset</figcaption>
</figure>
```

```
figure {
  border: thin #c0c0c0 solid;
  display: flex;
  flex-flow: column;
```

```
  padding: 5px;
  max-width: 220px;
  margin: auto;
}

img {
  max-width: 220px;
  max-height: 150px;
}

figcaption {
  background-color: #222;
  color: #fff;
  font: italic smaller sans-serif;
  padding: 3px;
  text-align: center;
}
```

Categorias de conteúdo	Nenhuma.
Conteúdo permitido	Flow content.
Omissão de Tag	Nenhuma, tanto a tag de início quanto a de fim são obrigatórias.
Elemento Pai Permitido	Um Elemento `<figure>`
Interface DOM	`HTMLElement`

Atributos

Esse elemento possui somente os atributos globais.

Exemplos

Por favor, consulte a página `<figure>` para visualizar o exemplos do elemento `<figcaption>`.

Veja também

- O elemento `<figure>`.

<figure>: O elemento Figure com Caption opcional

O elemento **`<figure>`** HTML representa conteúdo autocontido, potencialmente com uma legenda opcional, que é especificada usando o `<figcaption>` elemento. A figura, sua legenda e seu conteúdo são referenciados como uma única unidade.

Experimente

```
<figure>
  <img
    src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/figure//shared-
assets/images/examples/elephant.jpg"
    alt="Elephant at sunset" />
  <figcaption>An elephant at sunset</figcaption>
</figure>
```

```
figure {
  border: thin #c0c0c0 solid;
  display: flex;
  flex-flow: column;
  padding: 5px;
  max-width: 220px;
  margin: auto;
}

img {
  max-width: 220px;
  max-height: 150px;
```

```
}

figcaption {
  background-color: #222;
  color: #fff;
  font: italic smaller sans-serif;
  padding: 3px;
  text-align: center;
}
```

Categorias de conteúdo	Conteudo de fluxo, conteúdo palpável.
Conteúdo permitido	Um elemento `<figcaption>`, seguido por conteúdo de fluxo; ou conteúdo de fluxo seguido por um `<figcaption>` elemento; ou conteúdo de fluxo.
Omissão de tag	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Pais permitidos	Qualquer elemento que aceite Contúdo de fluxo.
Função ARIA implícita	figure
Funções ARIA permitidas	Com nenhum figcaption descendente: qualquer, caso contrário, não há papéis permitidos
interface DOM	`HTMLElement`

Atributos

Este elemento inclui apenas os atributos globais.

Notas de uso

- Normalmente uma `<figure>` é uma imagem, ilustração, diagrama, trecho de código, etc., que é referenciado no fluxo principal de um documento, mas que pode ser movido para outra parte do documento ou para um apêndice sem afetar o fluxo principal.
- Uma legenda pode ser associada ao elemento `<figure>` inserindo um `<figcaption>` dentro dele (como o primeiro ou o último filho). O primeiro elemento `<figcaption>` encontrado na figura é apresentado como legenda da figura.

Exemplos

Imagens

```
<!-- Apenas uma imagem -->
<figure>
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/figure/favicon-192x192.png" alt="O
lindo logotipo do MDN." />
</figure>

<!-- Imagem com legenda -->
<figure>
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/figure/favicon-192x192.png" alt="O
lindo logotipo do MDN." />
  <figcaption>Logotipo MDN</figcaption>
</figure>
```

Trechos de codigo

```
<figure>
  <figcaption>
    Obtenha detalhes do navegador usando o <code>navigator</code>.
  </figcaption>
  <pre>
function NavigatorExample() {
  var txt;
  txt = "Nome do código do navegador: " + navigator.appCodeName + "; ";
```

```
  txt+= "Nome do navegador: " + navigator.appName + "; ";
  txt+= "Versão do navegador: " + navigator.appVersion + "; ";
  txt+= "Cookies habilitados: " + navigator.cookieEnabled + "; ";
  txt+= "Plataforma: " + navigator.platform + "; ";
  txt+= "Cabeçalho do agente do usuário: " + navigator.userAgent + "; ";
  console.log("Exemplo de Navegador", txt);
}
  </pre>
</figure>
```

Citações

```
<figure>
  <figcaption><b>Edsger Dijkstra:</b></figcaption>
  <blockquote>
    Se a depuração é o processo de remoção de bugs de software, então a
    programação deve ser o processo de colocá-los.
  </blockquote>
</figure>
```

Poemas

```
<figure>
  <p style="white-space:pre">
    Ofereça-me um discurso, encantarei seu ouvido, Ou como uma viagem de fadas
    sobre o verde, Ou, como uma ninfa, com longos cabelos desgrenhados, Dança na
    areia, e mas nenhum fundamento visto: O amor é um espírito todo compacto de
    fogo, Não grosseiro para afundar, mas leve, e vai aspirar.
  </p>
  <figcaption><cite>Vênus e Adônis</cite>, de William Shakespeare</figcaption>
</figure>
```

Veja também

- O elemento `<figcaption>`.

footer (rodapé)

Resumo

O elemento HTML de Rodapé (<footer>) representa um rodapé para o seu sectioning content (conteúdo de seção) mais próximo ou sectioning root elemento (ou seja, seu parente mais próximo `<article>`, `<aside>`, `<nav>`, `<section>`, `<blockquote>`, `<body>`, `<details>`, `<fieldset>`, `<figure>`, `<td>`). Normalmente um rodapé contém informações sobre o autor da seção de dados, direitos autorais ou links para documentos relacionados.

> **Note: Notas de Uso:**
>
> - Coloque informações sobre o autor em um elemento `<address>` que pode ser incluído no elemento `<footer>`.
> - O elemento `<footer>` não é sectioning content portanto, não introduz uma nova seção no outline.

Contexto de uso

Conteúdo permitido	**Flow content, mas não o `<footer>` ou `<header>` descendentes.**
Omissão de tag	Nenhuma, ambas as tags de início e fim são obrigatórias.
Parent Elements permitidos	Qualquer elemento que aceite flow content. Note-se que um elemento <footer> não deve ser um descendente de um `<address>`, `<header>` ou outro elemento `<footer>`.
Documento normativo	HTML5, section 4.4.9

Atributos

Este elemento não tem outros atributos que os global attributes, comuns a todos os elementos.

DOM Interface

Este elemento implementa a interface `HTMLElement`.

Exemplos

```
<footer>
  Algumas informações de copyright ou talvez alguma informação do autor de um <article>?
</footer>
```

Veja também

- Outros elementos de seção relacionados `<body>`, `<nav>`, `<article>`, `<aside>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<hgroup>`, `<header>`, `<section>`, `<address>`;
- Sections and outlines of an HTML5 document.

<head>

O elemento **HTML `<head>` providencia informações gerais** (metadados) sobre o documento, incluindo seu título e links para scripts e folhas de estilos.

Categoria de conteúdo	Nenhuma.
Conteúdo permitido	Se o elemento é um `<iframe>` `srcdoc`, ou se a informação do título está sendo avaliada desde um nivel alto do protocolo, zero ou mais elementos de conteúdo de metadados. Por outro lado, um ou mais elementos de conteúdo de metadados onde exatamente um é um `<title>` elemento.
Omissão de tag	A tag inicial pode ser omitida se a primeira coisa dentro do head elemento é um elemento. A tag final pode ser omitida se a primeira coisa seguindo o elemento head não é um caractere espaço ou um comentário.
Elementos parentes permitidos	Um `<html>` elemento, como primeiro filho.
DOM interface	`HTMLHeadElement`

Attributos

Estes elementos incluem os atributos globais.

`profile`

A URI de um ou mais perfis de metadados, separado por espaço em branco.

Exemplo

```
<html>
  <head>
    <title>Document title</title>
  </head>
</html>
```

Notas

Moderno, HTML5-compatível browsers automaticamente constrói um elemento `<head>` se as tags forem omitidas na marcação. Este comportamento não pode ser garantido por browsers antigos.

See also

- Elementos que podem ser usados dentro de um `<head>` element: `<title>`, `<base>`, `<link>`, `<style>`, `<meta>`, `<script>`, `<noscript>`

header (cabeçalho)

Sumário

O **elemento HTML `<header>`** representa um grupo de suporte introdutório ou navegacional. Pode conter alguns elementos de cabeçalho mas também outros elementos como um logo, seções de cabeçalho, formulário de pesquisa, e outros.

Experimente

```
<header>
  <a class="logo" href="#">Cute Puppies Express!</a>
</header>

<article>
  <header>
    <h1>Beagles</h1>
    <time>08.12.2014</time>
  </header>
  <p>
    I love beagles <em>so</em> much! Like, really, a lot. They’re adorable and
    their ears are so, so snugly soft!
  </p>
</article>
```

```
.logo {
  background: left / cover
    url("/shared-assets/images/examples/puppy-header.jpg");
  display: flex;
  height: 120px;
  align-items: center;
  justify-content: center;
  font:
    bold calc(1em + 2 * (100vw - 120px) / 100) "Dancing Script",
    fantasy;
  color: #ff0083;
  text-shadow: #000 2px 2px 0.2rem;
}

header > h1 {
  margin-bottom: 0;
}

header > time {
  font: italic 0.7rem sans-serif;
}
```

> **Note: Notas de utilização:**
>
> - O elemento `<header>` não é separador de conteúdo (sectioning content), portanto, não introduz uma nova seção no outline.

- *Categoria de conteúdo* Flow content, conteúdo palpável.
- *Conteúdo permitido*Flow content, mas não com descendentes de `<header>` ou `<footer>`

- *Omissão da tag* Nenhuma, as tags de abertura e fechamento são obrigatórias.
- *Elementos-pai permitidos* Qualquer elemento que aceite flow content. Note que o elemento `<header>` não deve ser descendente de `<address>`, `<footer>` ou outro elemento `<header>` .
- *Interface DOM* `HTMLElement`

Atributos

Este elemento não possui outros atributos além dos atributos globais, comuns a todos os elementos.

Exemplos

```
<header>
  <h1>Título da Página Principal</h1>
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/header/mdn-logo-sm.png" alt="MDN
logo">
</header>
```

Veja também

- Uma descrição mais detalhada: O elemento header | HTML5 Doctor
- Outros elementos relacionados a seção: `<body>`, `<nav>`, `<article>`, `<aside>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<hgroup>`, `<footer>`, `<section>`, `<address>`;
- Seções e outlines de um documento HTML5.

<h1>–<h6>: Os elementos HTML de cabeçalho da seção

Os **elementos HTML `<h1>–<h6>`** representam seis níveis de título de seção. `<h1>` é o nível de seção mais alto e `<h6>` é o mais baixo.

Experimente

```
<h1>Beetles</h1>
<h2>External morphology</h2>
<h3>Head</h3>
<h4>Mouthparts</h4>
<h3>Thorax</h3>
<h4>Prothorax</h4>
<h4>Pterothorax</h4>
```

```
h1,
h2,
h3,
h4 {
  margin: 0.1rem 0;
}

h1 {
  font-size: 2rem;
}

h2 {
  font-size: 1.5rem;
  padding-left: 20px;
}

h3 {
  font-size: 1.2rem;
  padding-left: 40px;
}

h4 {
  font-size: 1rem;
  font-style: italic;
```

```
  padding-left: 60px;
}
```

Sumário

Elementos de **cabeçalho** são implementados em seis níveis, `<h1>` é o mais importante e `<h6>` é o de menor importância. Um elemento de cabeçalho descreve brevemente o tópico da seção em que ele está. As informações de cabeçalho podem ser usadas por agentes de usuário, por exemplo, para construir uma tabela de conteúdos para um documento automaticamente.

- *Categorias do conteúdo* Conteúdos de fluxo, conteúdos de cabeçalho, conteúdos palpáveis.
- *Conteúdo permitido* Conteúdo com texto.
- *Omissão de tag* Não há, porém, as tags de abertura e fechamento são obrigatórias.
- *Elementos pais permitidos* Qualquer elemento que aceite conteúdos de fluxo; ou como um elemento filho do grupo de elementos de cabeçalho `<hgroup>`
- *Interface DOM* `HTMLHeadingElement`

Atributos

Esses elementos incluem os atributos globais.

O atributo **align** está no HTML 4 e no HTML 5.

Exemplos

Todos os cabeçalhos

O seguinte código mostra, em uso, todos os níveis de cabeçalho.

```
<h1>Cabeçalho nível 1</h1>
<h2>Cabeçalho nível 2</h2>
<h3>Cabeçalho nível 3</h3>
<h4>Cabeçalho nível 4</h4>
<h5>Cabeçalho nível 5</h5>
<h6>Cabeçalho nível 6</h6>
```

Aqui está o resultado deste código:

Página de exemplo

O código seguinte mostra alguns cabeçalhos com um pouco de conteúdo abaixo deles.

```
<h1>Elementos de cabeçalho</h1>
<h2>Sumário</h2>
<p>Algum texto aqui...</p>

<h2>Exemplos</h2>
<h3>Exemplo 1</h3>
<p>Algum texto aqui...</p>

<h3>Exemplo 2</h3>
<p>Algum texto aqui...</p>

<h2>Veja também</h2>
<p>Algum texto aqui...</p>
```

Aqui está o resultado deste código:

Notas

Não use níveis menores para diminuir o tamanho da fonte do cabeçalho: use a propriedade CSS `font-size` .

Evite pular níveis de cabeçalhos: sempre comece com `<h1>`, depois `<h2>` e assim por diante. Tente, também, ter pelo menos um cabeçalho de primeiro nível em uma página..

Em HTML 5, use o elemento `<section>` para definir o outline de um documento. Cabeçalhos fornecem títulos para secções e subsecções. Você também pode agrupar um cabeçalho e seu conteúdo usando o elemento `<div>` .

Compatibilidade com navegadores

Veja também

- `<p>`
- `<div>`
- `<section>`

<hgroup>

> **Experimental: Esta é uma tecnologia experimental**
> Verifique a tabela de compatibilidade entre Navegadores cuidadosamente antes de usar essa funcionalidade em produção.

O **elemento HTML `<hgroup>` **destina-se a agrupar cabeçalhos de diferentes níveis para uma seção do documento. Ele agrupa (é um container para) um conjunto de elementos `<h1>-<h6>`.

Categorias de conteudo	Flow content, heading content, palpable content.
Conteúdo permitido	One or more `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, and/or `<h6>`.
Omissão de tag	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Elementos-pai permitidos	Any element that accepts flow content.
DOM interface	`HTMLElement`

atributos

Esse elemento admite apenas os global attributes (atributos globais)

Notas sobre o uso

> **Nota:** O elemento `<hgroup>` foi removido da especificação do W3C para a HTML5, mas ainda se encontra na especificação HTML do WHATWG. Foi parcialmente implementado na maioria dos navegadores, contudo é pouco provável que seja incluído na especificação. Considerando que a finalidade do elemento `<hgroup>` é a de definir a maneira como os títulos serão mostrados pelo outline algorithm definido na especificação da HTML e considerando ainda que o **outline algorithm da HTML não está implementado em nenhum navegador**, conclui-se que, na prática, a semântica do elemento `<hgroup>` é meramente teórica. A especificação do W3C para a HTML5 fornece algumas sugestões para a marcação de Subheadings, subtitles, alternative titles and taglines sem que se use o elemento `<hgroup>`.

`O elemento <hgroup> permite que o cabeçalho principal de uma seção de documento seja agrupado com quaisquer cabeçalhos secundarios-como subtitulos ou titulos alternativos-para formatar um cabeçalho de varios niveis.`

Em outras palavras o elemento <hgroup> impede que qualquer um dos seus filhos secundarios <h1>-<h6> crie seçoes separadas propias no esboço-como esses elementos <h1>-<h6> normalmene seriam se não fossem filhos de nenhum <hgroup>

Então, no esboço abstrato produzida pelo algortimo abstrato do HTML definido na especificação HTML, o <hgroup> como um todo forma uma posição lógica simples, com todo o conjunto de filhos <h1>-<h6> do <hgroup> entrando no esquema como uma unidade multinível, para compreender esse cabeçalho lógico único no resumo abstrato.

Para produzir alguma (não-abstrato) visualização não renderizada como um esboço, alguma escolhe deve ser feita no design para renderizar ferramentas sobre como renderizar <hgroup> cabeçalhos como um jeito para transmitir sua natureza multi-nível. Há uma varidade de jeitos que <hgroup> pode ser mostrado em um esboço renderizado; por exemplo:

- um <hgroup> pode ser mostrado em um esboço renderizado com dois pontos e espaço (": ") ou outra pontuação após o cabeçalho principal e antes do primeiro cabeçalho secundário (e como a mesma pontuaçã, ou parecida, antes de qualquer

cabeçalho secundário).
- um <hgroup> pode ser mostrado em um esboço renderizado com o primeiro cabeçalho seguido de parênteses ao redor do segundo cabeçalho (s).

Considere o documento HTML a seguir:

```
<!doctype html>
<title>HTML Standard</title>
<body>
  <hgroup id="document-title">
    <h1>HTML</h1>
    <h2>Living Standard — Last Updated 12 August 2016</h2>
  </hgroup>
  <p>Some intro to the document.</p>
  <h2>Table of contents</h2>
  <ol id="toc">
    ...
  </ol>
  <h2>First section</h2>
  <p>Some intro to the first section.</p>
</body>
```

Um esboço renderizado para esse documento seria como:

Isso é, o esboço renderizado deve ser o título primário, HTML, seguido por dois pontos e espaço, seguido pelo título secundário, Living Standard - Last Update 12 August 2016.

Ou, o esboço renderizado para aquele documento poderia ser como:

 Rendered outline that includes an 'hgroup' element, with parens around the secondary heading

Isso é, o esboço renderizado deve mostrar o título primário, HTML, seguido pelo título secundário mostrado entre parênteses: (Living Standard - Last Updated 12 August 2016).

Exemplos

```
<hgroup id="document-title">
  <h1>HTML</h1>
  <h2>Living Standard — Last Updated 12 August 2016</h2>
</hgroup>
```

See also

- Others section-related elements: `<body>`, `<article>`, `<section>`, `<aside>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<nav>`, `<header>`, `<footer>`, `<address>`;
- Sections and outlines of an HTML5 document.

<hr>

Sumário

O **elemento HTML** `<hr>` representa uma quebra temática entre elementos de nível de parágrafo (por exemplo , uma mudança da cena de uma história, ou uma mudança de tema com uma seção). Nas versões anteriores do HTML, representava uma linha horizontal. Pode continuar sendo exibida como uma linha horizontal nos navegadores, mas agora está definida em termos semânticos, em vez de termos de apresentação.

- *Categorias de conteúdo* Fluxo de conteúdo.
- *Conteúdo permitido* Nenhum, é um empty element.
- *Omissão de Tag* Deve conter a tag de início, mas não precisa ter uma tag de encerramento.
- *Elementos pai permitidos* Qualquer elemento que aceite fluxo de conteúdo.
- *Interface DOM* `HTMLHRElement`

Atributos

Este elemento inclui os atributos globais.

`align`

Define a regra de alinhamento na página. Se nenhum valor for especificado, o valor padrão é `left`.

`color`

Define a regra da cor através do nome da cor ou um valor hexadecimal.

`noshade`

Define a regra para não ter sombreamento.

`size`

Define a altura, em pixels.

`width`

Define a regra de comprimento na página através de um valor em pixel ou porcentagem.

Exemplo

```
<p>
  Este é o primeiro parágrafo do texto. Este é o primeiro parágrafo do texto.
  Este é o primeiro parágrafo do texto. Este é o primeiro parágrafo do texto.
</p>

<hr />

<p>
  Este é o segundo parágrafo do texto. Este é o segundo parágrafo do texto. Este
  é o segundo parágrafo do texto. Este é o segundo parágrafo do texto.
</p>
```

O HTML acima irá produzir:

Este é o primeiro parágrafo do texto. Este é o primeiro parágrafo do texto. Este é o primeiro parágrafo do texto. Este é o primeiro parágrafo do texto.

Este é o segundo parágrafo do texto. Este é o segundo parágrafo do texto. Este é o segundo parágrafo do texto. Este é o segundo parágrafo do texto.

Notas

Para alterar a aparencia da linha ou as lacunas entre ela e os parágrafos, utilize folhas de estilo em cascata.

Veja também

- Elemento HTML parágrafo

<html>

Sumário

O elemento **HTML <html> **(ou *HTML root element*) representa a raiz de um HTML ou XHTML documento. Todos os outros elementos devem ser descendentes desse elemento.

- *Categorias de conteúdo* Nenhuma.
- *Conteúdo permitido* Um elemento `<head>`, seguido por um elemento `<body>`.
- *Omissão de tag* A tag inicial pode ser omitida se a primeira coisa dentro do elemento `<html>` não for um comentário. A tag final pode ser omitida se o elemento `<html>` não for imediatamente seguido por um comentário, e conter um elemento

`<body>` ou que não está vazio, ou cuja tag inicial está presente.
- *Elementos pais permitidos* Como o elemento raiz do documento, ou qualquer fragmento de subdocumento é permitido em um documento composto.
- *DOM interface* `HTMLhtmlElement`

Atributos

Esse elemento inclui os atributos globais.

`version`

Especifica a versão da definição de tipo de documento HTML que governa o documento atual. Esse atributo não é necessário, porque é redundante com a informação de versão na declaração de tipo de documento.

Exemplo

O DOCTYPE usado no exemplo seguinte indica HTML5.

```
<!doctype html>
<html>
  <head>
    ...
  </head>
  <body>
    ...
  </body>
</html>
```

Notas

Desde que o elemento `<html>` seja o primeiro em documento outro que comenta, esse é chamado o elemento raiz. Embora essa tag possa ser implícita, ou não requerida, com HTML, essa é requerida para ser aberta e fechada em XHTML.

Veja também

- Elemento de alto nível MathML: `<math>`
- Elemento de alto nível SVG: `<svg>`

<i>

O **elemento HTML `<i>` **representa uma parte do texto que é destacada do restante por algum motivo, por exemplo, termos técnicos, expressões de outros idiomas ou pensamentos de personagens fictícios. Normalmente, é apresentado com o uso do tipo "itálico".

Categorias de conteúdo	Conteúdo de fluxo (flow content), conteúdo com texto (phrasing content), conteúdo palpável (palpable content).
Conteúdo permitido	Conteúdo com texto (phrasing content).
Omissão de tag	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Pais permitidos	Qualquer elemento que aceite conteúdo com texto (phrasing content).
ARIA roles permitidos	Qualquer um.
Interface DOM	`HTMLElement` Up to Gecko 1.9.2 (Firefox 4) inclusive, Firefox implements the `HTMLSpanElement` interface for this element.

Atributos

Esse elemento inclui apenas atributos globais.

Exemplo

```
<p>
  A expressão em latim <i class="latin">Veni, vidi, vici</i> é frequentemente
  mencionada na música, na arte e na literatura.
</p>
```

Resultado

A expressão em latim *Veni, vidi, vici* é frequentemente mencionada na música, na arte e na literatura.

Notas

Nas primeiras versões das especificações HTML, a tag `<i>` era usada apenas para apresentar textos em itálico, assim como a tag `<b>` tag era usada para mostrar textos em negrito. Isso não é mais verdade, já que, atualmente, essas tags definem a semântica, e não mais aparência tipográfica. A tag `<i>` pode representar uma parte do texto com uma diferença semântica, na qual a representação tipográfica padrão é no tipo itálico. Isso significa que os navegadores continuarão a apresentar o conteúdo em itálico, mas, de acordo com o que é definido, isso não é mais requerido.

Use esse elemento somente quando não existir mais nenhum outro elemento com semântica apropriada. Por exemplo:

- Use `<em>` para indicar ênfase ou estresse.
- Use `<strong>` para indicar importância.
- Use `<mark>` para indicar relevância.
- Use `<cite>` para citar o nome de um trabalho, tal como um livro, jogo ou música.
- Use `<dfn>` para marcar a instância de definição de um termo .

Uma boa ideia é usar o atributo **class** para identificar porque esse elemento está sendo usado. Com isso, caso a apresentação precise ser alterada com o tempo, isso pode ser feito de forma seletiva utilizando as folhas de estilo.

Ver também

- `<em>`

<iframe>

O **elemento HTML <iframe>** (ou *elemento HTML inline frame*) representa um contexto de navegação aninhado, efetivamente incorporando outra página HTML para a página atual. Em HTML 4.01, um documento pode conter uma cabeça e um corpo ou uma cabeça e um conjunto de quadros, mas não tanto um corpo e um conjunto de quadros. No entanto, um <iframe> pode ser usado dentro de um corpo de documento normal. Cada contexto de navegação tem sua própria história de sessão e o documento ativo. O contexto de navegação que contém o conteúdo incorporado é chamado o pai de contexto de navegação. O contexto de navegação de nível superior (que não tem um pai) normalmente é a janela do navegador.

- *Content categories* Flow content, phrasing content, embedded content, interactive content, palpable content.
- *Permitted content* Special, see prose
- *Tag omission* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Permitted parent elements* Any element that accepts embedded content.
- *DOM interface* `HTMLIFrameElement`

Atributos

Este elemento inclui os atributos globais.

`align`

> O alinhamento deste elemento em relação ao contexto.

`allowfullscreen`

> Esse atributo pode ser definido como true se o quadro é permitido para ser colocado em modo de tela cheia, chamando seu método de `element.mozRequestFullScreen()`. Se isto não estiver definido, o elemento não pode ser colocado em modo de tela cheia.

`frameborder`

O valor 1 (o padrão) pede ao navegador para desenhar uma borda entre este quadro e todos os outros quadros. O valor 0 informa ao navegador para não traçar uma fronteira entre este quadro e outros quadros.

`height`

Indica a altura do quadro HTML 5 em pixels CSS, ou HTML 4 em pixels ou porcentagem.

`longdesc`

Uma URI de uma longa descrição do quadro. Devido à má utilização generalizada, isto não é útil para navegadores não-visuais.

`marginheight`

A quantidade de espaço em pixels entre o conteúdo do quadro e suas margens superior e inferior.

`marginwidth`

A quantidade de espaço em pixels entre o conteúdo dos quadros e suas margens direita e esquerda.

`mozallowfullscreen`

Use **`allowfullscreen`** ao invés. Em Gecko 9.0 or later, this attribute can be set to `true` if the frame is allowed to be placed into full screen mode by calling its `element.mozRequestFullScreen()` method. If this isn't set, the element can't be placed into full screen mode.

`webkitallowfullscreen`

Use **`allowfullscreen`** ao invés. No Chrome 17 ou mais novo (e talvez mais antigo), this attribute can be set to `true` if the frame is allowed to be placed into full screen mode by calling its `element.webkitRequestFullScreen()` method. If this isn't set, the element can't be placed into full screen mode.

`mozapp`

For frames hosting an open web app, this specifies the URL of the app manifest. This ensures that the app is loaded with the right permissions. See Using the Browser API for details. Available in Gecko 13.0 and later.

`mozbrowser`

Indicates that the frame is to appear like a top-level browser window to the embedded content. This means that `window.top`, `window.parent`, `window.frameElement`, etc. will *not* reflect the frame hierarchy. This allows for a web browser UI to be implemented entirely with web technology, given the right permissions. See Using the Browser API for details. Available in Gecko 13.0 and later.

`name`

A name for the embedded browsing context (or frame). This can be used as the value of the **`target`** attribute of an `<a>` or `<form>` element, or the formtarget attribute of an `<input>` or `<button>` element.

`remote`

Carrega a página do *frame* em um processo separado.

`scrolling`

Enumerated attribute indicating when the browser should provide a scroll bar (or other scrolling device) for the frame:

- `auto`: Só quando necessário.
- `yes`: Sempre mostrar uma barra de rolagem.
- `no`: Nunca mostrar uma barra de rolagem.

`sandbox`

If specified as an empty string, this attribute enables extra restrictions on the content that can appear in the inline frame. The value of the attribute can either be an empty string (all the restrictions are applied), or a space-separated list of tokens that lift particular restrictions. Valid tokens are:

- `allow-same-origin`: Allows the content to be treated as being from its normal origin. If this keyword is not used, the embedded content is treated as being from a unique origin.
- `allow-top-navigation`: Allows the embedded browsing context to navigate (load) content to the top-level browsing context. If this keyword is not used, this operation is not allowed.
- `allow-forms`: Allows the embedded browsing context to submit forms. If this keyword is not used, this operation is not allowed.
- `allow-popups`: Allows popups (like from window.open).
- `allow-scripts`: Allows the embedded browsing context to run scripts (but not create pop-up windows). If this keyword is not used, this operation is not allowed.
- `allow-pointer-lock`: Allows the embedded browsing context to use the Pointer Lock API.

> **Nota:**
>
> - When the embedded document has the same origin as the main page, it is strongly discouraged to use both `allow-scripts` and `allow-same-origin` at the same time, as that allows the embedded document to programmatically remove the `sandbox` attribute. Although it is accepted, this case is no more secure than not using the `sandbox` attribute.
> - Sandboxing in general is only of minimal help if the attacker can arrange for the potentially hostile content to be displayed in the user's browser outside a sandboxed `iframe`. It is recommended that such content should be served from a *separate dedicated domain*, to limit the potential damage.
> - The `sandbox` attribute is not supported in Internet Explorer 9 and earlier versions, or in Opera.

`seamless`

This **Boolean attribute** indicates that the browser should render the inline frame in a way that makes it appear to be part of the containing document, for example by applying CSS styles that apply to the `<iframe>` to the contained document before styles specified in that document, and by opening links in the contained documents in the parent browsing context (unless another setting prevents this). In XHTML, attribute minimization is forbidden, and the seamless attribute must be defined as `<iframe seamless="seamless">`.

`src`

The URL of the page to embed.

`srcdoc`

The content of the page that the embedded context is to contain. This attribute is expected to be used together with the sandbox and seamless attributes. If a browser supports the `srcdoc` attribute, it will override the content specified in the `src` attribute (if present). If a browser does NOT support the `srcdoc` attribute, it will show the file specified in the `src` attribute instead (if present).

`width`

Indicates the width of the frame HTML 5 in CSS pixels, or HTML 4 in pixels or as a percentage.

Scripting

Inline frames, like `<frame>` elements, enter the `window.frames` pseudo-array.

From the DOM `iframe` element, scripts can get access to the `window` object of the included HTML page via the `contentWindow` property. The `contentDocument` property refers to the document element inside the `iframe` (this is equivalent to `contentWindow.document`), but is not supported by Internet Explorer versions before IE8.

From the inside of a frame, a script can get a reference to the parent window via `window.parent`.

Scripts trying to access a frame's content are subject to the same-origin policy, and cannot access most of the properties in the other window object if it was loaded from a different domain. This also applies to a script inside a frame trying to access its parent window. Cross-domain communication can still be achieved with `window.postMessage`.

Exemplo 1

```
<script>
  var iframe = window.getElementsByTagName("iframe")[0];
  alert("Frame title: " + iframe.contentWindow.title);
</script>

<iframe src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/iframe/page.html" width="300"
height="300">
  <p>Your browser does not support iframes.</p>
</iframe>
```

Exemplo 2: Open link inside iframe in another tab

```
<base target="_blank" />
<iframe
  width="400"
  height="215"
  frameborder="0"
  scrolling="no"
  marginheight="0"
  marginwidth="0"
  src="https://maps.google.com/maps?f=q&amp;source=s_q&amp;hl=es-
419&amp;geocode=&amp;q=buenos+aires&amp;sll=37.0625,-95.677068&amp;sspn=38.638819,80.859375&amp;t=h&amp;ie=UTF8&amp;hq
=&amp;hnear=Buenos+Aires,+Argentina&amp;z=11&amp;ll=-34.603723,-58.381593&amp;output=embed">
</iframe
><br />
<small>
  <a
    href="https://maps.google.com/maps?f=q&amp;source=embed&amp;hl=es-
419&amp;geocode=&amp;q=buenos+aires&amp;sll=37.0625,-95.677068&amp;sspn=38.638819,80.859375&amp;t=h&amp;ie=UTF8&amp;hq
=&amp;hnear=Buenos+Aires,+Argentina&amp;z=11&amp;ll=-34.603723,-58.381593"
    style="color:#0000FF;text-align:left">
    See bigger map
  </a>
</small>
```

Resultado

Live example

Notas

> **Nota:** Starting in Gecko 6.0, rendering of inline frames correctly respects the borders of their containing element when they're rounded using `border-radius`.

Resumo

O **elemento** **HTML `<img>` **(or *HTML Image Element*) representa a inserção de imagem no documento, sendo implementado também pelo HTML5 para uma melhor experiência com o elemento `<figure>` e `<figcaption>`.

Exemplo da implementação do HTML5

```
<figure>
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/img/imagem.jpg" alt="Minha Figura">
  <figcaption>Informações da Figura</figcaption>
</figure>
```

> **Note: Usage note:** Navegadores nem sempre exibem a imagem referenciada pelo elemento. Este é o caso para navegadores não gráficos (incluindo aqueles usados por pessoas com deficiência de visão), ou se o usuário optar por não exibir imagens

> ou se o navegador não conseguir exibir a imagem porque é inválido ou um tipo não suportado. Nesses casos, o navegador pode substituir a imagem pelo texto definido no atributo **alt** deste elemento.

- *Content categories* Flow content, phrasing content, embedded content, palpable content. If the element has a `usemap` attribute, it also is a part of the interactive content category.
- *Permitted content* None, it is an empty element.
- *Tag omission* Must have a start tag and must not have an end tag.
- *Permitted parent elements* Any element that accepts embedded content.
- *DOM interface* `HTMLImageElement`

Atributos

Este Elemeto inlcui o atributo global.

`align`

The alignment of the image with respect to its surrounding context. Use the `vertical-align` CSS property.

`alt`

Este atributo define um texto alternativo que descreve a imagem. Os Usuários irão ver o texto se a URL da imagem estiver errado, a imagem não está em um dos formatos suportados ou até a imagem ser baixada .

> **Note: Usage note:** Omitting this attribute indicates that the image *is* a key part of the content, but no textual equivalent is available. Setting this attribute to the empty string indicates that this image is *not* a key part of the content; non-visual browsers may omit it from rendering.

`border`

The width of a border around the image.

`crossorigin`

This enumerated attribute indicates if the fetching of the related image must be done using CORS or not. CORS-enabled images can be reused in the `<canvas>` element without being *tainted*. The allowed values are:

anonymous

A cross-origin request (i.e. with `Origin:` HTTP header) is performed. But no credential is sent (i.e. no cookie, no X.509 certificate and no HTTP Basic authentication is sent). If the server does not give credentials to the origin site (by not setting the `Access-Control-Allow-Origin:` HTTP header), the image will be *tainted* and its usage restricted..

use-credentials

A cross-origin request (i.e. with `Origin:` HTTP header) performed with credential is sent (i.e. a cookie, a certificate and HTTP Basic authentication is performed). If the server does not give credentials to the origin site (through `Access-Control-Allow-Credentials:` HTTP header), the image will be *tainted* and its usage restricted.

When not present, the resource is fetched without a CORS request (i.e. without sending the `Origin:` HTTP header), preventing its non-tainted usage in `<canvas>` elements. If invalid, it is handled as if the enumerated keyword **anonymous** was used. See CORS settings attributes for additional information.

`height`

The height of the image in HTML 5 CSS pixels, or HTML 4 in pixels or as a percentage.

`hspace`

The number of pixels of white space to insert to the left and right of the image.

`ismap`

This Boolean attribute indicates that the image is part of a server-side map. If so, the precise coordinates of a click are sent to the server.

> **Note: Usage note:** This attribute is allowed only if the `<img>` element is a descendant of an `<a>` element with a valid `href` attribute.

`longdesc`

A link to a more detailed description of the image. Possible values are a URL or an element `id`.

`name`

A name for the element. Use the `id` attribute instead.

`src`

Image URL, this attribute is obligatory for the `<img>` element. On browsers supporting **srcset**, **src** is ignored if this one is provided.

`srcset`

A list of one or more strings separated by commas indicating a set of possible images for the user agent to use. Each string is composed of:

1. one URL to an image,
2. a width descriptor, that is a positive integer directly followed by `'w'`. The default value, if missing, is the infinity.
3. a pixel density descriptor, that is a positive floating number directly followed by `'x'`. The default value, if missing, is `1x`.

Each string in the list must have at least a width descriptor or a pixel density descriptor to be valid. Among the list, there must be only one string containing the same tuple of width descriptor and pixel density descriptor. The browser chooses the most adequate image to display at a given point of time.

`width`

The width of the image in pixels or percent.

`usemap`

The partial URL (starting with '#') of an image map associated with the element.

> **Note: Usage note:** You cannot use this attribute if the `<img>` element is a descendant of an `<a>` or `<button>` element.

`vspace`

The number of pixels of white space to insert above and below the image.

Formato de imagens suportadas

O padrão HTML não fornece uma lista de formatos de imagem que devem ser suportados, portanto, cada agente de usuário oferece suporte a um conjunto diferente de formatos. Gecko suporta:

- JPEG
- GIF, incluindo GIFs animados
- PNG
- APNG
- SVG
- BMP
- BMP ICO
- PNG ICO

Interação com CSS

Em relação ao CSS, uma `<img>` é um elemento substituído. Ele não tem linha de base, ou seja, quando usado em um contexto de formatação em linha (inline) com `vertical-align`: baseline, a base inferior da imagem será colocada na linha de base do contêiner.

Depending of its type, an *image* may have an intrinsic dimension, but this is not a necessary condition: a SVG image has no intrinsic dimension, a raster image has one.

Exemplos

Usando o texto alternativo

O exemplo a seguir insere uma imagem na página e inclui o texto alternativo para acessibilidade, de forma que ele possa ser lido por programas leitores de tela ou exibido caso a imagem não carregue.

```
<img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/img//shared-
assets/images/examples/favicon144.png" alt="MDN logo" />
```

Link em imagem

Esse exemplo mostra como transformar uma imagem em um link. Para isso, insira a tag `<img>` dentro da tag do link `<a>`. Nesse caso, é interessante fazer o texto alternativo descrever o site para o qual o link aponta, como se fosse o texto usado dentro da tag `<a>`.

```
<a href="https://developer.mozilla.org">
  <img
    src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/img//shared-
assets/images/examples/favicon144.png"
    alt="Visit the MDN site" />
</a>
```

Usando o atributo <code>srcset</code>

Nesse exemplo, o atributo `srcset` inclui uma versão do logotipo com maior resolução, a qual vai ser carregada no lugar de `src` em aparelhos de alta resolução e cujo navegador tenha suporte à `srcset`.

A imagem presente no atributo `src` conta como um candidato `1x` em navegadores com suporte à `srcset`.

As imagens exibidas dependerão do tipo de tela.

`srcset` define o conjunto de imagens que nós iremos permitir ao navegador escolher, e qual tamanho cada imagem tem.

```
<img
  src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/img/mdn-logo-sm.png"
  alt="MD Logo"
  srcset="mdn-logo-HD.png 2x, mdn-logo-small.png 15w, mdn-banner-HD.png 100w 2x" />
```

Antes de cada vírgula nós escrevemos:

1. Um **nome do arquivo da imagem** (`mdn-logo-HD.png`).
2. Um espaço.
3. A **largura da imagem** ou **sua densidade de pixels**.

Acessibilidade

Utilizando textos alternativos relevantes

O valor do atributo `alt` deve descrever de maneira clara e concisa o conteúdo da imagem. Evite descrever a própria presença da imagem ou o nome de seu arquivo. Se o atributo `alt` for propositalmente deixado vazio porque a imagem não possui nenhum equivalente em texto, considere métodos alternativos para indicar o que a imagem deseja comunicar.

Evite

```
<img alt="imagem" src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/img/penguin.jpg" />
```

Prefira

```
<img alt="Um pinguim-saltador-da-rocha em pé numa praia." src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/img/penguin.jpg" />
```

Quando o atributo `alt` não estiver presente em uma imagem, alguns programas leitores de tela vão narrar o nome do arquivo da imagem em seu lugar, o que pode ser confuso caso o nome do arquivo não seja representativo do conteúdo da imagem.

- An alt Decision Tree • Images • WAI Web Accessibility Tutorials
- Alt-texts: The Ultimate Guide — Axess Lab
- How to Design Great Alt Text: An Introduction | Deque
- MDN Understanding WCAG, Guideline 1.1 explanations
- Understanding Success Criterion 1.1.1 | W3C Understanding WCAG 2.0

O atributo title

O atributo `title` não é um substituto aceitável para o atributo `alt`. Além disso, evite duplicar o valor do atributo `alt` no atributo `title` para uma mesma imagem. Isso pode fazer com que alguns programas leitores de tela narrem duas vezes a descrição, o que pode criar uma experiência confusa para usuários.

Evite usar o atributo `title` como uma forma suplementar de legenda para a descrição do `alt`. Caso a imagem precise de uma legenda, prefisa os elementos `figure` e `figcaption`.

O valor do atributo `title` é geralmente mostrado ao usuário como uma dica, que aparece após o usuário parar o cursor sobre a imagem. Apesar de *poder prover* informações adicionais ao usuário, não se deve assumir todos os usuários vão vê-lo, pois o mesmo pode possuir apenas um teclado ou uma tela de toque (touchscreen). Se você considera a informação particularmente importante para o usuário, prefira o uso de elementos inline.

- Using the HTML title attribute – updated | The Paciello Group

Veja também

- `<object>` and `<embed>` elements

<input>

O **elemento HTML `<input>`** é usado para criar controles interativos para formulários baseados na web para receber dados do usuário. A semântica de um `<input>` varia consideravelmente dependendo do valor de seu atributo `type`.

- *Categorias de conteúdo* Conteúdo de fluxo, listado, enviável, resetável, elemento associado a formulário, conteúdo fraseado. Se o atributo `type` não tiver o valor `hidden`, elemento rotulável, conteúdo palpável.
- *Conteúdo permitido* Nenhum, este é um elemento vazio.
- *Omissão de tags* Deve ter uma tag de abertura e não deve ter uma tag de fechamento.
- *Elementos pais permitidos* Qualquer elemento que aceite conteúdo fraseado.
- *Interface do DOM* `HTMLInputElement`

Atributos

Este elemento inclui os atributos globais.

`type`

O tipo de controle a ser exibido. O tipo padrão é **text**, se este atributo não for especificado. Os valores possíveis são:

- `button`: Um botão sem comportamento padrão.
- `checkbox`: Uma caixa de marcação. Você deve usar o atributo **value** para definir o valor enviado por este item. Use o atributo **checked** para indicar se o item está selecionado por padrão. Você também pode usar o atributo **indeterminate** para indicar que a caixa de marcação está em um estado indeterminado (na maioria das plataformas, isso desenha uma linha horizontal cortando a caixa).

- `color`: Um controle para especificar cores. A interface de um seletor de cores não tem nenhuma funcionalidade obrigatória a não ser aceitar cores simples em texto (mais informações — em inglês).
- `date`: Um controle para inserir uma data (ano, mês e dia, sem horário).
- `datetime`: Um controle para inserir data e horário (hora, minuto, segundo e fração de segundo) baseado no fuso horário UTC.
- `datetime-local`: Um controle para inserir data e horário, sem fuso horário.
- `email`: Um campo para editar um endereço de e-mail. O valor do campo é validado para estar vazio ou ter um único endereço de e-mail válido antes de ser enviado. As pseudoclasses CSS `:valid` e `:invalid` são aplicadas apropriadamente.
- `file`: Um controle que permite ao usuário selecionar um arquivo. Use o atributo**accept** para definir os tipos de arquivo que o controle pode selecionar.
- `hidden`: Um controle que não é exibido mas cujo valor é enviado ao servidor.
- `image`: Um botão gráfico para enviar o formulário. Você deve usar o atributo **src** para definir a fonte da imagem e o atributo **alt** para definir um texto alternativo. Você pode usar os atributos **height** e **width** para definir o tamanho da imagem em pixels.
- `month`: Um controle para inserir mês e ano, sem fuso horário.
- `number`: Um controle para inserir um número de ponto flutuante.
- `password`: Um campo de texto com uma só linha cujo valor é obscurecido. Use o atributo **maxlength** para especificar o comprimento máximo do valor que pode ser inserido.
- `radio`: Um botão de escolha. Você deve usar o atributo **value** para definir o valor a ser enviado por este item. Use o atributo **checked** para indicar se este item deve estar selecionado por padrão. Botões de escolha que têm o mesmo valor para o atributo **name** estão no mesmo "grupo de botões de escolha"; apenas um botão de escolha no grupo pode estar selecionado de cada vez..
- `range`: Um controle para inserir um número cujo valor exato não é importante. Este tipo de controle usa os seguintes valores padrão se os atributos correspondentes não forem especificados:
  - `min`: 0
  - `max`: 100
  - `value`: `min` + (`max`-`min`)/2, ou `min` se `max` for menos que `min`
  - `step`: 1
- `reset`: Um botão que faz o conteúdo do formulário voltar a ter seus valores padrão.
- `search`: Um campo de texto com uma só linha para digitar termos de busca; quebras de linha são automaticamente removidas do valor entrado.
- `submit`: Um botão que envia o formulário.
- `tel`: Um controle para inserir um número de telefone; quebras de linha são automaticamente removidas do valor entrado, mas nenhuma outra sintaxe é imposta. Você pode usar atributos como **pattern** e **maxlength** para restringir os valores inseridos no controle. As pseudoclasses CSS `:valid` e `:invalid` são aplicadas apropriadamente.
- `text`: Um campo de texto com uma só linha; quebras de linha são automaticamente removidas do valor entrado.
- `time`: Um controle para inserir um horário sem fuso horário.
- `url`: Um campo para editar uma URL. O valor inserido é validado para ser vazio ou uma URL absoluta válida antes de ser enviado. Quebras de linha e espaços em branco antes e após o valor inserido são automaticamente removidos. Você pode usar atributos como **pattern** e **maxlength** para restringir os valores inseridos no controle. As pseudoclasses CSS `:valid` e `:invalid` são aplicadas apropriadamente.
- `week`: Um controle para inserir uma data consistindo de ano da semana e número da semana sem fuso horário.

`accept`

Se o valor do atributo **type** for `file`, este atributo indica quais tipos de arquivo o servidor aceita; caso contrário, este atributo é ignorado. O valor deve ser uma lista de especificadores de tipo únicos separados por vírgula:

- Uma extensão de arquivo começando com o caractere STOP (U+002E). (Exemplo: ".jpg,.png,.doc")
- Um tipo MIME válido sem extensões
- `audio/*` representando arquivos de som
- `video/*` representando arquivos de vídeo
- `image/*` representando arquivos de imagem

`mozactionhint`

Especifica uma "dica de ação" usada para determinar como rotular a tecla Enter em dispositivos móveis com teclados virtuais. Os valores suportados são `go`, `done`, `next`, `search` e `send`; estes valores são automaticamente mapeados para o texto apropriado (maiúsculas e minúsculas não são diferenciadas).

`autocapitalize`

Este é um atributo não padronizado usado pelo iOS Safari Mobile que controla se e como o texto deveria ser automaticamente capitalizado conforme o usuário insere/edita. Os valores não obsoletos estão disponíveis no iOS 5 e posteriores. Os valores possíveis são:

- `none`: Desabilitar completamente a capitalização automática.
- `sentences`: Capitalizar automaticamente a primeira letra das frases.
- `words`: Capitalizar automaticamente a primeira letra das palavras.
- `characters`: Capitalizar automaticamente todas as letras.
- `on`: Obsoleto desde o iOS 5.
- `off`: Obsoleto desde o iOS 5.

`autocomplete`

Este atributo indica se o valor do controle pode ser completado automaticamente pelo navegador. Este atributo é ignorado se o valor do atributo **type** é `hidden`, `password`, `checkbox`, `radio`, `file` ou um tipo de botão (`button`, `submit`, `reset`, `image`). Os valores possíveis são:

- `off`: O usuário deve inserir explicitamente um valor neste campo a cada uso, ou o documento implementa seu próprio método de autocompletar; o navegador não completa a entrada automaticamente.
- `on`: O navegador pode completar automaticamente o valor baseado em valores que o usuário inseriu no passado.

Se o atributo **autocomplete** não está especificado num elemento `<input>`, o navegador usa o valor do atributo **autocomplete** do formulário dono do `<input>`. O formulário dono deste `<input>` é o elemento `<form>` que contém este `<input>` ou o elemento `<form>` cujo **id** é referenciado pelo atributo **form** do elemento `<input>`. Para mais informações, veja o atributo `autocomplete` no elemento `<form>`.

O atributo **autocomplete** também controla se o Firefox vai, ao contrário de outros navegadores, persistir o estado dinâmico de desabilitado e marcado (se aplicável) de um `<input>` entre carregamentos de página. A funcionalidade de persistir é habilitada por padrão. Definir o valor do atributo **autocomplete** para `off` desabilita esta funcionalidade; isto funciona mesmo quando o atributo **autocomplete** não se aplica ao `<input>` por causa de seu **type**. Veja Erro do Firefox 654072.

`autocorrect`

Este é um atributo não padronizado suportado pelo Safari que é usado para controle se a correção automática deveria ser habilitada quando o usuário entra/edita o texto do `<input>`. Os valores possíveis são:

- `on`: Habilita correção automática
- `off`: Desabilita correção automática

`autofocus`

Este atributo booleano permite que você especifique que um controle de formulário deve ter o foco de entrada de dados assim que a página carrega, a não ser que o usuário sobrescreva este comportamento digitando em um controle diferente, por exemplo. Apenas um elemento de formulário em um documento pode ter o atributo **autofocus**, que é booleano. Este

atributo não pode ser aplicado se o atributo **type** estiver definido como `hidden` (isto é, você não pode colocar automaticamente o foco em um controle escondido).

`autosave`

Este atributo deve ser definido com um valor único. Se o valor do atributo **type** for `search`, termos de busca anteriores serão persistidos no menu entre carregamentos de página.

`checked`

Quando o valor do atributo **type** é `radio` ou `checkbox`, a presença deste atributo booleano indica que o controle é selecionado por padrão; caso contrário, este atributo é ignorado.

O Firefox vai, por padrão, ao contrário de outros navegadores, persistir o estado dinâmico de marcado de um `<input>` entre carregamentos de página. Use o atributo `autocomplete` para controlar esta funcionalidade.

`disabled`

Este atributo booleano indica que o controle de formulário não está disponível para interação. Em particular o evento `click` não será disparado em controles desabilitados. Além disso, o valor de um controle desabilitado não é enviado com o formulário.

O Firefox vai, por padrão, ao contrário de outros navegadores, persistir o estado dinâmico de desabilitado de um `<input>` entre carregamentos de página. Use o atributo `autocomplete` para controlar esta funcionalidade.

`form`

O elemento `<form>` ao qual o elemento `<input>` está associado (seu *formulário dono*). O valor do atributo deve ser um **id** de um elemento `<form>`. Este atributo permite que você coloque elementos `<input>` em qualquer lugar num documento, não apenas como descendentes de seus elementos `<form>`. Um `<input>` só pode estar associado a um único formulário.

`formaction`

A URI de um programa que processa a informação enviada pelo elemento, se ele for um botão ou uma imagem de envio de formulário. Se especificado, ele sobrescreve o atributo `action` do formulário dono do elemento.

`formenctype`

Se o elemento for um botão ou uma imagem de envio de formulário, este atributo especifica o tipo de conteúdo que deve ser usado para enviar o formulário para o servidor. Os valores possíveis são:

- `application/x-www-form-urlencoded`: O valor padrão se o atributo não for especificado.
- `multipart/form-data`: Use este valor se você estiver usando um elemento `<input>` com o atributo `type` definido como `file`.
- `text/plain`

Se este atributo for especificado, ele sobrescreve o atributo `enctype` do formulário dono do elemento.

`formmethod`

Se o elemento for um botão ou uma imagem de envio de formulário, este atributo especifica o método HTTP que o navegador deve usar para enviar o formulário. Os valores possíveis são:

- `post`: Os dados do formulário são incluídos no corpo do formulário e enviados para o servidor.
- `get`: Os dados do formulário são acrescentados ao URI do atributo **form**, com um '?' como separador, e o URI resultante é enviado ao servidor. Use este método quando o formulário não tem efeitos colaterais e contém somente caracteres ASCII.

Se especificado, este atributo sobrescreve o atributo `method` do formulário dono do elemento.

`formnovalidate`

Se o elemento for um botão ou uma imagem de envio de formulário, este atributo booleano especifica que o formulário não deve ser validado quando enviado. Se especificado, este atributo sobrescreve o atributo `novalidate` do formulário

dono do elemento.

`formtarget`

Se o elemento for um botão ou uma imagem de envio de formulário, este atributo é um nome ou palavra-chave indicando onde mostrar a resposta recebida após a submissão do formulário. Este é o nome de, ou palavra-chave para, um *contexto de navegação* (por exemplo aba, janela ou frame inline). Se especificado, este atributo sobrescreve o atributo `target` do formulário dono do elemento. As seguintes palavras-chave têm significado especial:

- `_self`: Carrega a resposta nom mesmo contexto de navegação que o atual. Este é o valor padrão se o atributo não for especificado.
- `_blank`: Carrega a resposta num novo contexto de navegação anônimo.
- `_parent`: Carrega a resposta no contexto de navegação pai do atual. Se não houver pai, esta opção tem o mesmo comportamento de `_self`.
- `_top`: Carrega a resposta no contexto de navegação raiz (isto é, o contexto de navegação que é ancestral do atual e não tem pai). Se não houver pai, esta opção tem o mesmo comportamento de `_self`.

`height`

Se o valor do atributo **type** for `image`, este atributo define a altura da imagem mostrada para o botão.

`incremental`

Este é um atributo não padronizado suportado pelo Safari que é aplicado somente quando o **type** é `search`. Se o atributo estiver presente, independentemente do valor que ele tiver, o `<input>` dispara eventos `search` conforme o usuário edita o texto do campo. Este evento somente é disparado um tempo após a última tecla ter sido pressionada; este tempo é definido pela implementação e novas digitações zeram o contador de tempo. Em outras palavras, o disparo do evento é filtrado (*debounced*). Se o atributo não estiver presente, o evento `search` só é disparado quando o usuário iniciar explicitamente uma busca (por exemplo, pressionando a tecla Enter enquanto estiver no campo).

`inputmode`

Uma dica para o navegador de qual teclado deve ser mostrado. Este atributo se aplica quando o valor do atributo **type** é `text`, `password`, `email` ou `url`. Os valores possíveis são:

- `verbatim`: Conteúdo alfanumérico que não seja prosa, como nomes de usuário e senhas.
- `latin`: Conteúdo no alfabeto latino na língua preferida do usuário, com auxílios de entrada como predição de texto habilitados. Para interações humano-computador como caixas de busca.
- `latin-name`: Como *latin*, mas para nomes humanos.
- `latin-prose`: Como *latin*, mas com auxílios de entrada mais agressivos. Para comunicações entre humanos como mensagens instantâneas ou email.
- `full-width-latin`: Como *latin-prose*, mas para as línguas secundárias do usuário.
- `kana`: Entrada em kana ou romaji, tipicamente hiragana, usando caracteres de largura total, com suporte à conversão para kanji. Para entrada de texto em língua japonesa.
- `katakana`: Entrada em katakana, usando caracteres de largura total, com suporte à conversão para kanji. Para entrada de texto em língua japonesa.
- `numeric`: Entrada numérica, incluindo teclas para os dígitos de 0 a 9, o caractere separador de milhar preferido pelo usuário e o caractere para indicar números negativos. Para códigos numéricos como números de cartão de crédito. Para números de verdade, prefira usar `<input type="number">`
- `tel`: Entrada de números de telefone, incluindo as teclas asterisco e cerquilha. Use `<input type="tel">` se possível em vez disso.
- `email`: Entrada de email. Use `<input type="email">` se possível em vez disso.
- `url`: Entrada de URLs. Use `<input type="url">` se possível em vez disso.

`list`

Identifica uma lista de opções pré-definidas de sugestões para o usuário. O valor deve ser o **id** de um elemento `<datalist>` no mesmo documento. O navegador mostrará apenas opções que são valores válidos para este elemento. Este atributo é ignorado quando o valor do atributo **type** é `hidden`, `checkbox`, `radio`, `file` ou um tipo de botão.

`max`

O valor máximo (numérico ou data) para este item, que não deve ser menor que seu valor mínimo (atributo **min**).

`maxlength`

Se o valor do atributo **type** for `text`, `email`, `search`, `password`, `tel` ou `url`, este atributo especifica o número máximo de caracteres (em pontos de código Unicode) que o usuário pode inserir; para outros tipos de controle, este atributo é ignorado. Seu valor pode exceder o do atributo **size**. Se não for especificado, o usuário pode inserir uma quantidade ilimitada de caracteres. Especificar um valor negativo resulta no comportamento padrão, isto é, o usuário vai poder inserir um número ilimitado de caracteres. Esta regra só é validada quando o valor do atributo é modificado.

`min`

O valor mínimo (númerico ou data) para este item, que não deve ser maior do que seu máximo (atributo **max**).

`minlength`

Se o valor do atributo **type** for `text`, `email`, `search`, `password`, `tel` ou `url`, este atributo especifica o número mínimo de caracteres (em pontos de código Unicode) que o usuário pode inserir; para outros tipos de controle, este atributo é ignorado.

`multiple`

Este atributo booleano indica se o usuário pode inserir mais de um valor. Ele é aplicado quando o atributo **type** está definido como `email` ou `file`; em outros casos, é ignorado.

`name`

O nome do controle, que é enviado junto com os dados do formulário.

`pattern`

Uma expressão regular usada para validar o valor do controle. O padrão deve casar com o valor completo da entrada, não apenas uma parte. Use o atributo **title** para descrever o padrão para ajudar o usuário. Este atributo é aplicado quando o valor do atributo **type** é `text`, `search`, `tel`, `url` ou `email`; caso contrário ele é ignorado. A linguagem da expressão regular é a mesma do JavaScript. O padrão não deve estar entre barras.

`placeholder`

Uma dica para o usuário do que ele pode inserir no controle. O texto do atributo não deve conter quebras de linha. Este atributo é aplicado quando o valor do atributo **type** é `text`, `search`, `tel`, `url` ou `email`; caso contrário, ele é ignorado.

> **Nota:** Não use o atributo `placeholder` no lugar de um elemento `<label>`. Os propósitos de cada um são diferentes: o atributo `<label>` descreve o papel o elemento do formulário, isto é, ele indica que tipo de informação é esperada. Já o atributo `placeholder` é uma dica sobre o formato que o conteúdo deveria ter. Há casos em que o atributo `placeholder` nunca é exibido para o usuário, portanto o formulário deve ser inteligível sem ele.

`readonly`

Este atributo booleano indica que o usuário não pode modificar o valor do controle.

Este atributo é ignorado se o valor do atributo **type** for `hidden`, `range`, `color`, `checkbox`, `radio`, `file` ou um tipo de botão.

`required`

Este atributo especifica que o usuário deve preencher o campo com um valor antes de enviar o formulário. Ele não pode ser usado quando o atributo **type** está definido como `hidden`, `image` ou um tipo de botão (`submit`, `reset` ou `button`). As pseudoclasses CSS `:optional` e `:required` serão aplicadas ao campo como for apropriado.

`selectionDirection`

A direção na qual a seleção ocorreu. Vale `forward` se a seleção foi feita no mesmo sentido da escrita do idioma usado ou `backward` se a seleção foi feita na direção oposta. Pode ainda ser `none` se a direção da seleção não for conhecida.

`size`

O tamanho inicial do controle. Este valor é em pixels a não ser que o valor do atributo **type** seja `text` ou `password`. Neste caso, este valor é um número inteiro de caracteres. A partir do HTML5, este atributo é aplicado apenas quando o atributo **type** está definido como `text`, `search`, `tel`, `url`, `email` ou `password`; caso contrário, ele é ignorado. Além disso, o tamanho deve ser maior que zero. Se você não especificar um tamanho, o valor padrão 20 é usado.

`spellcheck`

Definir o valor deste atributo como `true` indica que o elemento deve passar por um corretor ortográfico e gramatical. O valor `default` indica que o elemento deve agir de acordo com seu comportamento padrão, possivelmente baseado no valor do atributo `spellcheck` de seu elemento pai. O valor `false` indica que o elemento não deve ser verificado.

`src`

Se o valor do atributo **type** for `image`, este atributo especifica uma URI para a localização de uma imagem a ser exibida no botão gráfico; caso contrário, este atributo é ignorado.

`step`

Trabalha junto com os atributos **min** e **max** para limitar os incrementos em que valores numéricos ou datas podem ser alterados. Seu valor pode ser `any` ou um número de ponto flutuante positivo. Se o valor não for `any`, o controle aceita somente valores múltiplos valor do incremento maiores que o mínimo.

`tabindex`

A posição do elemento na ordem de navegação pela tecla Tab no documento atual.

`value`

O valor inicial do controle. Este atributo é opcional exceto quando o valor do atributo **type** for `radio` ou `checkbox`. Note que, quando a página é recarregada, Gecko and IE ignorarão o valor especificado no código HTML se o valor tiver sido alterado antes do recarregamento.

`width`

Se o valor do atributo **type** for `image`, este atributo define a largura da imagem exibida para o botão.

`x-moz-errormessage`

Essa extensão da Mozilla permite que você especifique a mensagem de erro a ser exibida quando um campo não valida com sucesso.

Notas

Entradas de arquivo

> **Nota:** a partir do Gecko 2.0, chamar o método `click()` num elemento `<input>` do tipo `file` abre o seletor de arquivos e permite que o usuário selecione arquivos. Veja Usando arquivos a partir de aplicações web para um exemplo e mais detalhes.

Você não pode definir o valor de um seletor de arquivos a partir de um script; fazer algo como o seguinte não tem efeito:

```
var e = getElementById("algumaEntradaDeArquivo");
e.value = "foo";
```

Mensagens de erro

Se você quiser que o Firefox mostre uma mensagem de erro personalizada quando a validação de um campo falha, você pode usar o atributo `x-moz-errormessage` para isso:

```
<input
  type="email"
  x-moz-errormessage="Por favor, especifique um endereço de e-mail." />
```

Note, no entanto, que isso não é padronizado e não terá efeito em outros navegadores.

Exemplos

Uma caixa de entrada simples

```
<!-- Um campo básico -->
<input type="text" name="input" value="Digite aqui" />
```

Um caso de uso comum

```
<!-- Um formulário comum que inclui tags input -->
<form action="getform.php" method="get">
  Nome: <input type="text" name="nome" /><br />
  Sobrenome: <input type="text" name="sobrenome" /><br />
  E-mail: <input type="email" name="email_usuario" /><br />
  <input type="submit" value="Enviar" />
</form>
```

Usando o mozactionhint no Firefox mobile

Você pode usar o atributo `mozactionhint` para especificar o texto para o rótulo da tecla Enter no teclado virtual quando seu formulário é exibido no Firefox mobile. Por exemplo, para ter um rótulo "Próximo", você pode fazer o seguinte:

```
<input type="text" mozactionhint="next" name="sometext" />
```

Veja também

- Outros elementos relacionados a formulários: `<form>`, `<button>`, `<datalist>`, `<legend>`, `<label>`, `<select>`, `<optgroup>`, `<option>`, `<textarea>`, `<fieldset>`, `<output>`, `<progress>` e `<meter>`.
- Cross-browser HTML5 placeholder text

<input type="button">

Elementos `<input>` do tipo **`button`** são renderizados como um simples botão, que podem ser programados para controlar funcionalidades customizadas em qualquer lugar de uma página web quando for atribuído um evento (tipicamente para um evento `click`).

Experimente

```
<input class="styled" type="button" value="Add to favorites" />
```

```
.styled {
  border: 0;
  line-height: 2.5;
  padding: 0 20px;
  font-size: 1rem;
  text-align: center;
  color: #fff;
  text-shadow: 1px 1px 1px #000;
  border-radius: 10px;
  background-color: rgb(220 0 0 / 100%);
  background-image: linear-gradient(
    to top left,
    rgb(0 0 0 / 20%),
    rgb(0 0 0 / 20%) 30%,
    rgb(0 0 0 / 0%)
  );
  box-shadow:
    inset 2px 2px 3px rgb(255 255 255 / 60%),
    inset -2px -2px 3px rgb(0 0 0 / 60%);
}

.styled:hover {
```

```
  background-color: rgb(255 0 0 / 100%);
}

.styled:active {
  box-shadow:
    inset -2px -2px 3px rgb(255 255 255 / 60%),
    inset 2px 2px 3px rgb(0 0 0 / 60%);
}
```

> **Nota:** Enquanto elementos `<input>` do tipo `button` ainda são perfeitamente válidos, os novos elementos `<button>` são agora os favoráveis meios para criar botões. Uma etiqueta de texto (label) para um `<button>` pode ser inserida entre uma tag de abertura e outra de fechamento, podendo ser incluídas até imagens.

Value	Um `String` usado como uma etiqueta de botão.
Eventos	[`click`](/pt-BR/docs/Web/API/Element/click_event)
Atributos comuns suportados	`type`, e `value`
atributos IDL	`value`
Métodos	Nenhum

Value

Seu atributo `value` de um elemento `<input type="button">` contém uma `String` que é usado como uma etiqueta (label) de um botão

```
<input type="button" value="Click Me" />
```

Se você não especificar um `value`, você obtém um botão vazio:

```
<input type="button" />
```

Usando buttons

Elementos `<input type="button">` não possuem comportamento padrão (seu primos, `<input type="submit">` e `<input type="reset">` são usados para submeter e resetar formulários). Para que botões possam fazer algo, você tem de escrever um código em JavaScript para fazê-lo trabalhar.

Um simples botão

Nós iremos começar criando um simples botão com um evento `click` que inicia nossa máquina (bem, ele altera o `value` do botão e o contéudo texto do seguinte parágrafo):

```
<form>
  <input type="button" value="Start machine" />
</form>
<p>The machine is stopped.</p>
```

```
const button = document.querySelector("input");
const paragraph = document.querySelector("p");

button.addEventListener("click", updateButton);

function updateButton() {
  if (button.value === "Start machine") {
    button.value = "Stop machine";
    paragraph.textContent = "The machine has started!";
  } else {
    button.value = "Start machine";
    paragraph.textContent = "The machine is stopped.";
```

```
  }
}
```

O script recebe uma referência para o objeto `HTMLInputElement` representando o `<input>` no DOM, salvando esta referência na variável `button`. `addEventListener()` é então usado para criar uma função que será chamada quando o evento `click` for executado no botão.

Adicionando atalhos de teclados aos botões

Keyboard shortcuts, also known as access keys and keyboard equivalents, let the user trigger a button using a key or combination of keys on the keyboard. To add a keyboard shortcut to a button — just as you would with any `<input>` for which it makes sense — you use the `accesskey` global attribute.

In this example, `s` is specified as the access key (you'll need to press `s` plus the particular modifier keys for your browser/OS combination; see accesskey for a useful list of those).

```
<form>
  <input type="button" value="Start machine" accesskey="s" />
</form>
<p>The machine is stopped.</p>
```

> **Nota:** The problem with the above example of course is that the user will not know what the access key is! In a real site, you'd have to provide this information in a way that doesn't intefere with the site design (for example by providing an easily accessible link that points to information on what the site accesskeys are).

Desativando e ativando um botão

To disable a button, simply specify the `disabled` global attribute on it, like so:

```
<input type="button" value="Disable me" disabled />
```

You can enable and disable buttons at run time by simply setting `disabled` to `true` or `false`. In this example our button starts off enabled, but if you press it, it is disabled using `button.disabled = true`. A `setTimeout()` function is then used to reset the button back to its enabled state after two seconds.

If the `disabled` attribute isn't specified, the button inherits its `disabled` state from its parent element. This makes it possible to enable and disable groups of elements all at once by enclosing them in a container such as a `<fieldset>` element, and then setting `disabled` on the container.

The example below shows this in action. This is very similar to the previous example, except that the `disabled` attribute is set on the `<fieldset>` when the first button is pressed — this causes all three buttons to be disabled until the two second timeout has passed.

> **Nota:** Firefox will, unlike other browsers, by default, persist the dynamic disabled state of a `<button>` across page loads. Use the `autocomplete` attribute to control this feature.

Validação

Buttons não participam na validação; eles não tem um valor real para ser restringido.

Exemplos

The below example shows a very simple drawing app created using a `<canvas>` element and some simple CSS and JavaScript (we'll hide the CSS for brevity). The top two controls allow you to choose the color and size of the drawing pen. The button, when clicked, invokes a function that clears the canvas.

```
<div class="toolbar">
  <input type="color" aria-label="select pen color" />
  <input
```

```
    type="range"
    min="2"
    max="50"
    value="30"
    aria-label="select pen size" /><span class="output">30</span>
  <input type="button" value="Clear canvas" />
</div>

<canvas class="myCanvas">
  <p>Add suitable fallback here.</p>
</canvas>
```

```
var canvas = document.querySelector(".myCanvas");
var width = (canvas.width = window.innerWidth);
var height = (canvas.height = window.innerHeight - 85);
var ctx = canvas.getContext("2d");

ctx.fillStyle = "rgb(0,0,0)";
ctx.fillRect(0, 0, width, height);

var colorPicker = document.querySelector('input[type="color"]');
var sizePicker = document.querySelector('input[type="range"]');
var output = document.querySelector(".output");
var clearBtn = document.querySelector('input[type="button"]');

// covert degrees to radians
function degToRad(degrees) {
  return (degrees * Math.PI) / 180;
}

// update sizepicker output value

sizePicker.oninput = function () {
  output.textContent = sizePicker.value;
};

// store mouse pointer coordinates, and whether the button is pressed
var curX;
var curY;
var pressed = false;

// update mouse pointer coordinates
document.onmousemove = function (e) {
  curX = window.Event
    ? e.pageX
    : e.clientX +
      (document.documentElement.scrollLeft
        ? document.documentElement.scrollLeft
        : document.body.scrollLeft);
  curY = window.Event
    ? e.pageY
    : e.clientY +
      (document.documentElement.scrollTop
        ? document.documentElement.scrollTop
        : document.body.scrollTop);
};

canvas.onmousedown = function () {
  pressed = true;
};

canvas.onmouseup = function () {
  pressed = false;
};

clearBtn.onclick = function () {
  ctx.fillStyle = "rgb(0,0,0)";
```

```
  ctx.fillRect(0, 0, width, height);
};

function draw() {
  if (pressed) {
    ctx.fillStyle = colorPicker.value;
    ctx.beginPath();
    ctx.arc(
      curX,
      curY - 85,
      sizePicker.value,
      degToRad(0),
      degToRad(360),
      false,
    );
    ctx.fill();
  }

  requestAnimationFrame(draw);
}

draw();
```

Veja também

- `<input>` and the `HTMLInputElement` interface which implements it.
- The more modern `<button>` element.
- Compatibility of CSS properties

<input type="checkbox">

`<input>` elementos do tipo **`checkbox`** são renderizados por padrão como caixas quadradas que são marcadas (com uma marca de verificação) quando ativadas, como as que você veria em um formulário do governo. A aparência exata depende de da configuração de sistema operacional sobre o qual o navegador está sendo executado. Caixas de seleção permitem que você selecione valores únicos para envio em um formulário (ou não).

Experimente

```
<fieldset>
  <legend>Choose your monster's features:</legend>

  <div>
    <input type="checkbox" id="scales" name="scales" checked />
    <label for="scales">Scales</label>
  </div>

  <div>
    <input type="checkbox" id="horns" name="horns" />
    <label for="horns">Horns</label>
  </div>
</fieldset>
```

```
p,
label {
  font:
    1rem "Fira Sans",
    sans-serif;
}

input {
  margin: 0.4rem;
}
```

> **Note: Nota** : Os botões de opção são semelhantes às caixas de seleção, mas com uma distinção importante: os botões de opção são agrupados em um conjunto no qual apenas um botão pode ser selecionado por vez, enquanto as caixas de seleção permitem ativar e desativar valores únicos. Quando existem vários controles, os botões de opção permitem que um seja selecionado de todos, enquanto as caixas de seleção permitem que vários valores sejam selecionados.

Valor	Um `String` representando o valor da caixa de seleção.
Eventos	[`change`](/pt-BR/docs/Web/Events/change) e [`input`](/pt-BR/docs/Web/API/Element/input_event)
Atributos comuns suportados	`checked`
Atributos IDL	`checked` e `value`
Métodos	`select ()`

Valor

Um `String` representando o valor da caixa de seleção. Isso nunca é visto no lado do cliente, mas no servidor esse é o `value` atribuído aos dados enviados com o `name` da caixa de seleção. Tome o seguinte exemplo:

```
<form>
  <div>
    <input
      type="checkbox"
      id="subscribeNews"
      name="subscribe"
      value="newsletter" />
    <label for="subscribeNews"> Subscreva a newsletter? </label>
  </div>
  <div>
    <button type="submit">Subscrever</button>
  </div>
</form>
```

Neste exemplo, temos um nome `subscribe`e um valor de `newsletter`. Quando o formulário é enviado, o par de nome / valor de dados será `subscribe=newsletter`.

Se o atributo `value` for omitido, o valor padrão da caixa de seleção é `on` , portanto, os dados enviados nesse caso seriam `subscribe=on`.

> **Note: Nota** : Se uma caixa de seleção estiver desmarcada quando seu formulário for enviado, não haverá nenhum valor enviado ao servidor para representar seu estado desmarcado (por exemplo `value=unchecked`); o valor não é submetido ao servidor.

Usando entradas da caixa de seleção

Já cobrimos o uso mais básico de caixas de seleção acima. Vamos agora olhar para os outros recursos e técnicas relacionados à caixa de seleção que você precisa.

Lidando com várias caixas de seleção

O exemplo que vimos acima continha apenas uma caixa de seleção; em situações do mundo real, você provavelmente encontrará várias caixas de seleção. Se eles não estiverem relacionados, você pode lidar com todos eles separadamente, como mostrado acima. No entanto, se todos estiverem relacionados, as coisas não são tão simples.

Por exemplo, na demonstração a seguir, incluímos várias caixas de seleção para permitir que o usuário selecione seus interesses (veja a versão completa na seção Examples).

```
<fieldset>
  <legend>Escolha seus interesses</legend>
  <div>
    <input
      type="checkbox"
      id="codificação"
      name="interesse"
      value="codificação" />
    <label for="coding"> Codificação </label>
  </div>
  <div>
    <input type="checkbox" id="música" name="interesse" value="música" />
    <label for="music"> Música </label>
  </div>
</fieldset>
```

Neste exemplo, você verá que atribuímos a cada caixa de seleção o mesmo `name`. Se ambas as opções são verificados e, em seguida, o formulário é enviado, você terá uma série de pares nome / valor apresentado assim: `interest=coding&interest=music`. Quando esses dados atingirem o lado do servidor, você poderá capturá-los como uma matriz de valores relacionados e tratá-los adequadamente - consulte Lidar com várias caixas de seleção com uma única variável de servidor , por exemplo.

Caixas de verificação por padrão

Para fazer uma caixa de seleção marcada por padrão, você simplesmente atribui o atributo `checked`. Veja o exemplo abaixo:

```
<fieldset>
  <legend>Escolha seus interesses</legend>
  <div>
    <input
      type="checkbox"
      id="codificação"
      name="interesse"
      value="codificação"
      checked />
    <label for="coding"> Codificação </label>
  </div>
  <div>
    <input type="checkbox" id="música" name="interesse" value="música" />
    <label for="music"> Música </label>
  </div>
</fieldset>
```

Fornecendo uma área de acertos maior para suas caixas de seleção

Nos exemplos acima, você deve ter notado que pode alternar uma caixa de seleção clicando no elemento `<label>` associado e na própria caixa de seleção. Esse é um recurso realmente útil de rótulos de formulários HTML que facilitam o clique na opção desejada, especialmente em dispositivos de tela pequena, como smartphones.

Além da acessibilidade, esse é outro bom motivo para configurar corretamente os `<label>`elementos em seus formulários.

Caixas de seleção de estado indeterminado

Além dos estados marcado e não marcado, há um terceiro estado em que uma caixa de seleção pode estar: **indeterminada**. Este é um estado em que é impossível dizer se o item está ativado ou desativado. Isso é definido usando a propriedade `indeterminate` do objeto `HTMLInputElement` via JavaScript (não pode ser definido usando um atributo HTML):

```
inputInstance.indeterminate = true;
```

Uma caixa de seleção no estado indeterminado tem uma linha horizontal na caixa (parece um hífen ou sinal de menos) em vez de uma marca de verificação na maioria dos navegadores.

Não há muitos casos de uso para essa propriedade. O mais comum é quando uma caixa de seleção está disponível e "possui" um número de subopções (que também são caixas de seleção). Se todas as subopções estiverem marcadas, a caixa de seleção proprietária também será marcada e, se todas elas estiverem desmarcadas, a caixa de seleção proprietária será desmarcada. Se

uma ou mais das subopções tiverem um estado diferente das outras, a caixa de seleção proprietária está no estado indeterminado.

Isso pode ser visto no exemplo abaixo (graças aos CSS Tricks para a inspiração). Neste exemplo, acompanhamos os ingredientes que estamos coletando para uma receita. Quando você marca ou desmarca a caixa de seleção de um ingrediente, uma função JavaScript verifica o número total de ingredientes verificados:

- Se nenhum deles estiver marcado, a caixa de seleção do nome da receita está definida como desmarcada.
- Se um ou dois estiverem marcados, a caixa de seleção do nome da receita está definida como `indeterminate`.
- Se todos os três forem marcados, a caixa de seleção do nome da receita estará definida como `checked`.

Portanto, neste caso, o `indeterminate`estado é usado para afirmar que a coleta dos ingredientes foi iniciada, mas a receita ainda não está completa.

```
var overall = document.querySelector ('input[id = "EnchTbl"]');
var ingredients = document.querySelectorAll ('ul input');

overall.addEventListener ('clique', function (e) {
  e.preventDefault ();
});

for(var i = 0; i <ingredientes.length; i ++) {
  ingredientes [i] .addEventListener ('click', updateDisplay);
}

função updateDisplay () {
  var checkedCount = 1;
  for(var i = 0; i <ingredientes.length; i ++) {
    if (ingredientes [i] .checked) {
      checkedCount ++;
    }
  }

  if (checkedCount === ingredientes.length + 1) {
    overall.checked = true;
    overall.indeterminate = false;
  } else if (checkedCount <= ingredients.length + 1 && checkedCount> 1) {
    overall.checked = false;
    overall.indeterminate = true;
  } else {
    overall.checked = false;
    overall.indeterminate = false;
  }
}
```

> **Note: Nota** : Se você enviar um formulário com uma caixa de seleção indeterminada, a mesma coisa acontece como se o formulário estivesse desmarcado - nenhum dado é enviado para representar a caixa de seleção.

Validação

As caixas de seleção suportam validação (oferecidas para todos os `<input>`s). No entanto, a maioria dos `ValidityState` s sempre será `false`. Se a caixa de seleção tiver o `required` atributo, mas não estiver marcada, ela `ValidityState.valueMissing` será `true`.

Exemplos

O exemplo a seguir é uma versão estendida do exemplo "multiple checkboxes" que vimos acima - tem mais opções padrão, além de uma caixa de seleção "other" que quando marcada faz com que um campo de texto apareça para inserir um valor para a opção "other". Isto é conseguido com um simples bloco de JavaScript. O exemplo também inclui alguns CSS para melhorar o estilo.

HTML

```
<form>
  <fieldset>
    <legend>Escolha seus interesses</legend>
    <div>
      <label>
        <input type="checkbox" id="coding" name="interest" value="coding" />
        Codificação
      </label>
    </div>
    <div>
      <label>
        <input type="checkbox" id="music" name="interest" value="music" />
        Música
      </label>
    </div>
    <div>
      <label>
        <input type="checkbox" id="art" name="interest" value="art" />
        Arte
      </label>
    </div>
    <div>
      <label>
        <input type="checkbox" id="sports" name="interest" value="sports" />
        Esportes
      </label>
    </div>
    <div>
      <label>
        <input type="checkbox" id="cooking" name="interest" value="cooking" />
        Culinária
      </label>
    </div>
    <div>
      <label>
        <input type="checkbox" id="other" name="interest" value="other" />
        Outro
      </label>
      <input
        type="text"
        id="otherValue"
        name="other"
        aria-label="Other interest" />
    </div>
    <div>
      <button type="submit">Enviar formulário</button>
    </div>
  </fieldset>
</form>
```

CSS

```
html {
  font-family: sans-serif;
}

form {
  width: 600px;
  margin: 0 auto;
}

div {
  margin-bottom: 10px;
}

fieldset {
```

```
  background: cyan;
  border: 5px solid blue;
}

legend {
  padding: 10px;
  background: blue;
  color: cyan;
}
```

JavaScript

```
const otherCheckbox = document.querySelector("#other");
const otherText = document.querySelector("#otherValue");
otherText.style.visibility = "hidden";

otherCheckbox.addEventListener("change", () => {
  if (otherCheckbox.checked) {
    otherText.style.visibility = "visible";
    otherText.value = "";
  } else {
    otherText.style.visibility = "hidden";
  }
});
```

Compatibilidade com navegadores

Veja também

- `<input>` e a interface `HTMLInputElement` que o implementa.
- Os seletores CSS `:checked` e `:indeterminate` permitem que você marque as caixas de seleção com base em seu estado atual

<input type="date">

Os elementos `<input>` do tipo **`date`** cria campos de entrada que permite o usuário informar uma data, como também usar uma caixa de texto que valida automaticamente o conteúdo, ou usando uma interface de seleção de data especial. O valor resultante inclui ano, mês e dia, mas não o `horário`. Os tipos de entrada time e `datetime-local` permitem informar horário e data/hora.

A interface do usuário do controle varia geralmente de navegador para navegador; neste momento o suporte é irregular, veja Browser compatibility para maiores detalhes. Nos navegadores sem suporte, o controle é rebaixado graciosamente para um `<input type="text">` simples.

```
<input id="date" type="date" />
```

Entre os navegadores que suportam uma interface personalizada para selecionar datas é o controle de data do Chrome/Opera, que se parece com:



O controle de data do Edge se parece assim:

O controle de data do Firefox se parece assim:



Value	Um `String` que representa uma data no formato AAAA-MM-DD ou vazio
Eventos	[`change`](/pt-BR/docs/Web/Events/change) e [`input`](/pt-BR/docs/Web/API/Element/input_event)
Atributos Comuns Suportados	`autocomplete`, `list`, `readonly` e `step`
Atributos IDL	`list`, `value`, `valueAsDate`, `valueAsNumber`.
Métodos	`select()`, `stepDown()`, `stepUp()`

Valores

Um `String` representa o valor data informada na entrada. Você pode definir o valor padrão para a entrada incluindo uma data dentro do atributo `value`, como:

```
<input id="date" type="date" value="2017-06-01" />
```

Uma coisa para perceber é que o formato da data mostrada difere do `value` atual— o formato da data mostrada será escolhido baseado na localização definida no navegador do usuário, enquanto que a data em `value` sempre será formatado como `yyyy-mm-dd`.

Você pode, além disso, obter e definir o valor da data em JavaScript usando a propriedade `value` do elemento de entrada, por exemplo:

```
var dateControl = document.querySelector('input[type="date"]');
dateControl.value = "2017-06-01";
```

Este código localiza o primeiro elemento `<input>` que o `type` é `date` e define seu valor para a data 2017-06-01 (1 de junho de 2017).

Atributos adicionais

Além dos atributos disponíveis para todos os elementos `<input>`, as caixas de texto de data oferecem os seguintes atributos:

Atributo	Descrição
`max`	A maior data aceitável

Atributo	Descrição
`min`	A menor data aceitável
`readonly`	Se o conteúdo da caixa de texto é somente leitura
`step`	O intervalo a ser usado, quando clicar nos botões de seta para baixo e para cima, e também para validação

Usando caixas de texto de data

Caixas de texto de data se mostra conveniente à primeira vista — eles fornecem uma interface simples para escolha de datas e normalizam o formato da data enviado para o servidor, independentemente da localização do usuário. Contudo, há problemas com o `<input type="date">` por causa do suporte limitado do navegador.

Iremos dar uma olhada em usos básicos e mais complexos de `<input type="date">`, então aconselharemos sobre como atenuar os problemas de suporte dos navegadores (veja Handling browser support). Claro, esperamos que ao longo do tempo o supore dos navegadores sejam mais universal e este problema desapareça.

Uso básico da data

O uso mais simples de `<input type="date">` envolve a combinação de um `<input>` básico e o elemento `<label>`, como pode ser visto abaixo:

```
<form>
  <div>
    <label for="diaa">Informe a data do seu aniversário:</label>
    <input type="date" id="diaa" name="diaa" />
  </div>
</form>
```

Definindo data mínima e máxima

Você pode usar os atributos `min` e `max` para restringir as datas que podem ser escolhidas pelo usuário. No próximo exemplo nós definimos uma data mínima como `2017-04-01` e data máxima como `2017-04-30`:

```
<form>
  <div>
    <label for="festa">Escolha a sua data preferida da festa:</label>
    <input
      type="date"
      id="festa"
      name="festa"
      min="2017-04-01"
      max="2017-04-30" />
  </div>
</form>
```

O resultado aqui será apenas que as dias de Abril de 2017 serão selecionados — apenas a parte "dias" do texto será editável e datas fora de Abril não serão rolados na ferramenta de seleção de data.

> **Note: Observação**: Você deve conhecer o uso do atributo `step` para variar o número de dias pulados cada vez que a data é incrementada (ex.: talvez você queira deixar que os Sábados sejam selecionáveis). Contudo, isto não parece funcionar eficiente de qualquer implementação em tempo de escrita.

Controlando o tamanho da entrada

`<input type="date">` não suporta atributos de tamanho de formulário como `size`. Você poderá recorrer ao CSS para modificar o tamanho.

Validação

Por padrão `<input type="date">` não aplica nenhuma validação de entrada de valores. As implementações da interface geralmente não deixam você informar nada que não seja uma data — o que é útil — mas você pode continuar deixando o campo vazio ou (em navegadores onde a entrada converte para o tipo `text`) informar uma data inválida (ex.: o 32 de Abril).

Se você usa `min` e `max` para restringir datas disponíveis (ver Definindo data mínima e máxima), os navegadores suportados mostrarão um erro se você tentar submeter uma data fora da faixa. Contudo, você terá que verificar os resultados para ter certeza que o valor está entre estas datas, uma vez que são aplicadas apenas se o selecionador de data for totalmente suportado pelo dispositivo do usuário.

Adicionalmente, você pode usar o atributo `required` para tornar o preenchimento da data obrigatório — novamente, um erro será mostrado se você tentar submeter um campo de data vazia. Isto, finalmente, deve funcionar em muitos navegadores.

Vamos dar uma olhada em um exemplo — aqui nós definimos datas mínima e máxima e deixamos o campo como obrigatório:

```
<form>
  <div>
    <label for="festa"
      >Escolha sua data preferida da festa (obrigatório, de 1º a 20 de
      abril):</label
    >
    <input
      type="date"
      id="festa"
      name="festa"
      min="2017-04-01"
      max="2017-04-20"
      required />
    <span class="validity"></span>
  </div>
  <div>
    <input type="submit" />
  </div>
</form>
```

Se você tentar submeter o formulário com uma data imcompleta (ou com uma data fora da faixa), o navegador mostrará um erro. Tente executar o exemplo agora:

Aqui tem uma captura de tela que mostra o resultado se seu navegador não suporta:



Aqui tem o CSS utilizado no exemplo acima. Nós usamos as propriedades CSS `:valid` e `:invalid` para estilizar a caixa de texto dependendo se o valor atual é válido ou não. Colocamos ícones num `<span>` próximo a caixa de entrada, não dentro da caixa, porque no Chrome o conteúdo gerado é colocado dentro do controle do formulário, e não seria estilzado ou mostrado eficientemente.

```
div {
  margin-bottom: 10px;
  display: flex;
  align-items: center;
}

label {
  display: inline-block;
  width: 300px;
}

input:invalid + span:after {
  content: "✖";
  padding-left: 5px;
}
```

```
input:valid + span:after {
  content: "✓";
  padding-left: 5px;
}
```

> **Warning: Importante**: A validação do formulário HTML não subtitui scripts que validam se a entrada de dados está em um formato apropriado. É muito fácil para alguém fazer ajustes no HTML que permitam sobrepor a validação ou removê-lo inteiramente. Também é possível simplesmente sobrepor seu HTML inteiramente e submeter os dados diretamente ao seu servidor. Se seu código server-side falhar na validação do dado que recebe pode ocorreu um desaste quando os dados forem submetidos inapropriadamente formatado (ou dado pode ser muito grande, ou é do tipo errado e assim por diante).

Manipulação do suporte do navegador

Como mencionado acima, o maior problema em usar caixas de entrada de data em tempo de escrita é o suporte do navegador. Por exemplo, o selecionador de data no Firefox para Android se parece com isso:



Navegadores que não suportam graciosamente rebaixa para uma caixa de texto comum, mas criam problemas em termos de consistência da interface do usuário (o controle apresentado será diferente) e a manipulação do dado.

O segundo problema é mais sério que os anterirores; como mencionamos antes, com um campo de texto de data o valor atual sempre é normalizado pelo formato `yyyy-mm-dd`. Com a caixa de texto comum, por outro lado, por padrão do navegador não há reconhecimento de qual formato a data deve ter e há muitos modos diferentes em que as pessoas escrevem datas. Por exemplo:

- `ddmmyyyy`
- `dd/mm/yyyy`

- mm/dd/yyyy
- dd-mm-yyyy
- mm-dd-yyyy
- Month dd yyyy

Um jeito de contornar isso é colocar um atributo `pattern` na caixa de texto de data. Sempre que a caixa de texto de data não usá-lo, a caixa de texto devolverá um erro. Por exemplo, tente ver o que o seguinte exemplo faz num navegador sem suporte:

```
<form>
  <div>
    <label for="diaa">Informe a data do seu aniversário:</label>
    <input
      type="date"
      id="bday"
      name="diaa"
      required
      pattern="[0-9]{4}-[0-9]{2}-[0-9]{2}" />
    <span class="validity"></span>
  </div>
  <div>
    <input type="submit" />
  </div>
</form>
```

Se você submetê-lo, verá que o navegador agora mostra uma mensagem de erro (e destaca a caixa de texto como inválido) se o que foi informado não combinam com o padrão `nnnn-nn-nn`, onde `n` é um número entre 0 e 9. Claro, isto não parará as pessoas de informar datas inválidas ou datas formatadas incorretamente, como `yyyy-dd-mm` (onde nós precisamos de `yyyy-mm-dd`). Então continua sendo um problema.

A melhor maneira de lidar com datas nos formulários de um modo entre navegadores no momento é obter do usuário o dia, mês e ano em controles separados (elementos `<select>` são bem populares; veja abaixo uma implementação) ou use uma biblioteca JavaScript como o selecionador de data do jQuery.

Atributos

Beside the attributes listed below, this element can have any of the global attributes.

`mozactionhint`

> Specifies an "action hint" used to determine how to label the enter key on mobile devices with virtual keyboards. Supported values are `go`, `done`, `next`, `search`, and `send`; these automatically get mapped to the appropriate string (and are case-insensitive).

`autofocus`

> This Boolean attribute lets you specify that a form control should have input focus when the page loads, unless the user overrides it, for example by typing in a different control. Only one form element in a document can have the **autofocus** attribute, which is a Boolean. It cannot be applied if the **type** attribute is set to `hidden` (that is, you cannot automatically set focus to a hidden control).

`disabled`

> This Boolean attribute indicates that the form control is not available for interaction. In particular, the `click` event will not be dispatched on disabled controls. Also, a disabled control's value isn't submitted with the form.

`form`

> The form element that the input element is associated with (its *form owner*). The value of the attribute must be an **id** of a `<form>` element in the same document. If this attribute is not specified, this `<input>` element must be a descendant of a `<form>` element. This attribute enables you to place `<input>` elements anywhere within a document, not just as descendants of their form elements. An input can only be associated with one form.

`formaction`

The URI of a program that processes the information submitted by the input element, if it is a submit button or image. If specified, it overrides the `action` attribute of the element's form owner.

`formenctype`

If the input element is a submit button or image, this attribute specifies the type of content that is used to submit the form to the server. Possible values are:

- `application/x-www-form-urlencoded`: The default value if the attribute is not specified.
- `multipart/form-data`: Use this value if you are using an `<input>` element with the `type` attribute set to `file`.
- `text/plain`

If this attribute is specified, it overrides the `enctype` attribute of the element's form owner.

`formmethod`

If the input element is a submit button or image, this attribute specifies the HTTP method that the browser uses to submit the form. Possible values are:

- `post`: The data from the form is included in the body of the form and is sent to the server.
- `get`: The data from the form are appended to the **form** attribute URI, with a '?' as a separator, and the resulting URI is sent to the server. Use this method when the form has no side-effects and contains only ASCII characters.

If specified, this attribute overrides the `method` attribute of the element's form owner.

`formnovalidate`

If the input element is a submit button or image, this Boolean attribute specifies that the form is not to be validated when it is submitted. If this attribute is specified, it overrides the `novalidate` attribute of the element's form owner.

`formtarget`

If the input element is a submit button or image, this attribute is a name or keyword indicating where to display the response that is received after submitting the form. This is a name of, or keyword for, a *browsing context* (for example, tab, window, or inline frame). If this attribute is specified, it overrides the `target` attribute of the elements's form owner. The following keywords have special meanings:

- `_self`: Load the response into the same browsing context as the current one. This value is the default if the attribute is not specified.
- `_blank`: Load the response into a new unnamed browsing context.
- `_parent`: Load the response into the parent browsing context of the current one. If there is no parent, this option behaves the same way as `_self`.
- `_top`: Load the response into the top-level browsing context (that is, the browsing context that is an ancestor of the current one, and has no parent). If there is no parent, this option behaves the same way as `_self`.

`inputmode`

A hint to the browser for which keyboard to display. This attribute applies when the value of the **type** attribute is text, password, email, or url. Possible values are:

- `verbatim`: Alphanumeric, non-prose content such as usernames and passwords.
- `latin`: Latin-script input in the user's preferred language with typing aids such as text prediction enabled. For human-to-computer communication such as search boxes.
- `latin-name`: As *latin*, but for human names.
- `latin-prose`: As *latin*, but with more aggressive typing aids. For human-to-human communication such as instant messaging for email.
- `full-width-latin`: As *latin-prose*, but for the user's secondary languages.
- `kana`: Kana or romaji input, typically hiragana input, using full-width characters, with support for converting to kanji. Intended for Japanese text input.
- `katakana`: Katakana input, using full-width characters, with support for converting to kanji. Intended for Japanese text input.
- `numeric`: Numeric input, including keys for the digits 0 to 9, the user's preferred thousands separator character, and the character for indicating negative numbers. Intended for numeric codes, e.g. credit card numbers. For actual

numbers, prefer using <input type="number">
- `tel`: Telephone input, including asterisk and pound key. Use <input type="tel"> if possible instead.
- `email`: Email input. Use <input type="email"> if possible instead.
- `url`: URL input. Use <input type="url"> if possible instead.

`list`

Identifies a list of pre-defined options to suggest to the user. The value must be the **id** of a `<datalist>` element in the same document. The browser displays only options that are valid values for this input element.

`max`

The maximum value for this item, which must not be less than its minimum (**min** attribute) value.

`min`

The minimum value for this item, which must not be greater than its maximum (**max** attribute) value.

`name`

The name of the control, which is submitted with the form data.

`readonly`

This Boolean attribute indicates that the user cannot modify the value of the control.

`required`

This attribute specifies that the user must fill in a value before submitting a form. It cannot be used when the **type** attribute is `hidden`, `image`, or a button type (`submit`, `reset`, or `button`). The `:optional` and `:required` CSS pseudo-classes will be applied to the field as appropriate.

`selectionDirection`

The direction in which selection occurred. This is "forward" if the selection was made from left-to-right in an LTR locale or right-to-left in an RTL locale, or "backward" if the selection was made in the opposite direction. This can be "none" if the selection direction is unknown.

`spellcheck`

Setting the value of this attribute to `true` indicates that the element needs to have its spelling and grammar checked. The value `default` indicates that the element is to act according to a default behavior, possibly based on the parent element's own `spellcheck` value. The value `false` indicates that the element should not be checked.

`step`

Works with the **min** and **max** attributes to limit the increments at which a numeric or date-time value can be set. It can be the string `any` or a positive floating point number. If this attribute is not set to `any`, the control accepts only values at multiples of the step value greater than the minimum.

`value`

The initial value of the control. This attribute is optional. Note that when reloading the page, Gecko and IE will ignore the value specified in the HTML source, if the value was changed before the reload.

`x-moz-errormessage`

This Mozilla extension allows you to specify the error message to display when a field doesn't successfully validate.

Examples

To create a widget to display a date, use:

```
<input type="date" />
```

Veja também

- The generic `<input>` element and the interface used to manipulate it, `HTMLInputElement`
- Date and Time picker tutorial
- Bug 1283381 - Firefox bug to implement

<input type="password">

Elementos `<input>` do tipo **`"password"`** são uma maneira do usuário digitar senhas com segurança. O elemento é mostrado como um controle de edição de texto de uma linha, no qual o texto é omitido para que não possa ser lido, geralmente substituindo cada caractere por um símbolo como o astesrisco ("*") ou um ponto ("•"). Esse caracter varia dependendo do agente de usuário e do OS.

Experimente

```
<div>
  <label for="username">Username:</label>
  <input type="text" id="username" name="username" />
</div>

<div>
  <label for="pass">Password (8 characters minimum):</label>
  <input type="password" id="pass" name="password" minlength="8" required />
</div>

<input type="submit" value="Sign in" />
```

```
label {
  display: block;
}

input[type="submit"],
label {
  margin-top: 1rem;
}
```

Os detalhes de como o processo de inserção do texto funciona podem variar dependendo do navegador. Dispositivos móveis, por exemplo, frequentemente mostram o caractere digitado por um breve momento antes de ser ocultado, de forma que o usuário possa verificar se realmente digitou o caractere pretendido. Isso é útil devido ao tamanho reduzido das teclas e a facilidade de se pressionar a tecla errada, principalmente em teclados virtuais.

> **Nota:** Todo formulário que envolve informações sensíveis tais como senhas (ex.: formulários de login) deve ser servido usando HTTPS. Vários navegadores implementam mecanismos que avisam sobre formulários não protegidos - veja Senhas não Protegidas.

Value	Uma `String` representando uma senha, ou vazia
Eventos	[`change`](/pt-BR/docs/Web/Events/change) e [`input`](/pt-BR/docs/Web/API/Element/input_event)
Atributos comuns suportados	`autocomplete`, `inputmode`, `maxlength`, `minlength`, `pattern`, `placeholder`, `readonly`, `required`, e `size`
Atributos IDL	`selectionStart`, `selectionEnd`, `selectionDirection`, e `value`
Métodos	`select()`, `setRangeText()`, e `setSelectionRange()`

Valor

O atributo `value` contém uma `String` cujo valor é o conteúdo atual do controle de edição de texto usado para a inserção da senha. Se o usuário ainda não digitou nada, esse valor é uma string vazia (`""`). Se a propriedade `required` for especificada, então a caixa de edição de senha precisa conter um valor que não seja uma string vazia para ser válida.

Se o atributo `pattern` for especificado, o conteúdo de um controle `"password"` só é considerado válido se o valor passar na validação; veja Validação para mais informações.

> **Nota:** Os caracteres de quebra de linha *line feed* (U+000A) e *carriage return* (U+000D) não são permitidos em valores de `"password"`. Quando o valor de um campo de senha é definido, os caracteres acima são removidos do valor.

Usando campos de senha

Caixas de inserção de senha funcionam assim como outras caixas de inserção de texto; a principal diferença é que o conteúdo é ocultado para impedir que pessoas que estejam próximas ao usuário leiam a senha.

Um campo de senha simples

Aqui vemos o mais básico dos campos de senha, juntamente com um rótulo para ele usando o elemento `<label>`.

```
<label for="senhaUsuario">Senha: </label>
<input id="senhaUsuario" type="password" />
```

Permitindo preenchimento automático

Para permitir que o gerenciador de senhas do usuário digite a senha automaticamente, especifique o atributo `autocomplete`. Para senhas, o valor dele deve ser geralmente um dos seguintes:

`"on"`

> Permite que o navegador ou um gerenciador de senhas preencha automaticamente o campo de senha. Esse valor não é tão informativo quanto `"current-password"` ou `"new-password"`.

`"off"`

> Não permite que o navegador ou gerenciador de senhas preencha automaticamente o campo de senha. Vale observar que alguns softwares ignoram esse valor, já que ele costuma ser prejudicial para a capacidade dos usuários de manter senhas seguras.

`"current-password"`

> Permite que o navegador ou gerenciador de senhas preencha automaticamente o campo de senha. Esse valor fornece mais informações do que `"on"`, já que ele permite que o navegador ou gerenciador de senhas digite automaticamente a senha atualmente conhecida do site no campo, mas não que sugira uma nova.

`"new-password"`

> Permite que o navegador ou gerenciador de senhas digite automaticamente uma nova senha para o site. Esse valor é usado em formulários de "mudar senha" e de "novo usuário", no campo que pede por uma nova senha. A nova senha pode ser gerada de várias maneiras, dependendo do gerenciador de senhas usado. Ele pode simplificar o preenchimento de uma sugestão de nova senha, ou mesmo pode mostrar ao usuário uma interface para criar uma.

```
<label for="senhaUsuario">Senha:</label>
<input id="senhaUsuario" type="password" autocomplete="current-password" />
```

Tornando a senha obrigatória

Para dizer ao navegador do usuário que o campo de senha precisa ter um valor válido antes do formulário ser enviado, basta adicionar o atributo booleano `required`.

```
<label for="senhaUsuario">Senha: </label>
<input id="senhaUsuario" type="password" required />
<input type="submit" value="Enviar" />
```

Especificando um modo de inserção

Se as suas regras de sintaxe recomendadas (ou obrigatórias) para a senha poderiam obter vantagem de uma interface de digitação diferente da de um teclado padrão, você pode usar o atributo `inputmode` para pedir por uma interface específica. O

caso de uso mais óbvio para isso é o de a senha precisar ser numérica (como um PIN). Dispositivos móveis com teclados virtuais, por exemplo, podem optar por mostrar um teclado numérico ao invés de um teclado completo, facilitando a digitação da senha.

```
<label for="pin">PIN: </label>
<input id="pin" type="password" inputmode="numeric" />
```

Definindo restrições de tamanho

Como de praxe, você pode usar os atributos `minlength` e `maxlength` para estabelecer os tamanhos mínimo e máximo aceitáveis para a senha. Este exemplo continua o anterior, agora especificando que o PIN do usuário deva ter pelo menos quatro dígitos e não passe de oito dígitos. O atributo `size` é usado para garantir que o campo de senha aceite no máximo oito caracteres.

```
<label for="pin">PIN:</label>
<input
  id="pin"
  type="password"
  inputmode="numeric"
  minlength="4"
  maxlength="8"
  size="8" />
```

Selecionando o texto

Assim como em outros tipos de campos de texto, você pode usar o método `select()` para selecionar todo o texto no campo de senha.

HTML

```
<label for="senhaUsuario">Senha: </label>
<input id="senhaUsuario" type="password" size="12" />
<button id="selecionarTudo">Selecionar tudo</button>
```

JavaScript

```
document.getElementById("selecionarTudo").onclick = function () {
  document.getElementById("senhaUsuario").select();
};
```

Resultado

Você também pode usar `selectionStart` e `selectionEnd` para obter (ou definir) o intervalo dos caracteres no campo que estão selecionados no momento, e `selectionDirection` para saber em qual direção a seleção ocorreu (ou será estendida, dependendo da sua plataforma; veja a documentação dela para saber mais). Porém, como o texto está oculto, a utilidade desses métodos fica reduzida.

Validação

Se seu aplicativo tiver restrições de conjuntos de caracteres ou qualquer outro requisito para o conteúdo da senha digitada, você pode usar o atributo `pattern` para estabelecer que uma expressão regular seja usada para garantir automaticamente que as senhas cumpram com esses requisitos.

Neste exemplo, são válidos apenas valores consistindo de no mínimo quatro e no máximo oito dígitos hexadecimais.

```
<label for="hexId">Hex ID: </label>
<input
  id="hexId"
  type="password"
  pattern="[0-9a-fA-F]{4,8}"
  title="Digite um ID consistindo de 4-8 dígitos hexadecimais" />
```

`disabled`

Esse atributo booleano indica que o campo de senha não está disponível para interação. Além disso, os valores dos campos marcados com ele não são enviados com o formulário.

Exemplos

Pedindo por um CPF

Este exemplo aceita apenas valores que correspondam ao formato de um Cadastro de Pessoa Física. Este número, usado para fins de cobrança e identificação, segue o formato "123.456.789-09".

HTML

```
<label for="cpf">CPF:</label>
<input
  type="password"
  id="cpf"
  inputmode="number"
  minlength="11"
  maxlength="11"
  pattern="^[0-9]{3}.?[0-9]{3}.?[0-9]{3}-?[0-9]{2}"
  required
  autocomplete="off" />
<br />
<label for="cpf">Valor:</label>
<span id="atual"></span>
```

Ele usa um padrão `pattern` que limita o valor digitado a strings que representem números de CPF no formato válido. O padrão aceita tanto valores separados por pontos (".") e hífen ("-") como apenas os números.

O modo de inserção em `inputmode` está definido como `"number"` para indicar a dispositivos com teclados virtuais que eles devem mostrar um teclado numérico para uma digitação mais fácil. Os atributos `minlength` e `maxlength` foram definidos como 11 e 14, respectivamente, de forma a aceitar valores com no mínimo 11 e no máximo 14 caracteres (o primeiro para valores sem os caracteres separadores e o último para valores com esses caracteres). O atributo `required` é usado para indicar que este campo precisa ter um valor válido. Por fim, `autocomplete` foi definido como `"off"` para evitar que gerenciadores de senhas tentem usar ou definir o valor dele, já que ele não é nem mesmo uma senha.

Obviamente, apenas essa expressão regular não garante um CPF realmente válido, já que a validação deste tipo de número exige cálculos adicionais. Confira este artigo para ver exatamente como garantir um número de CPF válido.

JavaScript

A seguir temos um código simples para mostrar o CPF digitado na tela para que você possa vê-lo. É claro que isso vai contra o propósito de um campo de senha, mas ajuda na hora de experimentar o padrão `pattern`.

```
var cpf = document.getElementById("cpf");
var atual = document.getElementById("atual");

cpf.oninput = function (event) {
  atual.innerHTML = cpf.value;
};
```

Resultado

<input type="range">

`<input>` elementos do tipo **`"range"`** deixam o usuário especificar um valor numérico que não deve ser inferior a um determinado valor, e não mais do que um valor máximo especificado. O valor preciso, no entanto, não é considerado importante. Este geralmente é representado por um controle deslizante ou o mesmo tipo de controle de "number" input. Como este tipo de elemento é impreciso, não deve ser usado a menos que o valor exato do controle não seja importante.

Experimente

```
<p>Audio settings:</p>

<div>
  <input type="range" id="volume" name="volume" min="0" max="11" />
  <label for="volume">Volume</label>
</div>

<div>
  <input
    type="range"
    id="cowbell"
    name="cowbell"
    min="0"
    max="100"
    value="90"
    step="10" />
  <label for="cowbell">Cowbell</label>
</div>
```

```
p,
label {
  font:
    1rem "Fira Sans",
    sans-serif;
}

input {
  margin: 0.4rem;
}
```

Se o navegador do usuário não suportar o tipo `"range"`, este será tratado como um input do tipo `"text"`.

Value	Uma `String` contendo a a string que representa o valor numérico escolhido; use `valueAsNumber` para pegar o valor como um `Number`.
Events	[`change`](/pt-BR/docs/Web/Events/change) e [`input`](/pt-BR/docs/Web/API/Element/input_event)
Supported Common Attributes	`autocomplete`, `list`, `max`, `min`, e `step`
IDL attributes	`list`, `value`, e `valueAsNumber`
Methods	`stepDown()` e `stepUp()`

Value

O atributo `value` contém uma `String` que conte uma representação dos números selecionados em string. O valor nunca é uma string vazia (`""`). O valor padrão está entre o mínimo e o máximo especificados, a menos que o valor máximo seja menor que o mínimo, caso em que o padrão é definido como o atributo de valor mínimo. O algoritmo que determina o valor padrão é:

```
defaultValue =
  rangeElem.max < rangeElem.min
    ? rangeElem.min
    : rangeElem.min + (rangeElem.max - rangeElem.min) / 2;
```

Se for feito uma tentativa para definir como máximo um valor inferior ao mínimo, então este é definido como mínimo. Da mesma forma, uma tentativa de definir um valor maior do que o máximo sendo configurado então para o máximo.

Usando range inputs

Equanto o tipo `"number"` permite aos usuários inserir um número com restrições opcionais forçando-o a digitar um valor entre o valor mínimo e máximo, ele requer que digitem um valor específico. A input de tipo permite ao usuário que coloque um valor sem que preocupe ou precise saber o valor numério específico.

Alguns exemplos de situações que este tipo são comumente usados são:

- Controle de audio como volume, equilíbrio e ou controle de filtros.
- Controle de configuração de cores como canais de cores, transparência, brilho, etc.
- Controle de configuração de jogos, como dificuldade, distância de visibilidade, tamanho do mundo, entre outros.
- Comprimento de senha gerada por gerenciadores de senha.

Como regra geral, se o usuário tiver maior probabilidade de se interessar por porcentagens entre distâncias mínimas e máximas do que por um número real, este tipo de input é um forte candidato. Por exemplo, no caso de um controle de volume estéreo, "colocar o controlador no meio" em vez de "definir o volume como 0.5".

Especificando o mínimo e o máximo

Por padrão, o mínimo é 0 e o máximo é 100. Se não é o que você precisa, voce pode facilmente especificar limites diferentes alterando os valores dos atributos `min` e/ou `max`. Este pode ser um valor inteiro.

Por exemplo, para pedir ao usuário um valor entre -10 e 10, você pode usar:

```
<input type="range" min="-10" max="10" />
```

Definindo a escala entre valores

Por padrão, a escala entre o mínimo e o máximo é 1, significando que este sempre é um valor inteiro. você pode alterar o atributo `step` para controlar esta escala. Por exemplo, se você precisa de um valor de duas casas decimais entre os valores 5 e 10, voce deve definir o valor de `step` para 0.01:

Configurando o atributo step

```
<input type="range" min="5" max="10" step="0.01" />
```

Configurando step como <code>any</code>

Se você deseja aceitar um valor independente das casas decimais, voce pode especificar o valor `"any"` para o atributo `step`:

```
<input type="range" min="0" max="3.14" step="any" />
```

Este exemplo permite ao usuário selecionar qualquer valor entre 0 e π sem qualquer restrição na parte fracionada do valor selecionado.

Adicionando marcas de mapeamento e etiquetas

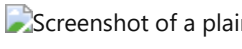
As especificações do HTML oferece aos navegadores uma certa flexibilidade sobre como apresentar o controle deslizante. Em parte alguma esta flexibilidade é mais aparente do que no mapeamento de marcas e, menos ainda, na etiqueta. A especificação descreve como adicionar pontos personalizados ao controle usando o atributo `list` e o elemento `<datalist>` , mas não tem requisitos nem padronizações para marcações ou mesmo pontos ao longo do controle deslizante.

Modelo de controle do range

Uma vez que os navegadores têm essa flexibilidade e, até à data, nenhum suporte a todos os recursos que o HTML define para controles de alcance, aqui estão alguns modelos para mostrar o que você pode obter no MacOS em um navegador que os suporta.

Um controle sem o atributo

Isto é o que você obtem se não especificar o atributo `list` , ou se o navegador não oferece suporte.

HTML	Screenshot
`<input type="range">`	 Screenshot of a plain slider control on macOS

Um controle com o Atributo

Este controle está usando um atributo `list` especificando o ID de uma `<datalist>` que define uma serie de marcações no controle. Há 11 deles, de modo que há um 0% bem como a adição de 10% em cada próximo. Cada ponto representa usando um elemento `<option>` com propriedade `value` definido para o valor do intervalo no qual uma marca deve ser desenhada.

HTML	Screenshot

`<input type="range" list="tickmarks">`
 	
`<datalist id="tickmarks">`
`<option value="0">`
`<option value="10">`
`<option value="20">`
`<option value="30">`
`<option value="40">`
`<option value="50">`
`<option value="60">`
`<option value="70">`
`<option value="80">`
`<option value="90">`
`<option value="100">`
`</datalist>`
	 Screenshot of a plain slider control on macOS

Um controle deslizante com marcas e rótulos

Você pode adicionar rótulos para seu controle usando o atributo `label` para o elemento `<option>` correspondente às marcas de marcação para as quais você deseja ter rótulos.

HTML	Screenshot

`<input type="range" list="tickmarks">`
 	
`<datalist id="tickmarks">`
`<option value="0" label="0%">`
`<option value="10">`
`<option value="20">`
`<option value="30">`
`<option value="40">`
`<option value="50" label="50%">`
`<option value="60">`
`<option value="70">`
`<option value="80">`
`<option value="90">`
`<option value="100" label="100%">`
`</datalist>`
	 Screenshot of a plain slider control on macOS

> **Nota:** Atualmente nenhum navegador suporta estes recursos totalmente. Firefox não suporta pontos e rótulos, por exemplo, enquanto o Chrome suporta as marcações de pontos, porém não suporta as etiquetas.

Criando controles de intervalo vertical

Controle de range horizontal

Considerando este controle:

```
<input type="range" id="volume" min="0" max="11" value="7" step="1" />
```

Esse controle é horizontal (pelo menos na maioria, senão em todos os principais navegadores, outros podem variar).

Usando a propriedade appearance

A propriedade `appearance` tem um valor não padrão de `slider-vertical` que, bem, torna os controles deslizantes verticais.

Usamos o mesmo HTML dos exemplos anteriores:

```
<input type="range" min="0" max="11" value="7" step="1" />
```

Visamos apenas as entradas com um tipo de range:

```
input[type="range"] {
  appearance: slider-vertical;
}
```

Controle de range vertical

As especificações HTML recomenda que os navegadores mudem automaticamente para um controle vertical quando a largura for menor que à altura. Infortuniamente nenhum dos principais navegadores atualmente oferece tal suporte. No entanto você pode cria-lo pelo seu lado. A maneira mais fácil para isto é usar o CSS: aplicando um: `transform` para rotacionar o elemento tornando-o vertical. Vamos dar uma olhada:

HTML

No HTML o elemento `<input>` precisa ser envolto em uma `<div>` para nos permitir corrigir o layout após a transformação ser executada, (uma vez que as transformações não afetam automaticamente o layout da página):

```
<div class="slider-wrapper">
  <input type="range" min="0" max="11" value="7" step="1" />
</div>
```

CSS

Agora precisamos de algum CSS. Primeiro é o CSS para o próprio wrapper; isso especifica o modo de exibição e o tamanho que queremos para que a página seja definida corretamente; em essência, está reservando uma área da página para que o controle deslizante gire e se encaixe no espaço reservado sem bagunçar outros elementos.

```
.slider-wrapper {
  display: inline-block;
  width: 20px;
  height: 150px;
  padding: 0;
}
```

Em seguida, vem a informação de estilo para o elemento `<input>` dentro do espaço reservado:

```
.slider-wrapper input {
  width: 150px;
  height: 20px;
  margin: 0;
  transform-origin: 75px 75px;
  transform: rotate(-90deg);
}
```

O tamanho do controle está configurado para ter 150 pixels de comprimento por 20 pixels de altura . As margins estão configuradas como 0 e a `transform-origin` é deslocada para o centro do espaço de giro do controle deslizante; uma vez que o controle deslizante está configurado para ter 150 pixels de largura, gira através de uma caixa que é de 150 pixels em cada lado. Compensando a origem por 75px em cada eixo significa que vamos girar em torno do centro desse espaço. Finalmente, giramos no sentido anti-horário em 90 °. O resultado: uma entrada de intervalo que é girada para que o valor máximo esteja na parte superior e o valor mínimo esteja na parte inferior.

Result

Validação

Não há padrões de validação disponível, no entanto, as seguintes formas de validações são realizadas automaticamente:

- Se `value` estiver definido para algo que não possa ser convertido em um número flutuante válido, a validação falhará porque a entrada está sofrendo de uma entrada incorreta.
- O valor não será inferior há `min`. O valor padrão é 0.
- O valor não pode ser maior que `max`. O valor padrão é 100.
- O valor será um multiplo de `step`. O valor padrão é 1.

Exemplos

Além dos exemplos variados acima, você encontrará as entradas de alcance demonstradas nestes artigos:

- Controlling multiple parameters with ConstantSourceNode

See also

- HTML Forms
- `<input>` and the `HTMLInputElement` interface it's based upon
- `<input type="number">`

<input type="time">

Elementos `<input>` do tipo **`time`** (hora) criam campos de inserção que permitem que o usuário digite horários facilmente (horas e minutos e, opcionalmente, segundos).

A interface de usuário deste tipo de campo varia de navegador para navegador. A maioria dos navegadores modernos é compatível com ele exceto pelo Safari, o único grande navegador que ainda não o implementou; no Safari (e em qualquer outro navegador que ainda não suporte `<time>`), ele regride para `<input type="text">`.

Experimente

```
<label for="appt">Choose a time for your meeting:</label>

<input type="time" id="appt" name="appt" min="09:00" max="18:00" required />

<small>Office hours are 9am to 6pm</small>
```

```
label {
  display: block;
  font:
    1rem "Fira Sans",
    sans-serif;
}

input,
label {
  margin: 0.4rem 0;
}
```

Aparência

Chrome e Opera

No Chrome/Opera, o campo de `time` é simples, com espaços onde o usuário pode inserir horas e minutos (no formato 24 horas), além de setas para cima e para baixo que servem para, respectivamente, incrementar e decrementar o componente atualmente selecionado. Um botão "X" também é apresentado para limpar o conteúdo do campo.

Firefox

O campo de `time` do Firefox é bem parecido com o do Chrome, exceto pela ausência das setas para cima e para baixo e por estar no formato 12 horas (com um espaço adicional para inserir AM ou PM).

Edge

O campo de `time` do Edge é melhor elaborado, abrindo um seletor de hora e minuto com rolagem infinita. Assim como o Chrome, ele também adota o formato 24 horas:

Value	Uma `String` representando um horário, ou vazia.
Eventos	[`change`](/pt-BR/docs/Web/Events/change) e [`input`](/pt-BR/docs/Web/API/Element/input_event).
Atributos comuns suportados	`autocomplete`, `list`, `readonly` e `step`.
Atributos IDL	`value`, `valueAsDate`, `valueAsNumber` e `list`.
Métodos	`select()`, `stepDown()` e `stepUp()`.

Valor

Uma `String` contendo o valor do horário inserido no campo. Você pode definir um valor padrão para o campo incluindo um horário válido no atributo `value` ao criar o elemento `<input>`, tipo assim:

```
<label for="hora-cons">Escolha o horário da consulta: </label>
<input id="hora-cons" type="time" name="hora-cons" value="13:30" />
```

Você também pode obter e definir o valor do campo via JavaScript usando a propriedade `HTMLInputElement.value`, por exemplo:

```
var campoHora = document.querySelector('input[type="time"]');
campoHora.value = "15:30";
```

Formato do valor do horário

O `value` do campo de `time` está sempre no formato 24 horas: `"hh:mm"`, não importa o formato usado na inserção. Esse formato muitas vezes é escolhido baseado no idioma/local do usuário (ou mesmo pelo agente de usuário). Se o horário incluir segundos (veja Usando o atributo step), o formato sempre é `"hh:mm:ss"`.

No próximo exemplo, você pode ver o modo como o valor do campo de horário varia entre o que foi digitado e o que ele retorna.

Mas antes, vamos dar uma conferida no HTML. Ele é bem simpless, contando com label (rótulo) e input (campo) do jeito que já vimos antes, com a adição de um elemento `<p>` com um `<span>` que mostra o valor do campo de `time`:

```
<form>
  <label for="horaInicio">Hora de início: </label>
  <input type="time" id="horaInicio" />
  <p>
    Valor do campo <code>time</code>:
    <code> "<span id="value">n/a</span>"</code>.
  </p>
</form>
```

No código JavaScript, chamamos um método que monitora o evento `input`, que é disparado sempre que o conteúdo de um elemento `<input>` muda. Toda vez que ele dispara, o conteúdo do `<span>` é substituído pelo novo valor do campo de horário.

```
var horaInicio = document.getElementById("horaInicio");
var valueSpan = document.getElementById("value");

startTime.addEventListener(
  "input",
  function () {
    valueSpan.innerText = startTime.value;
```

```
  },
  false,
);
```

Quando um formulário que inclui um campo de `time` é enviado, o valor é codificado antes de ser incluído nos dados do formulário. Nesse caso, o valor correspondente ao campo de horário sempre estará no formato `"name=hh%3Amm"`, ou `"name=hh%3Amm%3ass"` caso ele conte com segundos (veja Usando o atributo step).

Usando campos de horário

Apesar de que, entre os campos dos tipos data e horário, o de horário tenha a maior presença entre os navegadores, ele ainda não tem presença universal. Logo, é provável que você vá precisar fornecer uma maneira alternativa de se inserir data e hora, para que os usuários do Safari (e de outros navegadores sem suporte) ainda consigam inserir horários facilmente.

A seguir, vamos conferir casos de uso básicos e complexos do `<input type="time">`, além de dar conselhos sobre como contornar a questão dos navegadores na seção (see Lidando com navegadores sem suporte).

Usos básicos de horário

O caso de uso mais simples do `<input type="time">` envolve uma combinação básica dos elementos `<input>` e `<label>` como vemos abaixo:

```
<form>
  <label for="hora-cons">Escolha o horário da consulta: </label>
  <input id="hora-cons" type="time" name="hora-cons" />
</form>
```

Controlando o tamanho do campo

O elemento `<input type="time">` não é compatível com atributos de dimensionamento de formulários tais como `size`, já que horários quase sempre tem o mesmo número de caracteres. Você terá que recorrer ao CSS para ajustar tamanhos.

Usando o atributo step

Você pode usar o atributo `step` para variar a quantidade de tempo pulada sempre que o horário for incrementado/decrementado (por exemplo, para fazer com que o horário avance ou volte em 10 minutos ao clicar nas setinhas ao lado do campo).

> **Nota:** Esta propriedade pode se comportar de maneira inesperada em alguns navegadores. Por isso, ela não é 100% confiável.

O atributo recebe um valor igual ao número de segundos que você quer que o valor seja incrementado - o valor padrão é 60 segundos, ou 1 minuto. Se você especificar um valor menor que 60 segundos (1 minuto), o campo `time` vai mostrar uma área de inserção de segundos junto com as de hora e minuto:

```
<form>
  <label for="hora-cons">Escolha o horário da consulta: </label>
  <input id="hora-cons" type="time" name="hora-cons" step="2" />
</form>
```

No Chrome e no Opera, que são os únicos navegadores que mostram botões de setas para aumentar/diminuir o valor, clicar nas setinhas muda o valor em dois segundos, mas não afeta as horas e minutos. Para poder usar essa funcionalidade com minutos e horas, você precisa especificar o número de minutos (ou horas) em segundos (por exemplo, 120 para 2 minutos, ou 7200 para 2 horas).

No Firefox, não são mostrados botões de setas; logo, o valor de `step` não é usado. No entanto, adicionar esse atributo *por si só* já adiciona a área de inserção de segundos ao lado da de minutos.

O valor de `step` parece não ter efeito no Edge.

> **Nota:** Ao que parece, usar o atributo `step` faz com que a validação não funcione adequadamente (como podemos ver na seção seguinte).

Validação

Por padrão, `<input type="time">` não aplica nenhum tipo de validação nos valores inseridos (fora o próprio agente de usuário impedir que sejam inseridos valores diferentes de horários). Isso é útil (considerando que o agente de usuário seja completamente compatível com campos do tipo `time`), mas nem sempre devemos esperar que o valor desse campo seja realmente um horário, já que ele pode ser uma string vazia (`""`), o que é permitido. Também é possível que o valor se pareça com um horário válido, mas que esteja incorreto, como `"25:05"`.

Definindo horários mínimo e máximo

Você pode usar os atributos `min` e `max` para restringir a faixa de horário válida que o usuário pode escolher. No seguinte exemplo, definimos `12:00` como o horário mínimo e `18:00` como o horário máximo:

```
<form>
  <label for="hora-cons"
    >Escolha o horário da consulta (aberto das 12:00 às 18:00):
  </label>
  <input id="hora-cons" type="time" name="hora-cons" min="12:00" max="18:00" />
  <span class="validacao"></span>
</form>
```

Aqui está o CSS usado no exemplo acima. Nele, fazemos uso das propriedades CSS `:valid` e `:invalid` para estilizar o campo com base na validade ou não do conteúdo atual dele. Tivemos que colocar os ícones em um `<span>` ao lado do campo, e não no campo em si, porque no Chrome o conteúdo gerado é colocado denteo do campo do formulário e não pode ser efetivamente mostrado ou estilizado.

```
div {
  margin-bottom: 10px;
  position: relative;
}

input[type="number"] {
  width: 100px;
}

input + span {
  padding-right: 30px;
}

input:invalid + span:after {
  position: absolute;
  content: "✖";
  padding-left: 5px;
}

input:valid + span:after {
  position: absolute;
  content: "✓";
  padding-left: 5px;
}
```

O resultado disso é que:

- Apenas horários entre 12:00 e 18:00 serão vistos como válidos; horários fora dessa faixa serão tratados como inválidos.
- Dependendo do navegador usado, você pode perceber que os horários fora da faixa especificada não podem nem mesmo ser selecionados pelo seletor de hora (ex.: Edge).

Tornando campos de horário obrigatórios

Além do mais, você pode usar o atributo `required` para tornar obrigatória a inserção de um horário. Como resultado, os navegadores compatíves irão mostrar um erro se você tentar enviar um horário fora da proporção adequada ou que esteja em branco.

Vamos ver um exemplo. Nele, colocamos um horário mínimo e um máximo, além de tornarmos o campo obrigatório.

```
<form>
  <div>
    <label for="hora-cons"
      >Escolha o horário da consulta (aberto das 12:00 às 18:00):
    </label>
    <input
      id="hora-cons"
      type="time"
      name="hora-cons"
      min="12:00"
      max="18:00"
      required />
    <span class="validacao"></span>
  </div>
  <div>
    <input type="submit" value="Enviar formulário" />
  </div>
</form>
```

Se você tentar enviar o formulário com um horário incompleto (ou com um horário fora da faixa válida), o navegador vai mostrar um erro. Experimente brincar com esse exemplo agora:

Here's a screenshot for those of you who aren't using a browser that supports `time` inputs:



> **Warning: Important**: HTML form validation is *not* a substitute for scripts that ensure that the entered data is in the proper format. It's far too easy for someone to make adjustments to the HTML that allow them to bypass the validation, or to remove it entirely. It's also possible for someone to simply bypass your HTML entirely and submit the data directly to your server. If your server-side code fails to validate the data it receives, disaster could strike when improperly-formatted data is submitted (or data which is too large, of the wrong type, and so forth).

Handling browser support

As mentioned above, Safari and a few other, less common, browsers don't yet support time inputs natively. In general, otherwise, support is good — especially on mobile platforms, which tend to have very nice user interfaces for specifying a time value. For example, the `time` picker on Chrome for Android looks like this:

Browsers that don't support time inputs gracefully degrade to a text input, but this creates problems both in terms of consistency of user interface (the presented control will be different), and data handling.

The second problem is the more serious; as mentioned previously, `time` inputs' values are always normalized to the format `hh:mm` or `hh:mm:ss`. With a text input, on the other hand, by default the browser has no idea of what format the time should be in, and there multiple ways in which people write times, such as:

- `3.00 pm`
- `3:00pm`
- `15:00`
- `3 o'clock in the afternoon`

- etc.

One way around this is to put a `pattern` attribute on your `time` input. Even though the `time` input doesn't use it, the `text` input fallback will. For example, try viewing the following demo in a browser that doesn't support time inputs:

```
<form>
  <div>
    <label for="appt-time"
      >Choose an appointment time (opening hours 12:00 to 18:00):
    </label>
    <input
      id="appt-time"
      type="time"
      name="appt-time"
      min="12:00"
      max="18:00"
      required
      pattern="[0-9]{2}:[0-9]{2}" />
    <span class="validity"></span>
  </div>
  <div>
    <input type="submit" value="Submit form" />
  </div>
</form>
```

If you try submitting it, you'll see that non-supporting browsers now display an error message (and highlight the input as invalid) if your entry doesn't match the pattern `nn:nn`, where `n` is a number from 0 to 9. Of course, this doesn't stop people from entering invalid dates, or incorrectly formatted dates that follow the pattern.

Then there's the problem of the user having no idea exactly what format the time is expected to be in.

The best way to deal with times in forms in a cross-browser way, for the time being, is to get the user to enter the hours and minutes (and seconds if required) in separate controls (`<select>` elements are popular; see below for an example), or use JavaScript libraries such as the jQuery timepicker plugin.

Examples

In this example, we create two sets of interface elements for choosing times: a native picker created with `<input type="time">`, and a set of two `<select>` elements for choosing hours/minutes in older browsers that don't support the native input.

The HTML looks like so:

```
<form>
  <div class="nativeTimePicker">
    <label for="appt-time"
      >Choose an appointment time (opening hours 12:00 to 18:00):
    </label>
    <input
      id="appt-time"
      type="time"
      name="appt-time"
      min="12:00"
      max="18:00"
      required />
    <span class="validity"></span>
  </div>
  <p class="fallbackLabel">
    Choose an appointment time (opening hours 12:00 to 18:00):
  </p>
  <div class="fallbackTimePicker">
    <div>
      <span>
        <label for="hour">Hour:</label>
        <select id="hour" name="hour"></select>
      </span>
```

```
      <span>
        <label for="minute">Minute:</label>
        <select id="minute" name="minute"></select>
      </span>
    </div>
  </div>
</form>
```

The hour and minutes values for their `<select>` elements are dynamically generated.

The other part of the code that may be of interest is the feature detection code — to detect whether the browser supports `<input type="time">`, we create a new `<input>` element, set its `type` to `time`, then immediately check what its type is set to — non-supporting browsers will return `text`, because the `time` type falls back to type `text`. If `<input type="time">` is not supported, we hide the native picker and show the fallback picker UI (`<select>`s) instead.

```
// define variables
var nativePicker = document.querySelector(".nativeTimePicker");
var fallbackPicker = document.querySelector(".fallbackTimePicker");
var fallbackLabel = document.querySelector(".fallbackLabel");

var hourSelect = document.querySelector("#hour");
var minuteSelect = document.querySelector("#minute");

// hide fallback initially
fallbackPicker.style.display = "none";
fallbackLabel.style.display = "none";

// test whether a new date input falls back to a text input or not
var test = document.createElement("input");
test.type = "time";
// if it does, run the code inside the if() {} block
if (test.type === "text") {
  // hide the native picker and show the fallback
  nativePicker.style.display = "none";
  fallbackPicker.style.display = "block";
  fallbackLabel.style.display = "block";

  // populate the hours and minutes dynamically
  populateHours();
  populateMinutes();
}

function populateHours() {
  // populate the hours <select> with the 6 open hours of the day
  for (var i = 12; i <= 18; i++) {
    var option = document.createElement("option");
    option.textContent = i;
    hourSelect.appendChild(option);
  }
}

function populateMinutes() {
  // populate the minutes <select> with the 60 hours of each minute
  for (var i = 0; i <= 59; i++) {
    var option = document.createElement("option");
    option.textContent = i < 10 ? "0" + i : i;
    minuteSelect.appendChild(option);
  }
}

// make it so that if the hour is 18, the minutes value is set to 00
// — you can't select times past 18:00
function setMinutesToZero() {
  if (hourSelect.value === "18") {
    minuteSelect.value = "00";
  }
```

```
}

hourSelect.onchange = setMinutesToZero;
minuteSelect.onchange = setMinutesToZero;
```

See also

- The generic `<input>` element and the interface used to manipulate it, `HTMLInputElement`
- Date and Time picker tutorial
- `<input type="datetime-local">`, `<input type="date">`, `<input type="week">`, and `<input type="month">`

<label>

Resumo

Um **elemento** **HTML `<label>` **representa uma legenda para um item em uma interface de usuário. Ele pode estar associado com um elemento de controle, colocando este dentro do elemento `label`, ou usando o atributo `for`. Tal controle é chamado o *controle etiquetado* do elemento etiqueta. Um input pode ser associado a diversas etiquetas (<label>s).

- *As categorias de conteúdo* Flow content, phrasing content, interactive content, form-associated element, palpable content.
- *conteúdo permitido* Phrasing content, mas há elementos do rótulo descendente. Nenhum outro do que o controle rotulado elementos labelable são permitidos.
- *Omissão de tag* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos pai permitidas* Qualquer elemento que aceita phrasing content.
- *DOM interface* `HTMLLabelElement`

Atributos

A tecla de atalho para acessar este elemento a partir do teclado.

`accesskey`

A tecla de atalho para acessar este elemento a partir do teclado.

`for`

O ID de um elemento de formulário relacionados com labelable no mesmo documento como o elemento label. O primeiro elemento tal no documento com uma ID correspondente ao valor do atributo é o controle marcado for este elemento etiqueta.

> **Nota:** Um elemento etiqueta pode ter tanto um for o atributo e um elemento de controlo continham, enquanto os pontos de atributo para o elemento de controlo contido.

`form`

O elemento de forma que o elemento label está associado a (seu proprietário formulário). O valor do atributo deve ser uma identificação de um `<form>` elemento no mesmo documento. Se este atributo não for especificado, este elemento <label> deve ser um descendente de uma `<form>` elemento. Este atributo permite que você coloque elementos do rótulo em qualquer lugar dentro de um documento, e não apenas como descendentes de seus elementos de formulário.

Exemplo

Exemplo simples de label

```
<label>Click me <input type="text" id="User" name="Name" /></label>
```

Usando o atributo "for"

```
<label for="User">Click me</label> <input type="text" id="User" name="Name" />
```

See also

- Other form-related elements: `<form>`, `<button>`, `<datalist>`, `<legend>`, `<select>`, `<optgroup>`, `<option>`, `<textarea>`, `<fieldset>`, `<output>`, `<progress>` and `<meter>`.

<legend>

Resumo

O **Elemento HTML <legend> **(ou *Elemento HTML Campo "Legend"*) representa um rótulo para o conteúdo do seu ancestral `<fieldset>`.

- *Categorias de Conteúdo* Nenhuma.
- *Conteúdo permitido* Conteúdo Textual.
- *Omissão de Tag* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos ancestrais permitidos* Um `<fieldset>` cujo primeiro filho seja esse elemento <legend>
- *Interface DOM* `HTMLLegendElement`

Atributos

Esse elemento apenas inclui os atributos globais.

Exemplos

Veja `<form>` para exemplos sobre `<legend>`.

Veja Também

- Outros elementos relacionados a formulários: `<form>`, `<option>`, `<label>`, `<button>`, `<select>`, `<datalist>`, `<optgroup>`, `<fieldset>`, `<textarea>`, `<input>`, `<output>`, `<progress>` and `<meter>`.

Resumo

O ***elemento HTML*** **`<li>`** (ou a Lista dos Itens de um elemento HTML) é usado para representar um item que faz parte de uma lista. Este item deve estar contido em um elemento pai: uma lista ordenada (`<ol>`), uma lista desordenada (`<ul>`) , ou um menu (`<menu>`) e representa uma única entidade dessa lista. Em menus e listas desordenadas a relação de itens é exibida, normalmente, usando pontos de marcação (as bolinhas). Em listas ordenadas eles são, comumente, mostrados com algum contador ascendente - como um número, ou letra - à sua esquerda.

- *Categorias de conteúdo* Nenhuma.
- *Conteúdo permitido* Flutuantes.
- *Omissão de etiquetas (tags)* A etiqueta de fim pode ser omitida se esta for imediatamente seguida por um outro elemento `<li>`, ou se não houver mais conteúdo no seu elemento pai.
- *Elementos pai permitidos* Um elemento `<ul>`, `<ol>`, ou `<menu>`. Embora seja um uso incomum, o obsoleto `<dir>` pode ser pai, também.
- *Interface DOM* `HTMLLIElement`

Atributos

Este elemento inclui os atributos globais.

`value`

> Este atributo de número inteiro indica o valor ordinal atual do item na lista, definido pelo elemento `<ol>`. O único valor possível para este atributo é um número, ainda que a lista seja exibida com algarismos romanos, ou letras. A lista de itens que virá em seguida continuará a ser numerada a partir desta posição. O atributo **value** não tem significado para listas desordenadas (`<ul>`), nem para menus (`<menu>`).

> **Nota:** Este atributo, abandonado na HTML4, foi reintroduzido na HTML5.

> **Nota:** Antes de Gecko 9.0, os valores negativos eram, incorretamente, convertidos a 0. A partir de Gecko 9.0 todos os valores inteiros são analisados corretamente.

`type`

A característica deste atributo é indicar o tipo de numeração:

- `a`: letras minúsculas
- `A`: letras maiúsculas
- `i`: algarismos romanos minúsculos
- `I`: algarismos romanos maiúsculos
- `1`: números

Este tipo substitui o utilizado pelo seu elemento pai `<ol>`, qualquer que seja.

> **Note: Nota de utilização:** Este atributo foi preterido. Em seu lugar use a propriedade CSS `list-style-type`.

Exemplos

```
<ol>
  <li>primeiro item</li>
  <li>segundo item</li>
  <li>terceiro item</li>
</ol>
```

A saída HTML acima será:

1. primeiro item
2. segundo item
3. terceiro item

```
<ul>
  <li>primeiro item</li>
  <li>segundo item</li>
  <li>terceiro item</li>
</ul>
```

- primeiro item
- segundo item
- terceiro item

Para exemplos mais detalhados veja as páginas `<ol>` e `<ul>`.

Veja também

- Outros elementos HTML parentes: `<ul>`, `<li>`, `<menu>` e o obsoleto `<dir>`;
- Propriedades CSS que podem ser úteis, especialmente para dar um charme ao elemento `<li>`:
  - a propriedade `list-style`, para escolher a maneira como os posicionamentos serão exibidos,
  - contadores CSS para controlar listas aninhadas complexas,
  - a propriedade `margin`, para controlar a indentação dos itens da lista.

<link>

Resumo

O **Elemento HTML *<link>*** especifica as relações entre o documento atual e um recurso externo. Possíveis usos para este elemento incluem a definição de uma estrutura relacional para navegação. Este elemento é mais usado para vincular as folhas de estilo.

- *Categorias de conteúdo* Metadata content. If `itemprop` is present: flow content phrasing content

- *Conteúdo permitido* None, it is an empty element.
- *Omissão de tag* As it is a void element, the start tag must be present and the end tag must not be present
- *Elementos pai permitidos* Any element that accepts metadata elements. If `itemprop` is present: any element that accepts phrasing content.
- *Interface de DOM* `HTMLLinkElement`

Atributos

Este elemento inclui os atributos globais.

`as`

Esse atributo é usado apenas quando `rel="preload"` foi enviado no elemento `<link>`. Especifica o tipo de conteúdo carregado pelo `<link>`, que é necessário para a priorização do conteúdo, solicitação de correspondência, aplicação de política de segurança de conteúdo correta, e configuração do cabeçalho de pedido `Accept` correta.

`charset`

Este atributo define a codificação de caracteres do recurso vinculado. O valor é umespaço e/ou lista delimitada por vírgulas de conjuntos de caracteres, conformedefinido na RFC 2045. O valor padrão é ISO-8859-1.

> **Nota:** Este atributo é obsoleto em HTML5 e **não deve ser usada por autores**. Para atingir seu efeito, use o cabeçalho HTTP Content-Type sobre o recurso vinculado.

`crossorigin`

Este atributo enumerado indica se a busca da imagem relacionada deve ser feita usando CORS ou não. I-magens de CORS habilitado podem ser reutilizadas no elemento `<canvas>` sem ser conta-minado. Os valores permitidos são:

"anonymous"

A cross-origin request (i.e. with `Origin:` HTTP header) is performed. But no credential is sent (i.e. no cookie, no X.509 certificate and no HTTP Basic authentication is sent). If the server does not give credentials to the origin site (by not setting the `Access-Control-Allow-Origin:` HTTP header), the image will be *tainted* and its usage restricted.

use-credentials

A cross-origin request (i.e. with `Origin:` HTTP header) is performed with credential is sent (i.e. a cookie, a certificate and HTTP Basic authentication is performed). If the server does not give credentials to the origin site (through `Access-Control-Allow-Credentials:` HTTP header), the image will be *tainted* and its usage restricted.

When not present, the resource is fetched without a CORS request (i.e. without sending the `Origin:` HTTP header), preventing its non-tainted used in `<canvas>` elements. If invalid, it is handled as if the enumerated keyword **anonymous** was used. See CORS settings attributes for additional information.

`disabled`

Este atributo é usado para desativa uma relação com o link. Em conjunto com o script, esse atributo poderia ser usado para ligar e desligar várias relações com stylesheets.

> **Nota:** Embora, não haja nenhum atributo desativado no padrão HTML, há um atributo desabilitado no `HTMLLinkElement` DOM object.
>
> The use of `disabled` as an HTML attribute is non-standard and only used by some browsers (W3 #27677). **Do not use it**. To achieve a similar effect, use one of the following techniques:
>
> - If the `disabled` attribute has been added directly to the element on the page, do not include the `<link>` element instead;
> - Set the `disabled` **property** of the `StyleSheet` DOM object via scripting.

`href`

Esse atributo especifica a URL do recurso vinculado. Uma URL pode ser absolute or relative.

`hreflang`

Esse atributo indica o idioma do recurso vinculado. É meramente consultivo. Os valores permitidos são determindaos por BCP47 para HTML5 e por RFC1766 para HTML 4. Use esse atributo se apenas os atributos `href` estiverem presentes.

`media`

This attribute specifies the media which the linked resource applies to. Its value must be a media query. This attribute is mainly useful when linking to external stylesheets by allowing the user agent to pick the best adapted one for the device it runs on.

> **Nota:** In HTML 4, this can only be a simple white-space-separated list of media description literals, i.e., media types and groups, where defined and allowed as values for this attribute, such as `print`, `screen`, `aural`, `braille`. HTML5 extended this to any kind of media queries, which are a superset of the allowed values of HTML 4.
>
> - Browsers not supporting the CSS3 Media Queries won't necessarily recognize the adequate link; do not forget to set fallback links, the restricted set of media queries defined in HTML 4.

`methods`

The value of this attribute provides information about the functions that might be performed on an object. The values generally are given by the HTTP protocol when it is used, but it might (for similar reasons as for the **title** attribute) be useful to include advisory information in advance in the link. For example, the browser might choose a different rendering of a link as a function of the methods specified; something that is searchable might get a different icon, or an outside link might render with an indication of leaving the current site. This attribute is not well understood nor supported, even by the defining browser, Internet Explorer 4. See Methods Property (MSDN).

`rel`

This attribute names a relationship of the linked document to the current document. The attribute must be a space-separated list of the link types values. The most common use of this attribute is to specify a link to an external style sheet: the **rel** attribute is set to `stylesheet`, and the **href** attribute is set to the URL of an external style sheet to format the page. WebTV also supports the use of the value `next` for **rel** to preload the next page in a document series.

`rev`

The value of this attribute shows the relationship of the current document to the linked document, as defined by the `href` attribute. The attribute thus defines the reverse relationship compared to the value of the **rel** attribute. Link types values for the attribute are similar to the possible values for `rel`.

> **Nota:** This attribute is obsolete in HTML5. **Do not use it**. To achieve its effect, use the `rel` attribute with the opposite link types values, e.g. made should be replaced by author. Also this attribute doesn't mean *revision* and must not be used with a version number, which is unfortunately the case on numerous sites.

`sizes`

This attribute defines the sizes of the icons for visual media contained in the resource. It must be present only if the `rel` contains the icon link types value. It may have the following values:

- `any`, meaning that the icon can be scaled to any size as it is in a vector format, like `image/svg+xml`.
- a white-space separated list of sizes, each in the format `<width in pixels>x<height in pixels>` or `<width in pixels>X<height in pixels>`. Each of these sizes must be contained in the resource.

> **Nota:** Most icon formats are only able to store one single icon; therefore most of the time the `sizes` attribute contains only one entry. MS's ICO format does, as well as Apple's ICNS. ICO is more ubiquitous, so you should use this format if cross-browser support is a concern (especially for old IE versions).

`target`

Defines the frame or window name that has the defined linking relationship or that will show the rendering of any linked resource.

`type`

This attribute is used to define the type of the content linked to. The value of the attribute should be a MIME type such as **text/html**, **text/css**, and so on. The common use of this attribute is to define the type of style sheet linked and the most common current value is **text/css**, which indicates a Cascading Style Sheet format.

Exemplos

Incluindo um stylesheet

Para incluir um stylesheet em uma página, use a seguinte syntax:

```
<link href="style.css" rel="stylesheet" />
```

Fornecimento de stylesheets alternativas

Você também pode especificar stylesheet alternativas.

O usuário pode escolher qual stylesheet usar, escolhendo no View>Page Style menu. Isso fornece uma maneira para os usuários verem várias versões de uma página.

```
<link href="default.css" rel="stylesheet" title="Default Style" />
<link href="fancy.css" rel="alternate stylesheet" title="Fancy" />
<link href="basic.css" rel="alternate stylesheet" title="Basic" />
```

Stylesheet eventos carregados

Você pode determinar quando um stylesheet foi carregado observando um `load` event em ação; Da mesma forma, você pode detectar se ocorreu algum erro ao processar uma stylesheet, observando um `error` event:

```
<script>
  function sheetLoaded() {
    // Do something interesting; the sheet has been loaded
  }

  function sheetError() {
    alert("An error occurred loading the stylesheet!");
  }
</script>

<link
  rel="stylesheet"
  href="mystylesheet.css"
  onload="sheetLoaded()"
  onerror="sheetError()" />
```

> **Nota:** O `load` event é carregado quando a stylesheet e quando todo o conteúdo importado foi carregado e analisado, e imediatamente antes que styles comecem a ser aplicados ao conteúdo.

Notas

- Uma tag `<link>` pode ocorrer apenas no elemento head; No entanto, pode haver várias ocorrências de `<link>`.
- HTML 3.2 define apenas os atributos **href**, **rel**, **rev**, e **title** para o elemento link.
- HTML 2 define os atributos **href**, **methods**, **rel**, **rev**, **title**, e **urn** para o elemento `<link>`. O **methods** e **urn** foram atributos posteriormente removidos das especificações.
- As especificações HTML e XHTML definem manipulações de eventos para o elemento `<link>`, mas não está claro como eles seriam usados.
- Sob XHTML 1.0, elementos vazios como `<link>` requerem uma barra diagonal: `<link />`.

Veja também

- Ryan Grove's <script> and <link> node event compatibility chart

<main>

Resumo

O elemento **<main>** define o conteúdo principal dentro do <body> em seu documento ou aplicação. Entende-se como conteúdo principal aquele relacionado diretamente com o tópico central da página ou com a funcionalidade central da aplicação. O mesmo deverá ser único na página, ou seja, dentro do elemento <main> não deverão ser incluidas seções da página que sejam comuns a todo o site ou aplicação, tais como mecanismos de navegação, informações de copyright, logotipo e campos de busca (a não ser, é claro, caso a função principal do documento seja fazer algum tipo de busca).

> **Nota:** <main> **não pode** ser filho dos elementos <article>, <aside>, <footer>, <header>, ou <nav>.
>
> Autores **não devem** incluir mais de um elemento main no mesmo documento**.**

- *Categorias de conteúdo*Conteúdo de fluxo, conteúdo palpável.
- *Conteúdo permitido* Conteúdo de fluxo.
- *Omissão de tag* Nenhuma; ambas as tags de início e fim são obrigatórios.
- *Elementos pais permitidos* Qualquer elemento que aceite conteúdo de fluxo, mas não pode ser filho dos elementos <article>, <aside>, <footer>, <header>, ou <nav>.
- *DOM interface* HTMLElement

Atributos

Este elemento admite apenas os atributos globais.

Exemplo

```
<!-- outro conteúdo -->

<main>
  <h1>Maçãs</h1>
  <p>A maçã é a fruta pomácea da macieira.</p>

  <article>
    <h2>Vermelho delicioso</h2>
    <p>
      Estas maçãs vermelhas brilhantes são as mais comumente encontradas em
      muitos supermercados.
    </p>
    <p>...</p>
    <p>...</p>
  </article>

  <article>
    <h2>Granny Smith</h2>
    <p>
      Essas suculentas maçãs verdes, são um ótimo recheio para torta de maçã.
    </p>
    <p>...</p>
    <p>...</p>
  </article>
</main>

<!-- outro conteúdo -->
```

O elemento <main> é amplamente suportado (exceto pelo Internet Explorer). Sugere-se que, até que o elemento <main> seja suportado pelo Internet Explorer, adicione a ele o atributo role com o valor "main":

```
<main role="main">...</main>
```

Veja também

- Elementos estruturais básicos: `<html>`, `<head>`, `<body>`
- Elementos relacionados a seções: `<article>`, `<aside>`, `<footer>`, `<header>`, or `<nav>`

<map>

Resumo

O **elemento HTML** `<map>` é usado com os elementos `<area>` para definir um mapa de imagem (a área clicável do link).

- *Categorias de conteúdo* Conteúdo de fluxo, conteúdo fraseado, conteúdo palpável.
- *Conteúdo permitido* Qualquer elemento transparente.
- _Omissão da marcação_Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos pai permitidos* Qualquer elemento que aceita conteúdo fraseado.
- *Interface DOM* `HTMLMapElement`

Atributos

Este elemento inclui os atributos globais.

`name`

> O atributo name dá ao mapa de um nome, de modo que ela possa ser referenciada. O atributo deve estar presente e ter um valor não vazio, sem caracteres de espaço. O valor do atributo name não deve corresponder (independente da caixa) a um valor do atributo name de outro elemento no mesmo documento. Se o id de atributo também for especificado, ambos os atributos devem ter o mesmo valor.

Exemplos

```
<map>
  <area shape="circle" coords="200,250,25" href="another.htm" />
  <area shape="default" />
</map>
```

Veja também

- `<a>`
- `<area>`

<mark>

Sumário

O **Elemento HMTL `<mark>` **representa um trecho de destaque em um texto, por exemplo, uma sequência de texto marcado como referência, devido à sua relevância em um contexto particular. Por Exemplo, pode ser utilizado em uma página que mostra os resultados de uma busca onde todas as instâncias da palavra pesquisadas receberam destaque.

> **Note:** *Notas de Uso:*
>
> - Em uma citação ou outro tipo de bloco, O texto destacado normalmente marca o texto que é referenciado fora da citação, ou marcado para um analise específica, mesmo que o autor original não considerou importante.
> - No texto principal, o trecho destacado normalmente marca um conteúdo que pode ser de especial relevância para usuário corrente, como resultado de pesquisa.
> - Não use o o elemento `<mark>` para destaque de sintaxe; uso o elemento `<span>` para essa finalidade.
> - Não confunda o elemento `<mark>` com o elemento `<strong>` . O elemento `<strong>` é usado para denotar trechos de importância especial, quando o elemento `<mark>` é utilizado para denotar trechos de relevância especial.

- *Categorias de conteúdo* Conteúdo de Fluxo, phrasing content, palpable content.

- *Conteúdo Permitido* Phrasing content.
- *Omissão de Tag* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos parent permitidos* Qualquer elemento que aceite phrasing content.
- *DOM interface* `HTMLElement`

Atributos

Este elemento inclue apenas atributos Globais.

Exemplo

```
<p>
  O elemento &lt;mark&gt; é usado para <mark>destacar</mark> partes do texto
</p>
```

Veja Também

- Outros text-level semantics elements: `<a>`, `<em>`, `<strong>`, `<cite>`, `<q>`, `<dfn>`, `<abbr>`, `<time>`, `<code>`, `<var>`, `<samp>`, `<kbd>`, `<sub>`, `<sup>`, `<i>`, `<b>`, `<mark>`, `<ruby>`, `<rp>`, `<rt>`, `<bdo>`, `<span>`, `<br>`, `<wbr>`.

<marquee>

O elemento html <marquee> é usado para inserir uma área de rolagem de texto"scrolling" , similar a um letreiro. Você pode controlar o comportamento do conteúdo fornecendo alguns atributos extras.

Atributos

`behavior`

> Define como o texto é rolado dentro da área do letreiro. Os valores possíveis são scroll, slide e alternate. Se nenhum valor for especificado, o valor padrão será scroll.

`bgcolor`

> Define a cor do plano de fundo do letreiro através do nome da cor (ex: red, blue) ou de um valor hexadecimal.

`direction`

> Define a direção da rolagem do texto dentro do letreiro, os possíveis valores são: left, right, up & down. Se nenhum valor for especificado, o valor padrão será "left".

`height`

> Define a altura do letreiro em pixeis ou em um valor percentual.

`hspace`

> Aplica a margem horizontal.

`loop`

> Define o número de repetições da animação do letreiro. Se nenhum valor for especificado assumirá o valor padrão de -1, que significa que a animação será repetida infinitamente.

`scrollamount`

> Define em pixeis o tamanho de rolagem em cada intervalo, o valor padrão é 6.

`scrolldelay`

> Define o intervalo de tempo entre cada animação de rolagem em milissegundos. O valor padrão é 85. Qualquer valor menor que 60 será ignorado e o valor 60 será usado, a menos que seja especificado como truespeed.

`truespeed`

Por padrão, valores abaixo de 60 milissegundos são ignorados, a menos que o valor truespeed esteja presente, caso esteja estes valores são aceitos.

`vspace`

Aplica uma margem vertical em pixeis ou em valor percentual.

`width`

Define a largura em pixeis ou em um valor percentual.

Event Handlers

`onbounce`

Dispara quando o letreiro alcança o final da sua posição de rolagem. Ele apenas dispara quando o comportamento está configurado como `alternate`.

`onfinish`

Dispara quando o letreiro terminar a quantidade de repetições definida pelo atributo loop. Só pode disparar quando o atributo loop estiver definido para algum número maior que 0, obviamente.

`onstart`

Dispara quando o letreiro começa a se mover.

Métodos

`start()`

Começa a mover o letreiro.

`stop()`

Para de mover o letreiro.

Exemplos

```
<marquee>Este texto vai "rolar" da direita para esquerda</marquee>

<marquee direction="up">Este texto vai rolar de baixo para cima.</marquee>

<marquee direction="down" width="250" height="200" behavior="alternate" style="border:solid">
  <marquee behavior="alternate">
    Este texto vai pular
  </marquee>
</marquee>
```

<meta>

O elemento **HTML `<meta>` **define qualquer informação de metadados que não podem ser definidos por outros elementos **HTML.** (`<base>`, `<link>`, `<script>`, `<style>` ou `<title>`).

Categoria de conteúdo	Conteúdo de metadado. Se o `itemprop` atributo estiver presente: flow content, phrasing content.
Conteúdo permitido	Nenhum, é um elemento vazio.
Omissão de tag	Como é um elemento vazio, a marcação inicial deve estar presente e a de fechamento não.
Elementos pai permitidos	`<meta charset>`, `<meta http-equiv>`: um elemento `<head>`. Se o `http-equiv` não for uma declaração de codificação, ele pode estar dentro de um elemento `<noscript>`, dentro de um elemento `<head>`.

Papéis ARIA permitid	Nenhum
Interface DOM	`HTMLMetaElement`

Atributos

Esse elemento inclui os atributos globais.

> **Nota:** o atributo global `name` tem um significado específico para o elemento `<meta>`, e o atributo `itemprop` não deve ser definido no mesmo elemento`<meta>` que tem algum desses atributos existentes: `name`, `http-equiv` ou `charset`.

`charset`

Este atributo define a codificação de caracteres usada na página. Pode ser substituído localmente usando o atributo *lang* em qualquer elemento. Esse atributo é literalmente uma amarra e deve ser um dos *MIME names* preferidos para uma codificação de caracteres como definido pela IANA (defined by the IANA). Embora o padrão não solicite uma codificação específica, ele dá algumas sugestões:

- Autores são encorajados a usar UTF-8.
- Autores não devem usar codificações incompatíveis com ASCII (isto é, aqueles que não mapeiam os pontos de código de 8 bits 0x20 a 0x7E para os pontos de código Unicode 0x0020 a 0x007E), pois representam um risco de segurança: navegadores que não os suportam podem interpretar conteúdo maligno como Elementos HTML. Esse é o caso de pelo menos os seguintes caracteres: JIS_C6226-1983, JIS_X0212-1990, HZ-GB-2312, JOHAB, a família ISO-2022 e a família EBCDIC.
- Autores não devem usar CESU-8, UTF-7, BOCU-1 e SCSU, also falling in that category and not intended to be used on the web. Cross-scripting attacks with some of these encodings have been documented.
- Autores não devem usar UTF-32 pois nem todos algorítimos de codificação HTML5 conseguem distingui-lo do UTF-16.

> **Nota:**
>
> - O conjunto de caracteres declarado deve corresponder ao da página. Não há motivo válido para declarar um conjunto de caracteres imprecisos.
> - Esse elemento `<meta>` deve estar dentro do elemento `<head>` e dentro dos primeiros 1024 bytes da página, pois alguns navegadores só olham para esses primeiros bytes antes de escolher um caractere definido para a página.
> - Esse elemento `<meta>` é apenas uma parte do algoritmo para determinar o conjunto de caracteres de uma página que os navegadores aplicam. O cabeçalho HTTP Content-Type e quaisquer elementos da BOM têm precedência sobre esse elemento.
> - É uma boa prática, e altamente recomendável, definir o conjunto de caracteres usando este atributo. Se nenhum conjunto de caracteres estiver definido para uma página, várias técnicas de scripts cruzados poderão prejudicar o usuário da página, como a técnica de cross-scripting de fallback UTF-7. Sempre definindo esta meta irá proteger contra esses riscos.
> - Esse elemento `<meta>` é um sinônimo para o pre-HTML5 `<meta http-equiv="Content-Type" content="text/html; charset=IANAcharset">` onde o atributo `IANAcharset` corresponde ao valor equivalente de `charset`. Essa sintaxe ainda é permitida, embora obsoleta e não mais recomendada.

`content`

Esse atributo fornece o valor associado ao atributo `http-equiv` ou `name`, dependendo do contexto.

`http-equiv`

Este enumerado atributo define a pragma isso pode alterar o comportamento de servers e user-agents. o valor do pragma é definido usando `content` e pode ser um dos seguintes:

`"content-language"`

este pragma define a linguagem default da página

> **Nota:** não use este pragma, ele esta obsoleto. use o atributo global `lang` no `<html>` ao invés deste.

`"Content-Security-Policy"`

Este valor permite os administradores do web site definam uma política de conteúdo para a página atual. Com algumas exceções, politicas de conteúdo envolvem especificar origens de servidores e endpoints de scripts permitidos, isso ajuda na defesa de cross-server scripting attacks.

`"content-type"`

Esse atributo define o MIME type e o conjunto de caracteres do documento. Isso segue a mesma sintaxe como o HTTP `content-type` entity-header field, mas isto esta dentro de um elemento HTML, a maioria dos valores não é possível. Sendo assim a sintaxe válida para este conteúdo é a literal string '`text/html`' eventualmente seguido por estes caracteres com a seguinte sintaxe: '`; charset=`*`IANAcharset`*' onde `IANAcharset` é o *MIME preferido nome para um conjunto de caracteres como* definido pela IANA.

> **Nota:**
>
> - Nao use esta pragma ela esta obsoleta. use `charset` atributo `<meta>` element instead.
> - como o `<meta>` pode nao ser usado para mudar o tipo de documento no XHTML, ou em um documento de HTML5 seguindo uma syntax de XHTML, nunca marque MIME type para um XHTML MIME type desta forma. isso sera incoerente.
> - somente um documento HTML pode usar o content-type, então a maioria disto é redundante: isso porque esta obsoleto e trocado pelo atributo `charset`.

`"default-style"`

Este pragma preferencia de stylesheet a ser usado na página. o atributo `content` deve conter o **title** de um `<link>` elemento de quem `href` atributo link um CSS stylesheet, ou um **title** de um elemento `<style>` do qual contem um CSS stylesheet.

`"refresh"`

este pragma especifica:

- o numero de segundos até a página ser recarregada, se o atributo `content` contém apenas um número inteiro não negativo;
- o número de segundos até a página ser redirecionada para outro lugar, se o atributo `content` contém um inteiro não negativo seguido de uma string '`;url=`' e uma URL válida.

`"set-cookie"`

este pragma define um cookie para a página. este conteúdo deve seguir a sintaxe definida em IETF HTTP Cookie Specification.

> **Nota:** não use este pragma está obsoleto. Use HTTP header set-cookie instead.

`name`

Este atributo define o nome do document-level metadata. Isso não deve ser marcado se um dos atributos `itemprop`, `http-equiv` ou `charset` já estiver preparado. este document-level metadata name é associado a um valor, contido pelo atributo `content`. os possíveis valores para o elemento name são, com seu valor associado, guardado via `content` attribute:

- `application-name`, define o nome da aplicação que esta rodando na página;

> **Nota:**

    - Browsers podem usar isso para identificar a aplicação. isso é diferente do elemento `<title>`, que geralmente constituí no nome da aplicação, mas também contém informações específicas como o nome do documento ou status;
    - Webpages simples não deveriam definir application-name meta.

- `author`, definindo, em formato livre, o nome do author do documento;
- `description`, contém uma curta e precisa descrição do conteúdo da página. vários browsers, como o Firefox e o Opera, usam este meta como descrição padrão da página quando é marcada.
- `generator`, contendo, em um formato livre, o identificador do software que gerou a página;
- `keywords`, contendo, como strings separadas por vírgula, palavras relevantes associadas ao conteúdo da página;
- `referrer` controlando o conteúdo de um HTTP `Referer` HTTP header anexado a qualquer pedido enviado deste documento:

`no-referrer`	**não envia um HTTP `Referer` header.**
`origin`	envia a origin de um documento.
`no-referrer-when-downgrade`	Envia a origem como referenciador para um destino prioritariamente tão seguro (https-> https), mas não envie um referenciador para um destino menos seguro (https-> http). Este é o comportamento padrão.
`origin-when-crossorigin`	Envia uma URL completa (sem parâmetros) ao executar uma solicitação de mesma origem, mas envie apenas a origem do documento para outros casos.
`unsafe-URL`	Envia um URL completo (sem parâmetros) ao executar uma solicitação de mesma origem ou origem cruzada.

> **Nota:** alguns browsers suportam keywords `always`, `default`, e `never` para referenciar. estes valores estão descontinuados.

> **Nota:** dinamicamente inseridos `<meta name="referrer">` (por document.write ou appendChild) cria um nao-determinismo quando isso vem para enviar referências ou não. Note também quando muitas politicas conflitantes são definidas, o No-referrer politia é aplicada.

o atributo também pode ter um valor retirado de uma extensa lista definida em WHATWG Wiki MetaExtensions page. Embora nenhum tenha sido formalmente aceito ainda, alguns nomes comumente usados estão entre as propostas:

- `creator`, definindo, em um formato livre, o nome do criador do documento. Note que também pode ser o nome de uma instituição. se há mais de uma, vários elementos `<meta>` podem ser usados;
- `googlebot`, é um sinônimo de `robots`, mas só é seguido por Googlebot, o indexador crawler do Google;
- `publisher`, definido, em um formato livre, o nome do editor do documento. Note que também pode ser o nome de uma instituição;
- `robots`, definindo o comportamento que crawlers devem ter com a página. É separado por vírgula a lista de valores seguintes:

Value	Description	Used by
`index`	permite o robô indexar uma página	All
`noindex`	previne o robô de indexar uma página	All
`follow`	permite o robô a seguir os links da página	All
`nofollow`	previne o robô a seguir os links da página	All
`noodp`	previne o uso de descrição Open Directory Project, caso existam, como descrição da página o resultado na ferramenta de busca.	Google, Yahoo, Bing

Value	Description	Used by
`noarchive`	previne a ferramenta de busca pecar o conteúdo da página	Google, Yahoo
`nosnippet`	Impede a exibição de qualquer descrição da página na página de resultados do mecanismo de pesquisa	Google
`noimageindex`	previne esta página de aparecer como referencia de alguma imagem indexada	Google
`nocache`	sinônimo de `noarchive`	Bing

> **Nota:**
>
> - Somente robôs cooperativos seguirão as regras definidas pelo nome do robô. Não espere manter as colheitadeiras de e-mail à distância com isso.
> - O robô ainda precisa acessar a página para ler o valor meta. Se você quiser mantê-los sob controle, por exemplo, para evitar o consumo de largura de banda, use um arquivo robots.txt (ou em complemento).
> - Se você quiser remover a página de um índice, a alteração do meta para `noindex` funcionará, mas somente quando o robô visitar a página novamente. Verifique se o arquivo robots.txt não está impedindo essas visitas. Alguns mecanismos de busca possuem ferramentas para desenvolvedores, permitindo uma remoção rápida de algumas páginas.
> - Alguns valores possíveis são mutuamente exclusivos, como usar `index` e `noindex`, ou `follow` e `nofollow`, ao mesmo tempo. Nestes casos, o comportamento do robô é indefinido e pode variar de um para o outro. Então evite esses casos.
> - Alguns robôs rastreadores de mecanismos de pesquisa, como os do Google, Yahoo Search ou Bing, suportam os mesmos valores em uma diretiva `HTTP, X-Robot-Tags`: isso permite que eles usem esses pragma em documentos não HTML, como imagens

- `slurp`, qual é um sinônimo de `robots`, mas somente seguido por Slurp, o indexador crawler para Yahoo Search;

Finalmente, alguns nomes estão em uso comum, embora não estejam sendo padronizados:

- `viewport`, que dá dicas sobre o tamanho inicial do viewport. Este pragma é usado apenas por vários dispositivos móveis.

Value	Possible values	Description
`width`	um numero inteiro positivo ou o literal `device-width`	define a largura em pixel de um viewport
`height`	um numero inteiro positivo ou o literal `device-height`	define a altura em pixel de um viewport
`initial-scale`	um numero positivo entre `0.0` e `10.0`	define a relação entre a largura do dispositivo (largura do dispositivo no modo retrato ou altura do dispositivo no modo paisagem) e o tamanho da janela de visualização.
`maximum-scale`	um numero positivo entre `0.0` e `10.0`	define o valor máximo de zoom; deve ser maior ou igual ao `minimum-scale` ou o comportamento sera indeterminado.
`minimum-scale`	um numero positivo entre `0.0` e `10.0`	define o valor mínimo do zoom; deve ser menor ou igual ao `maximum-scale` ou o comportamento sera indeterminado
`user-scalable`	um valor booleano (`yes` ou `no`)	se setado `no`, o usuário não poderá usar zoom na página. o valor padrão é `yes`.

> **Nota:**

> - Embora não seja padronizado, esse atributo é usado por diferentes navegadores móveis, como o Safari Mobile, o Firefox para celular ou o Opera Mobile.
> - Os valores padrão podem mudar de um dispositivo e navegador para outro.
> - Para aprender sobre esse pragma no Firefox para celular, veja este artigo.

`scheme`

Este atributo define o esquema no qual os metadados são descritos. Um esquema é um contexto que leva às interpretações corretas dos `content` valores, como um formato.

> **Nota:** Não use este atributo pois esta obsoleto.Não há substituto para isso, pois não houve uso real para isso. Omitir completamente.

Notes

Dependendo do conjunto escolhido, o tipo de metadata pode ser uma das seguintes:

- se `name` é definido, isto é *document-level metadata*, aplicando para a página inteira.
- se `http-equiv` é definido, isto é um *pragma diretiva*, i.e. normalmente informação dada pelo web server sobre como a página web deve ser entregue.
- se `charset` é definido, isto é uma declaração de *charset*, i.e. o charset usado para o formulário serializado da página da Web
- se `itemprop` é definido, isto é *user-defined metadata*, transparente para o user-agent, pois a semântica dos metadados é específica do usuário.

Exemplos

```
<!-- In HTML5 -->
<meta charset="utf-8" />

<!-- redireciona a página depois de 3 segundos -->
<meta http-equiv="refresh" content="3;url=https://www.mozilla.org" />
```

See also

- The other elements containing metadata: `<base>`, `<head>`, `<link>`, `<style>`,`<title>`.

meter

Sumário

O elemento HTML *meter* (`<meter>`) pode representar um valor escalar dentro de um intervalo conhecido ou um valor fracionário.

> **Nota:** A não ser que o atributo **value** esteja entre 0 e 1(inclusive), o atributo **min** e o atributo **max** devem definir o intervalo de modo que o valor do atributo **value** esteja dentro dele.

Contexto de uso

Content categories	**flow content**, **phrasing content**, **Labelable Form-associated content**
Conteúdo permitido	Phrasing content, mas nenhum elemento `meter` descendente.
Tag omission	Nenhuma, ambas as tags iniciais e finais são obrigatórias.
Elementos pais permitidos	Qualquer elemento que aceite phrasing content.
Documento normativo	HTML5, section 4.10.8

Atributos

Como todos os elementos HTML, esse elemento suporta attributes.

`value`

O valor numérico atual. Ele deve estar entre os valores mínimos e máximo (o atributo **min** e o atributo **max**) se eles estiverem especificados. Se não especificado ou mal formatado, o valor é 0. Se especificado, mas fora do intervalo dado pelos atributos **min** e **max**, o valor é igual ao extremo do intervalo mais próximo.

`min`

O limite numérico mínimo do intervalo medido. Deve ser menor que o valor máximo (o atributo **max**), se especificado. Se não especificado, o valor mínimo é 0.

`max`

O limite numérico máximo do intervalo medido. Deve ser maior que o valor mínimo (o atributo **min**), se especificado. Se não especificado, o valor máximo é 1.

`low`

O limite numérico máximo da parte inferior do intervalo medido. Deve ser maior que o valor mínimo (o atibuto **min**), e também ser menor que o valor alto e o valor máximo (os atributos **high** e **max**, respectivamente), se estiver especificado. Se não especificado, ou se for menor que o valor mínimo, o valor de **low** é igual ao valor mínimo.

`high`

O limite numérico mínimo da parte superior do intervalo medido. Deve ser menor que o valor máximo (o atibuto **max**), e também ser maior que o valor baixo e o valor mínimo (os atributos **low** e **min**, respectivamente), se estiver especificado. Se não especificado, ou se for maior que o valor máximo, o valor de **high** é igual ao valor máximo.

`optimum`

Esse atributo indica o valor numérico ótimo. Deve estar dentro do intervalo (definido pelos atributos **min** e **max**). Quando com os atributos **low** e **high**, ele indica a região do intervalo qu é considerada preferível. Por exemplo, se estiver entre os atributos **min** e **low**, então a parte inferior do intervalo é considerada como ótima.

`form`

Esse atributo associa o elemento com um elemento `form` que é dono de um elemento `meter`. Por exemplo, um elemento `meter` pode estar mostrando um intervalo correspondente a um elemento `input` do **type**(tipo) *number*. Esse atributo só é utilizado se o elemento `meter` está sendo utilizado como um elemento associado a um formulário; mesmo assim, ele pode se romitido se o elemento for um descendente de um elemento `form`.

Exemplos

Exemplo simples

```
<p>
  Aqueça o forno para <meter min="200" max="500" value="350">350 graus</meter>.
</p>
```

No Google Chrome, o medidor aparece como:


meter1.png

Exemplos de intervalos com High e Low

Note que nesse exemplo o atributo **min** foi omitido; isso é permitido, pois ele irá ser, por padrão, 0.

```
<p>
  Ele recebeu <meter low="69" high="80" max="100" value="84">B</meter> no exame.
</p>
```

No Google Chrome, o medidor aparece como:



Veja também

- `<progress>`

nav

Sumário

O *Elemento HTML de Navegação* (`<nav>`) representa uma seção de uma página que aponta para outras páginas ou para outras áreas da página, ou seja, uma seção com links de navegação.

> **Note:** *Notas de utilização:*
>
> - Nem todos os links de um documento devem estar dentro de um elemento `<nav>`, o qual é destinado apenas para grupos importantes de links de navegação; tipicamente o elemento `<footer>` contém uma lista de links que não precisam estar em um elemento `<nav>`.
> - Um documento pode ter vários elementos `<nav>`, por exemplo, um para navegação no site e outro para navegação dentro da página.
> - Agentes de usuário, como leitores de tela para usuários deficientes, podem utilizar este elemento para determinar se a renderização inicial do conteúdo do mesmo deve ser omitida.

Contexto de uso

Conteúdo permitido	**Conteúdo de fluxo**
Omissão de tag	Nenhuma, tanto a tag de início quanto a de fim são obrigatórias
Elementos-pai permitidos	Quaisquer elementos que permitam conteúdo de fluxo. Note que um elemento `<nav>` não deve descender de um elemento `<address>`.
Documento normativo	HTML5, seção 4.4.3

Atributos

Este elemento não possui outros atributos além dos atributos globais, comuns a todos os elementos.

Interface DOM

Este elemento implementa a interface `HTMLElement`.

Exemplos

```
<nav>
  <ul>
    <li><a href="#">Página inicial</a></li>
    <li><a href="#">Sobre</a></li>
    <li><a href="#">Contato</a></li>
  </ul>
</nav>
```

Veja também

- Outros elementos relacionados à seções: `<body>`, `<article>`, `<section>`, `<aside>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<hgroup>`, `<header>`, `<footer>`, `<address>`;
- Seções e esboços de um documento HTML5

<nobr>

> **Não padrão:** Este recurso não é padronizado. Não recomendamos o uso de recursos não padronizados em produção, pois eles têm suporte limitado nos navegadores e podem mudar ou ser removidos. No entanto, podem ser uma alternativa viável em casos específicos onde não exista uma opção padrão.

Sumário

O elemento HTML `<nobr>` previne que um texto quebre em uma nova linha automaticamente, de forma que ele seja exibido em uma única grande linha, podendo tornar o *scroll* (horizontal) necessário. Esta tag não é padrão HTML e não deve ser usada. Como alternativa, use a propriedade CSS `white-space` como segue:

```
<span style="white-space: nowrap">Texto longo sem quebras de linha</span>
```

Ver também

- `white-space`
- `overflow`

<noscript>

Resumo

O **Elemento HTML `<noscript>`** define uma seção de html a ser inserida se um tipo de script não é suportado pela página ou se o script está desativado no navegador.

- *Categorias de conteúdo* Metadata content, flow content, phrasing content.
- *Conteúdo permitido* Quando scripting está desabilitado e quando este é um descendente do elemento `<head>` : em qualquer ordem, de zero ou mais elementos `<link>`, zero ou mais elementos `<style>`, e zero ou mais elementos `<meta>`. Quando o script está desativado e quando não é descendente do elemento `<head>` : qualquer conteúdo transparente, mas nenhum elemento `<noscript>` entre seus descendentes. Nos outros casos: conteúdo de fluxo ou conteúdo de texto.
- *Omissão da tag* Tags inicial e final obrigatórias.
- *Elemento pai permitidos* Qualquer elemento que aceite 'conteúdo de texto' (phrasing content), se não há um elemento `<noscript>` anterior ou em um elemento `<head>` (mas somente em um documento HTML), e também se não existe elemento `<noscript>` anterior.
- *Interface DOM* `HTMLElement`

Atributos

Este elemento só inclui os atributos globais.

Exemplo

```
<noscript>
  <!-- referência a arquivo externo -->
  <a href="http://www.mozilla.com/">Link Externo</a>
</noscript>
<p>Rocks!</p>
```

Resultado com o script ativado

Rocks!

Resultado com o script desativado

Link Externo

Rocks!

Sumário

O **Elemento HTML ** (ou *Elemento HTML de lista ordenada*) representa uma lista de itens ordenados. De forma característica esses itens ordenados em uma lista são mostrados com uma contagem que os precede, que pode ser de qualquer tipo, como numerais, letras, algarismos romanos, ou simples símbolos. Esse modelo numerado não é definido na descrição html da página, mas na folha de estilos CSS associada, pela propriedade `list-style-type`.

Não há limitação para a profundidade e a imbricação das listas definidas com os elementos `<ol>` e `<ul>`.

> **Nota:** Ambos os elementos `<ol>` e `<ul>`, representam uma lista de itens. Diferem porque, com o elemento `<ol>`, a ordem é significativa. Como regra de ouro para determinar qual deles usar, tente mudar a ordem dos itens da lista; se a significação for alterada, o elemento `<ol>` deve ser utilizado, senão o elemento `<ul>` é adequado.

- *Categorias de conteúdo*Flutuante e no caso dos elementos filhos de `<ol>` incluirem pelo menos um elemento de conteúdo `<li>` evidente.
- *Conteúdo permitido* Zero ou mais elementos `<li>`
- *Omissão de etiqueta (Tag)* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos pai permitidos* Qualquer elemento que aceite conteúdo flutuante.
- *Interface DOM* `HTMLOListElement`

Atributos

Este elemento inclui os atributos globais.

`compact`

Este atributo boleano sugere que a lista deve ser renderizada em um estilo compacto. A interpretação deste atributo depende do perfil de navegação (*user agent*) e não funciona em todos os navegadores.

> **Nota:** Não utilize este atributo, pois ele se tornou obsoleto. O elemento `<ol>` deve ser estilizado usando CSS. Para dar um efeito semelhante ao do atributo compacto, a propriedade CSS `line-height` pode ser utilizada com o valor de 80%.

`reversed`

Este atributo boleano especifica que as partes desta lista serão especificadas em ordem reversa, isto é, a menos importante será listada primeiro.

`start`

Este atributo inteiro especifica o valor inicial para a numeração dos itens da lista. Embora o tipo de ordenação dos elementos possa ser com algarismos romanos, tal como XXXI, ou letras, o valor inicial sempre é representado como um inteiro. Para iniciar a contagem a partir da letra "C", utilize <ol start="3">.

> **Nota:** Este atributo, obsoleto na HTML4, foi reintroduzido na HTML5.

`type`

Indica o tipo de numeração:

- `'a'` indica letras minúsculas,
- `'A'` indica letras maiúsculas,
- `'i'` indica algarismos romanos minúsculos,
- `'I'` indica algarismos romanos maiúsculos,
- e `'1'` indica números (padrão).

O tipo de marcação é usado na lista inteira, a menos que um atributo `type` diferente seja utilizado dentro do elemento `<li>`.

> **Nota:** Este atributo, obsoleto na HTML4, foi reintroduzido na HTML5. A menos que o valor do número na lista seja importante, a propriedade CSS `list-style-type` deve ser usada em seu lugar.

Exemplos

Exemplo simples

```
<ol>
  <li>primeiro item</li>
  <li>segundo item</li>
  <li>terceiro item</li>
</ol>
```

A saída HTML acima será:

1. primeiro item
2. segundo item
3. terceiro item

Usando o atributo start

```
<ol start="7">
  <li>primeiro item</li>
  <li>segundo item</li>
  <li>terceiro item</li>
</ol>
```

Listas aninhadas

```
<ol>
  <li>primeiro item</li>
  <li>
    segundo item
    <!-- Veja que a tag de fechamento </li> não é colocada aqui! -->
    <ol>
      <li>segundo item primeiro subitem</li>
      <li>segundo item segundo subitem</li>
      <li>segundo item terceiro subitem</li>
    </ol>
  </li>
  <!-- Aqui está a tag de fechamento </li> -->
  <li>terceiro item</li>
</ol>
```

A saída HTML acima será:

1. primeiro item
2. segundo item
    1. segundo item primeiro subitem
    2. segundo item segundo subitem
    3. segundo item terceiro subitem
3. terceiro item

 e <ul> aninhados

```
<ol>
  <li>primeiro item</li>
  <li>
    segundo item
    <!-- Observe que a tag de fechamento </li> não é colocada aqui! -->
```

```
    <ul>
      <li>segundo item primeiro subitem</li>
      <li>segundo item segundo subitem</li>
      <li>segundo item terceiro subitem</li>
    </ul>
  </li>
  <!-- Aqui está a tag de fechamento </li> -->
  <li>terceiro item</li>
</ol>
```

A saída HTML acima será:

1. primeiro item
2. segundo item
    - segundo item primeiro subitem
    - segundo item segundo subitem
    - segundo item terceiro subitem
3. terceiro item

Veja também

- Outros elementos HTML relacionados à lista: `<ul>`, `<li>`, `<menu>` e o obsoleto `<dir>`;
- Propriedades CSS que podem ser especialmente úteis para determinar o modelo do elemento `<ol>`:
    - a propriedade `list-style`, conveniente para escolher a forma como os ordinais são exibidos,
    - contadores CSS, útil para gerenciar listas complexas aninhadas,
    - a propriedade `line-height`, proficiente para simular o atributo obsoleto `compact`,
    - a propriedade `margin`, aplicável para controlar a indentação da lista.

<optgroup>

Sumário

Em um Formulário Web, o elemento HTML `<optgroup>` cria um agrupamento de opções dentro do elemento `<select>`.

- *Categorias de conteúdo* Nenhuma.
- *Conteúdo permitido* Nenhum ou mais elementos do tipo `<option>`.
- *Omissão da tag* A abertura da tag é obrigatória. O fechamento da tag é opcional se o elemento é imediatamente seguido de de outro elemento <optgroup>, ou se o elemento pai não tiver nenhum conteúdo.
- *Elemento pai permitido* Um elemento `<select>`.
- *Interface DOM* `HTMLOptGroupElement`

> **Nota:** Elementos do tipo **optgroup** não podem ser aninhados.

Atributos

Este elemento inclui os atributos globais.

`disabled`

Se este atributo booleano for definido, nenhum dos itens neste **optgroup** poderá ser selecionado. Muitos navegadores marcam como cinza e não permitem que o elemento receba nenhum evento de navegação, como cliques do mouse ou foco.

`label`

É o nome do optgroup, é o que os navegadores irão exibir como rótulo. Este atributo é obrigatório se o elemento for usado.

Exemplo

```
<select>
  <optgroup label="Grupo 1">
    <option>Opção 1.1</option>
  </optgroup>
  <optgroup label="Grupo 2">
    <option>Opção 2.1</option>
    <option>Opção 2.2</option>
  </optgroup>
  <optgroup label="Grupo 3" disabled>
    <option>Opção 3.1</option>
    <option>Opção 3.2</option>
    <option>Opção 3.3</option>
  </optgroup>
</select>
```

Resultado

Veja também

- Outros elementos de formularios relacionados: `<form>`, `<legend>`, `<label>`, `<button>`, `<select>`, `<datalist>`, `<option>`, `<fieldset>`, `<textarea>`, `<input>`, `<output>`, `<progress>` e `<meter>`.

<option>

Sumário

Em um formulário Web, o **elemento HTML** `<option>` é usado para criar um controle que representa um item dentro de um elemento HTML5 `<select>`, `<optgroup>` ou `<datalist>`.

- *Categorias de conteúdo* Nenhuma.
- *Conteúdo permitido* Texto com caracteres escapados (como `é`) ocasionalmente.
- *Omissão de tags* A tag de abertura é obrigatória. A tag de fechamento é opcional se este elemento é imediatamente seguido por um outro elemento `<option>` ou um `<optgroup>`, ou se o elemento pai não tem mais nenhum conteúdo.
- *Elementos pai permitidos* `<select>`, `<optgroup>` ou `<datalist>`.
- *Interface DOM* `HTMLOptionElement`

Atributos

Este elemento inclui os atributos globais.

`disabled`

> Se este atributo booleano estiver definido, esta opção não é selecionável. Frequentemente navegadores acinzentam este tipo de controle, e ele deixa de receber quaisquer eventos de navegação, como cliques do mouse ou eventos relacionados a foco. Se este atributo não estiver definido, o elemento ainda pode ser desabilitado se um de seus ancestrais é um elemento `<optgroup>` desabilitado.

`label`

> Este atributo é o texto para o rótulo indicando o significado da opção. Se o atributo **label** não estiver definido, seu valor é o conteúdo de texto do elemento.
>
> > **Nota:** O atributo **label** é projetado para conter um rótulo curto tipicamente usado num menu hierárquico. O atributo **value** descreve descreve um rótulo maior, designado para ser usado perto de um botão de escolha, por exemplo.

`selected`

> Se presente, este atributo booleano indica que a opção está selecionada inicialmente. Se o elemento `<option>` for descendente de um elemento `<select>` cujo atributo `multiple` não está definido, apenas um único `<option>` deste elemento `<select>` pode ter o atributo **selected**.

`value`

O conteúdo textual deste atributo representa o rótulo que explica a opção. Se ele não estiver definido, seu valor padrão é o texto contido no elemento.

> **Nota:** O atributo **label** é projetado para conter um rótulo curto tipicamente usado num menu hierárquico. O atributo **value** descreve descreve um rótulo maior, designado para ser usado perto de um botão de escolha, por exemplo.

Exemplos

Veja os exemplos do elemento `<select>`.

Veja também

- Outros elementos relacionados a formulários: `<form>`, `<legend>`, `<label>`, `<button>`, `<select>`, `<datalist>`, `<optgroup>`, `<fieldset>`, `<textarea>`, `<input>`, `<output>`, `<progress>` e `<meter>`.

Output

Sumário

O elemento de saída (<output>) é um elemento no qual um site ou aplicativo pode injetar os resultados de um cálculo ou o resultado de uma ação do usuário.

Categorias de conteúdo	**Flow content, phrasing content, listed, labelable, resettable form-associated element.**
Conteúdo permitido	Phrasing content.
Omissão de tag	Precisa ter ambas as tags de início e fim.
Documento normativo	HTML5, section 4.10.15

Atributos

Como qualquer elemento HTML, este elemento suporta os global attributes.

`for`

Uma lista de IDs de outros elementos, indicando que estes elementos contribuiram com valores de entrada (input) para o cálculo (ou outros afetados).

`form`

O elemento form ao qual este elemento está associado (seu "proprietário do formulário"). O valor do atributo deve ser um ID de um elemento form no mesmo documento. Se este atributo não está especificado, o elemento output deve ser descendente de um elemento form. Este atributo permite que você coloque elementos output em qualquer lugar em um documento, não apenas como descendentes de seus elementos form.

`name`

O nome do elemento.

Interface DOM

O elemento implementa a interface HTMLOutputElement .

Exemplo

```
<form oninput="result.value=parseInt(a.value)+parseInt(b.value)">
0<input type="range" name="b" value="50" />100 +<input type="number" name="a" value="10" /> =
<output name="result"></output>
</form>
```

Veja também

Outros elementos relacionados ao form: `<form>`, `<input>`, `<button>`, `<datalist>`, `<legend>`, `<label>`, `<select>`, `<optgroup>`, `<option>`, `<textarea>`, `<fieldset>`, `<progress>` and `<meter>`.

<p> O elemento Parágrafo

O **elemento HTML `<p>`** representa um parágrafo. Em mídias visuais, parágrafos são representados como blocos indentados de texto com a primeira letra avançada e separados por linhas em branco. Já em HTML, parágrafos são usados para agrupar conteúdos relacionados de qualquer tipo, como imagens e campos de um formulário.

Parágrafos são Elementos block-level, e fecharão automaticamente caso outro Elemento block-level inicie antes da tag de fechamento `</p>`. Veja "Omissão de tag" abaixo.

- *Categorias de conteúdo* Conteúdos de fluxo, conteúdos palpáveis.
- *Conteúdo permitido*Conteúdo frasal.
- *Omissão de tag* A tag de abertura é obrigatória. A tag de fechamento pode ser omitida caso o elemento `<p>` seja imediatamente sucedido pelos elementos `<address>`, `<article>`, `<aside>`, `<blockquote>`, `<div>`, `<dl>`, `<fieldset>`, `<footer>`, `<form>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<header>`, `<hr>`, `<menu>`, `<nav>`, `<ol>`, `<pre>`, `<section>`, `<table>`, `<ul>`, ou por outro elemento `<p>`, ou, caso não haja mais conteúdo no elemento pai e o elemento pai não for um elemento `<a>`.
- *Elementos pai permitidos* Qualquer elemento que aceite conteúdo de fluxo.
- *Interface DOM* `HTMLParagraphElement`

Atributos

Este elemento inclui atributos globais.

> **Note: Obs.:** O atributo `align` nas tags `<p>` está obsoleto e não deve ser usado.

Exemplo

HTML

```
<p>
  Este é o primeiro parágrafo do texto. Este é o primeiro parágrafo do texto.
  Este é o primeiro parágrafo do texto. Este é o primeiro parágrafo do texto.
</p>

<p>
  Este é o segundo parágrafo do texto. Este é o segundo parágrafo do texto. Este
  é o segundo parágrafo do texto. Este é o segundo parágrafo do texto.
</p>
```

Resultado

Estilizando Parágrafos

HTML

```
<p>
  Parágrafos separados por linhas em branco são mais legíveis. Porém, também
  podemos distinguir parágrafos indentando suas primeiras linhas. Uma abordagem
  que ocupa menos espaço e é usada para economizar papel quando impresso.
</p>

<p>
  Em textos que serão editados, como trabalhos escolares e rascunhos, usamos
  tanto as linhas em branco quanto a indentação para distinguirmos cada
  parágrafo. Já em publicações, usar ambas é considerado redundante e
  amadoresco.
</p>

<p>
  Em textos arcaicos usava-se o caractere especial: ¶, o <i>caldeirão</i>, para
```

```
  separar parágrafos. Atualmente, esta abordagem é considerada claustrofóbica e
  ilegível.
</p>

<p>
  Será mesmo que é ilegível? Clique para visualizar:
  <button data-toggle-text="Que horrível! Mudar de volta!">
    Usar caldeirão nos parágrafos
  </button>
</p>
```

CSS

```
p {
  margin: 0;
  text-indent: 3ch;
}

p.caldeirao {
  text-indent: 0;
  display: inline;
}
p.caldeirao + p.caldeirao::before {
  content: " ¶ ";
}
```

JavaScript

```
document.querySelector("button").addEventListener("click", function (event) {
  document.querySelectorAll("p").forEach(function (paragraph) {
    paragraph.classList.toggle("caldeirao");
  });
  var newButtonText = event.target.dataset.toggleText;
  var oldText = event.target.innerText;
  event.target.innerText = newButtonText;
  event.target.dataset.toggleText = oldText;
});
```

Result

Considerações acerca da Acessibilidade

Subdividir um conteúdo em parágrafos torna um texto mais acessível. Leitores de tela e outras tecnologias assistivas providenciam atalhos que permitem a navegação entre parágrafos. Possibilitando, então, uma leitura rápida do texto.

Utilizar elementos `<p>` vazios para adicionar linhas em branco entre parágrafos é uma abordagem problemática para os que dependem das tecnologias leitoras de tela. O leitor anunciará a existência de um parágrafo, mas não lerá nenhum conteúdo, pois não há. Isso confunde e frustra os que dependem dos leitures de tela.

Se desejas mais espaço, use CSS propriedades como `margin` para recriar o efeito:

```
p {
  margin-bottom: 2em; // aumenta o espaço em branco após um parágrafo
}
```

Veja também

- `<hr>`
- `<br>`

<picture>

O **elemento HTML** **`<picture>`** é um container usado para especificar múltiplos elementos `<source>` para um elemento específico `<img>` contido nele. O navegador irá escolher a imagem mais adequada de acordo com o layout atual da página, caracteristicas

do dispositivo em que será exibido (p.e. um dispositivo normal ou um hiDPI), e a habilidade do navegador de renderizar um certo tipo de imagem (p.e., envie uma imagem WebP para os navegadores baseados no Chromium ou PNG para navegadores não-Chromium); se não houver correspondência entre os elementos `<source>`, o arquivo especificado pelo elemento `<img>` será selecionado. A imagem selecionada é então exibida no espaço ocupado pelo elemento `<img>`.

Categorias de conteúdo	Conteúdo de fluxo, conteúdo fraseado, conteúdo embutido
Conteúdo permitido	Zero ou mais elementos `<source>`, seguidos de um elemento `<img>`, opcionalmente mesclado com elementos de suporte para scripts (por exemplo, `<script>` e `<template>`).
Omissão de tag	Nenhuma, as tags de abertura e fechamento são mandatórias
Parentes permitidos	Qualquer elemento que permita conteúdo embutido (embedded).
Papéis ARIA permitidos	Nenhum
Interface do DOM	`HTMLPictureElement`

Atributos

Esse elemento só inclui elementos globais.

Exemplo 1: Uso com atributo <code>media</code>

O atributo `media` permite você especificar uma media query que o navegador irá avaliar para selecionar um elemento `<source>` . Se a media query retornar `false`, o elemento `<source>` é ignorado.

```
<picture>
  <source srcset="mdn-logo-wide.png" media="(min-width: 600px)" />
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/picture/mdn-logo-narrow.png"
alt="MDN" />
</picture>
```

Exemplo 2: Uso com atributo type

O atributo `type` lhe permite especificar um tipo MIME para o(s) recurso(s) fornecido(s) no atributo srcset do elemento `<source>`. Se o navegador não suporta o tipo especificado, o elemento `<source>` é ignorado.

```
<picture>
  <source srcset="mdn-logo.svg" type="image/svg+xml" />
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/picture/mdn-logo.png" alt="MDN" />
</picture>
```

Veja também

- `<img>` element
- `<source>` element

<pre>

Resumo

HTML texto preformatado (**<pre>**) é a tag utilizada para representar texto pré-formatado. Um texto dentro desse elemento é tipicamente exibido em uma fonte não proporcional da mesma maneira em que o texto original foi disposto no arquivo. Espaços em branco são mantidos no texto da mesma forma em que este foi digitado.

Content categories Flow content, palpable content.

- *Conteúdo permitido* Phrasing content.
- *Tag omission* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos pai permitidos* Any element that accepts flow content.

- *interface DOM* `HTMLPreElement`

Atributos

Esse elemento apenas inclui atributos globais.

This element only includes the global attributes.

`cols`

> Contains the *preferred* count of characters that a line should have. It was a non-standard synonym of `width`. To achieve such an effect, use CSS styling instead.

`width`

> Contains the *preferred* count of characters that a line should have. Though technically still implemented, this attribute has no visual effect; to achieve such an effect, use CSS styling instead.

`wrap`

> Is a *hint* indicating how the overflow must happen. In modern browser this hint is ignored and no visual effect results in its present; to achieve such an effect, use CSS styling instead.

Exemplos

```
<!-- Um exemplo de código css -->
<pre>
body {
  color:red;
}
</pre>
```

Resultado

```
body {
  color:red;
}
```

Veja mais

- CSS: `white-space`, `word-break`

progress

o elemento HTML progress (<progress>) é usado para visualizar o progresso de uma tarefa. Embora as especifidades de como é mostrado ficam a cargo do desenvolvedor, tipicamente, é mostrado como uma barra de progresso.

Contexto de uso

Conteúdo permitido	**Conteúdo de texto**
Omissão de Tag	Nenhum, ambas as tags (abertura e encerramento) são obrigatórias
Elemento pai permitido	Conteúdo fraseado (phrasing content), mas não deve haver descendentes do elemento progress
Documento normativo	HTML5, seção 4.10.16

Atributos

Como todos os outros elementos HTML, este elemento tem os atributos globais (global attributes)

`max`

> Este atributo descreve quanto trabalho é demandado pela tarefa indicada pelo elemento progress.

`value`

Este atributo especifica quanto da tarefa foi concluído. Se este não existir, a barra de progresso é indeterminada; isso indica que uma atividade está em progresso sem previsão de quanto tempo é esperado para que seja concluída.

Você pode usar a propriedade `orient` para especificar se a barra de progresso deve ser renderizada horizontalmente (padrão) ou verticalmente. A pseudo-classe `:indeterminate` pode ser associada a barras de progresso indeterminadas.

Interface DOM

Este elemento implementa a interface HTMLProgressElement.

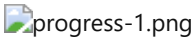
Exemplos

```
<progress value="70" max="100">70 %</progress>
```

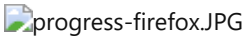
Resultado

View Live Examples

No Mac OS X, o progresso resultante deve aparecer assim:


progress-1.png

No Windows, o progresso resultante deve aparecer assim:


progress-firefox.JPG

Exemplos adicionais

Veja `orient`.

Veja também

- `orient`
- `:indeterminate`

<q>

O elemento HTML <q> indica que o texto dentro da tag é uma pequena citação. Este elemento destina-se a citações curtas que não requerem marcações de parágrafo; para citações maiores use o elemento `<blockquote>`.

Categorias de conteúdo	Fluxo de conteúdo, conteúdo textual, conteúdo palpável.
Conteúdo permitido	Conteúdo textual.
Omissão de tag	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Parents permitidos	Qualquer elemento que aceite conteúdo textual.
Atributos ARIA permitidos	Qualquer
Interface DOM	`HTMLQuoteElement`

> **Nota:** A maioria dos browsers modernos adicionará automaticamente marcas de citação em volta do texto dentro de um elemento <q>. Talvez seja necessário criar uma regra de estilo para adicionar marcas de citação em browsers antigos.

Atributos

Este elemento inclui os atributos globais.

`cite`

O valor deste atributo é uma URL que designa uma fonte ou mensagem para a informação citada. A intenção deste atributo é indicar a informação e explicar o contexto ou referência para a citação.

Exemplo

Conteúdo HTML

```
<p>
  According to Mozilla's website,
  <q cite="https://www.mozilla.org/pt-BR/about/history/details/"
    >Firefox 1.0 was released in 2004 and became a big success.</q
  >
</p>
```

Veja também

- O elemento `<blockquote>` para longas citações.
- O elemento `<cite>` para citar fontes.

<ruby>

Resumo

O **elemento** **HTML `<ruby>` **representa uma anotação ruby. Anotações ruby são para mostrar a pronúncia de caracteres do Leste Asiático

- *Categorias de conteúdo* Conteúdo de fluxo, conteúdo fraseado, conteúdo palpável.
- *Conteúdo permitido* Conteúdo fraseado.
- *Omissão de tag* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Elementos pais permitidos* Ver prosa
- *Interface DOM* `HTMLElement`

Atributos

Esse elemento somente inclui os atributos globais.

Exemplos

Exemplo 1: Caracter

```
<ruby>
  漢 <rp>(</rp><rt>Kan</rt><rp>)</rp> 字 <rp>(</rp><rt>ji</rt><rp>)</rp>
</ruby>
```

Exemplo 2: Palavra

```
<ruby> 明日 <rp>(</rp><rt>Ashita</rt><rp>)</rp> </ruby>
```

Veja também

- `<rt>`
- `<rp>`

<s>

O **elemento HTML `<s>`** renderiza um texto tachado ou uma linha cortando o texto. Use o elemento `<s>` para representar texto que não sejam relevante ou que não estam corretos. Não é apropriado o uso do `<s>` indicar edições no texto; para indicar edições no texto utilize `<del>` e `<ins>`, que são elementos mais apropriados.

Experimente

```
<p><s>There will be a few tickets available at the box office tonight.</s></p>

<p>SOLD OUT!</p>
```

```
/* stylelint-disable-next-line block-no-empty */
s {
}
```

Categorias de conteúdo	Conteúdo fraseado ou conteúdo de fluxo.
Conteúdo permitido	Conteúdo freaseado.
Omissão de Tag	Nenhuma, as tags de abertura e de fechamento são obrigatórias.
Pais permitidos	Qualquer elemento que aceite conteúdo fraseado.
Funções ARIA permitidas	Todas
DOM interface	`HTMLElement`

Atributos

Esse atributo incluí atributos globais.

> **Note: Notas de implementação:** Até Gecko 1.9.2 inclusive, Firefox implementa `HTMLSpanElement` para a interface deste elemento.

Exemplo

```
<s>Hoje é um dia especial: Salmon</s> ESGOTADO<br>
<span style="text-decoration:line-through;">Hoje é um dia especial:
  Salmon</span> ESGOTADO
```

Resultado

~~Hoje é um dia especial: Salmão~~ ESGOTADO ~~Hoje é um dia especial: Salmão~~ ESGOTADO

Preocupações com acessibilidade

A presença do elemento s não é anunciada pela maioria das tecnologias de leitura de tela em sua configuração padrão. Isso pode ser anunciado usando a propriedade CSS `content`, junto com os pseudoelementos `::before` and `::after`.

```
s::before,
s::after {
  clip-path: inset(100%);
  clip: rect(1px, 1px, 1px, 1px);
  height: 1px;
  overflow: hidden;
  position: absolute;
  white-space: nowrap;
  width: 1px;
}

s::before {
  content: " [início do texto utilizado] ";
}

s::after {
  content: " [fim do texto utilizado] ";
}
```

Algumas pessoas que usam leitores de tela que desativam deliberadamente o anúncio de conteúdo que criados com muita verbosidade. Por esse motivo, é importante não abusar dessa técnica e aplicá-la apenas nas situações em que o desconhecimento do conteúdo foi eliminado e que afetaria negativamente a compreensão.

- Nota curta sobre como deixar sua marca (mais acessível) | O Grupo Paciello
- Ajustando estilos de texto | Adrian Roselli

Veja mais

- O elemento `<strike>`, é o alter ego de `<s>` é obsoleto e não deve mais ser usado em sites.
- Elemento `<del>` deve ser usado quando o conteúdo for *deletado*.
- O CSS `text-decoration-line` propriedade deve ser usada para alcançar o antigo aspecto visual do elemento `<s>`.

<script>

O **elemento HTML** `<script>` é usado para incluir ou referenciar um script executável.

Categorias de conteúdo	Conteúdo de metadados, Fluxo de conteúdo, Conteúdo fraseado.
Conteúdo Permitido	Script dinâmico, como `text/javascript`.
Omissão de tag	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Pais permitidos	Qualquer elemento que aceite conteúdo de metadados, ou qualquer elemento que aceite conteúdo fraseado.
Regras ARIA permitidas	Nenhuma
Interface DOM	`HTMLScriptElement`

Atributos

Esse elemento inclui os atributos globais.

`async`

Um atributo booleano indicando que o navegador deve, se possível, executar o script assíncronamente.

> **Aviso:** Esse atributo não deve ser utilizado se o atributo `src` estiver ausente (ex. scripts embutidos). Se incluído, nesse caso, ele não terá nenhum efeito.

Scripts inseridos dinamicamente (usando `document.createElement`) são executados assincronamente por padrão, então para torná-lo uma execução síncrona (ex. executar scripts na ordem que eles foram carregados) atribua `async=false`.

Veja Browser compatibility para notas no suporte do navegador. Veja também Scripts assíncronos para asm.js.

`crossorigin`

Elementos `script` passam o mínimo de informação para `window.onerror` em scripts que não passem na checagem do CORS. Para permitir logs de erro para sites que usam domínios diferentes para arquivos estáticos, use esse atributo. Veja CORS settings attributes para uma explicação mais detalhada dos argumentos válidos.

`defer`

Esse atributo Boleano é usado para indicar ao navegador que o script deve ser executado depois que o documento tenha sido parseado, mas antes de disparar o evento `DOMContentLoaded`

Scripts com o atributo `defer` vão impedir que o evento DOMContentLoaded seja disparado até que o script seja carregado e tenha terminado de ser *avaliado*.

> **Aviso:** Esse atributo não deve ser usado se o atibuto `src` estiver ausente (ex. scripts inline), nesse caso ele não vai ter efeito.

Para conseguir um efeito similar para scripts inseridos dinamicamente use `async=false`. Scripts com o atributo `defer` vão ser executados na ordem em que aparecem no `document`.

`integrity`

Contains inline metadata that a user agent can use to verify that a fetched resource has been delivered free of unexpected manipulation. See Subresource Integrity.

`nomodule`

This Boolean attribute is set to indicate that the script should not be executed in browsers that support ES6 modules — in effect, this can be used to serve fallback scripts to older browsers that do not support modular JavaScript code.

`src`

This attribute specifies the URI of an external script; this can be used as an alternative to embedding a script directly within a document. If a `script` element has a `src` attribute specified, it should not have a script embedded inside its tags.

`text`

Like the `textContent`attribute, this attribute sets the text content of the element. Unlike the `textContent` attribute, however, this attribute is evaluated as executable code after the node is inserted into the DOM.

`type`

Indicates the type of script represented. The value of this attribute will be in one of the following categories:

- **Omitted or a JavaScript MIME type:** For HTML5-complient browsers this indicates the script is JavaScript. HTML5 spec urges authors to omit the attribute rather than provided a redundant MIME type. In earlier browsers, this identified the scripting language of the embedded or imported (via the `src` attribute) code. JavaScript MIME types are listed in the specification.
- **`module`:** For HTML5-complient browsers the code is treated as a JavaScript module. Processing of the script contents are not affected by the `charset` and `defer` attributes. For information on using `module`, see ES6 in Depth: Modules.
- **Any other value or MIME type:** Embedded content is treated as a data block which won't be processed by the browser. The `src` attribute will be ignored.

Note that in Firefox you can use advanced features such as let statements and other features in later JS versions, by using `type=application/javascript;version=1.8` . Beware, however, that as this is a non-standard feature, this will most likely break support for other browsers, in particular Chromium-based browsers.

For how to include *exotic programming languages*, read about Rosetta.

Atributos obsoletos

`charset`

If present, its value must be an ASCII case-insensitive match for "`utf-8`". Both it's unnecessary to specify the `charset` attribute, because documents must use UTF-8, and the `script` element inherits its character encoding from the document.

`language`

Like the `type` attribute, this attribute identifies the scripting language in use. Unlike the `type` attribute, however, this attribute's possible values were never standardized. The `type` attribute should be used instead.

Notas

Scripts without `async` or `defer` attributes, as well as inline scripts, are fetched and executed immediately, before the browser continues to parse the page.

The script should be served with the `text/javascript` MIME type, but browsers are lenient and only block them if the script is served with an image type (`image/*`), a video type (`video/*`), an audio (`audio/*`) type, or `text/csv`. If the script is blocked, an `error` is sent to the element, if not a `success` event is sent.

Exemplos

```
<!-- HTML4 and (x)HTML -->
<script type="text/javascript" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/script/javascript.js"></script>
```

```
<!-- HTML5 -->
<script src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/script/javascript.js"></script>
```

See also

- `document.currentScript`
- Ryan Grove's <script> and <link> node event compatibility chart

<section>

O **elemento HTML** `<section>` representa uma seção genérica contida em um documento HTML, geralmente com um título, quando não existir um elemento semântico mais específico para representá-lo.

Por exemplo, um menu de navegação deve estar dentro um elemento `<nav>`, mas uma lista de resultados de pesquisa ou a exibição de um mapa e seus controles não possuem elementos específicos, e podem ser colocados dentro de uma <section>.

Categorias de conteúdo	Conteúdo de fluxo , conteúdo de seção, conteúdo palpável.
Conteúdo permitido	Conteúdo de fluxo.
Omissão de tags	Sem omissão de tags
Tags-pai permitidas	Qualquer elemento que aceite conteúdo de fluxo. Note que um elemento `<section>` não deve ser um descendente de um elemento `<address>`.
Regras ARIA permitidas	`alert`, `alertdialog`, `application`, `banner`, `complementary`, `contentinfo`, `dialog`, `document`, `feed`, `log`, `main`, `marquee`, `navigation`, `search`, `status`, `tabpanel`
Interface DOM	`HTMLElement`

Atributos

Este elemento inclui apenas os atributos globais.

Notas de uso

- Cada `<section>` deve ser identificado, geralmente incluindo um cabeçalho (elemento `<h1>`-`<h6>`) como um filho do elemento `<section>`.
- Se faz sentido distribuir separadamente o conteúdo de um elemento `<section>`, use um elemento `<article>` em seu lugar.
- Não use o elemento `<section>` como um container genérico; para isso que a `<div>` serve, especialmente quando a seção é apenas com propósito de estilização. Uma regra de ouro é ,"usar quando uma seção deve aparecer logicamente na estrutura de um documento".

Exemplo 1

Antes

```
<div>
  <h1>Cabeçalho</h1>
  <p>Um monte de conteúdo incrível</p>
</div>
```

Depois

```
<section>
  <h1>Cabeçalho</h1>
  <p>Um monte de conteúdo incrível</p>
</section>
```

Exemplo 2

Antes

```
<div>
  <h2>Cabeçalho</h2>
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/section/passaro.jpg" alt="pássaro" />
</div>
```

Depois

```
<section>
  <h2>Cabeçalho</h2>
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/section/passaro.jpg" alt="pássaro" />
</section>
```

Veja também

- Outros elemtnos relacionados: `<body>`, `<nav>`, `<article>`, `<aside>`, `<h1>`, `<h2>`, `<h3>`, `<h4>`, `<h5>`, `<h6>`, `<hgroup>`, `<header>`, `<footer>`, `<address>`;
- Seções e estrurura do um documento HTML5.

<select>

Sumário

O elemento HTML *select* (<select>) representa um controle que apresenta um menu de opções. As opções dentro do menu são representadas pelo elemento `<option>`, que podem ser agrupados por elementos `<optgroup>`. As opções podem ser pré-selecionadas para o usuário.

Contexto de uso

Categorias de conteúdo	**conteúdo fluido**, **conteúdo fraseado**, **conteúdo interativo**, **listado**, **rotulável**, **resetável**, **e enviável associado a formulários**
Conteúdo permitido	Zero ou mais elementos `<option>` ou `<optgroup>`.
Omissão de tag	Nenhuma, tanto tag de início quanto de fim são obrigatórias
Elementos pai permitidos	qualquer elemento que aceite conteúdo fraseado

Atributos

Este elemento inclui os atributos globais.

`autofocus`

> Este atributo permite especificar que um controle de formulário deve ter foco de entrada quando a página é carregada, a não ser que o usuário o substitua, por exemplo digitando em um controle diferente. Somente um elemento de formulário em um documento pode ter o atributo `autofocus`, que é booleano.

`disabled`

> Este atributo booleano indica que o usuário não pode interagir com o controle. Caso esse atributo não seja especificado, o controle herda a configuração do elemento que o contém, por exemplo o elemento `fieldset`; se não há nenhum elemento que o contém com o atributo `disabled`, então o controle está habilitado.

`form`

> O elemento form ao qual o elemento select é associado (algo como "formulário dono" do select). Se este atributo for especificado, seu valor deve ser o ID de um elemento form no mesmo documento. Isso permite que você coloque elementos select em qualquer lugar dentro do documento, e não apenas como descendentes de elementos form.

`multiple`

> Este atributo booleano indica que várias opções podem ser selecionadas na lista. Se não for especificado, apenas uma opção poderá ser selecionada de cada vez.

`name`

O nome do controle

`required`

Um atributo booleano que indica que uma opção com um valor de string que não esteja vazia deve ser selecionada.

`size`

Se o controle é apresentado como uma list box com scroll, este atributo representa o número de linhas na list box que devem estar visíveis num determinado momento. Os navegadores não são obrigados a apresentar elementos select com uma list box scroll. O valor padrão é 0.

> **Note: Nota sobre o Firefox:** De acordo com a especificação HTML5, o valor padrão para o size deveria ser 1; no entanto, na prática, notou-se que isso quebrava alguns sites, e nenhum outro navegador faz isso no momento, então a Mozilla escolheu por continuar devolvendo 0 por enquanto no Firefox.

Interface DOM

Este elemento implementa a interface `HTMLSelectElement`.

Exemplos

```
<!-- O segundo valor estará selecionado inicialmente -->
<select name="select">
  <option value="valor1">Valor 1</option>
  <option value="valor2" selected>Valor 2</option>
  <option value="valor3">Valor 3</option>
</select>
```

Notas

O conteúdo deste elemento é estático e não editável.

A seguir um exemplo de como simular uma lista de seleção com opções editáveis, **mas esteja ciente** de que leitores de tela e dispositivos de acessibilidade *não* interpretarão o formulário corretamente; este exemplo seria HTML inválido se os elementos corretos fossem usados:

Este é um exemplo de um select editável usando um `<fieldset>` de botões de opção e caixas de texto (**escrito somente com CSS**, sem JavaScript),

Veja também

- Outros elementos relacionados a formulários: `<form>`, `<legend>`, `<label>`, `<button>`, `<option>`, `<datalist>`, `<optgroup>`, `<fieldset>`, `<textarea>`, `<input>`, `<output>`, `<progress>` e `<meter>`.

source

Sumário

O elemento `source` é utilizado para especificar múltiplos recursos de mídia de elementos `<picture>`, `<audio>` ou `<video>` em HTML5. É um elemento vazio. É normalmente usado para disponibilizar multiple formats supported by different browsers.

Contexto de uso

Conteúdo permitido	Nenhum; isso é um elemento vazio.
Omissão de tag	Deve ter uma tag de início, mas não deve ter uma tag de fim.
Elementos pais permitidos	`<picture>`, `<audio>`, `<video>`
Documento normativo	HTML5, section 4.8.8

Atributos

Como todos os outros elementos de HTML, esse elemento suporta os global attributes.

`src`

Requerido, endereço do arquivo de mídia.

`type`

O tipo MIME do arquivo, opcionalmente com um parametro de `codecs`. Veja o RFC 4281 para informações sobre como especificar codec.

`media`

Definição do tipo de mídia (Media query) pretendido.

Se o atributo **type** não está especificado, o tipo da mídia é obtido no servidor e é verificado se o Gecko consegue reproduzi-lo; se não for possível reproduzi-lo, o próximo **source** é verificado. Se o atributo **type** está definido, ele é comparado aos tipos que o Gecko consegue reproduzir, e se não for reconhecido, o servido não é solicitado; ao invés disso, o próximo elemento **source** é verificado.

Interface do DOM

Esse elemento implementa a interface `HTMLSourceElement`.

Exemplos

Esse exemplo demonstra como oferecer um vídeo no formato Ogg para usuários em que os navegadores suporta o formato Ogg, e um formato QuickTime para os usuários que o suporta. Se os elementos `audio` ou `video` não forem suportados pelo navegador, um aviso será mostrado. Se o navegador suportar o elemento, mas não suportar nenhum dos formatos especificados, um evento de `error` será lançado e os controles padrões de mídia (se ativados) indicarão o erro. Veja também a lista de media formats supported by the audio and video elements em vários navegadores.

```
<video controls>
  <source src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/source/foo.ogg" type="video/ogg"
/>
  <!-- Escolhido pelo Firefox -->
  <source src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/source/foo.mov"
type="video/quicktime" />
  <!-- Escolhido pelo Safari -->
  Desculpa; seu navegador não é compatível com vídeo em HTML5.
</video>
```

Para mais exemplos, veja Using audio and video in Firefox.

Resumo

O elemento **HTML `<span>` é um conteiner generico em linha para conteúdo fraseado** , que não representa nada por natureza. Ele pode ser usado para agrupar elementos para fins de estilo (usando os atributos `class` ou `id`), ou para compartilhar valores de atributos como `lang`. Ele deve ser usado somente quando nenhum outro elemento semântico for apropriado. `<span>` é muito parecido com o elemento `<div>` , entretando `<div>` é um elemento de nível de bloco enquanto `<span>` é um elemento em linha.

Contexto de Uso

Categorias de conteúdo	**Flow content, phrasing content**
Conteúdo permitido	Phrasing content
Omissão de Tag	Nenhum, deve ter tanto tag de início quanto tag final.
Elementos pai permitidos	Qualquer elemento que aceite phrasing content, ou flow content.
Padrões de documento	HTML5, section 4.6.24; HTML 4.01, section 7.5.4

Atributos

Esse elemento inclui apenas os atributos globais.

DOM interface

Este elemento implementa a interface `HTMLSpanElement`.

Exemplos

```
<p><span>Some text</span></p>
```

Resultado

Some text

Veja também

- Elemento HTML `<div>`


```
O elemento HTML <strong> dá ao texto uma forte importância, e é tipicamente mostrado em negrito.
```

Categorias do conteúdo	Flux content, Phrasing content, conteúdo palpável.
Conteúdo permitido	Phrasing content.
Omissão de tag	Nenhuma, deve ter uma tag de início e uma tag de encerramento.
Permitted parents	Qualquer elemento que aceite phrasing content, ou qualquer elemento que aceite flow content.
Permitted ARIA roles	Qualquer um
DOM interface	`HTMLElement` Até Gecko 1.9.2 (Firefox 4) inclusive, Firefox implementa a interface `HTMLSpanElement` para este elemento.

Atributos

Este elemento inclui apenas os atributos globais.

Notas de uso

O elemento `<strong>` é utilizado em conteúdos que são de "grande importância", incluindo coisas de urgentes (como alertas). Podem ser sentenças que são de grande importância para toda a página, ou, você pode meramente tentar pontuar que algumas palavras são de grande importância, comparado ao conteúdo próximo.

Tipicamente, estes elementos são renderizados por padrão, usando fontes em negrito. Contudo, ele não deve ser usado para simplesmente aplicar o estilo negrito; use o CSS `font-weight` para este propósito. Use o elemento `<b>` para chamar a atenção para certos textos sem a indicação de grande nível de importância. Use o elemento `<em>` para marcar textos que tem necessitam de ênfase.

Outro uso aceitavel para `<strong>` é denotado com o rótulo (label) de parágrafos, que representa notas ou avisos, dentro do texto da página.

Bold vs. Strong

Muitas vezes é confuso para novos desenvolvedores porque há tantas maneiras de expressar a mesma coisa em um website renderizado. Bold e Strong são talvez um dos casos mais comuns. Porque usar vs ? Você precisa digitar muito mais ao usar strong e ela produz o mesmo resultado, certo?

Talvez não; strong é um estado lógico, e bold é um estado físico. Estados lógicos separam apresentação do conteúdo, e ao fazer isso permitem que ele seja expresso de várias maneiras diferentes. Possivelmente em vez de renderizar um texto como negrito você queira renderizá-lo vermelho, ou num tamanho diferente, ou sublinhado, ou seja lá o que for. Faz mais sentido mudar as propriedades de apresentação de strong do que bold. Isto porque bold é um estado físico; não há separação entre a apresentação e o conteúdo, e fazer com que bold faça qualquer outra coisa diferente de deixar o texto em negrito seria confuso e ilógico.

É importante notar que tem outros usos, quando se quer chamar atenção sem aumentar a importância.

Emphasis vs. Strong

Enquanto no HTML4, Strong simplesmente indicava uma ênfase mais forte, em HTML5, o elemento é descrito como representando "uma forte importância para o seu conteúdo." Esta é uma importante distinção a se fazer. Enquanto Emphasis é usado para alterar o significado de uma sentença ("Eu *amo* cenouras" vs. "Eu amo *cenouras*"), Strong é usado para dar mais importância a porções de uma sentença (e.g., "**Cuidado!** Isso é **muito perigoso.**"). Ambos Strong e Emphasis podem ser aninhados para aumentar o grau relativo de importância ou reforçar ênfase, respectivamente.

Exemplo

Exemplo básico

```
<p>
  Ao fazer x é <strong>imperativo</strong> que se faça y antes de prosseguir.
</p>
```

Resultado

Ao fazer x é **imperativo** que se faça y antes de prosseguir.

Veja também

- HTML bold element

<style>

Resumo

O **elemento HTML <style>** contém informações de estilo para um documento ou uma parte do documento. As informações de estilo específico estão contidas dentro deste elemento, geralmente no CSS.

- *Content categories*Metadata content, e se o atributo `scoped` é apresentado: flow content.
- *Conteúdo Permitido* Style information matching the language of the `type` attribute.
- *Omissão de Tag* Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
- *Permitted parent elements* Qualquer elemento que aceite Metadata content.
- *Interface DOM* `HTMLStyleElement`

Atributos

Este elemento inclui os atributos globais.

type

> Este atributo define a linguagem de estilo como um tipo MIME (charset não deve ser especificado). ' text/css' presume se este atributo estiver ausente.

media

> Qual mídia deve aplicar-se a este estilo. Deve ser uma consulta de mídia, consulte http://dev.w3.org/csswg/css3-mediaqueries.

scoped

Se este atributo está presente, então o estilo aplica-se apenas ao seu elemento pai. Se está ausente, o estilo aplica-se ao documento inteiro.

title

Especifica os conjuntos de folhas de estilo alternativa.

disabled

Se definido, desativa (não se aplica) às regras de estilo para a `Document` que são especificadas no elemento.

Exemplos

Uma folha de estilo simples

```
<style type="text/css">
  body {
    color: red;
  }
</style>
```

Uma folha de estilos com escopo

```
<article>
  <div>
    The scoped attribute allows for you to include style elements mid-document.
    Inside rules only apply to the parent element.
  </div>
  <p>
    This text should be black. If it is red your browser does not support the
    scoped attribute.
  </p>
  <section>
    <style scoped>
      p {
        color: red;
      }
    </style>
    <p>This should be red.</p>
  </section>
</article>
```

Amostra

Veja também

- O elemento `<link>` que permite usar folhas de estilo externas.

summary

Sumário

O elemento HTML *summary* (`<summary>`) é utilizado como um sumário ou legenda para o conteúdo de um elemento `<details>`.

> **Nota:** Se o elemento `<summary>` for omitido, o cabeçalho "details" será utilizado.

Contexto de uso

Conteúdo permitido	Phrasing content
Omissão de tag	Nenhuma, tanto as tags iniciais quanto as finais devem estar presentes.
Elementos pai permitidos	O elemento `<details>`.

Conteúdo permitido	Phrasing content
Documentos normativos	HTML5, section 4.11.2

Atributos

Como todos os outros elementos HTML, esse elemento possui os atributos globais.

Interface do DOM

Esse elemento implementa a interface `HTMLElement`.

Exemplos

Veja a página `<details>` para ver exemplos com `<summary>`.

Veja também

- `<details>`

<table>: O elemento Table

O elemento **`<table>`** HTML representa dados tabulares — isto é, informações apresentadas em uma tabela bidimensional composta por linhas e colunas de células contendo dados .

Experimente

```
<table>
  <caption>
    Front-end web developer course 2021
  </caption>
  <thead>
    <tr>
      <th scope="col">Person</th>
      <th scope="col">Most interest in</th>
      <th scope="col">Age</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Chris</th>
      <td>HTML tables</td>
      <td>22</td>
    </tr>
    <tr>
      <th scope="row">Dennis</th>
      <td>Web accessibility</td>
      <td>45</td>
    </tr>
    <tr>
      <th scope="row">Sarah</th>
      <td>JavaScript frameworks</td>
      <td>29</td>
    </tr>
    <tr>
      <th scope="row">Karen</th>
      <td>Web performance</td>
      <td>36</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th scope="row" colspan="2">Average age</th>
      <td>33</td>
    </tr>
```

```
  </tfoot>
</table>
```

```
table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
  font-weight: bold;
}

thead,
tfoot {
  background-color: rgb(228 240 245);
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

td:last-of-type {
  text-align: center;
}

tbody > tr:nth-of-type(even) {
  background-color: rgb(237 238 242);
}

tfoot th {
  text-align: right;
}

tfoot td {
  font-weight: bold;
}
```

Contexto de uso

Categoria de conteúdo	Fluxo de conteúdo
Conteúdo permitido	Nesta ordem: 1. um elemento `<caption>` opcional, 2. zero ou mais `<colgroup>` elementos, 3. um elemento `<thead>` opcional, 4. uma das duas alternativas: ○ zero ou mais elementos `<tbody>` ○ um ou mais elementos `<tr>` 5. um elemento `<tfoot>` opcional
Omissão de tag	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Elementos pais permitidos	Qualquer elemento que aceite fluxo de conteúdo
Função ARIA implícita	`table`
Permitted ARIA roles	Any
Interface DOM	`HTMLTableElement`

Atributos

Este elemento inclui os atributos globais.

Atributos depreciados

`align`

Este atributo enumerado indica como a tabela deve ser alinhada dentro do documento que a contém. Pode ter os seguintes valores:

- `left`: a tabela é exibida no lado esquerdo do documento;
- `center`: a tabela é exibida no centro do documento;
- `right`: a tabela é exibida no lado direito do documento.

Defina `margin-left` e `margin-right` como `auto` ou `margin` como `0 auto` para obter um efeito que é semelhante ao atributo align.

`bgcolor`

A cor de fundo da tabela. É um código RGB hexadecimal de 6 dígitos, prefixado por um '#'. Uma das [palavras-chave de cor] predefinidas (/pt-BR/docs/Web/CSS/named-color) também pode ser usada.

Para obter um efeito semelhante, use a propriedade CSS `background-color`.

`border`

Este atributo inteiro define, em pixels, o tamanho do quadro ao redor da tabela. Se definido como 0, o atributo `frame` é definido como void.

Para obter um efeito semelhante, use a propriedade abreviada CSS `border`.

`cellpadding`

Este atributo define o espaço entre o conteúdo de uma célula e sua borda, exibida ou não. Se o comprimento do preenchimento da célula for definido em pixels, esse espaço de tamanho de pixel será aplicado a todos os quatro lados do conteúdo da célula. Se o comprimento for definido usando um valor percentual, o conteúdo será centralizado e o espaço vertical total (superior e inferior) representará este valor. O mesmo vale para o espaço horizontal total (esquerda e direita).

Para obter um efeito semelhante, aplique a propriedade `border-collapse` ao elemento `<table>`, com seu valor definido para recolher, e a propriedade `padding` aos elementos `<td>`.

`cellspacing`

Este atributo define o tamanho do espaço entre duas células em um valor percentual ou pixels. O atributo é aplicado horizontal e verticalmente, ao espaço entre o topo da tabela e as células da primeira linha, a esquerda da tabela e a primeira coluna, a direita da tabela e a última coluna e a parte inferior da tabela e a última linha.

Para obter um efeito semelhante, aplique a propriedade `border-spacing` ao elemento `<table>`. `border-spacing` não tem nenhum efeito se `border-collapse` estiver definido para recolher.

`frame`

Este atributo enumerado define qual lado do quadro ao redor da tabela deve ser exibido.

Para obter um efeito semelhante, use as propriedades `border-style` e `border-width`.

`rules`

Este atributo enumerado define onde as rules, ou seja, as linhas, devem aparecer em uma tabela. Pode ter os seguintes valores:

- `none`, que indica que nenhuma regra será exibida; é o valor padrão;
- `groups`, que fará com que as regras sejam exibidas entre os grupos de linhas (definidas por `<thead>`, `<tbody>` e `<tfoot>` elementos) e entre grupos de colunas (definidos apenas pelos elementos `<col>` e `<colgroup>`);

- `rows`, que fará com que as regras sejam exibidas entre as linhas;
- `cols`, que fará com que as regras sejam exibidas entre as colunas;
- `all`, que fará com que as regras sejam exibidas entre linhas e colunas.

Para obter um efeito semelhante, aplique a propriedade `border` ao apropriado `<thead>`, `<tbody>`, `<tfoot >`, `<col>`, ou `<colgroup>` elementos.

`summary`

Este atributo define um texto alternativo que resume o conteúdo da tabela. Use o elemento `<caption>` em vez disso.

`width`

Este atributo define a largura da tabela. Em vez disso, use a propriedade CSS `width`.

Exemplos

Table simples

```
<table>
  <tr>
    <td>John</td>
    <td>Doe</td>
  </tr>
  <tr>
    <td>Jane</td>
    <td>Doe</td>
  </tr>
</table>
```

Resultado

Outros exemplos simples

```
<p>Tabela simples com cabeçalho</p>
<table>
  <tr>
    <th>First name</th>
    <th>Last name</th>
  </tr>
  <tr>
    <td>John</td>
    <td>Doe</td>
  </tr>
  <tr>
    <td>Jane</td>
    <td>Doe</td>
  </tr>
</table>

<p>Tabela com thead, tfoot, e tbody</p>
<table>
  <thead>
    <tr>
      <th>Header content 1</th>
      <th>Header content 2</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Body content 1</td>
      <td>Body content 2</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
```

```
      <td>Footer content 1</td>
      <td>Footer content 2</td>
    </tr>
  </tfoot>
</table>

<p>Tabela com colgroup</p>
<table>
  <colgroup span="4"></colgroup>
  <tr>
    <th>Countries</th>
    <th>Capitals</th>
    <th>Population</th>
    <th>Language</th>
  </tr>
  <tr>
    <td>USA</td>
    <td>Washington, D.C.</td>
    <td>309 million</td>
    <td>English</td>
  </tr>
  <tr>
    <td>Sweden</td>
    <td>Stockholm</td>
    <td>9 million</td>
    <td>Swedish</td>
  </tr>
</table>

<p>Tabela com colgroup e col</p>
<table>
  <colgroup>
    <col style="background-color: #0f0" />
    <col span="2" />
  </colgroup>
  <tr>
    <th>Lime</th>
    <th>Lemon</th>
    <th>Orange</th>
  </tr>
  <tr>
    <td>Green</td>
    <td>Yellow</td>
    <td>Orange</td>
  </tr>
</table>

<p>Tabela simples com caption</p>
<table>
  <caption>
    Awesome caption
  </caption>
  <tr>
    <td>Awesome data</td>
  </tr>
</table>
```

Resultado

Ordenação de tabela

Ordenando as linhas da tabela

Não há métodos nativos para classificar as linhas (elementos `<tr>`) de uma tabela HTML. Mas usando `Array.prototype.slice()`, `Array.prototype.sort()`, `Node.removeChild()` e `Node.appendChild()`, você pode implementar sua própria função `sort()` para classificar uma `HTMLCollection` de elementos `<tr>`.

No exemplo abaixo, você pode ver tal exemplo. Estamos anexando-o ao elemento <tbody> para que ele classifique as células da tabela em ordem crescente de valor e atualize a exibição para se adequar.

HTML

```
<table>
  <tbody>
    <tr>
      <td>3</td>
    </tr>
    <tr>
      <td>2</td>
    </tr>
    <tr>
      <td>1</td>
    </tr>
  </tbody>
</table>
```

JavaScript

```
HTMLTableSectionElement.prototype.sort = function (cb) {
  Array.from(this.rows)
    .sort(cb)
    .forEach((e) => this.appendChild(this.removeChild(e)));
};

document
  .querySelector("table")
  .tBodies[0].sort((a, b) => a.textContent.localeCompare(b.textContent));
```

Resultado

Ordenando as linhas com um clique no elemento th

O exemplo a seguir adiciona um manipulador de eventos a cada elemento `<th>` de cada `<table>` no `document`; ele ordena todas as linhas do `<tbody>`, baseando a ordenação nas células `td` contidas nas linhas.

> **Nota:** Esta solução assume que os elementos `<td>` são preenchidos por texto bruto sem elementos descendentes.

HTML

```
<table>
  <thead>
    <tr>
      <th>Números</th>
      <th>Letras</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>3</td>
      <td>A</td>
    </tr>
    <tr>
      <td>2</td>
      <td>B</td>
    </tr>
    <tr>
      <td>1</td>
      <td>C</td>
    </tr>
```

```
  </tbody>
</table>
```

JavaScript

```
const allTables = document.querySelectorAll("table");

for (const table of allTables) {
  const tBody = table.tBodies[0];
  const rows = Array.from(tBody.rows);
  const headerCells = table.tHead.rows[0].cells;

  for (const th of headerCells) {
    const cellIndex = th.cellIndex;

    th.addEventListener("click", () => {
      rows.sort((tr1, tr2) => {
        const tr1Text = tr1.cells[cellIndex].textContent;
        const tr2Text = tr2.cells[cellIndex].textContent;
        return tr1Text.localeCompare(tr2Text);
      });

      tBody.append(...rows);
    });
  }
}
```

Resultado

Exibindo tabelas grandes em espaços pequenos

Um problema comum com tabelas na web é que elas não funcionam muito bem nativamente em telas pequenas quando a quantidade de conteúdo é grande, e a maneira de torná-las roláveis não é óbvia, especialmente quando a marcação pode vir de um CMS e não pode ser modificado para ter um wrapper.

Este exemplo fornece uma maneira de exibir tabelas em espaços pequenos. Ocultamos o conteúdo HTML porque ele é muito grande e não há nada de extraordinário nele. O CSS é mais útil para inspecionar neste exemplo.

Ao observar esses estilos, você notará que a propriedade `display` da tabela foi definida como `block`. Embora isso permita a rolagem, a tabela perde um pouco de sua integridade e as células da tabela tentam se tornar o menor possível. Para atenuar esse problema, definimos `white-space` como `nowrap` no `<tbody>`. No entanto, não fazemos isso para o `<thead>` para evitar que títulos longos forcem as colunas a serem mais largas do que o necessário para exibir os dados.

Para manter os cabeçalhos da tabela na página durante a rolagem para baixo, definimos `position` para fixar nos elementos `<th>`. Observe que **não** definimos `border-collapse` como `collapse`, pois se o fizermos, o cabeçalho não pode ser separado corretamente do restante da tabela.

```
table,
th,
td {
  border: 1px solid;
}

table {
  width: 100%;
  max-width: 400px;
  height: 240px;
  margin: 0 auto;
  display: block;
  overflow-x: auto;
  border-spacing: 0;
}

tbody {
  white-space: nowrap;
```

```
}

th,
td {
  padding: 5px 10px;
  border-top-width: 0;
  border-left-width: 0;
}

th {
  position: sticky;
  top: 0;
  background: #fff;
  vertical-align: bottom;
}

th:last-child,
td:last-child {
  border-right-width: 0;
}

tr:last-child td {
  border-bottom-width: 0;
}
```

Resultado

Preocupações com acessibilidade

Legendas

Ao fornecer um elemento `<caption>` cujo valor descreve de forma clara e concisa o propósito da tabela, ele ajuda as pessoas a decidirem se precisam ler o restante do conteúdo da tabela ou ignorá-lo.

Isso ajuda as pessoas a navegar com o auxílio de tecnologia assistiva, como um leitor de tela, pessoas com problemas de visão subnormal e pessoas com problemas cognitivos.

- MDN Adicionando uma legenda à sua tabela com <caption>
- Legenda e resumo • Tabelas • Tutoriais de acessibilidade na Web do W3C WAI

Escopo de linhas e colunas

O atributo `scope` em elementos de cabeçalho é redundante em contextos simples, porque o escopo é inferido. No entanto, algumas tecnologias assistivas podem não obter inferências corretas, portanto, especificar o escopo do cabeçalho pode melhorar a experiência do usuário. Em tabelas complexas, o escopo pode ser especificado para fornecer as informações necessárias sobre as células relacionadas a um cabeçalho.

Examples

```
<table>
  <caption>
    Nomes das cores e valores
  </caption>
  <tbody>
    <tr>
      <th scope="col">Nome</th>
      <th scope="col">HEX</th>
      <th scope="col">HSLa</th>
      <th scope="col">RGBa</th>
    </tr>
    <tr>
      <th scope="row">Teal</th>
      <td><code>#51F6F6</code></td>
      <td><code>hsl(180 90% 64% / 1)</code></td>
      <td><code>rgb(81 246 246 / 1)</code></td>
    </tr>
```

```
    <tr>
      <th scope="row">Goldenrod</th>
      <td><code>#F6BC57</code></td>
      <td><code>hsl(38 90% 65% / 1)</code></td>
      <td><code>rgba(246 188 87 / 1)</code></td>
    </tr>
  </tbody>
</table>
```

Resultado

Fornecer uma declaração de `scope="col"` em um elemento `<th>` ajudará a descrever que a célula está no topo de uma coluna. Fornecer uma declaração de `scope="row"` em um elemento `<th>` ajudará a descrever que a célula é a primeira em uma linha.

- Tabelas MDN para usuários com deficiência visual
- Tabelas com dois cabeçalhos • Tabelas • Tutoriais de acessibilidade na Web do W3C WAI
- Tabelas com cabeçalhos irregulares • Tabelas • Tutoriais de acessibilidade na Web W3C WAI
- H63: Usando o atributo scope para associar células de cabeçalho e células de dados em tabelas de dados | Técnicas W3C para WCAG 2.0

Tabelas complicadas

A tecnologia assistiva, como leitores de tela, pode ter dificuldade em analisar tabelas que são tão complexas que as células de cabeçalho não podem ser associadas de maneira estritamente horizontal ou vertical. Isso é normalmente indicado pela presença de `colspan` e [rowspan](/pt-BR/docs/Web/HTML/ Element/td#rowspan).

Idealmente, considere maneiras alternativas de apresentar o conteúdo da tabela, incluindo dividi-la em uma coleção de tabelas relacionadas menores que não precisam depender do uso dos atributos `colspan` e `rowspan`. Além de ajudar as pessoas que usam tecnologia assistiva a entender o conteúdo da tabela, isso também pode beneficiar pessoas com problemas cognitivos que podem ter dificuldade em entender as associações que o layout da tabela está descrevendo.

Se a tabela não puder ser separada, use uma combinação de `id` e [headers](/pt-BR/docs/Web/ HTML/Element/td#headers) para associar programaticamente cada célula da tabela aos cabeçalhos aos quais a célula está associada.

- Tabelas MDN para usuários com deficiência visual
- Tabelas com cabeçalhos de vários níveis • Tabelas • Tutoriais de acessibilidade na Web do W3C WAI
- H43: Usando os atributos id e headers para associar células de dados com células de cabeçalho em tabelas de dados | Técnicas para W3C WCAG 2.0

Veja também

- Tutorial de tabela de dados HTML
- Propriedades CSS que podem ser especialmente úteis para estilizar o elemento `<table>`:
  - `width` para controlar a largura da tabela;
  - `border`, `border-style`, `border-color`, `border-width` , `border-collapse`, `border-spacing` para controlar o aspecto das bordas das células, réguas e moldura;
  - `margin` e `padding` para estilizar o conteúdo da célula individual;
  - `text-align` e `vertical-align` para definir o alinhamento do texto e do conteúdo da célula.

<template>

Sumário

O elemento **HTML** **`<template>`** é um mecanismo para encapsular um conteúdo do lado do cliente que não é renderizado quando a página é carregada, mas que pode ser instanciado posteriormente em tempo de execução usando JavaScript.

Pense no template como um fragmento de conteúdo, que é armazenado para um possível uso futuro no documento. Enquanto o parser processa o conteúdo do elemento **`<template>`** ao carregar a página, isso apenas garante que o conteúdo é válido; porém, o conteúdo do elemento ainda não foi renderizado.

Conteúdo de categorias	Metadata content, flow content, phrasing content, script-supporting element

Conteúdo Permitido	Metadata content, flow content, qualquer conteúdo HTML válido que é permitido para ocorrer dentro do `<ol>`, `<dl>`, `<figure>`, `<ruby>`, `<object>`, `<video>`, `<audio>`, `<table>`, `<colgroup>`, `<thead>`, `<tbody>`, `<tfoot>`, `<tr>`, `<fieldset>`, `<select>`, `<details>` elemento e `<menu>` cujo tipo de atributo está em estado de menu de contexto.
Tags omitidas	Nenhuma, tanto a tag inicial quanto a final são obrigatórias.
Elementos pais permitidos	`<body>`, `<frameset>`, `<head>` and `<colgroup>` sem um atributo `span`
Interface do DOM	`HTMLTemplateElement`

Atributos

Esse elemento inclui os atributos globais.

Há também um atributo *content,* o qual é *read-only* e provê acesso ao conteúdo do template. A existência desse atributo *content* é frequentemente usada como um modo de determinar se o navegador do usuário suporta o element **`<template>`**.

Exemplo

Primeiro, nós começamos com um pedaço de HTML como exemplo:

```
<table id="producttable">
  <thead>
    <tr>
      <td>UPC_Code</td>
      <td>Product_Name</td>
    </tr>
  </thead>
  <tbody>
    <!-- existing data could optionally be included here -->
  </tbody>
</table>

<template id="productrow">
  <tr>
    <td class="record"></td>
    <td></td>
  </tr>
</template>
```

No início, nós temos uma tabela que irá receber conteúdo inserido via Javascript. Em seguida, vem o *template*, que descreve a estrutura de um fragmento de HTML que representa uma única linha da tabela.

Agora que a tabela foi criada e o *template* definido, nós usamos JavaScript para inserir linhas na tabela, cada linha é construída usando o *template* como base.

```
// Teste se seu navegador suporta o template HTML checando
// a presença do atribute content no elemento template .
if ("content" in document.createElement("template")) {
  // Instancie a tabela com o HTML tbody e a row com o template
  var t = document.querySelector("#productrow"),
    td = t.content.querySelectorAll("td");
  td[0].textContent = "1235646565";
  td[1].textContent = "Stuff";

  // Clone a nova row e insira-a na tabela
  var tb = document.getElementsByTagName("tbody");
  var clone = document.importNode(t.content, true);
  tb[0].appendChild(clone);

  // Crie uma nova row
  td[0].textContent = "0384928528";
```

```
  td[1].textContent = "Acme Kidney Beans";

  // Clone a nova row e insira-a na tabela
  var clone2 = document.importNode(t.content, true);
  tb[0].appendChild(clone2);
} else {
  // Ache outro modo de adicionar as rows na tabela, pois
  // o elemento HTML template não é suportado.
}
```

O resultado é a tabela original do HTML, com duas novas linhas inseridas via Javascript:

Veja também

- Web components: `<content>`, `<shadow>`

<textarea>

O **elemento** HTML `<textarea>` representa um controle de edição para uma caixa de texto, útil quando você quer permitir ao usuário informar um texto extenso em formato livre, como um comentário ou formulário de retorno.

Experimente

```
<label for="story">Tell us your story:</label>

<textarea id="story" name="story" rows="5" cols="33">
It was a dark and stormy night...
</textarea>
```

```
label,
textarea {
  font-size: 0.8rem;
  letter-spacing: 1px;
}

textarea {
  padding: 10px;
  max-width: 100%;
  line-height: 1.5;
  border-radius: 5px;
  border: 1px solid #ccc;
  box-shadow: 1px 1px 1px #999;
}

label {
  display: block;
  margin-bottom: 10px;
}
```

Os exemplos acima demonstram vários recursos do `<textarea>`. O primeiro exemplo mostra o uso mais simples, com apenas um `id` atributo para permitir que o`<textarea>` estar associado a um `<label>` elemento para fins de acessibilidade e um `name` atributo para definir o nome do ponto de dados associado enviado ao servidor quando o formulário é enviado.

O segundo exemplo mostra alguns recursos mais complexos:

- O `rows` e `cols` atributos permitem especificar um tamanho exato para o `<textarea>` pegar. Definir isso é uma boa idéia para manter a consistência, pois os padrões do navegador podem ser diferentes.
- `maxlength` especifica um número máximo de caracteres que o `<textarea>` tem permissão para conter. Você também pode definir um comprimento mínimo considerado válido usando o `minlength` atributo e especifique que o `<textarea>` não enviará (e é inválido) se estiver vazio, usando o`required` atributo. Isso fornece o `<textarea>` com validação simples, que é mais básica que os outros elementos do formulário (por exemplo, você não pode fornecer regexs específicas para validar o valor usando o método `pattern` atributo, como você pode com o `<input>` elemento).
- `wrap` especifica o comportamento de quebra automática do texto quando ele atinge a borda do `<textarea>`.

- Se você deseja conteúdo padrão para o seu `<textarea>`, você o insere entre as tags de abertura e fechamento. `<textarea>` não suporta o `value` atributo.

O `<textarea>` O elemento também aceita vários atributos comuns para formar `<input>`s, tal como`autocomplete`, `autofocus`, `disabled`, `placeholder`, `readonly`, e `required`.

Atributos

Este elemento inclui os atributos globais.

`autocapitalize`

Este é um atributo não padrão suportado pelo WebKit no iOS (portanto, quase todos os navegadores executados no iOS, incluindo Safari, Firefox e Chrome), que controla se e como o valor do texto deve ser automaticamente capitalizado quando é inserido / editado pelo do utilizador. Os valores não preteridos estão disponíveis no iOS 5 e posterior. Os valores possíveis são:

- `none`: Desativa completamente a capitalização automática.
- `sentences`: Coloque automaticamente em maiúscula a primeira letra das frases.
- `words`: Coloque em maiúscula automaticamente a primeira letra das palavras.
- `characters`: Colocar em maiúscula automaticamente todos os caracteres.
- `on`: Descontinuado desde o iOS 5.
- `off`: Descontinuado desde o iOS 5.

`autocomplete`

Este atributo indica se o valor do controle pode ser preenchido automaticamente pelo navegador. Os valores possíveis são:

- `off`: The user must explicitly enter a value into this field for every use, or the document provides its own auto-completion method; the browser does not automatically complete the entry.
- `on`: O navegador pode concluir automaticamente o valor com base nos valores inseridos pelo usuário durante os usos anteriores.

Se o `autocomplete` atributo não está especificado em um `<textarea>` elemento, o navegador usa o elemento `autocomplete` valor do atributo `<textarea>` proprietário do formulário do elemento. O proprietário do formulário é o `<form>` elemento que este`<textarea>` elemento é um descendente ou o elemento de formulário cuja `id` é especificado pelo `form` atributo do elemento de entrada. Para mais informações, consulte o `autocomplete` atribuno no `<form>`.

`autofocus`

Esse atributo booleano permite especificar que um controle de formulário tenha foco de entrada quando a página for carregada. Somente um elemento associado ao formulário em um documento pode ter esse atributo especificado.

`cols`

A largura visível do controle de texto, em larguras médias de caracteres. Se for especificado, deve ser um número inteiro positivo. Se não for especificado, o valor padrão é 20.

`disabled`

Esse atributo booleano indica que o usuário não pode interagir com o controle. Se esse atributo não for especificado, o controle herdará sua configuração do elemento que contém, por exemplo `<fieldset>`; se não houver elemento contendo quando o `disabled` atributo estiver definido, o controle está ativado.

`form`

O elemento do formulário que o `<textarea>` elemento está associado (seu "proprietário do formulário"). O valor do atributo deve ser o `id` de um elemento de formulário no mesmo documento. Se este atributo não for especificado, o atributo `<textarea>` O elemento deve ser um descendente de um elemento do formulário. Este atributo permite que você coloque`<textarea>` elementos em qualquer lugar do documento, não apenas como descendentes de elementos do formulário.

`maxlength`

O número máximo de caracteres (pontos de código unicode) que o usuário pode inserir. Se esse valor não for especificado, o usuário poderá inserir um número ilimitado de caracteres.

`minlength`

O número mínimo de caracteres (pontos de código unicode) exigidos pelo usuário.

`name`

O nome do controle.

`placeholder`

Uma dica para o usuário sobre o que pode ser inserido no controle. Retornos de carro ou feeds de linha no texto do espaço reservado devem ser tratados como quebras de linha ao renderizar a dica.

> **Note: Nota: Os espaços reservados devem ser usados apenas para mostrar um exemplo do tipo de dados que deve ser inserido em um formulário; eles não substituem uma adequada** `<label>` elemento vinculado à entrada. Veja Labels e placeholders para uma explicação completa.

`readonly`

Esse atributo booleano indica que o usuário não pode modificar o valor do controle. Ao contrário do `disabled` atributo, o`readonly` O atributo não impede o usuário de clicar ou selecionar no controle. O valor de um controle somente leitura ainda é enviado com o formulário.

`required`

Este atributo especifica que o usuário deve preencher um valor antes de enviar um formulário.

`rows`

O número de linhas de texto visíveis para o controle.

`spellcheck`

Especifica se o `<textarea>`está sujeito a verificação ortográfica pelo navegador / SO subjacente. o valor pode ser:

- `true`: Indica que o elemento precisa ter sua ortografia e gramática verificadas.
- `default` :Indica que o elemento deve agir de acordo com um comportamento padrão, possivelmente com base no próprio elemento pai `spellcheck` valor.
- `false` : Indica que o elemento não deve ter verificação ortográfica.

`wrap`

Indica como o controle quebra o texto. Os valores possíveis são:

- `hard`: O navegador insere automaticamente quebras de linha (CR + LF) para que cada linha não tenha mais que a largura do controle; a `cols` O atributo também deve ser especificado para que isso entre em vigor.
- `soft`: O navegador garante que todas as quebras de linha no valor consistam em um par CR + LF, mas não insira nenhuma quebra de linha adicional.
- `off` : Como `soft` mas muda a aparência para `white-space: pre` segmentos de linha que excedam `cols` não estão embrulhados e os `<textarea>` torna-se rolável horizontalmente.

Se este atributo não for especificado, `soft` é o seu valor padrão.

Estilo com CSS

`<textarea>` é um elemento substituído - possui dimensões intrínsecas, como uma imagem rasterizada. Por padrão, sua`display` o valor é bloco. Em comparação com outros elementos de formulário, é relativamente fácil de estilizar, com seu modelo de caixa, fontes, esquema de cores etc. sendo facilmente manipuláveis usando CSS comum.

O estilo de formulários HTML fornece algumas dicas úteis sobre estilo`<textarea>`s.

Inconsistência da linha de base

A especificação HTML não define onde a linha de base de um `<textarea>` ou seja, navegadores diferentes o definem em posições diferentes. Para Gecko, o `<textarea>` linha de base é definida na linha de base da primeira linha da primeira linha da área de texto, em outro navegador pode ser definida na parte inferior da `<textarea>`caixa. Não use `vertical-align: baseline` nele; o comportamento é imprevisível

Controlando se uma área de texto é redimensionável

Na maioria dos navegadores, `<textarea>`s são redimensionáveis - você notará a alça de arrasto no canto direito, que pode ser usada para alterar o tamanho do elemento na página. Isso é controlado pelo `resize` Propriedade CSS - o redimensionamento está ativado por padrão, mas você pode desativá-lo explicitamente usando um`resize` valor de`none`:

```
textarea {
  resize: none;
}
```

Estilo de valores válidos e inválidos

Valores válidos e inválidos de um `<textarea>` elemento (por exemplo, dentro e fora dos limites definidos por`minlength`, `maxlength`, ou`required`) pode ser destacado usando o `:valid` e`:invalid` pseudo-classes. Por exemplo, para atribuir à sua área de texto uma borda diferente, dependendo se é válida ou inválida:

```
textarea:invalid {
  border: 2px dashed red;
}

textarea:valid {
  border: 2px solid lime;
}
```

Exemplo

Exemplo básico

O exemplo a seguir mostra uma área de texto muito simples, com um número definido de linhas e colunas e algum conteúdo padrão.

```
<textarea name="textarea" rows="10" cols="50">Write something here</textarea>
```

Comprimento mínimo e máximo

Este exemplo tem um número mínimo e máximo de caracteres - de 10 e 20, respectivamente. Experimente e veja.

```
<textarea name="textarea" rows="5" cols="30" minlength="10" maxlength="20">
Write something here</textarea>
```

Observe que `minlength` não impede o usuário de remover caracteres para que o número digitado ultrapasse o mínimo, mas faz com que o valor digitado no `<textarea>` invalide.Observe também que, mesmo se você tiver um `minlength` conjunto de valores (3, por exemplo), um campo vazio `<textarea>` ainda é considerado válido, a menos que você também tenha o `required` conjunto de atributos.

Placeholder

Este exemplo tem um espaço reservado definido. Observe como ele desaparece quando você começa a digitar na caixa.

```
<textarea
  name="textarea"
  rows="5"
  cols="30"
  placeholder="Comment text."></textarea>
```

> **Note: Nota: Os espaços reservados devem ser usados apenas para mostrar um exemplo do tipo de dados que deve ser inserido em um formulário; eles não substituem uma adequada**`<label>` elemento vinculado à entrada. Veja Labels e placeholders para uma explicação completa.

Desativado e somente leitura

Este exemplo mostra dois `<textarea>`s — um dos quais é `disabled`, e o outro é `readonly`.Brinque com os dois e verá a diferença de comportamento - O `disabled` elemento não é selecionável de forma alguma (e seu valor não é enviado), enquanto o elemento `readonly` é selecionável e seu conteúdo copiável (e seu valor é enviado); você simplesmente não pode editar o conteúdo.

```
<textarea name="textarea" rows="5" cols="30" disabled>
I am a disabled textarea</textarea>
<textarea name="textarea" rows="5" cols="30" readonly>
I am a readonly textarea</textarea>
```

Resumo técnico

Categorias de conteúdo	Elemento associado ao formulário de conteúdo de fluxo, conteúdo de frases, conteúdo interativo, listado, rotulável, redefinível e submetível a envio.
Conteúdo permitido	Text
Omissão de tag	**Parents permitido**
Funções ARIA permitidas	None
DOM interface	`HTMLTextAreaElement`

Veja também

Outros elementos relacionados ao formulário:

- `<form>`
- `<button>`
- `<datalist>`
- `<legend>`
- `<label>`
- `<select>`
- `<optgroup>`
- `<option>`
- `<input>`
- `<fieldset>`
- `<output>`
- `<progress>`
- `<meter>`

<tfoot>: Elemento para o rodapé da tabela

O **`<tfoot>`** é um **elemento HTML** que define um conjunto de linhas as quais farão parte do rodapé de uma tabela HTML

Experimente

```
<table>
  <caption>
    Council budget (in £) 2018
  </caption>
  <thead>
```

```
    <tr>
      <th scope="col">Items</th>
      <th scope="col">Expenditure</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th scope="row">Donuts</th>
      <td>3,000</td>
    </tr>
    <tr>
      <th scope="row">Stationery</th>
      <td>18,000</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <th scope="row">Totals</th>
      <td>21,000</td>
    </tr>
  </tfoot>
</table>
```

```
thead,
tfoot {
  background-color: #2c5e77;
  color: #fff;
}

tbody {
  background-color: #e4f0f5;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
}

th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

td {
  text-align: center;
}
```

Categorias de conteúdo	Nenhum.
Conteúdo permitido	Zero ou mais `<tr>` elementos.
Omitir Tag	Iniciar a tag é obrigatório. O fechamento da tag pode ser omitida caso não exista mais conteúdo no elemento pai `<table>`.
Pais permitidos	O elemento `<table>`. O `<tfoot>` deve aparecer depois de qualquer `<caption>`, `<colgroup>`, `<thead>`, `<tbody>`, ou elementos `<tr>`. Note que isso é requerido no HTML5.

	O elemento `<tfoot>` não pode ser colocado depois de qualquer `<tbody>` e o elemento `<tr>`. Note que isso contradiz diretamento com as normas do HTML5.
Papeis de ARIA permitidos	Qualquer um
Interface DOM	`HTMLTableSectionElement`

Atributos

Este elemento inclui os atributos globais.

`align`

Estes atributos especificam como será o alinhamento horizontal de cada conteúdo da célula. Os possíveis valores são:

- `left`, alinha o conteúdo do elemento a esquerda da célula
- `center`, centraliza o conteúdo ao centro da célula
- `right`, alinha o conteúdo do elemento a direita da célula
- `justify`, insere espaços ao texto para que o mesmo fique centralizado na célula
- `char`, alinha o texto em um alinhamento especial com um deslocamento mínimo definido pelos atributos `char` e `charoff`.

Se nenhum atributo for definido o valor `left` é passado por padrão

> **Note:**Não use esse atributo o mesmo está obsoleto (não suportado) no último padrão.- Para obter os mesmo efeitos `left`, `center`, `right` ou `justify`, use a propriedade `text-align` do CSS.
>
> - Para obter o mesmo efeito no CSS3, você pode usar os valores `char` com o valor da propriedade `text-align`.

`bgcolor`

Este atributo define uma cor de fundo para cada célula da coluna. Cada um dos 6 dígitos define um código hexadecimal no sRGB, prefixado pelo '#'. Um desses nomes de cores predefinidos também podem ser usado:

	`black = "#000000"`		**`green = "#008000"`**
	`silver = "#C0C0C0"`		`lime = "#00FF00"`
	`gray = "#808080"`		`olive = "#808000"`
	`white = "#FFFFFF"`		`yellow = "#FFFF00"`
	`maroon = "#800000"`		`navy = "#000080"`
	`red = "#FF0000"`		`blue = "#0000FF"`
	`purple = "#800080"`		`teal = "#008080"`
	`fuchsia = "#FF00FF"`		`aqua = "#00FFFF"`

> **Nota:** Não use este atributo, como não é mais padrão ele não foi implementado em algumas versões do Microsoft Internet Explorer: o elemento `<tfoot>` pode ser estilizando com CSS. Para conseguir simular efeitos do atributo **bgcolor** use a propriedade `background-color` do CSS , nos elementos `<td>` ou `<th>`.

`char`

O elemento é usado para alinhar as células em uma columa. Valores típicos para isso inclui o periódico (.) quando se alinha valores monetários. Se `align` não é definido para `char`, este atributo é ignorado

> **Nota:** Não use esté atributo, ele é obsoleto (e não é mais suportado) desde da última versão padrão. Em vez disso use `char` no CSS3, você pode usar o atributo `char` com a propriedade `text-align`.

`charoff`

O atributo é usado para indicar um número de caracteres para compensar os dados da coluna dos caracteres de alinhamento especificados pelo atributo **char**.

> **Nota:** Não use esse atributo o mesmo está obsoleto (e não é mais suportado) na última versão padrão.

`valign`

O atributo específico para o alinhamento do texto dentro de cada linha da célula do cabeçalho de uma tabela. Possíveis valores para esse atributo são:

- `baseline`, coloca o texto o mais próximo possível da parte de baixo da célula, mas o alinha a baseline dos caracteres em vez do **bottom** deles. Caso os caracteres tenham todos os mesmo tamanho, eles terão o mesmo efeito que `bottom`.
- `bottom`, coloca o texto o mais próximo possível da parte de baixo da célula;
- `middle`, centraliza o texto na célula;
- e `top`, coloca o texto o mais próximo possível do topo da célula.

> **Nota:** Não use este atributo eles está obsoleto (e não é suportado) no último padrão: em vez disso use a propriedade `vertical-align` do CSS.

Exemplos

Por favor veja a página `<table>` para exemplos sobre `<tfoot>`.

Veja também

- Outros elemento HTML relacionados a tabela: `<caption>`, `<col>`, `<colgroup>`, `<table>`, `<tbody>`, `<td>`, `<th>`, `<thead>`, `<tr>`;
- Propriedades CSS e pseudo-classes são especialmente úteis para estilizar elementos `<tfoot>`:
  - a pseudo-class `:nth-child` para definir o alinhamento nas células de uma coluna;
  - a propriedade `text-align` para alinar as células do conteúdo que tenham um memso caractere como '.'.

`<th>`

O **elemento HTML `<th>`** define uma célula cabeçalho do grupo de células de sua tabela. A exatidão natural deste grupo é definida pelos atributos `scope` e `headers`.

Experimente

```
<table>
  <caption>
    Alien football stars
  </caption>
  <tr>
    <th scope="col">Player</th>
    <th scope="col">Gloobles</th>
    <th scope="col">Za'taak</th>
  </tr>
  <tr>
    <th scope="row">TR-7</th>
    <td>7</td>
    <td>4,569</td>
  </tr>
  <tr>
    <th scope="row">Khiresh Odo</th>
    <td>7</td>
    <td>7,223</td>
  </tr>
  <tr>
    <th scope="row">Mia Oolong</th>
    <td>9</td>
    <td>6,219</td>
```

```
  </tr>
</table>
```

```
th,
td {
  border: 1px solid rgb(160 160 160);
  padding: 8px 10px;
}

th[scope="col"] {
  background-color: #505050;
  color: #fff;
}

th[scope="row"] {
  background-color: #d6ecd4;
}

td {
  text-align: center;
}

tr:nth-of-type(even) {
  background-color: #eee;
}

table {
  border-collapse: collapse;
  border: 2px solid rgb(140 140 140);
  font-family: sans-serif;
  font-size: 0.8rem;
  letter-spacing: 1px;
}

caption {
  caption-side: bottom;
  padding: 10px;
}
```

Categorias do conteúdo	Nenhuma.
Conteúdo permitido	Conteúdo de fluxo, mas sem descendentes de cabeçalho, rodapé, conteúdo de seção ou conteúdo de cabeçalho.
Omissão de tag	A tag de abertura é obrigatória. A tag de fechamento pode ser omitida, se for imediatamente seguido por um `<th>`, `<td>` ou se não houver mais dados em seu elemento pai.
Elemento pai	Um elemento `<tr>`.
Funções ARIA permitidas	Qualquer
DOM interface	`HTMLTableHeaderCellElement`

Atributos

Esse elemento inclui os atributos globais.

`abbr`

> Este atributo contém uma breve descrição do conteúdo da célula. Alguns usuários-agentes, como leitores, pode apresentar essa descrição antes do próprio conteúdo.

`colspan`

Este atributo contém um valor inteiro não negativo que indica quantas colunas a célula ocupará. Valor padrão `1`. Valores acima de 1000 são considerados incorretos e serão modificados para o valor padrão `1`.

`headers`

Este atributo contém uma lista de palavras separadas por espaço, cada uma correspondendo ao atributo **id** dos elementos `<th>` que se aplicam a este elemento.

`rowspan`

Este atributo contém um valor inteiro não negativo que indica quantas linhas a célula estende. Valor padrão `1`. Se seu valor é definido como `0`, ele se estende até o final da tabela (`<thead>`, `<tbody>`, `<tfoot>`, mesmo que implicitamente definido), que a célula pertence. Não é possível colocar valores superiores a 65534.

`scope`

Este atributo define as células a que o cabeçalho (definido no elemento `<th>`) se relaciona. Possíveis valores:

- `row`: O cabeçalho se relaciona com todas as células da linha a que pertence.
- `col`: O cabeçalho se relaciona com todas as células da coluna a que pertence.
- `rowgroup`: O cabeçalho pertence a um grupo de linhas e se relaciona com todas as suas células. Essas células podem ser colocadas à direita ou à esquerda do cabeçalho, dependendo do valor do atributo `dir` no elemento `<table>`.
- `colgroup`: O cabeçalho pertence a um grupo de colgroup e se relaciona com todas as suas células.
- `auto`

O valor padrão quando este atributo não é especificado é `auto`.

Atributos depreciados

`align`

Este atributo especifica como o alinhamento horizontal do conteúdo da célula será tratado.

Possíveis valores:

- `left`: O conteúdo é alinhado a esquerda da célula.
- `center`: O conteúdo é centralizado da célula.
- `right`: O conteúdo é alinhado a direita da célula.
- `justify` (somente com texto): O conteúdo é estendido dentro da célula para cobrir toda a sua largura.
- `char` (somente com texto): O conteúdo está alinhado a um caractere dentro do elemento `<th>` com deslocamento mínimo. Esse caractere é definido pelos atributos `char` e `charoff`.

O valor padrão quando não especificado é `left`.

> **Note:**Não use esse atributo, pois ele está obsoleto no padrão mais recente.- Para alcançar o mesmo efeito que os valores `left`, `center`, `right` ou `justify`, aplique a propriedade CSS `text-align` no elemento.
>
> - Para alcançar o mesmo efeito que os valor `char`, atribue a `text-align` o mesmo valor que você usaria para `char`.

`axis`

Este atributo contém uma lista de palavras separadas por espaço. Cada palavra é o `id` de um grupo de células às quais este cabeçalho se aplica.

> **Nota:** Não use esse atributo, pois ele está obsoleto no padrão mais recente: use o atributo `scope`.

`bgcolor`

Esse atributo define a cor de fundo de cada célula na coluna. Ele consiste de 6 digitos hexadecimais definidos pelo sRGB com o prefixo '#. Nesse atributo pode ser usado dezesseis cores predefinidas:

	`black = "#000000"`		**`green = "#008000"`**
	`silver = "#C0C0C0"`		`lime = "#00FF00"`
	`gray = "#808080"`		`olive = "#808000"`
	`white = "#FFFFFF"`		`yellow = "#FFFF00"`
	`maroon = "#800000"`		`navy = "#000080"`
	`red = "#FF0000"`		`blue = "#0000FF"`
	`purple = "#800080"`		`teal = "#008080"`
	`fuchsia = "#FF00FF"`		`aqua = "#00FFFF"`

> **Nota:** Não use esse atributo como padrão pois não é implementado em algumas versões do Microsoft Internet Explorer: O elemento `<th>` deve ser estilizado usando CSS. Para criar um efeito semelhante, use a propriedade `background-color` do CSS.

`char`

O conteúdo da célula está alinhado a um caractere. Os valores típicos incluem um ponto (.) para alinhar números ou valores monetários. Se `align` não está definido no `char`, o atributo é ignorado.

> **Nota:** Não use esse atributo, pois ele está obsoleto no padrão mais recente. Para obter o mesmo efeito, você pode especificar o caractere como o primeiro valor da propriedade `text-align`.

`charoff`

Este atributo é usado para mudar os dados da coluna para a direita do caractere especificado pelo atributo **char**. Seu valor especifica o comprimento desse deslocamento.

> **Note:**Não use esse atributo, pois ele está obsoleto no padrão mais recente.

`height`

Esse atributo é usado para definir uma altura recomendada da célula.

> **Note:**Não use esse atributo, pois ele está obsoleto no padrão mais recente: use a propriedade CSS `height`.

`valign`

Esse atributo especifica como o texto é alinhado verticalmente na célula.

Possíveis valores:

- `baseline`: Posiciona o texto próximo à parte inferior da célula e o alinha ao roda pé. Se os caracteres não descerem abaixo da linha de base, o valor da linha de base alcançará o mesmo efeito que `bottom`.
- `bottom`: Posiciona o texto próximo à parte inferior da célula.
- `middle`: Centraliza o texto na célula.
- `top`: Posiciona o texto próximo à parte superior da célula.

> **Nota:** Não use esse atributo, pois ele está obsoleto no padrão mais recente: use a propriedade CSS `vertical-align`.

`width`

Este atributo é usado para definir uma largura de célula recomendada. Espaço adicional pode ser adicionado com as propriedades `cellspacing` e `cellpadding`, e a largura do elemento `<col>` pode criar largura extra. Mas, se a largura de uma coluna for muito estreita para mostrar uma célula específica corretamente, ela será ampliada quando exibida.

> **Nota:** Não use esse atributo, pois ele está obsoleto no padrão mais recente: use a propriedade CSS `width`.

Exemplos

Veja `<table>` para mais elementos `<th>`.

Veja mais

- Outros elementos HTML relacionados: `<caption>`, `<col>`, `<colgroup>`, `<table>`, `<tbody>`, `<td>`, `<tfoot>`, `<thead>`, `<tr>`.

<time>

Resumo

O elemento HTML *time* (`<time>`) representa o tempo tanto no formato de 24 horas ou como uma data precisa no calendário Gregoriano (com informações opcionais de tempo e fuso horário)

Este elemento é destinado a apresentar datas e horas no dispositivo em um formato legível. Isto pode ser útil aos agentes do usuário em oferecer qualquer programação de eventos ao calendário do utilizador.

> **Note: Notas de uso:** Este elemento não é adequado por exemplo, quando uma data específica não pode ser calculada, e nem deveria ser utilizado em datas anteriores ao calendário Gregoriano (devido as complicações com os cálculos dessas datas).

> **Note: Nota de estatus:** Este elemento foi concebido e está em discussão desde 2011 (http://blog.whatwg.org/weekly-time-data)

Contexto de uso

Categorias de conteúdo	flow content, phrasing content
Permitted content	Phrasing content, but no descendant `time` elements.
Tag omission	None, both the start tag and the end tag are mandatory.
Permitted parent elements	Any element that accepts phrasing content.
Normative document	HTML5, section 4.6.10

Atributos

Como todo outro elemento HTML, este elemento suporta os atributos globais.

`datetime`

Esse atributo indica a hora e a data do elemento e deve ser uma data válida com uma string de tempo opcional. Se o valor não puder ser analisado como uma data com uma string de tempo opcional, o elemento não possui um timestamp associado.

`pubdate`

(This feature is still being discussed by the WHATWG and the W3C HTML5 Working Group.) This Boolean attribute specifies that the date and time given by the element is the publication date of a document. The document it applies to is either the nearest ancestor article element or the document as a whole (if there is no ancestor article element). If true, the `time` element must have a corresponding date. Additionally, each `time` element indicating a publication date must be the only `time` element that does so for that document.

DOM interface

This element implements the `HTMLTimeElement` interface.

Exemplos

Simple example

```
<p>The concert starts at <time>20:00</time>.</p>
```

Above HTML will output:

The concert starts at 20:00.

<code>pubdate</code> example

```
<article>
  <p>This article was created on <time pubdate>2011-01-28</time>.</p>
</article>
```

Above HTML will output:

This article was created on 2011-01-28.

<code>datetime</code> example

```
<p>
  The concert took place on <time datetime="2001-05-15 19:00">May 15</time>.
</p>
```

Above HTML will output:

The concert took place on May 15.

Veja também

- The `time` element (HTML5 specification)

<title>: O titulo do documento.

O **elemento HTML** `<title>` (*Elemento HTML Título*) define o título do documento, mostrado na barra de título de um navegador ou na aba da página. Pode conter somente texto e quaisquer marcações contidas no texto não são interpretadas.

- *Categorias de conteúdo* Conteúdo de metadados.
- *Conteúdo permitido* Texto que não seja whitespace entre elementos.
- *Omissão de marcação* Todas as marcações são necessárias. Perceba que suprimir `</title>` fará o navegador ignorar o resto da página.
- *Elementos pais permitidos* Um elemento `<head>` que não contenha outro elemento `<title>`.
- *Funções ARIA permitidas* Nenhuma.
- *Interface DOM* `HTMLTitleElement`

Atributos

Este elemento inclui somente os atributos globais.

Notas de uso

O elemento `<title>` é sempre usado dentro da `<head>` da página.

Exemplo

```
<title>Título incrível</title>
```

Preocupações de acessibilidade

É importante informar um valor para o `title` que descreva o propósito da página.

Uma técnica comum de navegação para usuários de tecnologias assistivas é ler o título da página a fim de deduzir o que ela contém. Isso ocorre porque navegar pela página para determinar o conteúdo dela pode ser confuso e consumir muito tempo.

Exemplo

```
<title>
  Menu - Casa Azul Comida Chinesa - FoodYum: Faça seu pedido online!
</title>
```

Para ajudar o usuário, atualize o valor do `title` para refletir mudanças significativas no estado da página (como problemas de validação em formulários).

```
<title>2 erros - Seu pedido - Casa Azul Comida Chinesa - FoodYum: Faça seu pedido online!</title>
```

<track>

O **elemento HTML `<track>` **é usado como filho dos elementos de mídia`<audio>` e `<video>`. Ele permite que você especifique faixas de texto temporizadas (ou dados baseados em tempo), por exemplo, para lidar automaticamente com legendas. As faixas são formatadas em WebVTT format (arquivos `.vtt`) — Web Video Text Tracks or Timed Text Markup Language (TTML).

Experimente

```
<video controls src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/track//shared-
assets/videos/friday.mp4">
  <track
    default
    kind="captions"
    srclang="en"
    src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/track//shared-assets/misc/friday.vtt" />
  Download the
  <a href="https://developer.mozilla.org/shared-assets/videos/friday.mp4">MP4</a>
  video, and
  <a href="https://developer.mozilla.org/shared-assets/misc/friday.vtt">subtitles</a>.
</video>
```

```
video {
  width: 250px;
}

video::cue {
  font-size: 1rem;
}
```

Categorias de conteúdo	Nenhuma
Conteúdo permitido	Nenhum, é um empty element.
Omissão de tag	Como é um elemento vazio, a tag inicial deve estar presente e a tag final não deve estar presente.
Permite Parentes	Um elemento de mídia, antes de qualquer conteúdo de fluxo.
Funções ARIA permitidas	Nenhum
Interface DOM	`HTMLTrackElement`

Atributos

Este elemento inclui os atributos globais.

`default`

This attribute indicates that the track should be enabled unless the user's preferences indicate that another track is more appropriate. This may only be used on one `track` element per media element.

`kind`

How the text track is meant to be used. If omitted the default kind is `subtitles`. If the attribute is not present, it will use the `subtitles`. If the attribute contains an invalid value, it will use `metadata`. (Versions of Chrome earlier than 52 treated an invalid value as `subtitles`.) The following keywords are allowed:

- `subtitles`
  - Subtitles provide translation of content that cannot be understood by the viewer. For example dialogue or text that is not English in an English language film.
  - Subtitles may contain additional content, usually extra background information. For example the text at the beginning of the Star Wars films, or the date, time, and location of a scene.
- `captions`
  - Closed captions provide a transcription and possibly a translation of audio.
  - It may include important non-verbal information such as music cues or sound effects. It may indicate the cue's source (e.g. music, text, character).
  - Suitable for users who are deaf or when the sound is muted.
- `descriptions`
  - Textual description of the video content.
  - Suitable for users who are blind or where the video cannot be seen.
- `chapters`
  - Chapter titles are intended to be used when the user is navigating the media resource.
- `metadata`
  - Tracks used by scripts. Not visible to the user.

`label`

A user-readable title of the text track which is used by the browser when listing available text tracks.

`src`

Address of the track (`.vtt` file). Must be a valid URL. This attribute must be specified and its URL value must have the same origin as the document — unless the `<audio>` or `<video>` parent element of the `track` element has a `crossorigin` attribute.

`srclang`

Language of the track text data. It must be a valid BCP 47 language tag. If the `kind` attribute is set to `subtitles`, then `srclang` must be defined.

Usage notes

The type of data that `track` adds to the media is set in the `kind` attribute, which can take values of `subtitles`, `captions`, `descriptions`, `chapters` or `metadata`. The element points to a source file containing timed text that the browser exposes when the user requests additional data.

A `media` element cannot have more than one `track` with the same `kind`, `srclang`, and `label`.

Examples

```
<video controls poster="/images/sample.gif">
  <source src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/track/sample.mp4" type="video/mp4"
/>
  <source src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/track/sample.ogv" type="video/ogv"
/>
  <track kind="captions" src="https://developer.mozilla.org/pt-
```

```
BR/docs/Web/HTML/Reference/Elements/track/sampleCaptions.vtt" srclang="en" />
  <track kind="descriptions" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/track/sampleDescriptions.vtt" srclang="en" />
  <track kind="chapters" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/track/sampleChapters.vtt" srclang="en" />
  <track kind="subtitles" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/track/sampleSubtitles_de.vtt" srclang="de" />
  <track kind="subtitles" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/track/sampleSubtitles_en.vtt" srclang="en" />
  <track kind="subtitles" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/track/sampleSubtitles_ja.vtt" srclang="ja" />
  <track kind="subtitles" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/track/sampleSubtitles_oz.vtt" srclang="oz" />
  <track kind="metadata" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/track/keyStage1.vtt" srclang="en" label="Key Stage 1" />
  <track kind="metadata" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/track/keyStage2.vtt" srclang="en" label="Key Stage 2" />
  <track kind="metadata" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Elements/track/keyStage3.vtt" srclang="en" label="Key Stage 3" />
  <!-- Fallback -->
  ...
</video>
```

See also

- WebVTT text track format

O **elemento HTML`<ul>`** (ou *elemento HTML de Lista desordenada*) representa uma lista de itens sem ordem rígida, isto é, uma coleção de itens que não trazem uma ordenação numérica e as suas posições, nessa lista, são irrelevantes. Caracteristicamente, os itens em uma lista desordenada são exibidos com um marcador que pode ter várias formas, como um ponto, um círculo, ou um quadrado. O tipo de marcador não é definido na descrição HTML da página, mas na CSS associada, utilizando a propriedade `list-style-type`.

Não há nenhuma limitação para a profundidade e a imbricação das listas definidas com os elementos `<ol>` e `<ul>`.

> **Note: Nota sobre a utilização:** Ambos os elementos `<ol>` e `<ul>` representam uma lista de itens. São diferentes porque, com o elemento `<ol>`, a ordenação tem significado. Como regra de ouro para determinar qual dos dois usar, tente mudar a posição dos itens na lista; caso a significação seja alterada, o elemento `<ol>` deve ser utilizado mas, se não houver mudança, você pode aplicar `<ul>`.

Contexto de utilização

Categorias de conteúdo	Flutuantes
Conteúdo permitido	zero ou mais `<li>` elementos, eventualmente combinados com `<ol>` e elementos `<ul>`.
Omissão de etiqueta (*Tag*)	nenhuma, ambas as etiquetas - de início e de fim - são obrigatórias
Elementos pai permitidos	qualquer elemento que receba conteúdo flutuante
Interface DOM	`HTMLUListElement`

Atributos

Este elemento inclui os atributos globais.

`compact`

> Este atributo booleano sugere que a lista será processada em um modelo compacto. A interpretação deste atributo depende do perfil de navegação (*user agent*) e não funciona em todos os navegadores.

> **Note: Nota de utilização:** Não aplique este atributo, que foi preterido - o elemento `<ul>` deve ser definido utilizando-se a folha de estilos CSS. Para dar um efeito similar ao atributo compacto, a propriedade line-height (espaçamento), da CSS, pode ser utilizada com um valor de 80%.

`type`

Usados para estabelecer o tipo de marcador da lista. Os valores definidos durante a HTML3.2 e a versão de transição de HTML 4.0/4.01, são:

- `círculo`,
- `disco`,
- e `quadrado`.

Um quarto tipo de marcação está definido na interface *WebTV*, mas nem todos os navegadores o acolhem: `triângulo`.

Se não estiver presente e se nenhuma propriedade CSS `list-style-type` se aplicar ao elemento, o perfil de navegação decidirá qual tipo de marcador utilizar, dependendo do nível de aninhamento da lista.

> **Note: Nota de utilização:** Não use este atributo, pois está ultrapassado; em seu lugar, utilize a propriedade CSS `list-style-type`.

Exemplos

Exemplo simples

```
<ul>
  <li>primeiro item</li>
  <li>segundo item</li>
  <li>terceiro item</li>
</ul>
```

A HTML acima resulta em:

- primeiro item
- segundo item
- terceiro item

Lista aninhada

```
<ul>
  <li>primeiro item</li>
  <li>
    segundo item
    <!-- Observe que a tag de fechamento </li> não é colocada aqui! -->
    <ul>
      <li>segundo item primeiro subitem</li>
      <li>
        segundo item segundo subitem
        <!-- O mesmo para a segunda lista não ordenada aninhada (Same for the second nested unordered list)! -->
        <ul>
          <li>segundo item segundo subitem primeiro sub-subitem</li>
          <li>segundo item segundo subitem segundo sub-subitem</li>
          <li>segundo item segundo subitem terceiro sub-subitem</li>
        </ul>
      </li>
      <!-- A tag de fechamento </li>, que contém a terceira lista não ordenada (Closing </li> tag for the li that
contains the third unordered list) -->
      <li>segundo item terceiro subitem</li>
    </ul>
  </li>
  <!-- Aqui entra a tag de fechamento </li> (Here is the closing </li> tag) -->
```

```
  <li>terceiro item</li>
</ul>
```

A saída HTML acima, é:

- primeiro item
- segundo item
    - segundo item primeiro subitem
    - segundo item segundo subitem
        - segundo item segundo subitem primeiro sub-subitem
        - segundo item segundo subitem segundo sub-subitem
        - segundo item segundo subitem terceiro sub-subitem
    - segundo item terceiro subitem
- terceiro item

 e <ol> aninhados

```
<ul>
  <li>primeiro item</li>
  <li>
    segundo item
    <!-- Observe, a tag de fechamento </li> não é colocada aqui! (Look, the closing </li> tag is not placed here!) -->
    <ol>
      <li>segundo item primeiro subitem</li>
      <li>segundo item segundo subitem</li>
      <li>segundo item terceiro subitem</li>
    </ol>
  </li>
  <!-- Aqui está a tag de fechamento </li> (Here is the closing </li> tag) -->
  <li>terceiro item</li>
</ul>
```

A saída HTML acima será:

- primeiro item
- segundo item
    1. segundo item primeiro subitem
    2. segundo item segundo subitem
    3. segundo item terceiro subitem
- terceiro item

Veja também

- Outros elementos HTML relacionados à lista: `<ol>`, `<li>`, `<menu>` e o obsoleto `<dir>`;
- Propriedades CSS que podem ser especialmente úteis para determinar o modelo do elemento <ul>:
    - a propriedade list-style, conveniente para escolher a maneira como os ordinais serão mostrados,
    - CSS counters, eficientes para guiar listas complexas aninhadas,
    - a propriedade line-height, válida para simular o atributo ultrapassado `compact`,
    - a propriedade margin, proveitosa para controlar a indentação da lista.

<var>

Resumo

O elemento HTML Variable (`<var>`) representa uma variável em uma expressão matemática ou um contexto de programação.

Contexto de utilização

Permitted content	Phrasing content
Tag omission	Both start and end tags are required
Permitted parent elements	Any element that accepts phrasing content.
Normative document	HTML, "The `var` element"

Atributos

Este elemento somente inclui os global attributes.

Interface DOM

Este elemento implementa a interface `HTMLElement`.

> **Note: Implementation note:** up to Gecko 1.9.2 inclusive, Firefox implements the HTMLSpanElement interface for this element.

Propriedades típicas padrão de estilo

```
var {
  font-style: italic;
}
```

Exemplo

```
<p>A simple equation: <var>x</var> = <var>y</var> + 2</p>
```

A simple equation: *x* = *y* + 2

<video>: O elemento HTML de incorporação de Vídeo

O elemento HTML **`<video>`** incorpora um reprodutor de mídia que suporta a reprodução de vídeo no documento. Você também pode usar `<video>` para conteúdo de áudio, mas o elemento `<audio>` pode proporcionar uma experiência de usuário mais adequada.

Experimente

```
<video controls width="250">
  <source src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/video//shared-
assets/videos/flower.webm" type="video/webm" />

  <source src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/video//shared-
assets/videos/flower.mp4" type="video/mp4" />

  Download the
  <a href="https://developer.mozilla.org/shared-assets/videos/flower.webm">WEBM</a>
  or
  <a href="https://developer.mozilla.org/shared-assets/videos/flower.mp4">MP4</a>
  video.
</video>
```

O exemplo acima mostra um uso simples do elemento `<video>`. Semelhante ao elemento `<img>`, incluímos o caminho da mídia que queremos exibir dentro do atributo `src`; também podemos incluir outros atributos para especificar informações, como largura e altura do vídeo, se queremos que ele seja reproduzido automaticamente e em loop, se queremos mostrar os controles de vídeo padrão do navegador etc.

O conteúdo dentro das tags de abertura e fechamento `<video></video>` é exibido como uma alternativa (*fallback*) em navegadores que não suportam o elemento.

Atributos

Como qualquer elemento HTML, este elemento suporta os atributos globais.

`autoplay`

Um atributo Booleano; se especificado, o video vai ser executado assim que possível sem precisar de carregar todo o arquivo.

> **Nota:** Sites que reproduzem automaticamente áudio (ou vídeos com uma faixa de áudio) podem proporcionar uma experiência desagradável para os usuários, portanto, devem ser evitados sempre que possível. Se você precisar oferecer a funcionalidade de reprodução automática, é recomendável torná-la opcional (exigindo que o usuário a habilite especificamente). No entanto, isso pode ser útil ao criar elementos de mídia cuja fonte será definida posteriormente, sob controle do usuário. Consulte nosso guia sobre reprodução automática para obter informações adicionais sobre como usar a reprodução automática corretamente.

Para desativar a reprodução automática, `autoplay="false"` não vai funcionar; o vídeo será reproduzido automaticamente se o atributo estiver presente na tag `<video>`. Para remover a reprodução automática, o atributo deve ser removido por completo.

A reprodução automática não funciona em alguns navegadores (ex., Chrome 70.0) se o atributo `muted` estiver presente.

`controls`

Se esse atributo estiver presente, o navegador oferecerá controles para permitir o usuário controlar a reprodução do vídeo, incluindo volume, navegação (seek), e pausa/continuação da reprodução.

`controlslist`

O atributo `controlslist`, quando definido, ajuda o navegador a selecionar que controles mostrar no elemento `video` sempre que o navegador exibir o seu próprio conjunto de controles (ou seja, quando o atributo `controls` for especificado).

Os valores permitidos são `nodownload`, `nofullscreen` e `noremoteplayback`.

Use o atributo `disablepictureinpicture` se você quiser desabilitar o modo Picture-In-Picture (e o controle).

`crossorigin`

Esse atributo enumerado indica se deve usar CORS para buscar o vídeo relacionado. Recursos habilitados para CORS podem ser reutilizados no elemento `<canvas>` sem serem *contaminados*. Os valores permitidos são:

- `anonymous` Envia uma solicitação de origem cruzada sem uma credencial. Em outras palavras, ele envia o cabeçalho HTTP `Origin:` sem um cookie, certificado X.509 ou realizar uma autenticação HTTP Basic. Se o servidor não fornecer credenciais ao site de origem (não definindo o cabeçalho HTTP `Access-Control-Allow-Origin:`), o recurso será contaminado e seu uso restrito.
- `use-credentials`
  - : Envia uma solicitação de origem cruzada com uma credencial. Em outras palavras, envia o cabeçalho HTTP `Origin:` com um cookie, um certificado, ou realiza autenticação HTTP Basic. Se o servidor não fornecer credenciais ao o site de origem (por meio do cabeçalho HTTP `Access-Control-Allow-Credentials:`), o recurso será contaminado e seu uso será restrito.

Quando não está presente, o recurso é buscado sem uma solicitação CORS (ou seja, sem enviar o cabeçalho HTTP `Origin:`), impedindo seu uso não contaminado em elementos `<canvas>`. Se for inválido, é tratado como se a palavra-chave enumerada `anonymous` tivesse sido usada. Consulte os atributos de configuração de CORS para obter informações adicionais.

`disablepictureinpicture`

Impede o navegador de sugerir um menu de contexto Picture-in-Picture ou de solicitar automaticamente o Picture-in-Picture em alguns casos..

`disableremoteplayback`

Um atributo Booleano usado para desativar a capacidade de reprodução remota em dispositivos conectados usando tecnologias com fio (HDMI, DVI, etc.) e sem fio (Miracast, Chromecast, DLNA, AirPlay, etc.).

No Safari, você pode usar `x-webkit-airplay="deny"` como alternativa.

`height`

A altura da área de exibição do vídeo, em pixels de CSS (apenas valores absolutos; sem porcentagens).

`loop`

Um atributo Booleano; se especificado, ao chegar no fim do vídeo, ele voltará automaticamente para o começo.

`muted`

Um atributo Booleano que indica a configuração padrão do áudio contido no vídeo. Se definido, o áudio vai começar mudo. Seu valor padrão é falso, significando que o áudio será reproduzido juntamente com o vídeo.

`playsinline`

Um atributo booleano que indica que o vídeo deve ser reproduzido "em linha" (inline), ou seja, dentro da área de reprodução do elemento. Observe que a ausência deste atributo *não* implica que o vídeo será sempre reproduzido em tela cheia.

`poster`

Uma URL indicando uma imagem de prévia do vídeo até o usuário reproduzir ou navegar por ele. Se este atributo não estiver especificado, nada será mostrado até que o primeiro quadro esteja disponível; então o primeiro quadro será exibido como imagem de prévia.

`preload`

Esse atributo enumerado pretende dar uma sugestão ao navegador sobre o que o autor pensa que proporcionará uma melhor experiência do usuário. Ele pode ter os seguintes valores:

- `none`: indica que o usuário não necessitará consultar o vídeo ou que o servidor quer minimizar seu tráfego; em outros termos indica que o vídeo não deve ser pré-carregado.
- `metadata`: indica que embora o usuário não necessitará consultar o vídeo, pegar os meta-dados (ex: comprimento) é interessante.
- `auto`: indica que o usuário necessita ter prioridade; em outros termos isso indicou que, se necessário, o vídeo inteiro pode ser baixado, mesmo que não seja esperado a execução.
- a *string vazia*: é um sinônimo do valor `auto`.

Se não definido, seu valor padrão será definido pelo navegador (isto é, cada navegador pode escolher seu valor padrão), embora a especificação recomende que seja definido para o `metadata`.

> **Nota:**
>
> - O atributo `autoplay` tem precedência sobre o `preload`, pois se é necessário executar o vídeo automaticamente, o navegador obviamente o baixará. Definindo ambos `autoplay` e `preload` é permitido pela especificação.
> - O navegador não é forçado pela especificação a seguir o valor desse atributo; é apenas uma sugestão.

`src`

A URL do vídeo a ser incorporado. Isto é opcional; ao invés disso você pode usar o elemento `<source>` dentro do bloco do vídeo para especificar o vídeo a ser incorporado .

`width`

A largura da área de exibição do vídeo, em pixels de CSS (apenas valores absolutos; sem porcentagens).

Eventos

Nome do evento	Quando é disparado
`audioprocess`	O buffer de entrada de um `ScriptProcessorNode` está pronto para ser processado.
`canplay`	O navegador pode reproduzir a mídia, mas estima que não há dados suficientes carregados para reproduzir a mídia até o final sem ter que parar para carregar mais conteúdo.
`canplaythrough`	O navegador estima que pode reproduzir a mídia até o final sem precisar parar para carregar mais conteúdo.
`complete`	A renderização de um `OfflineAudioContext` foi concluída.
`durationchange`	O atributo `duration` foi atualizado.
`emptied`	A mídia ficou vazia; por exemplo, este evento é enviado se a mídia já foi carregada (ou parcialmente carregada), e o método `load()` é chamado para recarregá-la.
`ended`	A reprodução foi interrompida porque o final da mídia foi alcançado.
`error`	Ocorreu um erro ao buscar os dados da mídia, ou o tipo do recurso não é um formato de mídia suportado.
`loadeddata`	O primeiro quadro da mídia terminou de carregar.
`loadedmetadata`	Os metadados foram carregados.
`loadstart`	Disparado quando o navegador começou a carregar o recurso.
`pause`	A reprodução foi pausada.
`play`	A reprodução começou.
`playing`	A reprodução está pronta para começar depois de ter sido pausada ou atrasada devido à falta de dados.
`progress`	Disparado periodicamente conforme o navegador carrega um recurso.
`ratechange`	A taxa de reprodução foi alterada.
`seeked`	Uma operação de *navegação* foi concluída.
`seeking`	Uma operação de *navegação* foi iniciada.
`stalled`	O agente do usuário está tentando buscar dados de mídia, mas os dados não estão sendo recebidos conforme o esperado.
`suspend`	O carregamento dos dados da mídia foi suspenso.
`timeupdate`	O tempo indicado pelo atributo `currentTime` foi atualizado.
`volumechange`	O volume foi alterado.
`waiting`	A reprodução foi interrompida devido à falta temporária de dados.

Notas de uso

Os navegadores não dão suporte a todos os mesmos formatos de vídeo; você pode fornecer múltiplas fontes dentro de elementos `<source>` aninhados, e o navegador utilizará a primeira que tiver suporte.

```
<video controls>
  <source src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/video/meuVideo.webm"
type="video/webm" />
  <source src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Elements/video/meuVideo.mp4"
type="video/mp4" />
  <p>
    O seu navegador não tem suporte a vídeo HTML. Em vez disso, aqui está
    <a href="meuVideo.mp4" download="meuVideo.mp4">o link do vídeo</a>.
  </p>
</video>
```

Nós oferecemos um guia abrangente e detalhado sobre tipos de arquivos de mídia e o guia sobre os codecs compatíveis com vídeo. Também está disponível um guia sobre os codecs de áudio que podem ser usados com eles.

Outras notas de uso:

- Se você não especificar o atributo `controls`, o vídeo não incluirá os controles padrão do navegador; você pode criar seus próprios controles personalizados usando JavaScript e a API `HTMLMediaElement`. Consulte Criando um player de vídeo compatível com vários navegadores para obter mais detalhes.
- Para permitir o controle preciso do conteúdo do seu vídeo (e áudio), os elementos `HTMLMediaElement` disparam muitos eventos diferentes. Além de fornecer controlabilidade, esses eventos permitem que você monitore o progresso tanto do download quanto da reprodução da mídia, bem como o estado e a posição da reprodução.
- Você pode usar a propriedade `object-position` para ajustar o posicionamento do vídeo dentro do quadro do elemento e a propriedade `object-fit` para controlar como o tamanho do vídeo é ajustado para caber dentro do quadro.
- Para exibir legendas/legendas junto com o seu vídeo, você pode usar JavaScript junto com o elemento `<track>` e o formato WebVTT. Consulte Adicionando legendas em um vídeo HTML para obter mais informações.
- Você pode reproduzir arquivos de áudio usando um elemento `<video>`. Isso pode ser útil, por exemplo, se você precisar executar áudio com uma transcrição WebVTT, já que o elemento `<audio>` não permite legendas usando WebVTT.
- Para testar o conteúdo de alternativa (*fallback*) em navegadores que suportam o elemento, você pode substituir `<video>` por um elemento que não exista, como `<notavideo>`.

Uma boa fonte geral de informações sobre o uso de HTML `<video>` é o tutorial para iniciantes em Conteúdo de vídeo e áudio.

Estilização com CSS

O elemento `<video>` é um replaced element — seu valor `display` é `inline` por padrão, mas sua largura e altura padrão na viewport são definidas pelo vídeo que está sendo incorporado.

Não há considerações especiais para estilizar `<video>`; uma estratégia comum é atribuir a ele um valor `display` de `block` para facilitar o posicionamento, o tamanho etc. e, em seguida, fornecer informações de estilo e layout conforme necessário. Noções básicas de estilo do player de vídeo fornece algumas técnicas de estilo úteis.

Detectando a adição e a remoção de faixas

Você pode detectar quando as faixas são adicionadas e removidas de um elemento `<video>` usando os eventos `addtrack` e `removetrack`. Entretanto, esses eventos não são enviados diretamente para o próprio elemento `<video>`. Em vez disso, eles são enviados para o objeto da lista de faixas dentro do `HTMLMediaElement` do elemento `<video>` que corresponde ao tipo de faixas que foi adicionada ao elemento:

`HTMLMediaElement.audioTracks`

> Um `AudioTrackList` contendo todas as faixas de áudio do elemento de mídia. Você pode adicionar um listener no evento `addtrack` deste objeto para ser alertado quando novas faixas de áudio forem adicionadas ao elemento.

`HTMLMediaElement.videoTracks`

> Adicione um listener `addtrack` a este objeto `VideoTrackList` para ser informado quando faixas de vídeo forem adicionadas ao elemento.

`HTMLMediaElement.textTracks`

> Adicione um listener de evento `addtrack` a este `TextTrackList` para ser notificado quando novas faixas de texto forem adicionadas ao elemento.

Por exemplo, para detectar quando faixas de áudio são adicionadas ou removidas de um elemento `<video>`, você pode usar um código como este:

```
const elem = document.querySelector("video");

elem.audioTracks.onaddtrack = (event) => {
  trackEditor.addTrack(event.track);
};

elem.audioTracks.onremovetrack = (event) => {
  trackEditor.removeTrack(event.track);
};
```

Este código monitora as faixas de áudio que foram adicionadas ou removidas do elemento e chama uma função hipotética em um editor de faixas para registrar e remover a faixa da lista de faixas disponíveis do editor.

Você também pode usar `addEventListener()` para ouvir os eventos `addtrack` e `removetrack`.

Suporte do servidor para vídeo

Se o tipo MIME do vídeo não estiver definido corretamente no servidor, o vídeo poderá não ser mostrado ou uma caixa cinza contendo um X (se o JavaScript estiver habilitado) será exibida em seu lugar.

Se você disponibiliza seu vídeos como Ogg Theora, você pode corrigir esse problema para o servidor web Apache adicionando a extensão utilizada por seu arquivos de vídeo (".ogm", ".ogv", e ".ogg" são as mais comuns) para o tipo MIME "video/ogg" através do arquivo "mime.types" localizado em "/etc/apache" ou através da configuração de diretiva "AddType" no httpd.conf.

```
AddType video/ogg .ogm
AddType video/ogg .ogv
AddType video/ogg .ogg
```

Se você disponibilizar seu vídeos como WebM, você pode corrigir esse problema para o servidor web Apache adicionando a extensão usada por seu arquivos de vídeo (".webm" é a mais comum) para o tipo MIME "video/webm" através do arquivo "mime.types" localizado em "/etc/apache" ou através da configuração de diretiva "AddType" no httpd.conf.

```
AddType video/webm .webm
```

Seu serviço de hospedagem talvez forneça uma interface simples para mudar a configuração dos tipos MIME para as novas tecnologias até que uma atualização global ocorra naturalmente.

Exemplos

Fonte única

Esse exemplo reproduz um vídeo quando ativado, fornecendo ao usuário os controles de vídeo padrão do navegador para controlar a reprodução.

HTML

```
<!-- Exemplo de vídeo simples -->
<!-- 'Big Buck Bunny' licensed under CC 3.0 by the Blender foundation. Hosted by archive.org -->
<!-- Poster de peach.blender.org -->
<video
  controls
  src="https://archive.org/download/BigBuckBunny_124/Content/big_buck_bunny_720p_surround.mp4"
  poster="https://peach.blender.org/wp-content/uploads/title_anouncement.jpg?x11217"
  width="620">
  Desculpe, seu navegador não suporta vídeos incorporados, mas não se preocupe,
  você pode
  <a href="https://archive.org/details/BigBuckBunny_124">baixá-lo</a>
  e assisti-lo com seu player de vídeo favorito!
</video>
```

Resultado

Até que o vídeo comece a ser reproduzido, a imagem fornecida no atributo `poster` é exibida em seu lugar. Se o navegador não tiver suporte para reprodução de vídeo, o texto de alternativa (*fallback*) será exibido.

Múltiplas fontes

Este exemplo se baseia no anterior, oferecendo três fontes diferentes para a mídia; isso permite que o vídeo seja assistido independentemente dos codecs de vídeo suportados pelo navegador.

HTML

```
<!-- Usando multiplas fontes como alternativa para a tag video -->
<!-- 'Elephants Dream' by Orange Open Movie Project Studio, licensed under CC-3.0, hosted by archive.org -->
<!-- Poster hospedado por Wikimedia -->
<video
  width="620"
  controls
  poster="https://upload.wikimedia.org/wikipedia/commons/e/e8/Elephants_Dream_s5_both.jpg">
  <source
    src="https://archive.org/download/ElephantsDream/ed_hd.ogv"
    type="video/ogg" />
  <source
    src="https://archive.org/download/ElephantsDream/ed_hd.avi"
    type="video/avi" />
  <source
    src="https://archive.org/download/ElephantsDream/ed_1024_512kb.mp4"
    type="video/mp4" />

  Desculpe, seu navegador não suporta vídeos incorporados, mas não se preocupe,
  você pode fazer o
  <a
    href="https://archive.org/download/ElephantsDream/ed_1024_512kb.mp4"
    download="ed_1024_512kb.mp4">
    download do MP4
  </a>
  e assisti-lo com seu player de vídeo favorito!
</video>
```

Resultado

Primeiro, é tentado o formato Ogg. Se não for possível reproduzi-lo, será tentado o AVI. Por fim, é tentado o MP4. Uma mensagem de fallback é exibida se o vídeo não puder ser reproduzido, mas a mesma não será exibida se todas as fontes falharem.

Alguns tipos de arquivos de mídia permitem que você forneça informações mais específicas usando o parâmetro `codecs` como parte da string de tipo do arquivo. Um exemplo relativamente simples é `video/webm; codecs="vp8, vorbis"`, que indica que o arquivo é um vídeo WebM usando VP8 para o vídeo e Vorbis para o áudio.

Preocupações com a acessibilidade

Os vídeos devem fornecer legendas e transcrições que descrevam com precisão o seu conteúdo (consulte Adicionando legendas e subtítulos a vídeos em HTML para obter mais informações sobre como implementá-las). As legendas permitem que pessoas com perda auditiva compreendam o conteúdo de áudio de um vídeo enquanto ele está sendo reproduzido, enquanto as transcrições permitem que pessoas que precisam de mais tempo possam revisar o conteúdo de áudio em um ritmo e formato que seja confortável para elas.

Vale ressaltar que, embora seja possível legendar mídia somente de áudio, isso só pode ser feito ao reproduzir áudio em um elemento `<video>`, uma vez que a região de vídeo do elemento é usada para apresentar as legendas. Este é um dos cenários especiais em que é útil reproduzir áudio em um elemento de vídeo.

Se serviços de legendagem automática forem usados, é importante revisar o conteúdo gerado para garantir que ele represente com precisão o vídeo original.

Além do diálogo falado, legendas e transcrições também devem identificar músicas e efeitos sonoros que transmitam informações importantes. Isso inclui emoção e tom:

```
14
00:03:14 --> 00:03:18
[Música rock dramática]

15
00:03:19 --> 00:03:21
[sussurrando] O que é aquilo lá longe?

16
```

```
00:03:22 --> 00:03:24
É… é um…

16 00:03:25 --> 00:03:32
[Barulho alto]
[Barulho de pratos]
```

As legendas não devem obstruir o assunto principal do vídeo. Elas podem ser posicionadas usando a configuração `align` VTT cue.

- Web Video Text Tracks Format (WebVTT)
- WebAIM: Captions, Transcripts, and Audio Descriptions
- MDN Understanding WCAG, Guideline 1.2 explanations
- Understanding Success Criterion 1.2.1 | W3C Understanding WCAG 2.0
- Understanding Success Criterion 1.2.2 | W3C Understanding WCAG 2.0

Sumário técnico

Categorias de conteúdo	Conteúdo de fluxo, conteúdo de fraseado, conteúdo incorporado. Se possui o atributo `controls`: interactive conteúdo interativo e conteúdo palpável.
Conteúdo permitido	Se o elemento possui um atributo `src` : zero ou mais elementos `<track>`, seguido de conteúdo transparente que não contém elementos de mídia – ou seja nenhum `<audio>` ou `<video>`. Caso contrário: zero ou mais elementos `<source>`, seguido de zero ou mais elementos `<track>`, seguido de conteúdo transparente que não contém elementos de mídia – ou seja, nenhum `<audio>` ou `<video>`.
Omissão de tag	Nenhuma, ambas as tags de abertura e fechamento são obrigatórias.
Pais permitidos	Qualquer elemento que aceite conteúdo embutido.
ARIA role implícito	Nenhuma ARIA role correspondente
ARIA roles permitidas	`application`
Interface DOM	`HTMLVideoElement`

Veja também

- Guia para tipos de mídia e formatos para web
  - Formatos de contêiner de mídia (tipos de arquivo)
  - Guia de codecs de vídeo na Web
  - Guia de codecs de áudio da Web
- Posicionamento e dimensionamento da imagem em seu quadro: `object-position` and `object-fit`
- `<audio>`
- Usando áudio e vídeo em HTML
- Manipulando vídeo usando canvas
- Configurando servidores para mídia Ogg

<wbr>

Resumo

O elemento HTML `<wbr>` representa uma posição no texto onde o navegador pode, opcionalmente, quebrar uma linha, embora suas regras de quebra de linha de outra forma não criar uma ruptura naquele local.

Em UTF-8 páginas codificadas, `<wbr>` se comporta como o `U+200B de tamanho zero ESPAÇO` ponto de código. Em particular, ele se comporta como um bidi Unicode BN ponto de código, o que significa que não tem efeito sobre bidi-encomenda: `<div dir=rtl> 123, <wbr> 456 </ div>` exibe, quando não quebrado em duas linhas, `123.456` e Não `456,123` .

Pela mesma razão, o `<wbr>` elemento não introduzir um hífen no ponto de quebra de linha. Para fazer um hífen só aparecem no final de uma linha, use a entidade suave caractere hífen (``) em seu lugar.

Este elemento foi implementado pela primeira vez no Internet Explorer 5.5 e foi oficialmente definido no HTML5.

Contexto de uso

Permitida conteúdo	Vazio
Tag omissão	Este é um elemento vazio; ele deve ter uma marca de início, mas não deve ter uma tag de fechamento.
Elementos pai permitidas	Qualquer elemento que aceita conteúdo fraseado .
Documento normativo	HTML 5, seção 4.6.24

Atributos

Este elemento só inclui os atributos globais.

DOM interface

Este elemento implementa o HTMLElement interface.

Exemplo

O Guia de Estilo Yahoo recomenda quebrar uma URL *antes de* pontuação , para não deixar uma marca de pontuação no final da linha, o que o leitor pode confundir com o fim da URL.

```
<p>
  https://this<wbr />.is<wbr />.a<wbr />.really<wbr />.long<wbr />.example<wbr />.com/With<wbr />/deeper<wbr
/>/level<wbr />/pages<wbr />/deeper<wbr />/level<wbr />/pages<wbr />/deeper<wbr />/level<wbr />/pages<wbr
/>/deeper<wbr />/level<wbr />/pages<wbr />/deeper<wbr />/level<wbr />/pages
</p>
```

Atributos Globais

Atributos Globais são atributos comuns a todos elementos HTML; eles podem ser usados em todos os elementos, embora os atributos não tenham efeito em alguns elementos.

Os atributos globais podem ser especificados em todos os elementos HTML, mesmo aqueles não especificados no padrão. Isso significa que qualquer elemento não padrão ainda deve permitir esses atributos, mesmo que usar esses elementos significa que o documento não é mais compatível com HTML5. Por exemplo, navegadores compatíveis com HTML5 escondem o conteúdo marcado como `<foo hidden>...<foo>`, mesmo que `<foo>` não seja um elemento HTML válido.

Além dos atributos globais HTML básicos, os seguintes atributos globais também existem:

- **`xml:lang`** e **`xml:base`** — Estes são herdados das especificações XHTML e obsoletos, mas mantidos para fins de compatibilidade.
- Os vários atributos **`aria-*`**, são utilizados para promover a acessibilidade.
- Os atributos manipuladores de eventos:: **`onabort`**, **`onautocomplete`**, **`onautocompleteerror`**, **`onblur`**, **`oncancel`**, **`oncanplay`**, **`oncanplaythrough`**, **`onchange`**, **`onclick`**, **`onclose`**, **`oncontextmenu`**, **`oncuechange`**, **`ondblclick`**, **`ondrag`**, **`ondragend`**, **`ondragenter`**, **`ondragexit`**, **`ondragleave`**, **`ondragover`**, **`ondragstart`**, **`ondrop`**, **`ondurationchange`**, **`onemptied`**, **`onended`**, **`onerror`**, **`onfocus`**, **`oninput`**, **`oninvalid`**, **`onkeydown`**, **`onkeypress`**, **`onkeyup`**, **`onload`**, **`onloadeddata`**, **`onloadedmetadata`**, **`onloadstart`**, **`onmousedown`**, **`onmouseenter`**, **`onmouseleave`**, **`onmousemove`**, **`onmouseout`**, **`onmouseover`**, **`onmouseup`**, **`onmousewheel`**, **`onpause`**, **`onplay`**, **`onplaying`**, **`onprogress`**, **`onratechange`**, **`onreset`**, **`onresize`**, **`onscroll`**, **`onseeked`**, **`onseeking`**, **`onselect`**, **`onshow`**, **`onsort`**, **`onstalled`**, **`onsubmit`**, **`onsuspend`**, **`ontimeupdate`**, **`ontoggle`**, **`onvolumechange`**, **`onwaiting`**.

Lista dos Atributos Globais

`accesskey`

Fornece uma dica para gerar um atalho de teclado para o elemento atual. Este atributo consiste em uma lista de caracteres separados por espaços. O navegador deve usar o primeiro que existe no layout do teclado do computador.

`class`

É uma lista separada por espaço das classes do elemento. as classes permitem ao CSS e ao JavaScript selecionar e acessar elementos específicos atraves dos seletores de classe ou funciona como um método `Document.getElementsByClassName()`.

`contenteditable`

É um atributo enumerado que indica se o elemento deve ser editado pelo usuário. Se assim for, o navegador modifica seu widget para permitir a edição. O atributo deve ter um dos seguintes valores:

- `true` ou uma string vazia, indica que o elemento deve ser editável;
- `false`, indica que o elemento não deve ser editável.

`contextmenu`

É o **`id`** de um `<menu>` para usar como o menu contextual para este elemento.

`data-*`

Forma uma classe de atributos, denominado de dados personalizados, que permite troca de informações proprietárias entre o HTML e a sua representação no DOM pode ser usada por scripts. Todos esses dados personalizados estão disponíveis através da interface do elemento `HTMLElement` em que o atributo está definido. A propriedade `HTMLElement.dataset` dá acesso a eles.

`dir`

É um atributo enumerado que indica a direcionalidade do texto do elemento. Pode ter os seguintes valores:

- `ltr`, significa da esquerda para a direita e deve ser usado para idiomas que são escritos da esquerda para a direita (como o português do Brasil, inglês, espanhol, etc);
- `rtl`, significa da direita para a esquerda e deve ser usado para idiomas que são escritos da direita para a esquerda (como o árabe);
- `auto`, que permite que o agente do usuário decida. Ele usa um algoritmo básico, pois analisa os caracteres dentro do elemento até encontrar um caractere com uma direcionalidade forte e, em seguida, aplica essa direcionalidade para todo o elemento.

`draggable`

É um atributo enumerado que indica se o elemento pode ser arrastado, usando a API Drag and Drop. Pode ter os seguintes valores:

- `true`, que indica que o elemento pode ser arrastado
- `false`, que indica que o elemento não pode ser arrastado.

`dropzone`

É um atributo enumerado que indica quais tipos de conteúdo podem ser descartados em um elemento, usando a API Drag and Drop. Pode ter os seguintes valores:

- `copy`, que indica que o lançamento criará uma cópia do elemento que foi arrastado.
- `move`, que indica que o elemento que foi arrastado será movido para esta nova localização.
- `link`, irá criar um link para os dados arrastados.

`hidden`

Um atributo booleano indica que o elemento ainda não é relevante ou não é mais relevante. Por exemplo, ele pode ser usado para ocultar elementos da página que não podem ser usados até que o processo de login tenha sido concluído. O navegador não renderizará esses elementos. Este atributo não deve ser usado para ocultar conteúdo que possa ser legível.

`id`

Define um identificador exclusivo (ID) que deve ser único em todo o documento. Seu objetivo é identificar o elemento ao vincular (usando um identificador de fragmento), um script ou estilo (com CSS).

> **Nota:** Os 5 atributos seguintes são partes do Recursos de Microdados WHATWG HTML.

`itemid`

O identificador único e global de um item.

`itemprop`

Usado para adicionar propriedades a um item. Cada elemento HTML pode ter um atributo itemprop especificado, onde um itemprop consiste em um par de nomes e valores.

`itemref`

As propriedades que não são descendentes de um elemento com o atributo `itemscope` podem ser associadas ao item usando um **itemref**. Itemref fornece uma lista de ids de elementos (não `itemid`) com propriedades adicionais em outro lugar do documento.

`itemscope`

Este atributo funciona, em geral, com o atributo `itemtype` para especificar o HTML dentro de um bloco é sobre um item particular. `itemscope` cria o item e define o escopo do `itemtype` associado a ele. `itemtype` é uma URL válida de um vocabulário (como o schema.org) que descreve o item e o context de seus atributos.

`itemtype`

Especifica a URL do vocabulário que será usado para definir as propriedades do item na estrutura de dados. `itemscope` é usado para alterar o escopo na estrutura de dados onde o vocabulário definido por `itemtype` estará ativo.

`lang`

Participa da definição do idioma do elemento, o idioma no qual os elementos não-editáveis estão escritos, ou o idioma no qual elementos editáveis devem ser escritos. A Tag contém um único valor no formato definido no documento da IEFT Tags para Identificação de Idiomas (BCP47). `xml:lang` tem prioridade mais alta que `lang`.

`spellcheck`

É um atributo enumerado que define se o elemento pode ser verificado por errors de ortografia. Pode ter os seguintes valores:

- `true`, indica que o elemento deve ser, se possível, verificado por errors ortográficos;
- `false`, indica que o elemento não deve ser verificado quanto à ortogratia do texto.

`style`

Contém regras de declarações CSS para aplicar no elemento. Note que é recomendado que as regras CSS fiquem num arquivo, ou vários arquivos, separado do HTML. Este atributo e o elemento `<style>` tem principalmente o propósito para rápida estilização do elemento, como por exemplo para testes.

`tabindex`

É um atributo que recebe valores inteiros e indica se um elemento pode receber o foco de entrada de valores (é focável), se ele deve entrar na sequência da navegação pelo teclado (através da tecla TAB), e qual posição da sequência. Sobre seus valores:

- um *valor negativo* indica que o elemento **é focável**, mas **não é alcançável** pela navegação sequencial do teclado (através da tecla TAB);
- `0` indica que o elemento **é focável** e **alcançavel** pela navegação sequencial do teclado, mas sua ordem de seleção relativa é definida pela convenção da plataforma (ou do navegador);
- um *valor positivo* indica que o elemento **é focável** e **alcançável** pela navegação sequencial do teclado; sua ordem de seleção relativa é definida pelo valor colocado: a sequência segue em ordem crescente definida no `tabindex`. Se

vários elementos possuem o mesmo valor de `tabindex`, sua ordem relativa é estabelecida pela posição relativa no documento HTML.

`title`

Contém um texto representativo sobre a informação relacionada ao elemento ao qual este atributo pertence. Tal informação pode, mas não necessariamente, ser apresentada através de um *tooltip*.

`translate`

É um atributo enumerado, usado para especificar se um atributo de um elemento e os valores dos seus nós descendentes (filhos) `Text` serão traduzidos quando a página for localizada, ou se não serão alterados. Pode ter os seguintes valores:

- `yes` ou string vazia: indicam que o elemento será traduzido;
- `no`: indica que o elemento não será traduzido.

Veja também

- `Element` e `GlobalEventHandlers` interfaces que permitem acessar a maioria dos atributos globais.

accesskey

O atributo global **accesskey** fornece uma dica para gerar um atalho de teclado no elemento atual. O valor do atributo deve consistir em um único caracter imprimível (que inclui caracteres acentuados e outros caracteres que podem ser gerados pelo teclado).

Experimente

```
<p>If you need to relax, press the <b>S</b>tress reliever!</p>
<button accesskey="s">Stress reliever</button>
```

```
b {
  text-decoration: underline;
}
```

> **Nota:** Na especificação do WHATWG, diz que você pode especificar vários caracteres separados por espaço, e o navegador usará primeiro o que ele suportar. No entanto, isso não funciona na maioria dos browsers. O IE/Edge usa o primeiro que ele suporta sem problemas, desde que não haja conflitos com outros comandos.

A maneira de ativar a tecla de acesso depende do navegador e de sua plataforma:

	Windows	Linux	Mac
Firefox	`Alt` + `Shift` + *`key`*		No Firefox 57 ou mais recente: `Control` + `Option` + *`key`* or `Control` + `Alt` + *`key`* No Firefox 14 ou mais recente: `Control` + `Alt` + *`key`* No Firefox 13 ou mais antigo: `Control` + *`key`*
Edge	`Alt` + *`key`*	N/A	
Internet Explorer	`Alt` + *`key`*	N/A	
Google Chrome	`Alt` + *`key`*		`Control` + `Alt` + *`key`*
Safari	`Alt` + *`key`*	N/A	`Control` + `Alt` + *`key`*
Opera 15+	`Alt` + *`key`*		`Control` + `Alt` + *`key`*
Opera 12	`Shift` + `Esc` abre uma lista de conteúdos acessíveis pela tecla de acesso e, em seguida, pode escolher um item pressionando *`key`*		

Preocupações com acessibilidade

Além do baixo suporte ao navegador, existem inúmeras preocupações com o atributo `accesskey`:

- Um valor de `accesskey` pode entrar em conflito com uma tecla de atalho do sistema ou do navegador, ou alguma funcionalidade de tecnologia assistida. O que pode funcionar para uma combinação de sistema operacional, tecnologia assistida e navegador pode não funcionar com outras combinações.
- Certos valores de `accesskey` podem não estar presentes em alguns teclados, especialmente quando a internacionalização é uma preocupação. Portanto, a adaptação a idiomas específicos pode causar problemas adicionais.
- Os valores de `accesskey` que dependem de números podem ser confusos para indivíduos com problemas cognitivos, onde o número não tem uma associação lógica com a funcionalidade que aciona.
- Informar o usuário que `accesskey` estão presentes, para que eles estejam cientes da funcionalidade. Se o sistema não tiver o método de o usuário dessa funcionalidade, ele poderá ativar acidentalmente as teclas de acesso (`accesskeys`).

Devido a esses problemas, geralmente não é aconselhável usar o `accesskeys` para a maioria dos sites e aplicativos da web.

WebAIM: Keyboard Accessibility - Accesskey

Veja também

- `Element.accessKey`
- `HTMLElement.accessKeyLabel`
- Todos os atributos globais.

autocapitalize

O atributo global **`autocapitalize`** é um atributo enumerado que controla quando e como entradas de texto são automaticamente capitalizadas confome são escritas/editadas pelo usuário. O atributo deve receber um dos seguintes valores:

- `off` ou `none`: Nenhuma capitalização é aplicada (por padrão as letras serão minúsculas)
- `on` ou `sentences`: A primeira letra de cada sentença será por padrão uma letra maíuscula; todas as demais letras serão por padrão minúsculas.
- `words`: A letra inicial de cada palavra serão por padrão maiúsculas; todas as demais letras serão por padrão minúsculas.
- `characters`: Todas as letras deverão ser maiúsculas por padrão.

O atributo `autocapitalize` não afeta comportamentos quando digitando em um teclado físico. Ao invés disso, afeta o comportamento de outros mecanismos de entrada, tais quais teclados virtuais em dispositivos móveis e entradas por voz. O comportamento de tais mecanismos é que frequentemente auxiliam usuários aos capitalizar automaticamente a primeira letra de sentenças. O atributo `autocapitalize` permite a autores sobrescreverem esse comportamento por elemento.

O atributo `autocapitalize` nunca ativa a autocapitalização para um elemento `<input>` com um atributo `type` cujo valor seja `url`, `email`, ou `password`.

class

O atributo global **class** é uma lista das classes de um elemento, separada por espaços. Classes permitem a CSS e Javascript selecionar e acessar elementos específicos através dos seletores de classe ou funções como o método DOM `document.getElementsByClassName`.

Apesar da especificação não impor requesitos sobre os nomes de classes, é considerada boa prática usar nomes que descrevam o propósito semântico do elemento, em vez de sua representação (e.g. *atributo* para descrever um atributo em vez de *ítalico*, mesmo que um elemento desta classe possa ser representado por *ítalico)*. Nomes baseados em semântica permanecem coerentes mesmo se a representação da página mude.

Veja também

- Todos os atributos globais.

contenteditable

O atributo global **`contenteditable`** é um atributo enumerado indicando se o elemento deve ser editável pelo usuário. Se assim for, o navegador modifica seu widget para permitir a edição.

Experimente

```
<blockquote contenteditable="true">
  <p>Edit this content to add your own quote</p>
</blockquote>

<cite contenteditable="true">-- Write your own name here</cite>
```

```
blockquote {
  background: #eee;
  border-radius: 5px;
  margin: 16px 0;
}

blockquote p {
  padding: 15px;
}

cite {
  margin: 16px 32px;
  font-weight: bold;
}

blockquote p::before {
  content: "\201C";
}

blockquote p::after {
  content: "\201D";
}

[contenteditable="true"] {
  caret-color: red;
}
```

O atributo deve ter um dos seguintes valores:

- `true` ou uma string vazia, o que indica que o elemento é editável.
- `false`, o que indica que o elemento não é editável.

Se o atributo for aplicado sem um valor, como `<label contenteditable>Rótulo de Exemplo</label>`, seu valor é tratado como uma *string* vazia.

Se esse atributo estiver faltando ou seu valor for inválido, seu valor é herdado de seu elemento pai: para que o elemento seja editável se seu pai for editável.

Observe que, embora seus valores permitidos incluam *true* e *false*, este atributo é um enumerado e não um *Booleano*.

Você pode definir a cor usada para desenhar a inserção de texto caret com a propriedade `caret-color` do CSS.

Veja também

- Tornando o conteúdo editável
- Todos os atributos globais
- `HTMLElement.contentEditable` e `HTMLElement.isContentEditable`
- A propriedade `caret-color` do CSS.
- Evento de `input` - `HTMLElement`

hidden

O atributo global **`hidden`** é um atributo booleano que indica se um elemento é ou não *relevante*. Por exemplo, ele pode ser usado para esconder elementos de página que não podem ser usados até que o processo de login seja completo. Os navegadores não renderizarão elementos com o atributo `hidden` setado.

O atributo `hidden` não deve ser utilizado para esconder conteúdo que poderia legitimamente ser mostrado em outra apresentação, por exemplo, não é correto usar `hidden` para esconder abas de formulários em paineis, porque interfaces separadas por abas são meramente um tipo de apresentação em fluxo — o que pode ser igualmente mostrado em forma vertical com barra de rolagem. É similarmente incorreto usar este atributo para esconder conteúdo de apenas uma apresentação — se algo está marcado como `hidden`, ele está escondido de todas as apresentações incluindo, por exemplo, leitores de tela.

Elementos com `hidden` não devem estar ligados a outros elementos sem `hidden`. Elementos filhos de um elemento com o atributo `hidden` ainda estão ativos, ou seja, é possível executar a ação `submit` de um formulário ou uma tag `script` pode ser executada se em ambos os casos estes elementos forem filhos de um elemento com o atributo `hidden`.

Por exemplo, é incorreto o uso do atributo `href` referenciando a uma sessão marcada com o atributo `hidden`. Se o conteúdo não é aplicável ou relevante, não faz sentido fazer a referência.

Pode ser correto, entretanto, usar o atributo ARIA `aria-describedby`para referenciar descrições que estejam, elas mesmas, escondidas. Esconder as descrições implica que elas não são úteis por si só. Mas elas podem ser escritas de forma que sejam úteis no contexto específico do elemento que descrevem

Similarmente, um elemento canvas com atributo `hidden` poderia ser usado como buffer por um script de gráficos, e um controle de formulário, usando seu atributo form, poderia referir-se a um elemento `form` com atributo `hidden`.

> **Nota:** Mudar o valor da propriedade `display` no CSS de um elemento com o atributo `hidden` sobrescreve o comportamento. Por exemplo, elementos com o estilo `display: flex;` serão mostrados independente da presença do atributo `hidden`.

Veja também

- Todos os atributos globais.

id

O atributo global **id** define um identificador exclusivo (ID) que deve ser único por todo o documento. Seu objetivo é identificar o elemento ao navegar por âncoras (usando um identificador de fragmento), quando utilizar scripts ou estilizando (com CSS).

O valor deste atributo é uma string opaca: isso significa que os desenvolvedores não devem utilizá-lo para transmitir nenhuma informação. Significados específicos como, por exemplo, significado semântico não devem ser provenientes desta string.

O valor deste atributo não deve conter lacunas (espaços, tabulações etc.). Navegadores tratam IDs inadequadas que contenham lacunas como se as lacunas fossem parte do ID. Em contraste com o atributo **class**, que permite múltiplos valores separados por espaço, os elementos podem ter somente um único ID.

> **Nota:** Utilizar caracteres exceto letras e dígitos ASCII, `'_'`, `'-'` e `'.'` pode causar problemas de compatibilidade, já que eles não eram permitidos no HTML 4. Embora esta restrição fora suspensa no HTML 5, um ID deve iniciar com uma letra para fins de compatibilidade.

Veja também

- Todos os atributos globais.
- `Element.id` para explorar este atributo.

itemprop

O atributo global **`itemprop`** é usado para adicionar propriedades para um item. Todo elemento HTML pode ter um atributo `itemprop` especificado, e um `itemprop` consiste em um par de nome e valor. Cada par de nome e valor é chamado de **propriedade,** e um grupo de uma ou mais propriedades forma um **item**. Os valores das propriedades são ou um texto ou uma URL e pode ser associada a uma grande variedade de elementos incluindo `<audio>`, `<embed>`, `<iframe>`, `<img>`, `<link>`, `<object>`, `<source>` , `<track>`, and `<video>`.

Exemplos

The example below shows the source for a set of elements marked up with `itemprop` attributes, followed by a table showing the resulting structured data.

HTML

```
<div itemscope itemtype="http://schema.org/Movie">
  <h1 itemprop="name">Avatar</h1>
  <span
    >Director:
    <span itemprop="director">James Cameron</span>
    (born August 16, 1954)</span
  >
  <span itemprop="genre">Science fiction</span>
  <a href="../movies/avatar-theatrical-trailer.html" itemprop="trailer"
    >Trailer</a
  >
</div>
```

Structured data

	Item	
	itemprop name	**itemprop value**
itemprop	name	Avatar
itemprop	director	James Cameron
itemprop	genre	Science fiction
itemprop	trailer	../movies/avatar-theatrical-trailer.html

Properties

Properties have values that are either a string or a URL. When a string value is a URL, it is expressed using the `<a>` element and its `href` attribute, the `<img>` element and its `src` attribute, or other elements that link to or embed external resources.

Three properties with values that are strings

```
<div itemscope>
  <p>My name is <span itemprop="name">Neil</span>.</p>
  <p>My band is called <span itemprop="band">Four Parts Water</span>.</p>
  <p>I am <span itemprop="nationality">British</span>.</p>
</div>
```

One property, "image", whose value is a URL

```
<div itemscope>
  <img itemprop="image" src="https://developer.mozilla.org/pt-
BR/docs/Web/HTML/Reference/Global_attributes/itemprop/google-logo.png" alt="Google" />
</div>
```

When a string value can't be easily read and understood by a person (e.g., a long string of numbers and letters), it can be displayed using the value attribute of the data element, with the more easily-understood-by-a human-version given in the element's contents (which is not part of the structured data - see example below).

An item with a property whose value is a product ID

The ID is not human-friendly, so the product's name is used the human-visible text instead of the ID.

```
<h1 itemscope>
  <data itemprop="product-id" value="9678AOU879">The Instigator 2000</data>
</h1>
```

For numeric data, the meter element and its value attribute can be used.

A meter element

```
<div itemscope itemtype="http://schema.org/Product">
  <span itemprop="name">Panasonic White 60L Refrigerator</span>
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Global_attributes/itemprop/panasonic-fridge-
60l-white.jpg" alt="" />
  <div
    itemprop="aggregateRating"
    itemscope
    itemtype="http://schema.org/AggregateRating">
    <meter itemprop="ratingValue" min="0" value="3.5" max="5">
      Rated 3.5/5
    </meter>
    (based on <span itemprop="reviewCount">11</span>
    customer reviews)
  </div>
</div>
```

Similarly, for date- and time-related data, the time element and its datetime attribute can be used.

An item with one property, "birthday", whose value is a date

```
<div itemscope>
  I was born on
  <time itemprop="birthday" datetime="2009-05-10">May 10th 2009</time>.
</div>
```

Properties can also be groups of name-value pairs, by putting the itemscope attribute on the element that declares the property. Each value is either a string or a group of name-value pairs (i.e. an item).

An outer item representing a person, and an inner one representing a band

```
<div itemscope>
  <p>Name: <span itemprop="name">Amanda</span></p>
  <p>
    Band:
    <span itemprop="band" itemscope>
      <span itemprop="name">Jazz Band</span>
      (<span itemprop="size">12</span> players)</span
    >
  </p>
</div>
```

The outer item above has two properties, "name" and "band". The "name" is "Amanda", and the "band" is an item in its own right, with two properties, "name" and "size". The "name" of the band is "Jazz Band", and the "size" is "12". The outer item in this example is a top-level microdata item. Items that are not part of others are called top-level microdata items.

All the properties separated from their items

This example is the same as the previous one, but all the properties are separated from their items

```
<div itemscope id="amanda" itemref="a b"></div>
<p id="a">Name: <span itemprop="name">Amanda</span></p>
<div id="b" itemprop="band" itemscope itemref="c"></div>
<div id="c">
  <p>Band: <span itemprop="name">Jazz Band</span></p>
  <p>Size: <span itemprop="size">12</span> players</p>
</div>
```

This gives the same result as the previous example. The first item has two properties, "name", set to "Amanda", and "band", set to another item. That second item has two further properties, "name", set to "Jazz Band", and "size", set to "12".

An item can have multiple properties with the same name and different values.

Ice cream with two flavors

```
<div itemscope>
  <p>Flavors in my favorite ice cream:</p>
  <ul>
    <li itemprop="flavor">Lemon sorbet</li>
    <li itemprop="flavor">Apricot sorbet</li>
  </ul>
</div>
```

This results in an item with two properties, both with the name "flavor" and having the values "Lemon sorbet" and "Apricot sorbet".

An element introducing a property can also introduce multiple properties at once, to avoid duplication when some of the properties have the same value.

An item with two properties, "favorite-color" and "favorite-fruit", both set to the value "orange"

```
<div itemscope>
  <span
    itemprop="favorite-color
    favorite-fruit"
    >orange</span
  >
</div>
```

> **Nota:** There is no relationship between the microdata and the content of the document where the microdata is marked up.

Same structured data marked up in two different ways

There is no semantic difference between the following two examples

```
<figure>
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Global_attributes/itemprop/castle.jpeg" />
  <figcaption>
    <span itemscope><span itemprop="name">The Castle</span></span> (1986)
  </figcaption>
</figure>
```

```
<span itemscope><meta itemprop="name" content="The Castle" /></span>
<figure>
  <img src="https://developer.mozilla.org/pt-BR/docs/Web/HTML/Reference/Global_attributes/itemprop/castle.jpeg" />
  <figcaption>The Castle (1986)</figcaption>
</figure>
```

Both have a figure with a caption, and both, completely unrelated to the figure, have an item with a name-value pair with the name "name" and the value "The Castle". The only difference is that if the user drags the figcaption out of the document, the item will be included in the drag-and-drop data. The image associated with the item won't be included.

Names and values

A property is an unordered set of unique tokens that are case-sensitive and represent the name-value pairs. The property value must have at least one token. In the example below, each data cell is a token.

Names examples

	Item	
	itemprop name	**itemprop value**
itemprop	country	Ireland
itemprop	Option	2

	Item	
	itemprop name	**itemprop value**
itemprop	https://www.flickr.com/photos/nlireland/6992065114/	Ring of Kerry
itemprop	img	https://www.flickr.com/photos/nlireland/6992065114/
itemprop	website	flickr
itemprop	(token)	(token)

Tokens are either strings or URL's. An item is called a **typed item** if it is a URL. Otherwise, it is a string. Strings cannot contain a period or a colon (see below).

1. If the item is a typed item it must be either:
    1. A defined property name, or
    2. A valid URL, which refers to the vocabulary definition, or
    3. A valid URL that is used as a proprietary item property name (i.e. one not defined in a public specification), or
2. If the item is not a typed item it must be:
    1. A string that contains no "**.**" (U+002E FULL STOP) characters and no "**:**" characters (U+003A COLON) and is used as a proprietary item property name (again, one not defined in a public specification).

> **Nota:** The rules above disallow ":" characters in non-URL values because otherwise they could not be distinguished from URLs. Values with "." characters are reserved for future extensions. Space characters are disallowed because otherwise the values would be parsed as multiple tokens.

Values

The property value of a name-value pair is as given for the first matching case in the following list:

- If the element has an **itemscope** attribute
    - The value is the **item** created by the element.
- If the element is a **meta** element
    - The value is the value of the element's **content** attribute
- If the element is an **audio**, **embed**, **iframe**, **img**, **source**, **track**, or **video** element
    - The value is the resulting URL string that results from parsing the value of the element's src attribute relative to the node document (part of the Microdata DOM API) of the element at the time the attribute is set
- If the element is an **a**, **area**, or **link** element
    - The value is the resulting URL string that results from parsing the value of the element's href attribute relative to the node document of the element at the time the attribute is set
- If the element is an **object** element
    - The value is the resulting URL string that results from parsing the value of the element's data attribute relative to the node document of the element at the time the attribute is set
- If the element is a **data** element
    - The value is the value of the element's value attribute
- If the element is a **meter** element
    - The value is the value of the element's **value** attribute
- If the element is a **time** element

    - The value is the element's **datetime** value

Otherwise

- The value is the element's **textContent**.

If a property's value is a **URL**, the property must be specified using a URL property element. The URL property elements are the **a**, **area**, **audio**, **embed**, **iframe**, **img**, **link**, **object**, **source**, **track**, and **video** elements.

Name order

Names are unordered relative to each other, but if a particular name has multiple values, they do have a relative order.

In the following example, the "a" property has the values "1" and "2", *in that order*, but whether the "a" property comes before the "b" property or not is not important

```
<div itemscope>
  <p itemprop="a">1</p>
  <p itemprop="a">2</p>
  <p itemprop="b">test</p>
</div>
```

The following is equivalent

```
<div itemscope>
  <p itemprop="b">test</p>
  <p itemprop="a">1</p>
  <p itemprop="a">2</p>
</div>
```

As is the following

```
<div itemscope>
  <p itemprop="a">1</p>
  <p itemprop="b">test</p>
  <p itemprop="a">2</p>
</div>
```

And the following

```
<div id="x">
  <p itemprop="a">1</p>
</div>
<div itemscope itemref="x">
  <p itemprop="b">test</p>
  <p itemprop="a">2</p>
</div>
```

Other examples

HTML

```
<dl
  itemscope
  itemtype="http://vocab.example.net/book"
  itemid="urn:isbn:0-330-34032-8">
  <dt>Title</dt>
  <dd itemprop="title">The Reality Dysfunction</dd>
  <dt>Author</dt>
  <dd itemprop="author">Peter F. Hamilton</dd>
  <dt>Publication date</dt>
  <dd><time itemprop="pubdate" datetime="1996-01-26">26 January 1996</time></dd>
</dl>
```

Structured data

itemscope	itemtype: itemid	http://vocab.example.net/book: urn:isbn:0-330-34032-8	
	itemprop	title	The Reality Dysfunction
	itemprop	author	Peter F. Hamilton
	itemprop	pubdate	1996-01-26

Result

Specifications

Specification	Status	Comment
`itemprop`		WG Note - No longer being actively developed

Compatibilidade com navegadores

See also

- Other different global attributes
- Other, microdata related, global attributes:
  - `itemid`
  - `itemref`
  - `itemscope`
  - `itemtype`

lang

O atributo global **`lang`** ajuda a definir o idioma de um elemento: a língua em que elementos não-editáveis são escritos, ou a língua em que elementos editáveis devem ser escritos pelo usuário. O atributo contém uma uma única "tag de idioma" em um formato definido em *Tags para identificar linguagens(BCP47)*.

Experimente

```
<p>This paragraph is English, but the language is not specifically defined.</p>

<p lang="en-GB">This paragraph is defined as British English.</p>

<p lang="fr">Ce paragraphe est défini en français.</p>
```

```
p::before {
  padding-right: 5px;
}

[lang="en-GB"]::before {
  content: "(In British English) ";
}

[lang="fr"]::before {
  content: "(In French) ";
}
```

Se o valor do atributo é uma *string vazia* (`lang=""`), o idioma é definido como *unknown (desconhecido)*; se a tag de idioma não é válida conforme o BCP47, ela é definida como *invalid (invalida)*.

Sintaxe da tag de idioma

A sintaxe completa do BCP47 é profunda o suficente para caracterizar dialetos de linguagem extremamente expecíficos, mas a maioria dos usos são bem mais simples.Uma tag de idioma é feita de *subtags de idioma* separadas por hífen, onde cada subtag indica uma certa propriedade da língua. As três subtags mais comuns são:- Subtag de idioma - : Necessária. Um código de 2 ou

3 caracteres que define o idioma base, tipicamente escrita em letras minúsculas. Por exemplo, o código para Inglês é `en` e o código para Português é `pt`

> Subtag de script
>
> Opcional. Essa subtag define o sistema de escrita usado para a linguagem, e sempre tem 4 caracteres, com a primeira letra maiúscula. Por exemplo, Francês em Braile é `fr-Brai` e `ja-Kana` é Japonês escrito no alfabeto Katakana. Se a língua é escrita em uma maneira tipicamente comum, como Inglês no alfabeto Latino, não é necessário o uso da subtag.
>
> Subtag de região
>
> Opcional. Essa subtag define um dialeto do idioma base em uma localização particular e tem um código de duas letras maiúsculas referentes ao país, ou um código de três números referentes à uma área fora do país. Por exemplo, `es-ES` é para espanhol falado na Espanha, e `es-013` é o espanhol falado na America Central. "Espanhol Internacional" seria `es`.A subtag de script precede a região da subtag quando ambos estão presentes— `ru-Cyrl-BY` é Russo, escrito no alfabeto Cyrillic, falado em Belarus.Para encontrar os códigos corretos para uma subtag de idioma, acesse Language Subtag Lookup.

Mesmo que o atributo **lang** esteja bem definido ele ainda pode ser ignorado, pois o atributo **xml:lang** tem prioridade.

Para as pseudo-classes CSS `:lang`, dois nomes de idiomas inválidos são diferentes se seus nomes forem diferentes. Por tanto, enquanto `:lang(es)` corresponde a ambos `lang="es-ES"` e `lang="es-419"`, `:lang(xyzzy)` não corresponde a `lang="xyzzy-Zorp!"`.

Veja também

- Todos atributos globais.

Controlando a verificação ortográfica em formulários HTML

Firefox 2 introduz suporte à verificação ortográfica para áreas de texto e campos de texto em formulários web. O usuário pode especificar usando a interface about:config se a verificação ortográfica é ou não habilitada e se checará áreas de texto e campos de texto ou somente áreas de texto.

Por padrão, áreas de texto e documentos `designMode` tem ortografia verificada e caixas de texto de uma única linha não tem. Isto é assim porque os usuários do Firefox podem se distrair ou se incomodar se o Firefox marcar coisas como IDs de usuários ou endereços de e-mail como erros de ortografia.

Porém, podem haver situações nas quais este comportamento não é necessariamente apropriado. Por exemplo, se uma área de texto tem o objetivo de ser usada para editar HTML ou servir de entrada para outro tipo de texto que não seja semântico, a verificação ortográfica seria um entrave em vez de uma ajuda. Da mesma forma, podem haver casos nos quais um site faça uma recomendação de que o Firefox habilite a verificação ortográfica para um campo de texto específico, como campos de busca ou assunto/título de e-mail, mesmo estes sendo geralmente campos de texto de uma única linha.

Se um site deseja recomendar o uso ou não de verificação ortográfica para um elemento `<input>` específico, ele pode usar o atributo `spellcheck`, espefcificando o valor `true` para recomendar o uso da verificação ortográfica ou `false` para recomendar o não uso.

Tenha em mente que a recomendação do site pode ser ignorada pelo usuário se o mesmo tiver desativado a verificação ortográfica setando a configuração `layout.spellcheckDefault` para 0. Se a configuração `layout.spellcheckDefault` tiver qualquer outro valor, as recomendações serão consideradas.

Você pode codificar um campo de texto linha-única (elemento HTML `<input>`) habilitando a verificação ortográfica da seguinte forma:

```
<input type="text" size="50" spellcheck="true">
```

Da mesma forma, você pode desabilitar a verificação ortográfica em uma área de texto (elemento `<textarea>`) da seguinte forma:

```
<textarea spellcheck="false"></textarea>
```

Você pode controlar um documento em seu `designMode` (tipicamente usado para implementar edição de texto rica) setando o atributo `spellcheck` no elemento `<body>` de um documento.

Você também pode aplicar o atributo `spellcheck` em outros elementos, tais como os elementos `<span>` e `<div>`, e nesse caso todos os elementos `<input>` dentro dessas tags irão herdar esta configuração; elementos `<input>` que não tem um atributo `spellcheck` setado, irão herdar a configuração de verificação ortográfica de seu elemento pai. Se não houver nenhuma configuração setada na cadeia antecessora de elementos, a configuração padrão será usada.

Por exemplo:

```
<div spellcheck="true">
  <label>Escreva algo: </label><input type="text" size="50" />
  <br />
  <label>Escreva outra coisa: </label><input type="text" size="50" />
</div>
<br />
<label>Mais alguma coisa: </label><input type="text" size="50" />
```

Neste exemplo HTML acima, os dois primeiros campos de texto terão a verificação ortográfica e o terceiro não terá.

Iniciando no Gecko 9.0, a verificação ortográfica usa o atributo `lang` do elemento `<input>` para determinar o idioma padrão da verificação ortográfica. Se o `<input>` não tiver o atributo `lang` setado, esse atributo é procurado em cada elemento pai superior até chegar ao elemento raiz do documento.

Fazendo assim, se o usuário tem os dicionários de Português e Inglês instalados, e um elemento editável tiver o atributo `lang="en"`, o dicionário inglês será automaticamente usado para este elemento.

Por exemplo:

```
<html lang="pt-BR">
  <body>
    <textarea></textarea>
    <textarea lang="en"></textarea>
    <div lang="ru">
      <textarea></textarea>
    </div>
  </body>
</html>
```

No exemplo HTML acima, o primeiro `<textarea>` terá ortografia checada em Português, o segundo em Inglês e o terceiro em Russo.

Se um elemento especifica o idioma e o usuário não tem dicionário instalado para este idioma, a verificação ortográfica fica desabilitada por padrão, embora o usuário possa escolher por habilitá-la manualmente.

tabindex

O atributo global **tabindex** indica se um elemento pode receber foco de entrada (se ele é *focável*), se e em qual posição ele deve fazer parte da navegação sequencial do teclado (geralmente com a tecla `Tab`, daí seu nome).

Experimente

```
<p>Click anywhere in this pane, then try tabbing through the elements.</p>

<label>First in tab order:<input type="text" /></label>

<div tabindex="0">Tabbable due to tabindex.</div>

<div>Not tabbable: no tabindex.</div>

<label>Third in tab order:<input type="text" /></label>
```

```
p {
  font-style: italic;
```

```
  font-weight: bold;
}

div,
label {
  display: block;
  letter-spacing: 0.5px;
  margin-bottom: 1rem;
}

div:focus {
  font-weight: bold;
}
```

Ele aceita valores inteiros, com diferentes resultados dependendo do valor desse inteiro:

- um *valor negativo* (geralmente `tabindex="-1"`) significa que o elemento não deve ser localizado pela navegação sequencial do teclado, mas pode ser focável visualmente ou com JavaScript. Isso é mais útil para criar widgets accessíveis com JavaScript.

  > **Nota:** Um valor negativo é útil quando você tem conteúdo fora da tela que aparece com um evento específico. O usuário não poder focar em nenhum elemento com um `tabindex` negativo utilizando o teclado, porém um script pode o fazer utilizando o método `focus()`.

- `tabindex="0"` significa que o elemento deve ser focável e que pode ser localizado pela navegação sequencial do teclado, mas a sua posição será definida pela ordem no código-fonte do documento;

- um *valor positivo* significa que o elemento deve ser focável e que pode ser localizado pela navegação sequencial do teclado, e a sua posição definida pelo valor do número. Sendo assim, `tabindex="4"` seria focado antes de `tabindex="5"`, mas depois de `tabindex="3"`. Se vários elementos possuírem o mesmo valor positivo de `tabindex`, a sua posição na ordenação será definida pela sua posição no código-fonte do documento. O valor máximo do `tabindex` é 32767. Se não especificado, assume o valor padrão 0.

  > **Aviso:** Evite usar valores do `tabindex` maiores que 0. Ao fazer isso dificulta para pessoas que dependem de tecnologias assistivas para navegar e operar o conteúdo da página. Ao invés disso, escreva o documento com os elementos em uma sequência lógica.

Se o atributo `tabindex` for definido em um elemento div `<div>`, seu conteúdo filho (interno) não poderá ser rolado utilizando as teclas direcionais a menos que seja definido `tabindex` no conteúdo também. Veja este fiddle para entender melhor os efeitos de rolagem do tabindex.

Veja também

- Todos os atributos globais.
- `HTMLElement.tabIndex` que reflete este atributo.
- Problemas de acessibilidade com tabindex: veja Don't Use Tabindex Greater than 0 | Adrian Roselli (em inglês).

title

O atributo global **title** contém textos representando informações de orientação, relacionados ao elemento a que pertence. Geralmente, mas não obrigatoriamente, esta informação pode ser apresentada ao usuário como uma dica. Alguns usos comuns:

- Link: título ou descrição do documento vinculado
- Elemento de mídia como uma imagem: descrição ou créditos associados
- Parágrafo: nota de rodapé ou um comentário sobre ele
- Citação: informação sobre o autor, e assim por diante.

Se este atributo for omitido, significa que o título do antecessor mais próximo desse elemento ainda é relevante (e poderá ser usado como dica para este elemento). Se este atributo estiver difinido como uma *string vazia*, significa explicitamente que o

título do antecessor mais próximo não é relevante (e não deve ser usado como dica para este elemento).

Semânticas adicionais são anexadas aos atributos **title** dos elementos `<link>`, `<abbr>`, `<input>` e `<menuitem>`.

O atributo **title** pode conter várias linhas. Cada `U+000A LINE FEED (LF)` inserida representa uma nova linha. Alguns cuidados devem ser tomados, como a seguir:

```
<p>
  Novas linhas em título devem ser levadas em conta, como este
  <abbr
    title="Este é um
título de multiplas linhas"
    >exemplo</abbr
  >.
</p>
```

Este exemplo define um título de duas linhas.

Veja também

- Todos os atributos globais.
- `HTMLElement.title` que reflete este atributo.